

Optimizing Quantum Computing

Table of Contents

MLGO: a Machine Learning Guided Compiler Optimizations Framework	2
Selecting the Best Compiler Optimizations: A Bayesian Network Approach	13
Learning Compiler Pass Orders using Coreset and Normalized Value Prediction	41
Quantum circuit optimization with deep reinforcement learning	54
Compiler Optimization for Quantum Computing Using Reinforcement Learning	64
Achieving Pareto-Optimality in Quantum Circuit Compilation via a Multi-Objective Heuristic Optimization Approach	70
Leveraging Machine Learning for Quantum Circuit Optimization	74
MQT Predictor: Automatic Device Selection with Device-Specific Circuit Compilation for Quantum Computing	110
MQT Bench: Benchmarking Software and Design Automation Tools for Quantum Computing	132
END	141

MLGO: a Machine Learning Guided Compiler Optimizations Framework

Mircea Trofin*
Google, Inc.
mtrofin@google.com

Yundi Qian*
Google, Inc.
yundi@google.com

Eugene Brevdo
Google, Inc.
ebrevdo@google.com

Zinan Lin
Carnegie Mellon University
zinanl@andrew.cmu.edu

Krzysztof Choromanski
Google, Inc.
kchoro@google.com

David Li
Google, Inc.
davidxl@google.com

Abstract

Leveraging machine-learning (ML) techniques for compiler optimizations has been widely studied and explored in academia. However, the adoption of ML in general-purpose, industry strength compilers has yet to happen.

We propose MLGO¹, a framework for integrating ML techniques systematically in an industrial compiler – LLVM. As a case study, we present the details and results of replacing the heuristics-based inlining-for-size optimization in LLVM with machine learned models. To the best of our knowledge, this work is the first full integration of ML in a complex compiler pass in a real-world setting. It is available in the main LLVM repository.

We use two different ML algorithms: Policy Gradient and Evolution Strategies, to train the inlining-for-size model, and achieve up to 7% size reduction, when compared to state of the art LLVM -Oz. The same model, trained on one corpus, generalizes well to a diversity of real-world targets, as well as to the same set of targets after months of active development. This property of the trained models is beneficial to deploy ML techniques in real-world settings.

1 Introduction

Previous work [13, 25] has shown promise in replacing compiler optimization heuristics with machine-learned policies. Heuristics are algorithms that, empirically, produce reasonably optimal results for hard problems, within pragmatic constraints (e.g. "reasonably fast"). In the compiler case, heuristics are widely used in optimization passes, even those leveraging profile feedback, such as inlining and register allocation. Such passes have significant impact on the performance of a broad variety of programs. These problems are often NP-hard and searching for optimal solutions may require exponential time or memory. Reinforcement Learning (RL) is a family of machine learning techniques that may be applied to find increasingly optimal solutions through an automated iterative exploration and training process.

Our focus is ahead-of-time (AOT) compilers, specifically, C/C++. In a real-world setting, we expect two main benefits from machine learning techniques: first, heuristics are human-trained based on a human-manageable set of benchmarks and regression cases. Machine learning easily scales to large corpora of training examples - which we expect to increase the likelihood of obtaining policies that generalize well. This is important because, as we will explore in detail, we do not want to retrain policies too frequently (it is an adoption blocker), nor do we want to train 'online', while the compiler is running in production (it would affect determinism). Second, heuristics are human-written code that needs to be maintained. This places a downward pressure on the number of program properties ("features") and the combinations between them that can be practically leveraged. We believe using more features and feature combinations would result in better optimization decisions. ML scales well with the addition of features, and can discover profitable feature combinations. While ML techniques may be able to address these two points, a trade-off is that maintaining and evolving them requires practices and approaches different from those used for heuristics.

As pointed out, applying ML to compiler optimizations has been explored by academia, but it has not been adopted in production environments. To explore why, we chose a pilot optimization problem and approached it with the intention to deploy in production. The goal of the pilot is to inform problem framing and design choices. Other than performing better than the tip-of-tree production compiler, we did not aim to advance the state of the art for the pilot problem.

The chosen problem is inlining-for-size in LLVM, and in particular, the inlining decision heuristics. The expectation was that this would offer representative challenges - size optimization is important for real-world scenarios, such as mobile software, and inlining is a particularly challenging optimization (see Section 2.2).

We chose size rather than speed for the pilot because size is relatively easy to measure and non-noisy, which we expected to aid in rapid prototyping by removing one source of potential problems (noisy rewards). We acknowledge that translating our experience to performance problems may

*These authors contributed equally.

¹We welcome your feedback! Please open an issue at <https://github.com/google/ml-compiler-opt> with the label *paper*.

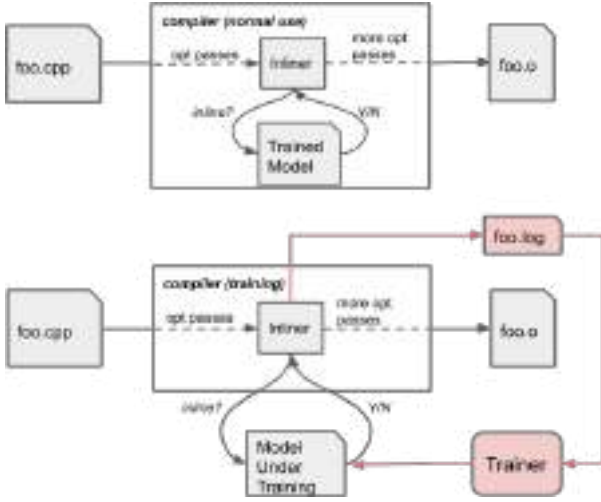


Figure 1. MLGO Overview

seem non-immediate at a first glance, but we believe that not to be the case: for example, inlining for speed can be understood partially as a “for-size” problem, with respect to the instruction working set (more in Section 8).

From a very high level, our MLGO framework separates the *use* of the compiler from the *training* of policies as shown in Figure 1. Day-to-day production use is unchanged; as an implementation detail, a trained model embedded in the compiler is used to make decisions (in this case, inlining) that were previously handled by a manual heuristic. Training happens separately using a large, representative corpus of intermediate representation (IR) modules. The training process is iterative, each step using an updated policy, so the policy is not embedded in the compiler. During training, the inliner produces a log that records the inlining process (features, decisions, etc). The logs are collected and fed to the training algorithm to produce a new model.

The paper is organized as follows: Section 2 provides an overview of the relevant ML techniques and the LLVM inliner. Section 3 gives an overview of MLGO: our framing of the problem of applying ML to compiler optimizations, and our methodology. Section 4 describes the policy training in MLGO for the inlining problem. Section 5 details the implementation, in LLVM, of our pilot project. The results are presented in section 6 and the related work is described in Section 7. Finally in Section 8, we discuss our plans for applying the lessons learned so far to speed problems, as well as next steps in ML techniques that we are considering. Section 9 concludes the paper.

A note: throughout this paper, we use interchangeably the terms **policy** and **model**. A compiler optimization policy is a decision rule that takes actions inside the optimization pipeline (e.g., “should we inline this call graph edge or not?”).

“Model” refers to a neural network implementing an optimization policy. Also, we use the term **heuristic** to refer to manually-crafted decision rules.

2 Background

2.1 Machine Learning Techniques for Replacing Compiler Optimization Heuristics

There are two characteristics that make reinforcement learning (RL) a suitable tool for replacing compiler optimization heuristics: 1) there are no examples showing optimal strategies for these heuristics — in the inlining problem, we don’t know whether inlining or not for a certain call site is the optimal choice; 2) we can efficiently explore different strategies, and improve strategies from those experiences. The absence of examples (“labels”) means we cannot use supervised learning. In contrast, RL is an area of machine learning that learns from trial and error instead of given labels. It has proven its success in robotics, playing Atari games, playing the game of Go, etc [16, 18, 24]. In RL, an agent (i.e., the compiler) learns by repeatedly interacting with the environment (i.e., compiling) and gradually improves its policy (i.e., decision rules). More specifically, by compiling software again and again with different strategies, the compiler will come up with better and better policy on its own with RL algorithms.

Previous work has shown Evolution Strategies (ES) to be a competitive alternative to RL algorithms in MuJoCo and Atari tasks [8, 22]. Motivated by this, we also tried this method in compiler optimization problems. ES are a class of black box optimization techniques. Like RL, ES training is also able to gradually improve the strategy with trial and error, and thus is also a suitable tool for compiler optimization problems.

2.2 High-level Overview of the Current LLVM Inlining Pass

Today’s LLVM inliner is a pass operating on a strongly-connected component (SCC) of the static call graph in a module², at a time in bottom up order. The inlined callee’s call sites are added to a work list and iteratively considered for inlining in a top down fashion. A pipeline of optimizations (Inst combine, scalar replacement of aggregates (SROA), loop optimizations, etc) is then applied on each function in the SCC³, after the SCC was processed. The effects of these optimizations impact inlining decisions of call sites in the SCC calling into the last one.

LLVM inlining consists of many heuristics: the choice of call site traversal, the set of “cleanup” function passes run on functions after they are modified because call sites were

²in LTO or ThinLTO mode, a module can consist of IR from multiple source files

³The DAG walk is repeated up to a set of times if de-virtualization happens in cleanups

inlined, the timing of these cleanups, and finally, the decision to inline or not a specific call site.

The decision to inline or not a call site is itself built on top of a rich set of heuristics. The compiler first computes the static "cost" of the callee post inlining by traversing the callee body simulating post-inline cleanup passes. If some call site arguments are known to be constant at compile time, that information is used to evaluate what instructions / basic blocks would be simplified, should the inlining be carried out. The computed cost is then compared with a threshold. The threshold value is based on things like call site hotness, inline keyword, etc. Bonuses are also given to callees with a single basic block or high percentage of SIMD instructions. In certain cases, the compiler may also choose to defer inlining if inlining the caller itself to its own callers first may result in better savings - it may be better to make a local non-optimal decision that, later, would open the opportunity for better optimizations due to more context being available, such as call parameters propagating from callers further up in the call graph.

These sets of heuristics have been tuned for years and our pilot project replaces the manual decision process described above with ML models.

3 MLGO

MLGO is a set of guidelines and requirements derived from our understanding of the problem of leveraging ML techniques for replacing manual optimization heuristics. We start with our understanding of the participating personas and their scenarios, which then motivates MLGO guidelines and design decisions.

3.1 Persona: Compiler User

This user wants to benefit from improved compiler optimizations. They care about: correctness and performance of the generated code; compilation determinism (i.e. identical output for identical input) - to leverage incremental builds; avoiding the added cost and complexity of new infrastructure requirements on build and release pipelines, such as new compiler run-time dependencies or new steps (like training); and timeliness of the build, as it impacts hardware resource planning and developer productivity⁴. **Our goal is to introduce no changes to this user.**

To achieve this, the MLGO guidelines are:

1. To maintain correctness guarantees, we replace heuristics, not semantics-preserving code. For example, we change the decision making process for carrying out function inlining, not how the inlining action is implemented. This is along the insight of separation of correctness vs policy observed earlier in [27]

⁴There are non-ML driven optimization alternatives that trade off significantly increasing compilation time for improved optimizations. An ML alternative needs to be competitive to justify its other trade-offs

2. 'Online' training - meaning, training while the compiler is executing in production - is an anti-goal for us: it would hurt determinism and compilation performance. Instead, policy training happens offline. Trained policies are embedded in the compiler as statically linked native code, and the resulting compiler is subjected to the same release process it currently is. Build and release infrastructures and pipelines of targets using the compiler do not need to be changed. Build determinism using the ML-enabled compiler is ensured because the policies are fixed - no training happens when the compiler runs, only inference. While native compilation doesn't guarantee timeliness, it eliminates one source of concern. Due engineering diligence still needs to be applied to ensure timely feature extraction, for instance.

3. We require ML techniques that yield policies that generalize well over different code bases and code changes, and do not need frequent retraining. The compiler user doesn't *have* to worry about policy training (although they are free to do so and potentially get better results). This is akin to how, in the context of manual heuristics, a compiler user doesn't have to fine-tune passes (or author code in them) to get reasonable results. In particular, we do not see automated tuning of existing heuristic parameters as a viable solution. Tuning parameters have been available in compilers for a long time, and the experience has been that a set of values does not translate well from target to target. The policy, while adjustable, is still dominated by combinations/evaluations identified manually. In addition, requiring re-tuning would complicate product build and release pipelines, which we want to avoid on behalf of our user.

We refer to this use of policies as **release mode** (since it is encountered by users of a released/shipped compiler).

3.2 Persona: Compiler Engineer

This user wants to drive better optimizations in the compiler, diagnose regressions, and incorporate findings:

1. **Policy Creation.** The engineer wants to incorporate ML techniques in a compiler optimization pass.
2. **Policy Improvement.** Here, they investigate a specific regression encountered in production, or want to improve a ML-enabled pass.
3. **"The Ship Blocker".** The engineer must quickly resolve a ship-blocking regression introduced by a hot patch, and caused by a misbehaving RL-enabled policy.

In all of these cases, the *compiler engineer* improves a policy through repeated exploration and training (see Section 4.4). They want flexibility in replacing the model under training, and have less concern with timeliness and determinism, especially since models under training may use small random

perturbations, to facilitate exploration. We refer to this use of policies as **development mode**. Here, models are loaded via a command line option, the compiler may have extra runtime dependencies, and model evaluation may involve changes to the runtime behavior of the compiler — because, for example, the model evaluators may be multi-threaded and/or JIT-ing.

Because of the tension between heuristic code complexity and hypothesized ability to improve the heuristic by incorporating more features (as discussed in the introduction), MLGO forgoes goals of human comprehensibility of the resulting policy (in contrast to [25]). Instead, we focus on developing and evaluating alternative methodologies to address the above scenarios. We discuss our current understanding of the trade-offs, and expect that more clarity will arise as we apply the approach through the lifetime of a number of diverse projects. Section 4 will detail our experience with developing *Policy Creation*. We have less experience, at this point, with *Policy Improvement* and *"The Ship Blocker"*, and derive our direction from experiences in other domains.

Policy Improvement is currently (i.e. for manual heuristics) an iterative engineering process. The trigger is typically regressions identified in the field. The compiler engineer diagnoses the problem, hypothesizes a solution, then ensures that the solution does not introduce regressions in some corpus of benchmarks; if regressions happen, the process is repeated. In the MLGO methodology, we envision a gradual process. We do not believe it presents significant negative trade-offs compared to the state of the art:

1. Start by incorporating regression use-case(s) into training corpus and retrain the policy.
2. If that fails, hypothesize missing features. This requires some manual diagnosing of the current policy behavior. While we treat the policy as a black box, we do observe its effects, and can formulate hypotheses as to what information may be missing - since the information we provide (features) is also observable. The needed skill set is close to what compiler engineers currently employ for manual heuristic development, and, just like for manual heuristics, evolving the feature set is likely an iterative process. Typically, adding features and retraining shouldn't result in regressions for the previous training corpus, which is a benefit of our approach over the manual heuristic case. We should note that, if applying feature auto-extraction [10, 13] proves feasible in production, this step collapses to the previous step.
3. If the above also fails, involve an ML expert to investigate alternative training algorithms. This is akin to today's (rare) full pass rewrites (for example: new register allocation pass). The difference is the need for cross-disciplinary interaction. Our hope is that, with time and experience, MLGO offer a reusable library of

best practices and training solutions available "off the shelf" to compiler engineers.

Ship blockers are those cases where the compiler engineer doesn't have the luxury to do deep investigations into compiler behavior, since they are on a tight time budget. Assuming the pathological case is identified (i.e. which compilation unit causes the compiler to misbehave), in the case of manual heuristics, the levers of control are: trial-and-error with different compilation flag values (change policy thresholds, for instance); modify user code (use inlining directives, for instance); or disable the specific optimization for a specific module.

In MLGO, the picture is similar. Other than policy thresholds, the control levers available to the engineer are the same. In addition, the engineer may choose to revert to an earlier version of the policy, or manual heuristics for the problematic module, and, if needed, experiment with threshold flags. Specific to ML-based policies, we are exploring with local training and overfitting: as we will detail, our experience so far shows that it is possible to train a policy on a single modern, multi-core workstation, and obtain a reasonably good result within a day. An engineer could attempt to specialize a policy to overfit for the pathological case, and compile that case with the specialized policy (while compiling the rest of the project with the non-overfitted policy). This is similar to "experimenting with flags", with the exception that the exploration is directed by a training algorithm and more likely to quickly converge to a solution. The trade-off is that changing heuristic flag values does not require a training infrastructure — even if that infrastructure could be run locally.

4 MLGO Policy Training

In this section, we show how we use reinforcement learning (RL) and evolution strategies (ES) to train inlining policies in the MLGO framework. Sections 4.1 and 4.2 present how we train the inlining policy for the inlining-for-size problem with RL and ES algorithms, and Section 4.3 compares the pros and cons of the two algorithms. Section 4.4 concludes this section by giving an overview of our policy training infrastructure⁵.

4.1 Reinforcement Learning

4.1.1 RL Problem Formulation

RL aims to find an optimal policy for a Markov Decision Process (MDP). MDP is a mathematical framework that models sequential decision making — in inlining-for-size problem, we make sequential decisions whether to inline or not. An MDP can be represented by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R} \rangle$ with state space $\mathcal{S}$, action space $\mathcal{A}$, state transition distribution $\mathcal{P}(s'|s, a)$, and reward function $\mathcal{R}(s, a)$. In the MDP formalism, at time t , the agent observes the state $s_t \in \mathcal{S}$ of the

⁵<https://github.com/google/ml-compiler-opt>

environment, then decides to take an action $a_t \in \mathcal{A}$. It also receives the reward $r_t = \mathcal{R}(s_t, a_t)$. The environment state then transitions to $s_{t+1} \in \mathcal{S}$ by sampling from the probability distribution $\mathcal{P}(s_{t+1}|s_t, a_t)$. This process repeats until the agent reaches a termination state at time T . The agent's decisions are a function (we call it policy) $\pi = Pr(a|s)$ that maps observed state s to a distribution over actions. In our case, π is a neural network and we call it policy network. RL algorithms aim to find the optimal policy π^* to maximize the total reward⁶ $R = \sum_{t=0}^T r_t$.

We first formulate the inlining-for-size problem as an MDP. The inlining pass traverses over the call sites in the call graph in a deterministic order and decides at each call site whether to inline or not. Every inlining operation changes the call graph. We treat this as a sequential decision process, and we formulate it into an MDP as:

state $\mathcal{S}$: we define the current call graph and the call site being visited to be the state.

action $\mathcal{A}$: $\mathcal{A} = \{0, 1\}$, where 1 means inline and 0 means do not inline.

state transition probability $\mathcal{P}$: unlike usual MDPs, the state transition is deterministic (no randomness) in the inlining problem. After an action is taken (inline or not inline), the compiler determines what the next state is (updates the call graph and decides the next call site to visit).

reward $\mathcal{R}$: reward is defined to be the native size reduction after the action is taken. If $a = 0$ (do not inline), the reward is 0 since nothing changes; if $a = 1$ (do inline), the reward is defined as:

$$S(Caller_{before}) - S(Caller_{after}) + \begin{cases} S(Callee), & \text{callee deleted} \\ 0, & \text{callee remains} \end{cases} \quad (1)$$

where $S(f)$ is the native size of function f . Note that we do not actually know what the native size would be for a certain function while performing inlining, since inlining operates at the IR level. The definition for reward here is not practical for training. We will discuss how we tackle this challenge next.

4.1.2 Policy Gradient Algorithm

Policy Gradient (PG) [28] is a family of RL algorithms derived from REINFORCE [34]. Though we use Proximal Policy Optimization (PPO) [23], we first briefly introduce REINFORCE — as PPO is an enhancement to REINFORCE and they work in very similar ways.

On a high level, all PG algorithms gradually improve the policy π_θ by computing the gradients of the parameters θ in the policy network w.r.t. the total reward R , and then update θ with the gradient to improve the policy. With $J(\theta)$ denoting the expected reward under policy π_θ , the gradient

$\nabla_\theta J(\theta)$ in REINFORCE is computed as:

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\sum_{t=0}^T R \nabla_\theta \log \pi_\theta(a_t|s_t) \right] \quad (2)$$

Here $\mathbb{E}$ is an expectation over the policy π_θ being applied to an inlining pass. In practice, this expectation is approximated with Monte Carlo methods — with n trajectories⁷ collected from compiling with policy π_θ , the parameter θ is updated with:

$$\theta \leftarrow \theta + \alpha \frac{1}{n} \sum_{i=1}^n \left\{ \sum_{t=0}^T R_i \nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t}) \right\} \quad (3)$$

where α is the learning rate. As θ is updated, the policy $\pi(\theta)$ tends to evolve in the direction that increases the total reward. Algorithm 1 describes the process — as training progresses, the policy gradually improves on its own by iterating between two stages: 1) compile with a new policy and collect fresh trajectories; 2) update policy network parameters θ .

Algorithm 1 MLGO PG Training Algorithm

- 1: Initialize θ
 - 2: **for** iteration = 1, 2, ... **do**
 - 3: Compile with policy π_θ to collect n trajectories
 - 4: Update θ using Equation 3
 - 5: **end for**
-

The details of training with PPO, which has several additional terms in the loss function, are available in [23]. One core improvement of PPO is to subtract a baseline B from the reward to reduce the variance. Equation 2 is modified as:

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\sum_{t=0}^T (R - B) \nabla_\theta \log \pi_\theta(a_t|s_t) \right] \quad (4)$$

Here the baseline B describes what the R is supposed to be (irrelevant to policy). By subtracting it, $R - B$ provides better information about the effectiveness of the policy π_θ .

The total reward R in these equations can be replaced with the returns following action a_t : $\sum_{t'=t}^T r_{t'}$. In this case, the baseline B is a value network $V(s_t)$ predicting the future returns $\sum_{t'=t}^T r_{t'}$ from the state s_t . We choose to use the total reward R since: 1) it is directly available in the inlining-for-size problem, while partial returns would have to be approximated; 2) it is difficult to build the value network $V(s_t)$ with the reduced state. We will discuss the details in the next section.

⁶In general, the total discounted reward is $R = \sum_{t=0}^T \gamma^t r_t$, where γ is the discounting factor; but we take $\gamma = 1$ so we ignore it here.

⁷A trajectory is defined as $(s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$, and total reward $R = \sum_{t=0}^T r_t$

4.1.3 Challenges and Implementation Details

We run into two challenges when applying PPO to the inlining-for-size problem: 1) complex state space; 2) impractical reward definition.

Complex state space: Our MDP formulation defines the state as the current call graph and the call site being visited. Unfortunately, encoding and processing a call graph at each decision point may not be computationally practical for a general-purpose compiler to afford.

Impractical reward definition: It is difficult to know a function’s native size $S(f)$ during the inlining pass because native code lowering happens in a later pass, and because its structure may change due to more of its call sites being inlined.

To tackle the first challenge, we approximate the true state by distilling the state space to 11 numerical features as listed in Table 1. These features describe the local call site and provide some global information about the call graph. Section 5.1 details the features we use. We considered, but rejected for now, the use of (IR) code embedding techniques[3]; this allows us to minimize additional computational/memory costs. We plan to consider such techniques in the future.

Type	Feature
caller feature	caller_basic_block_count
	caller_conditionally_executed_blocks
	caller_users
callee feature	callee_basic_block_count
	callee_conditionally_executed_blocks
	callee_users
call site feature	callsite_height
	cost_estimate
	number_constant_params
call graph feature	edge_count
	node_count

Table 1. Features for Inlining for Size

One drawback of the simplified state is that it greatly reduces information available to the policy — it only contains a part of the local call site information, and limited global call graph information. We do not expect this to hurt the policy network because it is roughly the same information available to the current inlining heuristic. However, this reduced state vector does not allow us to build the value network $V(s_t)$ baseline — at a certain time t , the simplified state s_t is not informative enough to predict the future return $\sum_{t'=t}^T r_{t'}$.

A simple approach to side-stepping the lack of partial reward information and the side effect of reduced state representation is to use the total reward R instead of the partial return as shown in Equation 2. While the per-step reward is difficult to get, the total reward is the sum of the individual

(unknown) rewards — it is relatively easy to evaluate: evaluate native size with / without inlining, and subtract. In the total reward setup, the baseline B is defined as the estimated native size reduction of the module after the inlining pass. We can use the native size reduction under the heuristic policy as the baseline B .

Using the total reward instead of partial rewards has its drawbacks: 1) more data needs to be collected to achieve the same performance; 2) the final model quality may be worse.

4.1.4 Warmstart with Behavioral Cloning Policy

Instead of having the RL algorithm learn from the scratch (initialize θ randomly), we facilitate training by initializing θ from some "warmstart" policy. To facilitate the RL training, we need to have a "warmstart" policy that already performs reasonably well. An intuitive idea is the heuristic inlining decisions in LLVM. Therefore, we train the warmstart policy to imitate the heuristic inlining decisions in LLVM using behavioral cloning algorithm[4]. The behavioral cloning algorithm essentially views the problem as a supervised learning problem where the features are the same as the RL training and the label is the heuristic inlining decision — it trains a neural network that makes inlining decisions as close as the heuristic inliner does. In this way, we get a policy that makes decisions similar to LLVM’s current inlining heuristics and thus can serve as the warmstart policy to make our RL training much faster.

4.2 Evolution Strategies

Previous work has shown that ES, as a gradient-free black box optimization technique, is a competitive alternative to RL algorithms on MuJoCo and Atari tasks [22].

ES focuses on black-box optimization problems of the form $\max_{\theta} F(\theta)$, where F can be any *black-box function* that can be evaluated. Given θ , we essentially assume we have an oracle that calculates $F(\theta)$. In our specific case, θ are the parameters of the policy network π_{θ} and $F(\theta)$ is the total reward R — the native size reduction after inlining under policy π_{θ} for a certain module.

Instead of directly optimizing $F(\theta)$, ES focuses on optimizing $J(\theta)$, which is a smoothed version of $F(\theta)$:

$$\max_{\theta} J(\theta) = \max_{\theta} \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} F(\theta + \sigma \epsilon). \quad (5)$$

Here $\mathcal{N}(0, I)$ denotes the multivariate normal distribution with zero mean and identity covariance matrix. Similar to PG, ES also takes the gradient of the parameter θ w.r.t. $J(\theta)$:

$$\nabla_{\theta} J(\theta) = \frac{1}{\sigma} \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} \{F(\theta + \sigma \epsilon) \epsilon\}, \quad (6)$$

and uses Monte Carlo approximation of the gradient to update θ to improve the policy:

$$\theta \leftarrow \theta + \alpha \frac{1}{n\sigma} \sum_{i=1}^n \{F(\theta + \sigma \epsilon_i) \epsilon_i\} \quad (7)$$

where α is the learning rate, and ε_i are vectors sampled from $\mathcal{N}(0, I)$.

Algorithm 2 describes the ES algorithm. Similar to PG, ES also iterates between data collection and policy update to gradually improve the policy.

Algorithm 2 MLGO ES Training Algorithm

```

1: initialize  $\theta$ 
2: for iteration = 1, 2, ... do
3:   Sample  $\varepsilon_1, \varepsilon_2, \dots, \varepsilon_n \sim \mathcal{N}(0, I)$ 
4:   Compile with policy  $\pi_{\theta+\sigma\varepsilon_i}$  to get  $F(\theta + \sigma\varepsilon_i)$ 
5:   Update  $\theta$  based on Equation 7
6: end for
  
```

4.3 PG v.s. ES

While the policy gradient algorithm and the evolution strategies algorithm are similar on a high level, they are different in many ways and have their pros/cons.

Complexity: The key advantage of ES is that it is conceptually simpler: 1) it requires less engineering complexity — unlike the PG algorithm where we need the logged trajectory during inlining ($s_1, a_1, s_2, a_2, \dots, s_T, a_T$) and the total reward R for training, the ES algorithm only needs the total reward. Thus we do not need to log the trajectory while doing compilation; this reduces both engineering complexity and storage/network requirements; 2) it has less requirements on the problem structure as long as there is an oracle telling the total reward $F(\theta)$ under the policy parameters θ . As a result, it is easier to apply ES to other compiler optimization problems as PG requires formulating the optimization problem as an MDP.

Sample Efficiency: Sample efficiency quantifies the amount of data required for training. The key advantage of PG is that it has much higher sample efficiency than ES. In the inlining-for-size problem, we observed that even though PG is trained using total reward, over 20X computational resources are needed to train an ES policy of similar quality. In problems where partial reward information is available after every decision point, we expect the sample efficiency gap to be even larger.

4.4 Training Infrastructure

PG and ES algorithms are very similar in policy training on a high level — both of them improve the policy π_θ by iterating between compiling with policy π_θ to collect data and update parameters θ . Figure 2 demonstrates their training workflow. Before the training, we prepare an IR corpus consisting of pre-inlining IR files extracted from some software. At each iteration, the trainer sends the policy π_θ to data collector, the data collector samples several IR files from the IR corpus, does compilation to collect training data, and sends the training data back to the trainer for training. The training

is done after several iterations, and the trainer exports the trained policy. We use TF-Agents[29] — an RL library in TensorFlow[1] for training and the policy is in the format of TensorFlow SavedModel.

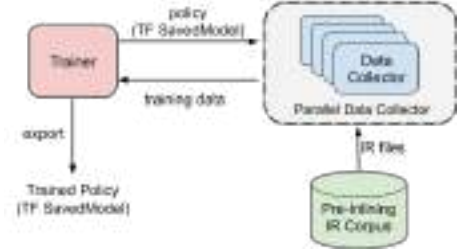


Figure 2. System Overview: Policy Training

The bottleneck of training for the inlining problem is data collection. Therefore, the data collection is carried out in a parallel way to improve the overall training efficiency.

Figure 3 details how the data collector module works. It is supported by the development mode in MLGO framework. It takes a pre-inlining IR file and a policy (optional) as inputs, conducts inlining on the IR file based on the policy, has the post-inlining IR file optimized by other opt passes after inlining, converts the optimized IR file into native code, and gets the native size of this module. The native size, together with the log file generated during inlining by MLGO that contains the trajectory ($s_1, a_1, s_2, a_2, \dots, s_T, a_T$), composes the output of the data collector module — training data. If the policy is not given, the inliner will conduct the current heuristic inlining and log the trace. It has two use-cases as discussed in Section 4.1: 1) collect data to train the warmstart policy with behavioral cloning algorithm; 2) use the heuristic inlining as the baseline. The ES algorithm only needs the reward (native size) for training so the log file is not needed.

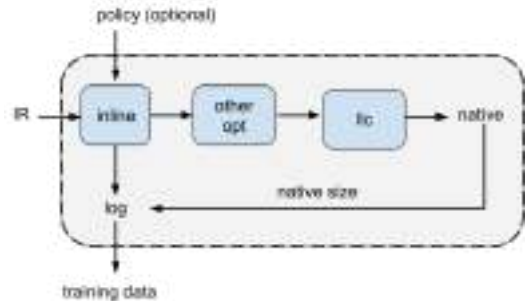


Figure 3. Data Collection for Inlining-for-Size

5 LLVM Implementation

We implemented the pilot project in LLVM⁸, together with reusable support for release and development modes, as well as continuous integration build bots.

5.1 RL Driven Inlining (for Size) in LLVM

We introduced an abstraction for the inline decision-making policy, the *InlineAdvisor*, and a module analysis, *InlineAdvisorAnalysis*, that may be used to retrieve the *InlineAdvisor*. The analysis can not be accidentally invalidated by other passes. This is necessary, since the inliner pass is interleaved with the execution of function passes, as previously discussed, and we want to track module-wide features throughout the performance of inlining and related passes over a module. Instead, the analysis is managed explicitly - see *ModuleInlinerWrapperPass* (`llvm/Transforms/IPO/Inliner.h`). The specific implementation of the advisor is chosen through a LLVM flag (`-enable-ml-inliner`). By default, the implementation is the manual heuristic. Passing `'release'` or `'development'` to the flag selects the respective mode, if the compiler was built with support for that mode.

Feature extraction is modeled as a separate analysis, *FunctionPropertiesAnalysis*, and reused by the release and development implementations. The full feature set is captured in `llvm/Analysis/InlineModelFeatureMaps.h`. We capture some call site-local information, as well as global information, such as module-wide number of functions and static calls, caller/-callee user counts; position in the original call graph, as the distance of the call site to the farthest SCC; and an estimate of removed instructions given the call site context.

We use TensorFlow [1] as the model training and inference framework. In both modes, the interface between LLVM and the model is defined solely in terms of input and output tensor specifications (tensor name, type, and shape). The internal structure of the model is an implementation detail. This means that during training, the compiler engineer is free to explore hyper-parameters or add/modify hidden layers. Also, ingesting a new model with a different internal structure, in release mode, is just a matter of recompiling LLVM.

Refer to `lib/Analysis/{MLInlineAdvisor | ReleaseModeModelRunner | DevelopmentModeInlineAdvisor}.cpp` for more implementation details. At a high level, both release and development modes:

- Handle user inlining directives and correctness aspects (these are done without model evaluation)
- Extract the features *associated* with a call site and form fixed-sized tensors (primitive data type vectors), and efficiently maintain the module-wide features
- Pass the tensors to the model evaluator, and request it perform an evaluation

- Take the result of the evaluation as advice (i.e. inline/don't) and make that available to the inliner pass.
- It is possible that a policy misbehaves in unforeseen circumstances (which, as a note, should then be incorporated into the training loop). The resulting IR, while correct, could become increasingly expensive to process by subsequent passes. To avoid this, we set a hard threshold to the amount by which the number of instructions may grow in a compilation unit.

5.2 Release Mode Implementation in LLVM

The model is encoded in the TensorFlow serialization format, *SavedModel* [31], which is compiled into native code by the `saved_model_cli` tool [32]. To use this tool, we added a build rule (see `llvm/cmake/modules/TensorFlowCompile.cmake`) to the LLVM build system. Applying the rule to a model generates a header file and an object file. From here, the model may be consumed as a C function; for simplicity, the *SavedModel* compiler provides a thin C++ wrapper, exposing plain C/C++ APIs (primitive types), which is compiled as part of the LLVM build process. The *SavedModel* is checked in as source⁹.

To build with support for the release mode, the *SavedModel* compiler must be available during the build time of LLVM. The compiler may be installed through a python pip package¹⁰. Once installed, its location is provided to the LLVM build via the `TENSORFLOW_AOT_PATH` cmake flag. Specifying that flag also defines a conditional compilation flag, `HAVE_TF_AOT`, which enables the compilation as part of the Analysis component of the support for release mode.

Note that this mechanism would build the release mode implementations of *all* optimization passes that have RL-driven policies, meaning that implementers just need to reuse the same mechanisms - conditional compilation flag, build rule, etc - to plug in a ML-based policy replacement.

5.3 Development Mode Implementation in LLVM

As discussed, in development mode, we want to support loading models from the command line. For the development mode, the build time dependency is to the TensorFlow C API library, instead of the TensorFlow pip package. Model loading, initialization, and evaluation is performed via a reusable C++ API wrapper (see `lib/Analysis/TFUtils.cpp`) that simplifies the programming model of this aspect of development mode implementations.

⁹The *SavedModel* separates the evaluation graph structure from the value of the trained weights used for evaluation. The graph is stored as text. The weights/serialized float arrays are stored as a binary blob. Their evolution (due to training) does not diff well, so the compactness of a binary format is more economical for the project repository.

¹⁰See the buildbot setup script available at https://github.com/google/ml-compiler-opt/blob/58bf347286c21519b3cc418f659c485cbb7ad82f/buildbot/buildbot_init.sh

⁸Code references are made in the context of commit 71059257bd4.

In addition to facilitating a different model ingestion mechanism, the development mode is responsible for producing traces necessary for training ("training logs")¹¹. These logs capture the succession of feature values observed when the policy is asked to make a decision, and the decision made afterwards ("trajectories"). Training logs may be produced for both the heuristic policy (for bootstrapping training - "warmstart") as well as for the ML policy currently under training. Exploration - i.e. deviating from policy, with the purpose of finding new learning opportunities - is delegated to a TensorFlow mechanism that introduces some randomness in decisions. This mechanism is an implementation detail of the model as produced by the training algorithm, and is outside the control of the compiler. Care must be taken to remove such randomness before shipping a model, and re-validate its effectiveness. We encode the training logs as textual SequenceExamples[30] proto-buffers, the typical abstraction Tensorflow training algorithms would expect. We produce a textual output to avoid an additional dependency to LLVM, and to simplify diagnostics and testing of the feature.

Analogous to the release mode, enabling development mode in LLVM requires the dependency be made available to the build system. In this case we use the `TENSORFLOW_C_API` flag, which in turn defines the `HAVE_TF_API` conditional compilation flag. More details may be obtained from the previously noted build bot scripts. Also similar to the release mode, this mechanism enables all cases that have the TensorFlow C API library dependency. Unlike release mode, the development mode's use of the TensorFlow C library is a run-time dependency, and needs to be on the loader path.

6 Evaluation

6.1 Compilation Overhead

Model evaluation in release mode has **fixed cost**, both in terms of compiler run-time memory utilization, as well as CPU utilization. This is because models are fixed size graphs connecting functional operators, taking fixed sized inputs, using constant weights, and producing fixed sized outputs. For the current model, we observed 0.65% increase in memory utilization at run-time. When inlining a large IR module (33MB), we measured a 10% increase in **inlining** time, mostly attributable to feature extraction; since inlining tends to represent 10-15% of **total** compile time, the net contribution of the release mode is only 1%. Finally, clang binary size increase due to the inclusion of the compiled model was 115KB, representing 0.08% size increase.

We did not formally measure the overhead of the development mode, mainly because timeliness is less of a concern here, and also because model evaluation may happen through a variety of means, including JIT-ing, which makes measurements more unstable. We did want to validate that

¹¹RL algorithms require these. ES algorithms do not

	PG	ES	ES (L)
Size Reduction	4.95%	3.74%	5.94%
Parallelism in Data Collection	100	488	488
Training Time	~12h	~60h	~150h

Table 2. Policy Gradient v.s. Evolution Strategies

the solution is practically usable in training loops, and observed 26K IR modules being inlined in parallel on a 72 thread machine, 192GB RAM, in around 10 minutes, and without going past half of the available RAM¹².

6.2 Inlining for Size Results

We trained the inlining for size policy on an internal search application containing over 28000 IR modules with a variety of different code patterns¹³. The rich set of patterns will improve generalizability, across both time and software domain, of the trained policy. As mentioned, this is important for real-world deployment.

We trained the policy using both PG and ES on the internal search software. Table 2 compares their effectiveness in terms of reduction of the .text section compared with heuristic-driven -Oz. We trained 3 policies: PG and ES with a 2 hidden layer (40, 20) neural network, and ES(L) with a deeper 4 layer (20, 20, 20, 20) neural network¹⁴. We can see that: 1) PG has better sample efficiency than ES — it consumes ~5% training resources of ES (100 * 12 v.s. 488 * 60); 2) better policies may be achieved with a larger neural network at the cost of more training resources; 3) we can train a reasonable PG policy on a single multi-core (e.g. 72 threads) high-performance machine well within a day.

6.2.1 Generalizability across Software

We deploy the trained PG and ES policies to a wide range of software to evaluate their generalizability. Figure 4 shows how the 3 models we trained on the search application perform on 3 different internal applications and on Clang¹⁵. Figure 5 shows their effectiveness on SPEC 2006. We can see that all the 3 policies show good generalizability — they are able to reduce the native size to some extent. Policy effectiveness is ES(L)>PG>ES for most software, which is the same as what we see on the search application. It also suggests good generalizability as a policy performs better on a certain software is likely to also perform better on other software.

¹²Anecdotally, we were able to build Fuchsia using a development mode clang, and timeliness was not a noticeable issue.

¹³We also have an end-to-end demo at <https://github.com/google/ml-compiler-opt/blob/main/docs/demo/demo.md> that trains on publicly available code and achieves similar performance.

¹⁴Detailed hyper-parameters at https://github.com/google/ml-compiler-opt/tree/main/compiler_opt/rl/inlining/gin_configs

¹⁵Specifically, clang @4ca60915bcc (2020/8/28) building clang @d469133f95b (2020/4/25).

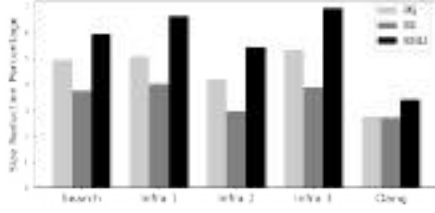


Figure 4. Generalizability across Software

6.2.2 Generalizability across Time

To evaluate the trained policies’ generalizability across time, we deploy the 3 trained policies on the same software as in Figure 4 4 months later. We also use the LLVM 4 months later¹⁶. Both the software and the compiler have been under active development in that period. Figure 6 demonstrates the results. We can see that their effectiveness may degrade somewhat (compared with Figure 4), but they still have decent wins compared with the current -Oz.

7 Related Work

There have been many academic efforts in using machine learning and related techniques to replace hand-crafted heuristics in compilers. Our contribution is identifying the problem framing and design constraints that enable applying these techniques to production.

Wang and O’Boyle [33] present an extensive survey of the use of machine learning in compiler optimizations. Most, however, employ supervised learning techniques, which, as explained, are not in our scope. The closest, Cavazos et al. [6] used unsupervised learning to automatically tune the inlining parameters (thresholds) of a research Java Virtual Machine (JVM), which features a very simple manually written heuristic. In subsequent work, Simon et al. [25] construct a heuristic as a decision tree, to address maintainability and evolvability. While the ML techniques are similar to what we are using in MLGO, both parameter tuning and direct policy comprehensibility are counter to our goals, as described in section 3.

Adams et al. [2] employed ML to train a cost model to automatically schedule Halide programs for image processing. With runtime sampling, the cost model is used to find the optimal schedule parameters using beam search. Similarly, Chen et al. [7] used deep learning to train a statistic model for TensorFlow programs.

Inlining-specific, Dean et al. [11] build a database of observed decisions and their effects, and consult it in subsequent compiler runs - while this is not learning, it is a precursor of efforts in this area. In [9], Cooper et al. presented a scheme to parameterize the inline heuristics (decision tree) and the hill-climbing parameter space search.

¹⁶Clang selfhost @4ca60915bcc (2020/8/28).

Haj-Ali et al. [13] use reinforcement learning to instrument source code with pragma directives to drive the vectorization pass. The policy does not replace a compiler heuristic, rather it informs one, by augmenting source code as a pre-build step. This does not make the technique transparently deployable for compiler users. That being said, we currently see no fundamental reason their solution cannot be adapted to MLGO. The main practical issue we see is understanding trade-offs of automated feature extraction, which we intend to explore as a next step as well.

Supervised learning is used by Stephenson and Amarasinghe [26] to predict loop unrolling factors and by Eliot et al. [19] to train a local (single basic block) instruction scheduler. It uses a machine model to predict the so called preference relationship given a partial schedule and two candidate/ready instructions. Cummins et al. [10] automatically extract features from source code, and use supervised learning to learn heuristics for predicting optimal mapping for heterogeneous parallelism and GPU thread coarsening factors.

Instruction scheduling is a hard problem in the compiler that extensively uses heuristics. The application of learning to instruction scheduling within straight line code has been explored by Moss et al. [20] and McGovern et al. [17].

Data prefetching plays a similar role in bringing *data* into the processor without stalls. In [15], Hashemi et al. treated the memory prefetching strategies as an n-gram classification problem in natural language processing, and used LSTM based Recurrent Neural Network (RNN) to do the prediction. Peled et al. [21] define the notion of *semantic locality* and use reinforcement learning techniques to build a context-based memory prefetcher that approximates semantic locality.

Another approach to optimize programs without dealing with specific optimizations is *super-optimization*. This refers to the process of finding a better version of a given program that is semantically equivalent. Early efforts in super-optimization relied on brute force search. Recent efforts have focused on using stochastic search to improve the efficiency. Bunel et al. [5] have used reinforcement learning to optimize stochastic search based super-optimization techniques.

Milepost GCC [12] is a self-tuning GCC-based compiler, where program features are used to predict compiler flags beneficial to some goal (such as performance or size). It does not use ML-trained policies as part of its implementation.

8 Future Directions

8.1 Applying MLGO to Speed Optimizations

The immediately-observable difference between our pilot project and speed problems is that the reward is measured differently: speed is measured through benchmark runs, which are more time consuming and more noisy than size measurements. Using benchmark runs results as reward for speed optimization will have difficulties scaling, so our current

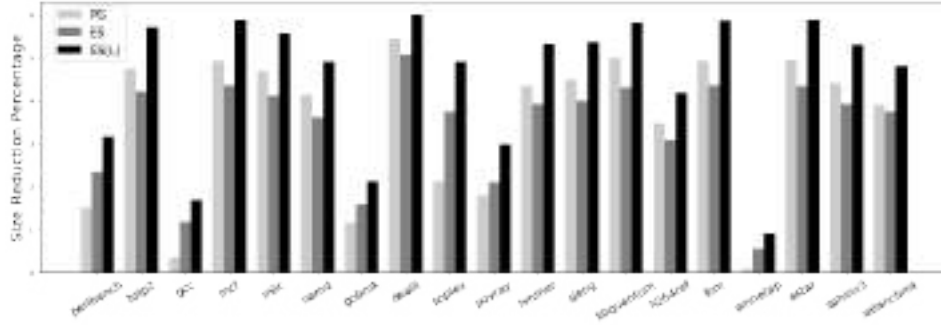


Figure 5. SPEC 2006 Size Reduction

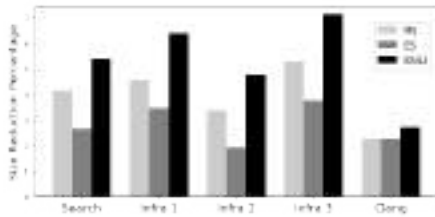


Figure 6. Generalizability across Time

preference is to avoid benchmark runs altogether, and focus instead on using problem-specific reward approximations.

For register allocation, for example, a natural reward is calculating, per function, the block frequency-weighted sum of introduced moves. For inlining for speed, we plan to use a linear combination of a per-critical call graph estimate of working set (i.e. cache lines needed for execution) and dynamic instruction count. Both approaches require profiling information for carrying out the analysis, which we assume as a pre-requisite for workloads that are concerned with speed.

8.2 ML Techniques

There are multiple directions to pursue in terms of the ML techniques:

Richer State Representations: instead of using the 11 numerical features to represent the state, we can have richer state representations. For example, we can use code embedding techniques [3] to embed the caller/callee to get more detailed information about the call site; or we can use graph neural network techniques [14] on the neighboring sub-graph of the call site to get more information about the call graph.

PG with Partial Reward: PG with partial reward would greatly improve the sample efficiency and trainability. However, there are two challenges to tackle: 1) find an efficient way to encode the global call graph information into state; 2) train a supervised model to predict a function’s native size from its IR.

9 Conclusion

We investigated the problem of leveraging ML techniques for compiler optimization in a real-world setting. We proposed a particular understanding of the problem space, and derived the MLGO framework. We applied it to inlining-for-size and described the resulting implementation, available in LLVM as a build-time opt-in, as well as the training methodology, two training algorithms and their trade-offs, and results. We are currently applying the same principles to addressing inlining for speed and register allocation policies, and hope that, through our experience, as well as that of the community, we can further refine MLGO and eventually mature it to a solution that compiler engineers can broadly apply and leverage machine learning for compiler optimizations in real-world settings.

References

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. Tensorflow: A system for large-scale machine learning. In *12th {USENIX} symposium on operating systems design and implementation ({OSDI} 16)*. 265–283.
- [2] Andrew Adams, Karima Ma, Luke Anderson, Riyadh Baghdadi, Tzu-mao Li, Michael Charbi, Benoit Steiner, Steven Johnson, Kayvon Fatahian, Fredo Durand, and Jonathan Regan-kelley. 2019. Learning to optimize Halide with tree search and random programs (*ACM Transactions on Graphics*). 2–12.
- [3] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. 2019. code2vec: Learning distributed representations of code. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 1–29.
- [4] Michael Bain and Claude Sammut. 1995. A Framework for Behavioural Cloning.. In *Machine Intelligence* 15. 103–129.
- [5] Rudy Bunel, Alban Desmaison, M Pawan Kumar, Philip HS Torr, and Pushmeet Kohli. 2016. Learning to superoptimize programs. *arXiv preprint arXiv:1611.01787* (2016).
- [6] John Cavazos and Michael FP O’Boyle. 2005. Automatic tuning of inlining heuristics. In *SC’05: Proceedings of the 2005 ACM/IEEE Conference on Supercomputing*. IEEE, 14–14.
- [7] Tianqi Chen, Lianmin Zheng, Eddie Q. Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. Learning to Optimize Tensor Programs. *CoRR* abs/1805.08166 (2018). arXiv:1805.08166 <http://arxiv.org/abs/1805.08166>

Chapter 3

Selecting the Best Compiler Optimizations: A Bayesian Network Approach

Abstract After presenting our DSE approach for finding good compiler optimizations, we present our autotuning framework to tackle the problem of selecting the best compiler passes. It leverages machine learning and an application characterization to find the most promising optimization passes given an application. This chapter proposes *COBAYN*: Compiler autotuning framework using Bayesian Networks. An autotuning methodology based on machine learning to speed up application performance and to reduce the cost of the compiler optimization phases. The proposed framework is based on the application characterization done dynamically by using independent micro-architecture features and Bayesian networks. The chapter also presents an evaluation based on static analysis and hybrid feature collection approaches. Besides, we compare our approach against several state-of-the-art machine-learning models. Experiments are carried out on an ARM embedded platform and GCC compiler by considering two benchmark suites with 39 applications. The set of compiler configurations selected by the model (less than 7% of the search space), demonstrated an application performance speedup of up to $4.6\times$ on Polybench ($1.85\times$ on average) and $3.1\times$ on Cbench ($1.54\times$ on average) with respect to standard optimization levels. Moreover, the comparison of the proposed technique with (i) random iterative compilation, (ii) machine learning-based iterative compilation and (iii) non-iterative predictive modeling techniques, shows on average, $1.2\times$, $1.37\times$ and $1.48\times$ speedup, respectively. Finally, the proposed method demonstrates $4\times$ and $3\times$ speedup, respectively on cBench and Polybench, in terms of exploration efficiency given the same quality of the solutions generated by the random iterative compilation model.

3.1 Introduction

Usually, software applications are developed in a high-level programming language (e.g. C, C++) and then passed through the compilation phase to get the executable binary. Optimizing the second phase (*compiler optimization*) plays an important role for the performance metrics. In other words, enabling compiler optimization parameters (e.g., loop unrolling, register allocation, etc.) might lead to substantial benefits in performance metrics. Depending on the strategy, these performance metrics could

be *execution time*, *code size* or *power consumption*. A holistic exploration approach to trade-off these metrics also represents a challenging problem [1].

Application developers usually rely on compiler intelligence for software optimization, but they are unaware of *how* the compiler itself does the job. Compiler interface usually has some standard optimization levels which enable the user to automatically include a set of predefined optimization sequences for the compilation process [2]. These standard optimizations (e.g. -O1, -O2, -O3 or -Os) are known to be beneficial for performance (or code size) in most cases. In addition to the above-mentioned standard optimizations, there are other compiler optimizations which are not included in the predefined optimization levels. Their effects on the software are quite complex and mostly depend on the features of the target application. Therefore, it is rather hard to decide whether to enable specific compiler optimizations on the target code. Considering an application-specific embedded system domain; constructing an optimized compiler optimization heuristic becomes even more crucial because the application is compiled once and then deployed on millions of devices on the market.

So far, researchers proposed two main approaches for tackling the problem of identifying the best compiler optimizations: (i) *iterative compilation* [3] and (ii) *machine-learning predictive modeling* [4–6]. The former approach relies on several recompilation phases and then selecting the best set of optimizations. Despite being effective, this approach has high overhead as it needs to be evaluated iteratively over all the enumerations of an optimization space. The latter approach focuses on building machine-learning predictive models to predict the best set of compiler optimizations. It relies on software features that are collected either *offline* or *online*. Once the model has been trained, given a target application, it can predict a sequence of compiler optimization options to maximize performance. Machine learning approaches normally require fewer compilation try-outs, however, their downside is the predicted optimizations lead to an execution binary that performs worse than the one found with iterative compilation.

In this chapter, we propose an approach to tackle the problem of identifying the compiler optimizations that maximize the performance of a target application. In contrast, our proposed methodology starts by applying a statistical methodology to infer the probability distribution of the compiler optimizations to be enabled. Then, we perform an iterative compilation process by sampling from this probability distribution. We use two major sets of training application suites to learn the statistical relations between application features and compiler optimizations. To the best of our knowledge, in this work, *Bayesian Networks* (BN) are used for the first time in this field to build the statistical model. Given a new application, its features are fed into the machine-learning algorithm as *evidence* on the distribution. This evidence imposes a bias on the distribution, and because compiler optimizations are correlated with the software features, we can iteratively sample the distribution obtaining the most promising compiler optimizations, by then exploiting an *iterative compilation* process.

The experiments carried out on an embedded ARM-based platform outperformed both standard optimization levels and the state-of-the-art iterative and not iterative

(based on prediction models) compilation techniques while using the same number of evaluations. Moreover, the proposed techniques demonstrated significant exploration efficiency improvement of up to $4\times$ speedup compared with random iterative compilation when targeting the same performance. To summarize, here are our contributions:

- The introduction of a BN capable of capturing the correlation between the application features and the compiler optimizations. BN allows to represent the relation by an acyclic graph, which can be easily analyzed graphically.
- The integration of the BN model in a compiler optimization framework. Given a new program, the probability distribution of the best compiler optimizations can be inferred by means of BN to focus on the optimization itself.
- The integration of both dynamic and static analysis feature collections in the framework as hybrid features.

Furthermore, the experimental evaluation section reports the assessment of the proposed methodology on an embedded ARM-based platform and the comparison of the proposed methodology against several state-of-the-art machine learning algorithms on 39 different benchmark applications.

The remainder of the chapter is organized as follows. Section 3.2 presents a quick review of the recent related literature. Readers are referred to Chap. 1 for a more comprehensive review on the state-of-the-art. Section 3.3 presents how the BN model can infer the probability of the distribution. Section 3.3.1 presents different techniques for collecting program features. Section 3.4 elaborates on the proposed framework. Sections 3.4.4 and 3.4.5 will introduce the results obtained on the application suites selected. Finally, Sect. 3.4.6 presents the comparison of the proposed methodology with state-of-the-art models.

3.2 Previous Work

Optimizations carried out at compilation have been broadly used, mainly in embedded computing applications. We refer to the holistic review on the survey in the Chap. 1. However, for the chapter completeness, we would like to remind a few related works. There are two major classes of optimization in the field of compilation research: (i) The problem of *selecting the best compiler optimizations* and (ii) The *phase-ordering* problem of compiler optimizations. As the target of this work is in the scope of *selection*, here we mostly refer to these areas. However, there are notable works to be mentioned that support the seminal concepts of the current work.

The related work in this field can be categorized into two sub-classes: (a) *iterative compilation* [7] and (b) *machine-learning* based approaches [8, 9]. Nonetheless, these two approaches have also been combined in many ways [4] that they cannot be distinguished easily.

Iterative compilation techniques were introduced as a mean to outperforming static handcrafted optimization sequences, those usually exposed by compiler interfaces as *optimization levels*. Since its introduction [7, 10], the goal of iterative compilation has been to identify the most appropriate compiler passes for a target application.

Other authors exploit iterative compilation jointly with architectural design space exploration for VLIW architectures [11]. The intuition was that the performance of a computer architecture depends on the executable binary which in turn, depends on the optimizations applied at compilation time. Thus, by studying the two problems jointly, the final architecture is optimal and the effects of different compiler optimizations are identified at the early design stages.

Given that compilation is a time-consuming task, several groups proposed techniques to predict the best compiler optimization sequences rather than applying a trial-and-error process, such as in iterative compilation. These prediction methodologies are generally based on *machine-learning* techniques [4, 6, 8, 12, 13].

More recent literature on using different program features with machine learning have been proposed by [14] where the authors collected *IR (intermediate representation)* of the kernels and utilized *graph-kernels* to derived the similarities between those fetched *IRs*. Our approach is significantly different from the previous ones given that it applies a statistical methodology to learn the relationships between application features and compiler optimizations as well as between different compiler optimizations where *machine-learning* techniques are used to capture the probability distribution of different compiler transformations. In this work, we propose the use of *BN* as a framework enabling statistical inference on the probability distribution given the evidence of application features. Given a target application, its features are fed to *Bayesian Networks* to induce an application-specific bias on the probability distribution of compiler optimizations.

Most recent machine-learning works aim at the generation of prediction models that, given a target application, predict the performance of the application for any set of compiler transformations applied to it. In contrast, in our work the machine-learning methodology aims directly at predicting the best compiler optimizations to be applied for a target application without going through the predictions of the resulting application performance.

Additionally, in our approach, program features are dynamic and obtained through micro-architecture-independent characterization [15] and compared with the results using the static profiling [16]. The adoption of dynamic profiling provides insight into the actual program execution with the purpose of giving more weight to the code segments executed more often (i.e. code segments whose optimization would lead to higher benefits according to Amdahl’s law).

3.3 Proposed Methodology

The main goal of the proposed approach is to identify the best compiler optimizations to be applied to a target application. Each application is passed through a characterization phase that generates a parametric representation of the application

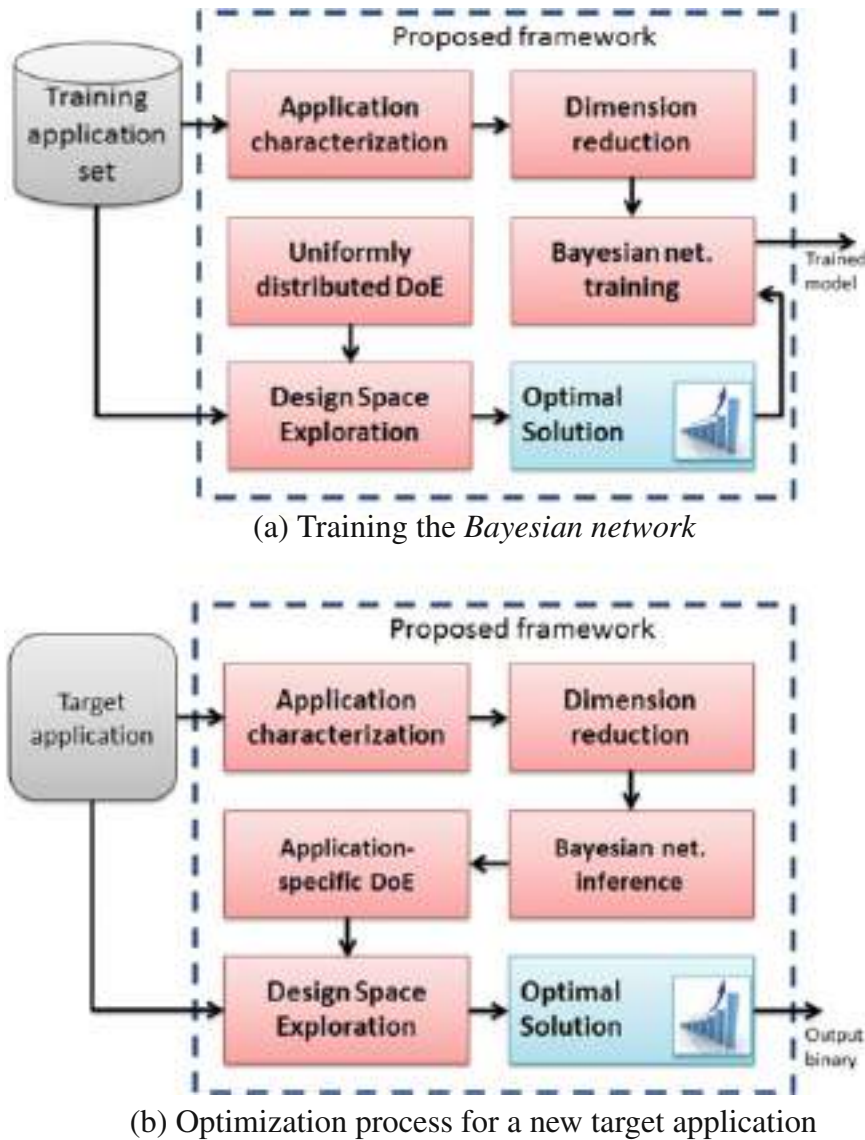


Fig. 3.1 Overview of the proposed methodology

under analysis in terms of its main features. With little information loss, these features are pre-processed by means of statistical *dimension reduction* techniques to identify a more compact representation. A statistical model based on *BN* correlates these reduced representations to the compiler optimizations to maximize application performance.

The optimization flow is shown in Fig. 3.1 and consists of two main phases. During the initial *training phase*, the *Bayesian network* is learned on the base of a set of training applications (see Fig. 5.1a). During the *exploitation phase*, new applications are optimized by exploiting the knowledge stored in the *Bayesian Network* (see an example of a BN topology in Fig. 3.3).

During both phases, an optimization process is necessary to identify the best compiler optimizations to achieve the best performance. This is done for learning purposes during the *training phase* and for optimization purposes during the *exploitation*

phase. To implement the optimization process, a Design Space Exploration (DSE) engine has been used. The DSE engine automatically compiles, executes and measures application performance by enabling/disabling different compiler optimizations. Which compiler optimizations will be enabled is decided in the Design of Experiments (DoE) phase. In our approach, the DoE is obtained by sampling from a given probability distribution that is either a *uniform distribution* (during the training phase as in Fig. 5.1a) or an *application-specific distribution* inferred through the BN (during the *exploitation phase* as in Fig. 5.1b).

The *uniform distribution* adopted during the *training phase* allows us to uniformly explore the compiler optimization space $\mathcal{O}$ to learn about the most promising regions of the space. The *application-specific distribution* during the *exploitation phase* allows us to speed up the optimization by focusing on the most promising region of the compiler optimization space $\mathcal{O}$.

3.3.1 Applying Program Characterization

The classic *supervised* Machine Learning (ML) approach deals with fitting a model exploiting a function f of program characterization. Function f might use a variety of comparison/similarity functions, such as *nearest-neighbor* and *graph-kernels*. To obtain a more accurate fitting, compiler researchers have been trying to understand the behavior of programs/kernels better and derive a *feature vector* that represents pair functionality efficiently. As a rule of thumb, the derived feature vector must be (i) representative enough of its program/kernel, and (ii) different programs/kernels must not have the same feature vectors as this will confuse the subsequent machine-learning process. Thus, building a huge non-efficient feature vector slows down the ML process and obtain less-precision.

Another goal of this work is to exploit the efficient use of different program characterization techniques and demonstrate their performance and effectiveness. Three characterization techniques have been selected among state-of-the-art works, namely, (i) *dynamic feature selection* using *MICA* [15], (ii) *static analysis* using *MilePost* [16] framework, and (iii) our handcrafted combination of those two as hybrid analysis.

MICA. *Microarchitecture-independent workload characterization* represents a recent work on dynamic workload characterization [15]. It is a plugin for the Linux-PIN tool [17] and is capable of characterizing the fed kernels *independently* from its running architecture as it monitors the *non-hardware* features of the kernels. This feature is of interest for targeting embedded domain as one might not be able to exploit PIN tools on the board. The main categories of MICA include *Instruction-Level-Parallelism (ILP)*, *Instruction Mix (ITypes)*, *Branch Predictability (PPM)*, *Register Traffic (REG)*, *Data Stream Stride (Stride)*, *Instruction and Data Memory Footprint (MEMFootprint)* and *Memory Reuse Distances (MEMReusedist)*.

MilePost. This recent tool [16, 18] was built as a plugin on top of *GCC* to capture static features of the programs. One advantage of *static analysis* is that the

compiler researchers do not have to run the actual binary just like what they have to do by a dynamic feature technique. On the other hand, *static analysis* fails to capture any correlations between the source code and memory hierarchy and different data streams fed as input dataset.

Hybrid. The third characterization technique consists of the combination of the two previous ones. We believe that, in some cases, hybrid feature selection can capture the kernel behaviors better as it takes into account both feature-selection methods.

3.3.2 Dimension-Reduction Techniques

In the proposed approach, the *dimension-reduction* process is important for two main reasons: (a) it eliminates the noise that might perturb further analyses, and (b) it significantly reduces the training time of the BN. The techniques used are *Principal Component Analysis* (PCA) and *Exploratory Factor Analysis* (EFA). The experimental results show that the selection of a good dimension-reduction technique has a significant impact on the final model quality. In the original work proposed in [5], PCA was used. In this chapter, we changed the model by exploiting *Exploratory Factor Analysis* (EFA) as explained in the following paragraphs. Experimental results will show the benefits of using EFA with respect to PCA for the specific problem addressed. For a quantitative comparison the readers can refer to Sect. 3.4.4, Table 3.5.

Let γ be a characterization vector storing all data of an application run. This vector stores l variables to account for either the static, dynamic or both analyses. Let us consider a set of known application profiles A consisting of m vectors γ . The application profiles can be organized in a matrix P with m rows and l columns. Each vector γ (i.e. a row in P) includes a large set of characteristics, such as the instruction count per instruction type (for both static and dynamic analysis), information on the memory access pattern and information characterizing the control flow (e.g. the number and length of the basic blocks, average and maximum loop nesting, etc.). Many of these application characteristics (columns of matrix P) are correlated to each other in a complex way. A simple example of this correlation is the instruction mix information collected during the static analysis and the instruction mix information collected during the dynamic profiling (even though these are not completely the same). A less intuitive example is between the distribution of basic block lengths and data related to the instruction memory reuse distance. The presence of many correlated columns in P implies that the information stored in a vector γ can be well represented with a vector α of smaller size.

Both PCA and EFA are statistical techniques aimed at identifying a way to represent γ with a shorter vector α while minimizing the information loss. Nevertheless, they rely on different concepts for organizing this reduction [19, 20]. In both cases, output values are derived by applying the dimension reduction and are no longer directly representing a certain feature. While in PCA the components are given by a combination of the observed features, in EFA the factors are represent-

ing the hidden process behind the feature generation. In both techniques, the output columns cannot be called by their header name and are not directly observable.

In PCA, the goal is to identify a summary of γ . To this end, a second vector ρ of the same length of γ (i.e. l) is organized by a variable change. Specifically, the elements of ρ are obtained through a linear combination of the elements in γ . The way to combine the elements of γ for obtaining ρ is decided upon the analysis of the matrix P , and is such that all elements in ρ are orthogonal (i.e. uncorrelated) and are sorted by their variance. Thus the first elements of ρ (also named principal components) carry most of the information of γ . The reduction can be obtained by generating a vector α to keep only the first most significant principal components in ρ , because the least significant ones carry little information content. Note that principal components in ρ (thus in α) are not meant to have a meaning; they are only used to summarize the vector γ as a signature.

In EFA, the elements in the vector of reduced size α are meant to explain the structure underlying the variables γ , while α , represents a vector of *latent variables* that cannot be directly observed. The variables γ are expected to be a linear combination of the variables in α . In EFA, this relationship explains the correlation between the different variables in γ ; that is, correlated variables in γ are likely to depend on the same hidden variable in α . The relationship between the latent α and the observed variables is regressed by exploiting the maximum likely method based on the data in matrix P .

When adopting PCA, each variable in α tends to be a mixture of all variables in γ . Therefore, it is hard to tell what a component represents. When adopting EFA instead, the components α tend to depend on a smaller set of elements in γ that are correlated with each other. That is, when applying EFA, α is a compressed representation of γ , where elements in γ that are correlated (i.e. that carry the same information) are compressed into a reduced number of elements in α . Note that reducing the profile size by means of EFA results in a α that better describes the type of application under analysis in reference to PCA [21].

Consequently, having obtained γ through any of the characterization techniques, a pre-processing filtering should be applied to ensure that the least noise has come through and the final P is eligible to be summarized by EFA. That implies manually (i) removing the zero columns in l and (ii) removing the redundant columns of l given that no column l is a linear combination of another l . In contrast, the algorithmic approach is that P needs to be transformed, as to obtain the final γ in *positive-definite covariance* form [22]. Different techniques have been described in the literature on how to transform a *non-positive-definite* matrix to a *positive-definite* one which exceeds the scope of this chapter, but interested readers can refer to [23, 24] or use packages in *R* statistical tool [25] i.e., *nearPD* to compute nearest positive definite matrix.

3.3.3 Bayesian Networks

Bayesian Networks are powerful to represent the probability distribution of different variables that characterize a certain phenomenon. The phenomenon to be investigated in this work is the optimality of compiler optimization sequences.

Let us define a Boolean vector o , whose elements o_i are the different compiler optimizations. Each optimization o_i can be either enabled, $o_i = 1$, or disabled, $o_i = 0$. In this chapter, the *phase ordering* problem [26] is not taken into account. We rather consider how different optimizations o_i are organized in a predefined order embedded in the compiler. A compiler optimization sequence represented by the vector o belongs to the n size Boolean space $\mathcal{O} = \{0, 1\}^n$, where n represents the number of compiler optimizations under study.

An application is parametrically represented by the vector α of the k reduced components computed either via PCA or EFA from its software features. Elements α_i in vector α generally belong to the continuous domain.

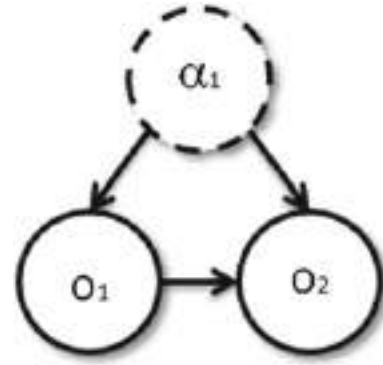
The optimal compiler optimization sequence $\bar{o} \in \mathcal{O}$ that maximizes the performance of an application is unknown. However, it is known that the effects of a compiler optimization o_i might depend on whether another optimization o_j has been applied in the optimization sequence. Additionally, it is known that the compiler optimization sequence that maximizes the performance of a given application depends on the application itself.

The reason why the optimal compiler optimization sequence $\bar{o}$ is unknown a priori is because it is not possible to capture, in a deterministic way, the dependencies among the variables in the vectors $\bar{o}$ and α . There is no way to identify an analytic model to exactly fit the vector function $\bar{o}(\alpha)$. As a matter of fact, the best optimization sequence $\bar{o}$ depends also on other factors that are somewhat outside our comprehension, *the unknown*. It is exactly to deal with *the unknown* that we propose not to predict the best optimization sequence $\bar{o}$ but rather to infer its probability distribution. The uncertainty stored in the probability distribution models the effects of *the unknown*.

As underlying probabilistic model, we selected *BN* because of the following features of interest for the target problem:

- Their expressiveness allows one to include heterogeneous variables in the same framework such as Boolean variables (in the optimization vector o) and continuous variables (in the application characterization α).
- Their capabilities to model *cause-effect* dependencies. Representing these dependencies is suitable for the target problem, as we expect that the benefits of some compiler optimizations (effects) are due to the presence of some application features (causes).
- It is possible to graphically investigate the model to visualize the dependencies among different compiler optimizations. If needed, it is even possible to manually edit the graph for including some a priori knowledge.
- It is possible to bias the probability distribution of some variables (the optimization vector o) given the *evidence* on other variables (the application characterization α).

Fig. 3.2 A Bayesian Network example



This enables us to infer an *application-specific distribution* for the vector o from the vector α observed by analyzing the target application.

A *Bayesian Network* is a direct acyclic graph whose nodes represent variables and whose edges represent the dependencies between these variables. Figure 3.2 reports a simple example with one variable α_1 representing the application features and two variables o_1, o_2 representing different compiler optimizations. In this example, the probability distributions of the two optimizations depend on the program features represented by α . Additionally, the probability distribution of o_2 depends on whether the optimization o_1 is applied. Dashed lines are used for nodes representing observed variables whose value can be input as evidence to the network. In this example, the variable α_1 can be observed and, by introducing its evidence, it is possible to bias the probability distributions of other variables.

Training the Bayesian model. Tools exist to construct *BN* automatically by fitting the distribution of some training data [27]. To do so, first the graph topology is identified and then the probability distribution of the variables including their dependencies is estimated.

The identification of the graph topology is particularly complex and time consuming. The *dimension reduction* technique applied on the SW features plays a key role in obtaining reasonable training times by limiting to k elements in the vector α , thus reducing the number of nodes in the graph.

For efficiency reasons, the algorithm used for selecting the graph topology is a heuristic algorithm, named *K2*, initialized with the *Maximum Weight Spanning Tree* (MWST) ordering method as suggested in the Matlab toolbox in use [27]. The initial ordering of the nodes for the MWST algorithm is given to let the elements α to appear first and then the elements of o . Even if the final topological sorting of the nodes changes according to the algorithm described in [28], by using this initialization criterion, it always happens that the dependencies are directed from elements of α to elements of o and not vice versa. When using the *K2* algorithm, the network topology is selected as follows. The graph is initialized with no edges to represent the fact that each variable is independent. Then, for each variable i , following their initial ordering, each possible edge from j to i (where $j < i$) is considered as a candidate to be added to the network. A candidate edge is added to the topology if it increases the probability that the training data were generated from the probability distribution

the new topology describes. This method has a polynomial complexity with respect to the number of variables involved and the number of lines in the training data set.

During the model training, we consider the *softmax* function for modeling the cumulative probability distribution of the Boolean elements in vector o [27]. This is a mathematical necessity to map in the *Bayesian framework* the dependencies of Boolean variables in o with respect to continuous variables in α . In particular, thanks to the use of *softmax* variables, we can express the conditional probability $P(o_i = b \mid \alpha_j = x)$, where o_i is a Boolean variable and α_j is a continuous variable.

The coefficients of the functions describing the probability distribution of each variable as well as their dependencies are tuned automatically to fit the distribution in the training data [27]. Training data are gathered by analyzing a set A of training applications (Fig. 5.1a). First, application features are computed for each application $a \in A$ to enable the principal component analysis. Thus, each application is characterized by its own principal component vector α . Then, an experimental compilation campaign is carried out for each application by sampling several compiler optimization sequences from the compiler optimization space $\mathcal{O}$ with a uniform distribution. For each application, we select the 15% best-performing compilation sequences among the sampled ones. The distribution of these sequences is learned by the *Bayesian Network* framework in relation to vector α characterizing the application.

Inferring an application-specific distribution. Once the *Bayesian Network* has been trained, the principal component vector α obtained for a new application can be fed as evidence to the framework to bias the distribution of the compiler optimization vector o . To sample a compiler optimization sequence from this biased distribution, we proceed as follows. The nodes in the direct acyclic graph describing the *Bayesian Network* are sorted in topological order, i.e. if a node at position i has some predecessors, those appear at positions j , $j < i$. At this point, all nodes representing the variables α appear at the first positions.¹ The value of each compiler optimization o_i is sampled in sequence by following the topological order such that all its parent nodes have been decided. Thus, the marginal probability $P(o_i = 0 \mid \mathcal{P})$ and $P(o_i = 1 \mid \mathcal{P})$ can be computed on the basis of the parent node vector value $\mathcal{P}$ (each parent being either an evidence α_j or a previously sampled compiler optimization o_j). Similarly, by using the maximum likelihood method, it is possible to compute the most probable vector from this biased probability distribution. When sampling from the application-specific probability distribution inferred through the *Bayesian Network*, we always consider finding the most probable optimization sequence as the first sample.

¹This is by construction due to the initialization of the MWST and the K2 algorithms used to discover the network topology.

3.4 Experimental Evaluation

The goal of this section is to assess the benefits of the proposed methodology. In this work, we run the experimental campaign on an ARMv7 Cortex-A9 architecture as part of a TI-OMAP 4430 processor [29] with *ArchLinux* and *GCC-ARM 4.6.3*.

3.4.1 Benchmark Suites

To assess the proposed methodology, we have used two major benchmark suites: (i) *cBench* [30], and (ii) *PolyBench* [31, 32]. Each consists of different classes of applications and kernels ranging from security and cryptography algorithms to office and image-processing applications. Readers can refer to Table 3.1 for the list of applications selected in the two benchmark suites.

3.4.1.1 cBench

The *cBench* suite [30] is a collection of open-source programs with multiple data sets assembled by the community to enable realistic workload execution and targeted by many different compilers such as *GCC*, *LLVM*, etc. The source code of individual programs is simplified to facilitate portability; therefore, it has been targeted in *autotuning* and *iterative compilation* research work. Of the available data sets for every individual kernel, we have selected five and sorted them in a way that dataset1 is always the smallest and dataset5 the largest. This ensures that for every kernel we have exposed enough of the input load to be able to measure fair runtime executions.

3.4.1.2 PolyBench

The *PolyBench* benchmark suite [31, 32] consists of benchmarks with static control parts. The purpose is to make the execution and monitoring of applications uniform. One of the main features of the *PolyBench* suite is that there is a single file per application, tunable at compile-time and used for kernel instrumentation. It performs extra operations such as cache flushing before the execution and can set real-time scheduling to prevent OS interference. We have defined two different data sets for each individual application to expose the main function with different input loads. PolyBench has a variety of benchmarks, i.e. 2D and 3D matrix multiplication, vector decomposition, etc. This suite is also suitable for parallel programming, which is beyond the focus of this work.

Table 3.1 Benchmark suites used in this work

cBench list	Description
(a) <i>cBench</i> applications selected for this work	
automotive_bitcount	Bit counter
automotive_qsort1	Quick sort
automotive_susan_c	Smallest univalue segment assimilating nucleus corner
automotive_susan_e	Smallest univalue segment assimilating nucleus edge
automotive_susan_s	Smallest univalue segment assimilating nucleus S
security_blowfish_d	Symmetric-key block cipher decoder
security_blowfish_e	Symmetric-key block cipher encoder
security_rijndael_d	AES algorithm Rijndael decoder
security_rijndael_e	AES algorithm Rijndael encoder
security_sha	NIST secure hash algorithm
telecom_adpcm_c	Intel/dvi adpcm coder/decoder coder
telecom_adpcm_d	Intel/dvi adpcm coder/decoder decoder
telecom_CRC32	32 BIT ANSI X3.66 crc checksum files
consumer_jpeg_c	JPEG kernel
consumer_jpeg_d	JPEG kernel
consumer_tiff2bw	convert a color TIFF image to grey scale
consumer_tiff2rgba	convert a TIFF image to RGBA color space
consumer_tiffdither	convert a TIFF image to dither noisespace
consumer_tiffmedian	convert a color TIFF image to create a TIFF palette file
network_dijkstra	Dijkstra's algorithm
network_patricia	Patricia Trie data structure
office_stringsearch1	Boyer-Moore-Horspool pattern match
bzip2d	Burrows—Wheeler compression algorithm
bzip2e	Burrows—Wheeler compression algorithm

(continued)

Table 3.1 (continued)

PolyBench list	Description
(b) Linear-algebra/applications of the <i>PolyBench</i> suite selected for this work	
2 mm	2 Matrix multiplications ($D = A \times B$; $E = C \times D$)
3 mm	3 Matrix multiplications ($E = A \times B$; $F = C \times D$; $G = E \times F$)
atax	Matrix transpose and vector multiplication
bicg	BiCG Sub Kernel of BiCGStab linear solver
cholesky	Cholesky decomposition
doitgen	Correlation computation
gemm	Matrix-multiply $C = \alpha A \times B + \beta C$
gemver	Vector multiplication and matrix addition
gesummv	Scalar, vector and matrix multiplication
mvt	Matrix vector product and transpose
symm	Symmetric matrix-multiply
syr2k	Symmetric rank—2k operations
syrk	Symmetric rank—k operations
trisolv	Triangular solver
trmm	Triangular matrix-multiply

3.4.2 Compiler Transformations

The compiler transformations analyzed have been reported in Table 3.2. We based our design space on the work of [3]. The authors implemented sensitivity analysis over a vast majority of the compiler optimizations and defined with a list of promising passes. Building upon their work, we selected the compiler optimizations with a speedup factor greater than 1.10. They are applied to improve application performance beyond the standard optimization level -O3 and have not yet been included in any prior optimization level. The optimizations can be enabled or disabled using their respective compiler optimization flags. The standard optimization level -O3 has been also used to collect the dynamic software-features for each application on both *training* and *inference* phases.

The application execution time has been estimated by using the Linux-perf tool. The execution time is done by averaging five loop-wraps of the specific compiled binary with one second of sleep in between five different executions of those loop-wraps. Therefore, in total, each transformed binary has been executed 25 times as five packages of five loop-wraps to ensure better accuracy of estimations and fairness among the generation of executions. This technique is used both in the *training* and the *inference* phases.

Table 3.2 Compiler optimizations under analysis (beyond -O3)

Compiler transformation	Abbreviation	Short description
-funsafe-math-optimizations	<i>math-opt</i>	Allow optimizations for floating-point arithmetic that (a) assume valid arguments and results and (b) may violate IEEE or ANSI standards
-fno-guess-branch-probability	<i>fn-gss-br</i>	Do not guess branch probabilities using heuristics
-fno-ivopts	<i>fn-ivopt</i>	Disable induction variable optimizations on trees
-fno-tree-loop-optimize	<i>fn-tree-br</i>	Disable loop optimizations on trees
-fno-inline-functions	<i>fn-inline</i>	Disable optimization that inline all simple functions
-funroll-all-loops	<i>funroll-lo</i>	Unroll all loops, even if their number of iterations is uncertain
-O2	<i>O2</i>	Overwrite the -O3 optimization level by disabling some optimizations involving a space-speed trade-off

3.4.3 Bayesian Network Results

In this work, Matlab environment [27, 33] has been used to train the *Bayesian Network*. We have used *Exploratory Factor Analysis* (EFA) of application features for the seven compiler optimization flags listed in Table 5.2. As stated in Sect. 3.3.2, one of the features of using EFA is that the factors are linear combinations that maximize the shared portion of the variance. Therefore, as a prerequisite, the covariance matrix should be positive definite. The pre-processing helps purify the highly correlated application characterization columns that are linearly correlated. In theory, *PCA* accepts any matrix ignoring the aforementioned condition and that is why we think applying factor analysis as our dimension reduction technique tends to obtain the most important factors and correlate them with the compiler optimizations. The decision on the numbers of factors has been derived from the *Kaiser* test [34]. The test implies taking only the factors having greater than 1 in the covariance matrix. In other words, the Kaiser rule is to drop all components with eigenvalues under 1, this being the eigenvalue equal to the information accounted for by an average single item. Table 3.3 reports the factors derived for each benchmark and characterization method.

Table 3.3 represents the number of features that have been produced both originally by the different feature selection techniques and by the Kaiser test. The third column is the *original number of features* and the last one refers to *range of selected factors* in each specific benchmark suite/feature selection method. Note that the last column reports the range of selected factors rather than a number as we have used cross-validation approach in the experimental campaign, thus different applications/datasets/feature selection techniques can result in a different number of factors to be used in COBAYN's framework.

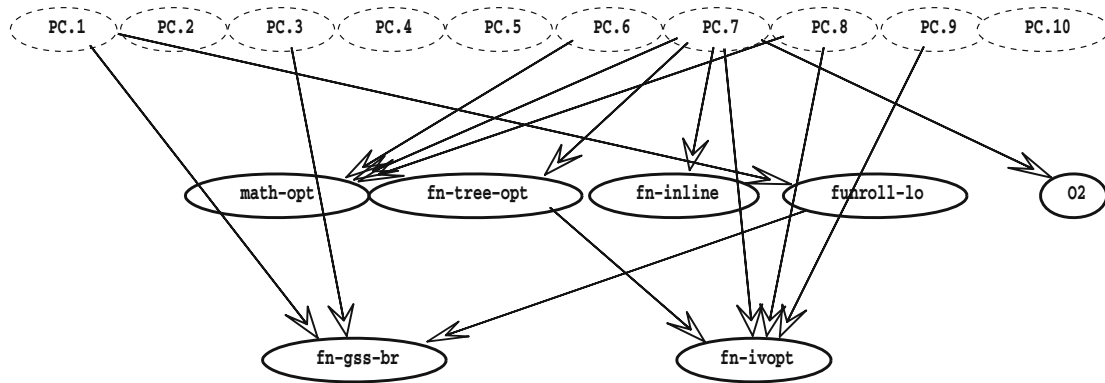
Table 3.3 Kaiser test results

Application	Characterization method	Original no. of factors	Range of selected factors by Kaiser test
cBench	MICA (Dynamic)	99	[7–11]
cBench	MILEPOST (Static)	53	[4–6]
cBench	Hybrid	143	[8–10]
polyBench	MICA (Dynamic)	99	[5–7]
polyBench	MILEPOST (Static)	53	[4–6]
polyBench	Hybrid	143	[4–5]

In this work, while training has been carried out using each application/dataset pair separately, the validation has been done through an application-level cross-validation (*Leave-One-Out* cross-validation, LOO). We train different BNs, each by excluding an applications (together with all its input dataset) from the training set.

Using *BN* enables us to investigate graphically the dependencies between the variables involved in the compiler optimization problem and to correlate them with the selected factors of the program characterization. We train a final *Bayesian Network* including all applications in the training set. The resulting network topology is a directed acyclic graph *DAG*, as shown in Fig. 3.3. By removing *security_rijndael_e* application from the training set, the graph topology slightly changes, mainly in terms of the different edges connecting the *Principal Components (PC)/program factors (FA)* nodes to the compiler optimization nodes. This is due to the change in the program features and its factors, which are computed in a different way. For the sake of conciseness, we do not report all graph topologies derived by the LOO technique for each individual trained *Bayesian Network*.

The nodes of the topology graph reported in Fig. 3.3 are organized in layers. The first layer reports the FAs that are the observable variables (reported as dashed lines). The second layer contains the compiler optimizations whose parents are the PC nodes (or FA nodes depending whether PCA or EFA is used). Therefore the effects

**Fig. 3.3** Topology of the *Bayesian Network* if *security_rijndael_e* is left out of the training set

of these compiler optimizations depend only on the application characterization in terms of its features. In the third layer, the compiler optimization nodes whose parents include optimization nodes from the second layer are listed. Once a new application is characterized for a target application data set, the evidence related to the PCs (or FAs) of its features is fed to the network in the first layer. Then, the probability distributions of other nodes can be inferred in turn on the second and third layers. There are two nodes in the third layer of Fig. 3.3. The first one is the *fn-gss-br* node that depends on *funroll-lo* because unrolling loops impacts the predictability of the branches implementing these loops. Moreover, *funroll-lo* impacts the effectiveness of the heuristic branch probability estimation, thus *fn-gss-br*. The second node in the third layer is the *fn-ivopt* node, which depends on *fn-tree-opt* as parent node in the second layer. Both these optimizations work on trees and therefore their effects are interdependent. While sampling compiler optimizations from the *Bayesian Network*, the decisions of whether to apply *fn-gss-br* and *fn-ivopt* are taken after deciding whether to apply *funroll-lo* and *fn-tree-opt*.

Table 3.4 shows the fine-grain breakdown of the timing when we use COBAYN framework. We have reported the time spent for each phase of the proposed technique, both on the training phase (done offline) and on the inference phase (done online). Constructing COBAYN's network is a one-time process and depends on the number of applications in the training set. The time needed to collect the training data is on the other side, depends not only on the number but also on the applications and data-set used for the training. The same applies to the time needed for compiling and executing the target application during the online compiler autotuning phase. To this end, Table 3.4 reports the numbers for each specific phase considering the Cbench as training set and Susan as the target application. During the offline training-phase, the time needed for data collection on the case of Cbench, is around 2 days. It includes the time needed for each benchmark to compile and execute, considering all set of configurations and the feature collection phase. The time needed to post-process the data and to generate the Bayesian Network model is around 70 s.

During the online phase (inference phase), the time needed for extracting the software features from the target application is 14.4 s while querying BN is less than 1 s. The compilation and execution-time on the target platform for Susan are 4.5

Table 3.4 COBAYN timing breakdown for offline training and online inference for *Susan* application

Phase	Tag and category	Time (s)
Offline training	(A) Offline data-collection	2 days
	(B) Construct BN	70 s
Online inference	(C) SW Feature collection	14.4 s
	(D) BN Inference	0.85 s
	(E) Susan compilation	4.5 s
	(F) Susan execution	8.9 s

and 8.9 s, respectively. Those numbers show that the initial overhead in adopting the proposed methodology on the user-side (composed of the software feature extraction and BN inference) is less than 2 compilation/executions pairs, for this specific example.

3.4.4 Comparison Results

It is well known that *Random Iterative Compilation* (RIC) can improve application performance compared with static handcrafted compiler optimization sequences [4]. Additionally, given the complexity of the *iterative compilation* problem, it has been proved that drawing compiler optimization sequences at random is as good as applying other optimization algorithms such as genetic algorithms or simulated annealing [3–5, 12]. Accordingly, to evaluate the proposed approach, we compared our results with (1) standard optimization levels -O2 and -O3 (2) the Random Iterative Compilation (RIC) methodology that samples compiler optimization sequences from the uniform distribution, and (3) two state-of-the-art methodologies coupling machine learning with an iterative methodology, and a optimization speedup predictor methodology, respectively.

The proposed methodology samples different compiler optimization sequences from the BN. The performance achieved by the best application binary depends on the number of sequences sampled from the model. In this section, the results of applying the proposed methodology using two benchmark suites with respect to standard optimization levels and the *random iterative compilation* have been reported. The performance speedup on the first comparison section is measured in reference to -O2 and -O3, which are the optimization levels available for GCC. In addition, we show the speedup of the proposed methodology with respect to our previous work [5].

3.4.4.1 Bayesian Networks Performance Evaluation

Table 3.5 reports COBAYN’s speedup achieved over the standard optimization levels of -O2 and -O3 and *Random Iterative Compilation* (RIC). The last column represents the average speedup achieved by revising our *Bayesian Network* engine and using the *Explanatory Factor Analysis* (EFA) described in the Sect. 3.3.2 with respect to PCA in [5]. Note that all speedup values have been averaged using harmonic mean. It is observed that in all categories, COBAYN outperforms standard optimization levels and the previous approach. The comparison with respect to the RIC has been reported by the harmonic average over the speedup data derived by dividing the COBAYN’s performance data by the RIC data in full space. It can be seen that dynamic feature selection brings best results followed by the hybrid and static method. However, in certain cases (cBench using hybrid SW features), it narrowly reaches the performance of dynamic feature selection.

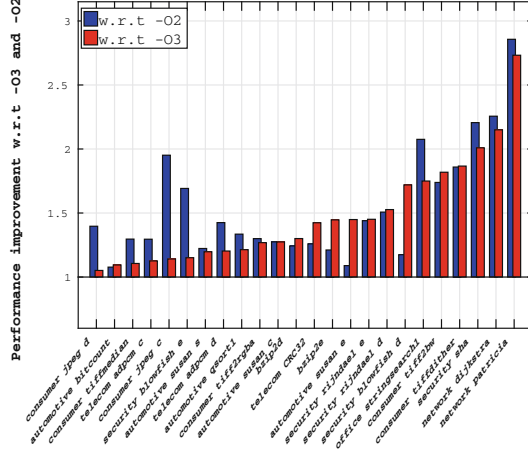
Table 3.5 COBAYN (BN using EFA) speedup w.r.t standard optimization levels (-O2 and -O3) and *Random Iterative Compilation* (RIC) and our previous approach of BN in [5] using PCA

Benchmarks	Features	COBAYN Speedup w.r.t			
		-O2	-O3	RIC	BN w/ PCA
Cbench	Dynamic	1.6093	1.528	1.2029	1.0744
Cbench	Static	1.5447	1.478	1.1143	1.0543
Cbench	Hybrid	1.5858	1.5066	1.2086	1.0654
Cbench average		1.5795	1.5035	1.1743	1.0617
PolyBench	Dynamic	1.9845	1.8387	1.3230	1.0921
PolyBench	Static	1.9353	1.8215	1.1518	1.0724
PolyBench	Hybrid	1.9441	1.7726	1.2333	1.1078
PolyBench average		1.9541	1.8101	1.2350	1.0901
Overall (Harmonic mean)		1.7669	1.6571	1.2052	1.0771

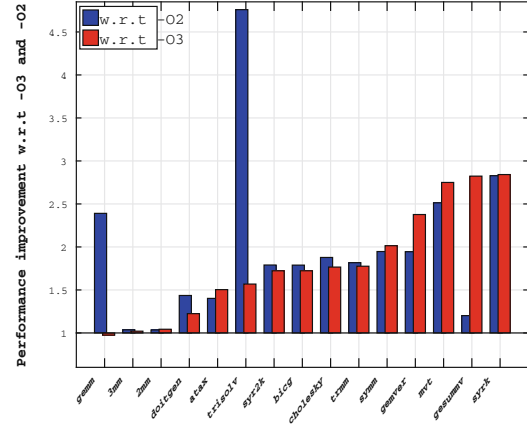
Using two major benchmarks and three different application characterization techniques, we report six different plots showcasing the benefits of the proposed methodology with respect to the GCC standard optimization levels. Figure 3.4, reports the speedups by considering a sample of eight different compiler optimization sequences. For each benchmark, the results have been averaged on the different data sets. All results have been sorted by the speedup values of -O3 and have been matched with their corresponding -O2 value. The bar plot is colored in blue and red, respectively, for the speedup achieved with respect to -O2 and -O3. All applications have achieved a speedup in reference to the performance of -O2 and -O3. This happens with the exception of *gemm* in reference to -O3 for static and hybrid feature-selection techniques and *consumer-jpeg-d* in reference to -O3 when using the dynamic method for feature selection. These applications reach their best performance using -O3 for two data sets out of five, and it was not possible to surpass this maximum by relying on the compiler transformations under consideration. On average for *Cbench*, the speedups are of **1.57** and **1.5** in reference to -O2 and -O3, respectively, and **1.95** and **1.81** for *PolyBench*. The maximum speedup observed is **3.1 $\times$** and **4.7 $\times$** . Table 3.5 reports the speedup gained using COBAYN compared with the standard optimization levels, *Random Iterative Compilation* (RIC) and our previous approach exploiting PCA as dimension-reduction method.

3.4.4.2 COBAYN's Portability Analysis

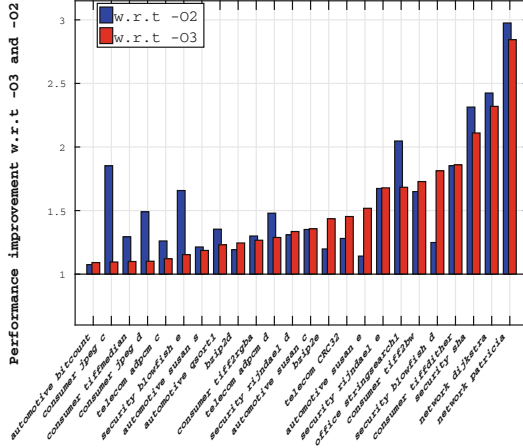
The results reported in this section are computed by means of LOO cross-validation on the two individual benchmark suites separately, one with 24 and the other with 15 applications. As the nature of these two benchmark suites is totally different, we believed it would be unfair to train on one and test on the other, so we analyzed the feasibility of mixing these applications in a fair heterogeneous set of *BigSet* so



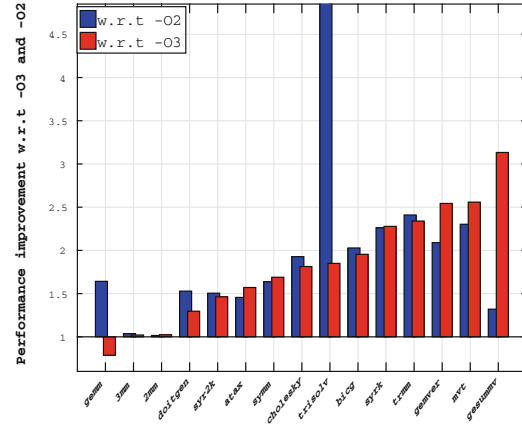
(a) BN with dynamic features on cBench



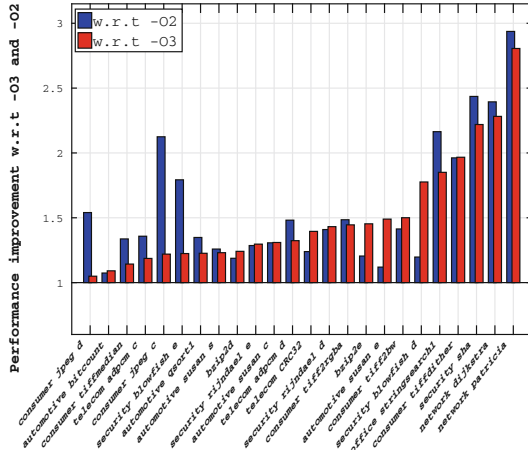
(b) BN with dynamic features on PolyBench



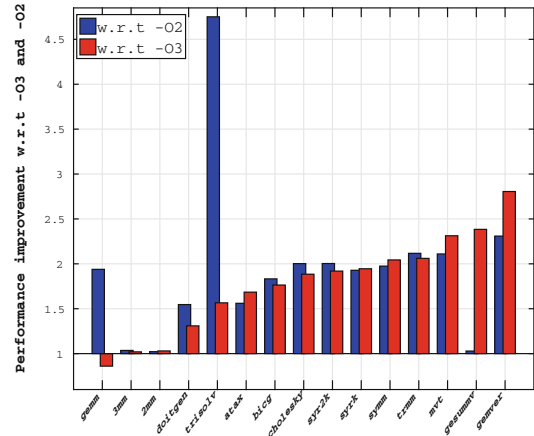
(c) BN with static features on cBench



(d) BN with static features on PolyBench



(e) BN with dynamic+static features on cBench



(f) BN with dynamic+static features on PolyBench

Fig. 3.4 Performance speedup w.r.t -O2 and -O3

Table 3.6 Evaluation of different BigSet formation in COBAYN Model construction. Note that COBAYN’s default refers to the version of COBAYN trained on a single benchmark set

BigSet combination		Speedup w.r.t. COBAYN’s default
Cbench	PolyBench	
24 (All)	15 (All)	1.1143
15	15	1.0743
10	10	1.0432
5	5	0.9896

that COBAYN’s engine gets evaluated. To this end, we tried 4 different scenarios, where the BigSet is obtained by: (i) including all 39 available applications, (ii) 15 applications of Cbench and 15 applications of PolyBench, (iii) selecting 10 Cbench and 10 PolyBench and finally (iv) 5 applications from each of those. Therefore, the BigSet was initialized with 39, 30, 20 and 10 different applications, and LOO cross-validation was carried-out. Table 3.6 reports the speedup gained in these scenarios. It is observed that COBAYN framework benefits from having (a) more applications, and (b) heterogeneous applications in the training set. The speedup listed in Table 3.6 is higher when BigSet accounts for more applications and, even just 10 applications per benchmark suite, it is higher than one (the default setting for the experimental results in this work refers to the COBAYN trained only on one of the two benchmark suites).

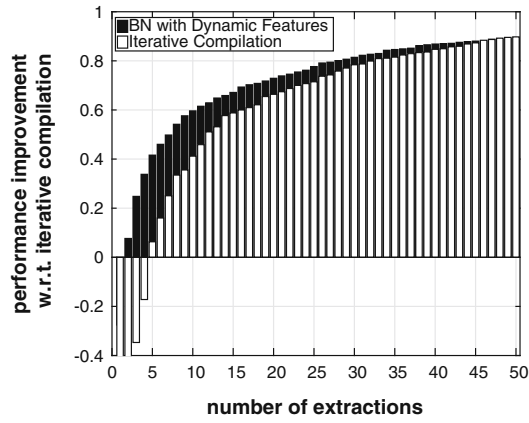
3.4.4.3 Performance Improvement

Let us define the *Normalized Performance Improvement* (NPI) as the ratio of the performance improvement achieved over the potential performance improvement:

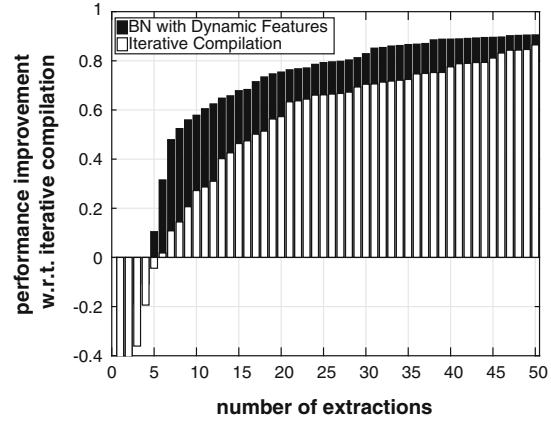
$$NPI = \frac{E_{ref} - E}{E_{ref} - E_{best}} \quad (3.1)$$

where E is the execution time achieved by the methodology under consideration, E_{ref} is the execution time achieved with a reference compilation methodology and E_{best} is the best execution time computed through an exhaustive exploration of all possible compiler optimization sequences (in our case 128 different sequences). As the execution time E of the iterative compilation methodology under analysis gets closer to the reference execution time E_{ref} , the NPI gets closer to 0, reporting that no improvement is returned. In the same way, as E gets closer to the best execution time E_{best} , while NPI gets close to 1, reporting that the entire potential performance improvement has been achieved.

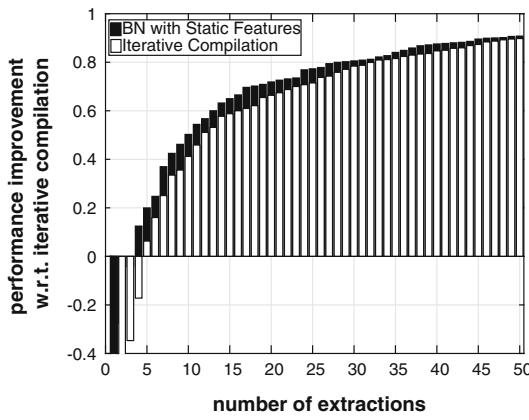
Figure 3.5 reports for six different benchmark/feature selection methods. The NPI achieved by the *proposed optimization* technique and by the RIC technique in refer-



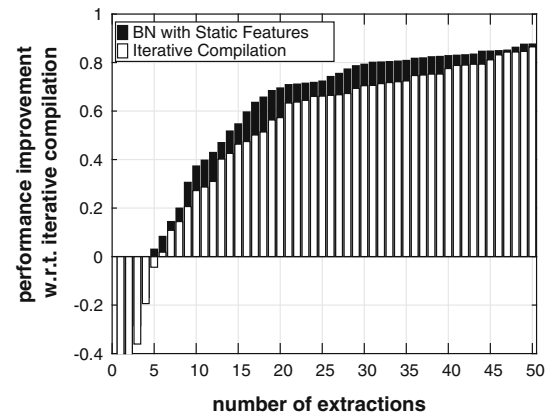
(a) BN with dynamic features on cBench



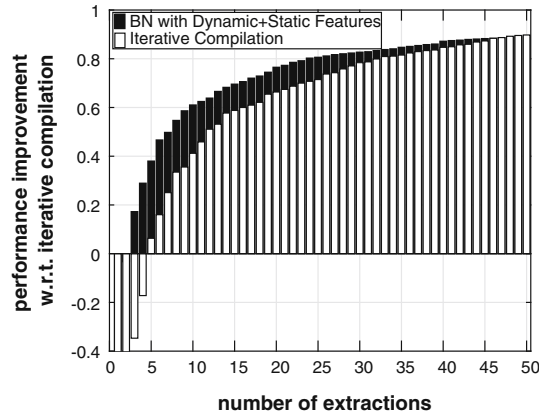
(b) BN with dynamic features on PolyBench



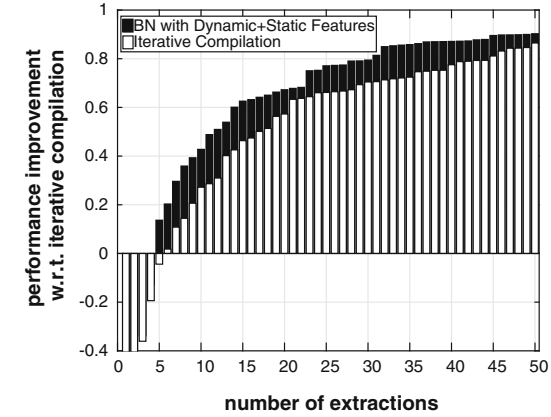
(c) BN with static features on cBench



(d) BN with static features on PolyBench



(e) BN with hybrid features on cBench



(f) BN with hybrid features on PolyBench

Fig. 3.5 Normalized performance improvement (NPI) w.r.t. RIC model

ence to the execution time obtained by -O3 (E_{ref}). It is noticeable that NPI has the upper-hand on performance on every number of extractions with respect to RIC. For readability purposes, we have only reported the first 50 extraction of the design space. The trend is continuously applied to the rest of the extractions until both get the maximum performance value of 1 at extraction no. 128, which accounts for the optimal compiler sequence given the specific application (also it is the optimal performance using exhaustive search). The comparisons reported in Fig. 3.5 were carried out by considering the same number of compiler optimization sequences sampled for both the RIC and the *proposed* approach. We acknowledge the fact that there is still room for improvement in future work. However, NPI figures show that in all cases, the proposed method was superior in terms of performance and that 30 extractions, on the current scale, reach 80% of the optimality.

3.4.5 A Practical Usage Assessment

When using *iterative compilation* in realistic cases, we need to decide how much effort should be spent on the optimization itself. This effort can be measured in terms of optimization time, which is directly proportional to the number of compilations to be executed. Thus, in this section, we evaluate the proposed optimization approach in terms of the application performance reached after a fixed number of compilations. In particular, we fix this number to eight which represents 6.25% of the overall optimization space. Our model has been compared with RIC and in Fig. 3.6, we report the *violin* plot for application speedup, while keeping the compilation effort of the proposed methodology to eight compilations (or extractions) and varying the compilation efforts of the RIC to explore more compiler space in the long run. Each individual distribution in Fig. 3.6 represents the performance of the proposed work with respect to *RIC* across different extractions. The red cross marks the *mean* and the green square marks the *median* of each violin distribution. It can be seen that the proposed methodology with BN inference achieved an at least $3\times$ reduction in exploration process effort compared with the same extraction of RIC. Here we define *exploration speedup* as the factor measuring the aforementioned metrics, enabling the researchers to traverse the compiler design space more efficiently.

Accordingly, by increasing the compilation efforts on *RIC*, while keeping the exploration efforts of the proposed approach constant, the application speedup of COBAYN decreases. On average, RIC needs 24-32 extractions to achieve the application performance obtained with eight extractions by COBAYN. This means that COBAYN provides a speedup of $3-4\times$ in terms of optimization efforts, that is only slightly impacted by the initial overhead (less than 2 evaluations) reported in Sect. 3.4.3. Furthermore, at the most extreme case, when RIC exhaustively enumerates and explores the full-space, 8 extractions of COBAYN, on average, still could gain up to 91% of the optimal solution. This is shown on the final distribution of each *violin* plot separated by a vertical dashed-line.

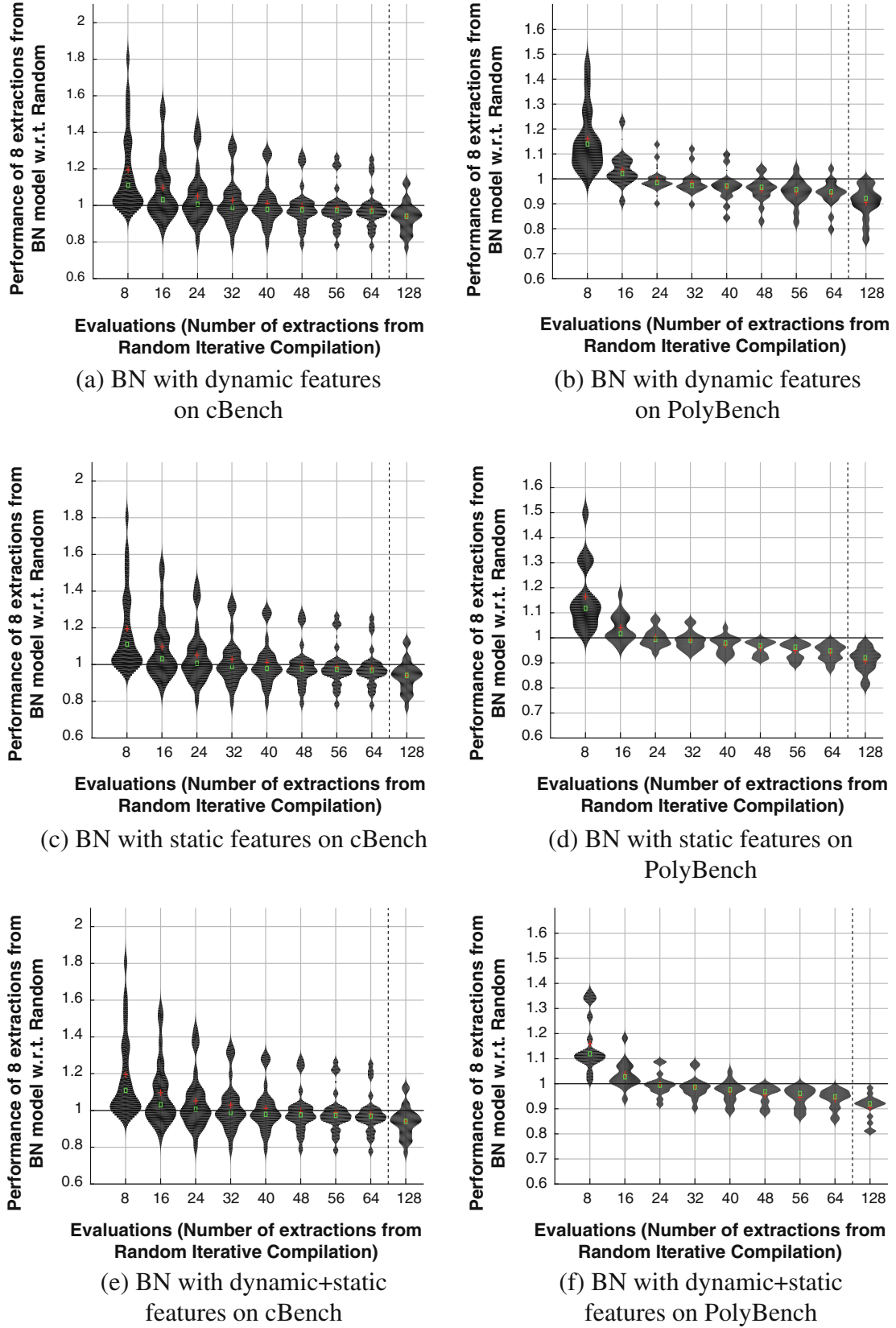


Fig. 3.6 Exploration speedup of 8 extractions w.r.t different evaluations of RIC

3.4.6 Comparison with State-of-the-Art Techniques

In this section, we compare the quality of the COBAYN results with respect to approaches that derived from (A) an *iterative compilation* and (B) a *non-iterative compilation* methodology.

3.4.6.1 Comparison to a Iterative Compilation Methodology

Agakov et al. [4] leveraged machine-learning models to focus on the exploration of the compiler optimization. Their methodology exploits a Markov chain oracle and an independent identically distributed (IID) probability distribution oracle. These two models learned offline bias certain optimizations over others and replace the uniform probability distribution we applied earlier for the RIC reference methodology. Their work reports significant speedup by coupling these machine-learning models with a nearest-neighbor-classifier. When predicting the probability distribution of the best compiler optimizations for a new application, the classifier first selects the training application having the smallest Euclidean distance in the feature vector space (derived by PCA). Then it learns the probability distribution of the best compiler optimizations for this neighboring application either by means of the Markov chain model or by using an IID model. These probability distribution learnt is then used as the predicted optimal distribution for the new application. It has been reported that the Markov chain oracle outperforms the IID oracle, followed by the RIC methodology using a uniform probability distribution.

We constructs the $P(S)$ probability matrix reported in Sects. 4.2 and 4.3 of [4] as:

$$P(S_{IID}) = s_1, s_1, \dots, s_L = \prod_{i=1}^L P(s_i) \quad (3.2)$$

$$P(S_{Markov}) = P(s_1) \prod_{i=2}^L P(s_i | s_{i-1}) \quad (3.3)$$

where $P(S_{IID})$ and $P(S_{Markov})$ define the probability of the specific sequence with IID and Markovian property for the optimization $t_1, t_1, \dots, t_L$. Using LOO cross-validation, we find the closest neighbor for each cBench application trained by the two oracles, and we sample from their probability distributions. To comply with the original work in [4], we consider only the five most relevant principal components PCs and account only for the *static program features* (also when applying COBAYN). The results are depicted in Fig. 3.7. It shows that COBAYN is faster in reaching higher speedup values. The results are scaled and normalized with respect to -O3 by using the NPI value (Eq. 5.5). COBAYN is able to capture a more realistic probability matrix of the compiler optimization problem and achieves with faster convergence

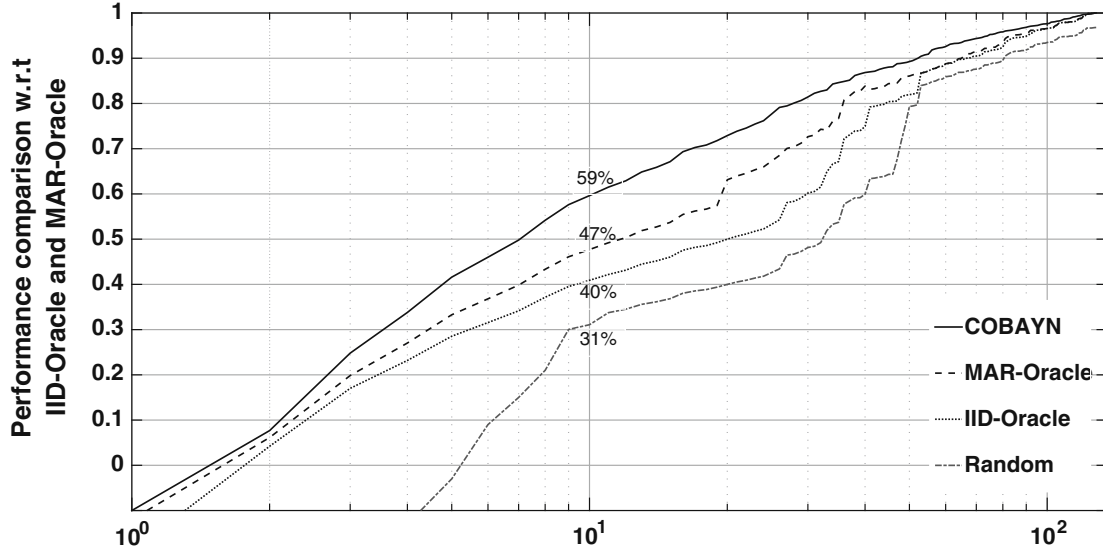


Fig. 3.7 NRI-scales speedup comparison of COBAYN with IID-oracle and MAR-oracle reported in [4]

towards the optimal result. It brings $1.25\times$ and $1.47\times$ speedup with respect to IID and the Markov oracle, respectively.

3.4.6.2 Comparison to a Non-iterative Compilation Methodology

Park et al. [35] used a polyhedral compiler framework capable of predicting the speedup for an unseen application. They used certain loop-optimizations in their *design-space* and surfed the *full-search* of the space. They reported the average speedup gained with respect to standard optimization O3 by using different machine-learning models on WEKA [36] machine-learning environment.

(i) Their predictive models are based on performance counters that are collected from the underlying architecture while running the applications. Therefore, the program features to be exposed to the model are architecture dependent, and the model loses its portability when it is used for a different architecture.

(ii) We also explore a different compiler optimization space. They have explored polyhedral optimization space including loop transformations, whereas we focus on GCC optimization space including loop transformations and other optimizations such as *inlining*, *math optimizations*, etc.

(iii) Furthermore, our model is based on a statistical analysis and BN, whereas they use a different set of machine-learning techniques, namely, predictive models. [35].

Nonetheless, we compare their methodology by applying it to the problem at hand. Table 3.7 reports the results obtained by using the machine-learning models in [35] on our compiler optimization space. We reproduced the data on both 1-shot and 8-shot scenarios to conform with the number of inference (predictions) COBAYN has in the current work. We use the Harmonic mean to average the speedup here.

Table 3.7 COBAYN speedup w.r.t. the average speedup gained with predictive modeling in both 1-shot and 8-shot scenarios reported in [35]

Algorithm and parameter configuration	COBAYN Speedup	
	w.r.t 1-shot	w.r.t 8-shot
LR -S 0	1.7551	1.6673
LR -S 1	1.7590	1.6710
LR -S 2	1.7191	1.6331
SVM NormalizedPolykernel -C 1.0 -E 8.0	1.5437	1.4665
SVM RBFKernel -C 2.0 -G 0.0	1.5206	1.4445
SVM RBFKernel -C 2.0 -G 25.0	1.5082	1.4327
SVM RBFKernel -C 2.0 -G 50.0	1.5045	1.4292
SVM RBFKernel -C 2.0 -G 75.0	1.5029	1.4277
SVM RBFKernel -C 2.0 -G 30.0	1.4927	1.4180
SVM RBFKernel -C 4.0 -G 30.0	1.5073	1.4319
SVM RBFKernel -C 0.01 -G 30.0	1.5073	1.4374
SVM RBFKernel -C 4.0 -G 50.0	1.5045	1.4292
IBk -K 1	1.4447	1.3724
IBk -K 2	1.4667	1.3933
IBk -K 5	1.4887	1.4142
M5P -M 1.0	1.4281	1.3566
M5P -M 2.0	1.4282	1.3567
M5P -M 4.0	1.4282	1.3568
M5P -M 10.0	1.4575	1.3846
M5P -M 50.0	1.4913	1.4167
K* -B 0 -M a	1.5192	1.4432
K* -B 20 -M a	1.5216	1.4455
K* -B 25 -M a	1.5258	1.4495
K* -B 50 -M a	1.4740	1.4003
K* -B 75 -M a	1.4737	1.4001
K* -B 100 -M a	1.5172	1.4413
K* -B 0 -M n	1.5208	1.4447
K* -B 20 -M n	1.5216	1.4455
K* -B 25 -M n	1.5258	1.4495
K* -B 50 -M n	1.4740	1.4003
MLP -L 0.3 -N 500 -H a	2.0435	1.9413
MLP -L 0.05 -N 500 -H a	1.6738	1.5901
MLP -L 0.1 -N 500 -H a	1.7246	1.6383
MLP -L 0.5 -N 500 -H a	1.8138	1.7231
MLP -L 0.9 -N 500 -H a	1.5250	1.4487

(continued)

Table 3.7 (continued)

Algorithm and parameter configuration	COBAYN Speedup	
	w.r.t 1-shot	w.r.t 8-shot
MLP -L 0.4 -N 500 -H a	1.9426	1.8454
MLP -L 0.5 -N 1000 -H a	1.8535	1.7608
MLP -L 0.5 -N 1500 -H a	1.7388	1.6518
MLP -L 0.5 -N 500 -H t	1.5579	1.4801
AVERAGE (Harmonic mean)	1.5622	1.4841

Note that Harmonic mean is always less than or equal to the arithmetic mean [37]. In all cases, COBAYN outperforms the reference methodology; specifically we have at least $1.3\times$ and up to $2.04\times$ speedup compared with the best achieved results reported in [35].

3.5 Conclusions

This chapter presented COBAYN, a methodology to infer by means of a *Bayesian framework* the best compiler optimizations to be applied for optimizing the performance of a target application. The methodology uses target independent software features to sample a statistical model built using Bayesian Networks to extract a set of suitable compiler configurations. Feature reduction techniques have been adopted to reduce the complexity and training time of the Bayesian model while also eliminating possible noise in the data and improving the quality of the results. The proposed approach has been evaluated on an ARM-based platform, using GCC compiler. The experimental results demonstrated that the proposed technique outperforms both standard optimization levels and state-of-the-art iterative and not iterative compilation techniques while using the same number of evaluations.

In the following chapter, we move towards tackling the phase-ordering problem of compiler optimizations using machine learning.

References

1. Palermo G, Silvano C, Zaccaria V (2005) Multi-objective design space exploration of embedded systems. *J Embed Comput* 1(3):305–316
2. Hoste K, Eeckhout L (2008) Cole: compiler optimization level exploration. In: *Proceedings of the 6th annual IEEE/ACM international symposium on code generation and optimization*, pp 165–174
3. Chen Y, Fang S, Huang Y, Eeckhout L, Fursin G, Temam O, Chengyong W (2012) Deconstructing iterative optimization. *ACM Trans Archit Code Optim (TACO)* 9(3):21

Learning Compiler Pass Orders using Coreset and Normalized Value Prediction

Youwei Liang^{*1} Kevin Stone^{*2} Ali Shameli² Chris Cummins² Mostafa Elhoushi² Jiadong Guo²
Benoit Steiner² Xiaomeng Yang² Pengtao Xie¹ Hugh Leather² Yuandong Tian²

Abstract

Finding the optimal pass sequence of compilation can lead to a significant reduction in program size and/or improvement in program efficiency. Prior works on compilation pass ordering have two major drawbacks. They either require an excessive budget (in terms of compilation steps) at compile time or fail to generalize to unseen programs. In this paper, for code-size reduction tasks, we propose a novel pipeline to find program-dependent pass sequences within 45 compilation calls. It first identifies a *coreset* of 50 pass sequences via greedy optimization of a submodular function, and then learns a policy with Graph Neural Network (GNN) to pick the optimal sequence by predicting the normalized values of the pass sequences in the coreset. Despite its simplicity, our pipeline outperforms the default `-Oz` flag by an average of 4.7% over a large collection (4683) of unseen code repositories from diverse domains across 14 datasets. In comparison, previous approaches like reinforcement learning on the raw pass sequence space may take days to train due to sparse reward, and may not generalize well in held-out ones from different domains. Our results demonstrate that existing human-designed compiler flags can be improved with a simple yet effective technique that transforms the raw action space into a small one with denser rewards.

1. Introduction

For more efficient execution with fewer resources (e.g., memory, CPU, and storage), applying the right ordering for compiler optimization passes to a given program, i.e., *pass ordering*, is an important yet challenging problem. Manual efforts require expert knowledge and are time-consuming, error-prone, and often yield sub-par results, due to the huge

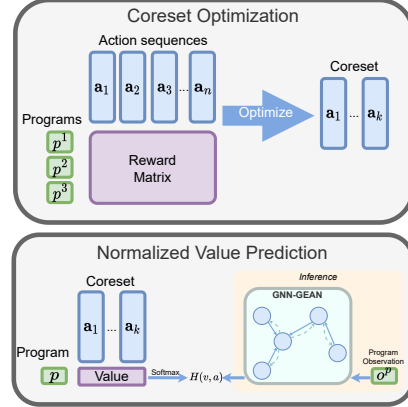


Figure 1. A depiction of our main contributions. (Top) **Coreset Optimization**: A process for discovering a small set of pass sequences (*coreset*) that generalizes. (Bottom) **Normalized Value Prediction**: A process where our model learns to predict the normalized value of pass sequences from the coreset. (Bottom inset) Our model, Graph Edge Attention Network (**GEAN**), for encoding program observations.

size of the search space. For example, the LLVM compiler has 124 different compilation flags. If the pass sequences have a length of 45, then the possible number of sequences ($124^{45} \sim 10^{94}$) is already more than the atoms in the universe ($\sim 10^{80}$ (Planck Collaboration et al., 2016)).

In recent years, machine learning (ML)-guided pass ordering has emerged as an interesting field to replace this laborious process (Wang & O’Boyle, 2018). Along this line, many works show promising results using various optimization and/or ML techniques (e.g., reinforcement learning (Haj-Ali et al., 2020a), language modelling (Cummins et al., 2017), evolutionary algorithms (Kulkarni & Cavazos, 2012), etc).

However, there are several limitations. Some previous works (e.g., MLGO (Trofin et al., 2021), MLGoPerf (Ashouri et al., 2022)) run adaptive search algorithms to optimize a set of programs for many hours. While this achieves strong performance gain, the procedure can be slow and does not distill knowledge from past experience and requires searching from scratch for unseen programs. Recent works like Autophase (Haj-Ali et al., 2020b) learn a *pass policy* via re-

^{*}Equal second author contribution, listed alphabetically.

¹University of California, San Diego ²Meta AI. Correspondence to: Kevin Stone <kevinleestone@meta.com>.

inforcement learning, and applies it to unseen programs without further search procedure, and GO (Zhou et al., 2020) fine-tunes the models for unseen programs. These approaches work for unseen programs from the same/similar domain, but can still be quite slow in the training stage, and do not show good generalization to programs from very different domains.

In this work, we propose a novel pass ordering optimization pipeline to reduce the *code size* of a program. As a first contribution, we formulate the search for a *universal* core set of pass sequences (termed **coreset**) as an optimization problem of a submodular reward function and use a greedy algorithm to approximately solve it. The resulting coreset consists of 50 pass sequences with a total number of passes of 625. Importantly, it leads to very strong performance across programs from diverse domains, ranging from the Linux Kernel to BLAS. Specifically, for one unseen program in the evaluation set, there exists one of the 50 pass sequences that leads to an average code size reduction of 5.8% compared to the default `-Oz` setting, across 10 diverse codebases (e.g. Cbench (Fursin, 2014), MiBench (Guthaus et al., 2001), NPB (Bailey et al., 1995), CHStone (Hara et al., 2008), and Anghabench (Da Silva et al., 2021)) of over one million programs in total. Considering the huge search space of compiler flags, this is a very surprising finding.

While it is time-consuming to find an optimal pass sequence with an exhaustive enumeration of the core subset of pass sequences, as a second contribution, we find that the (near) optimal pass sequence can be directly predicted with high accuracy via our *Graph Edge Attention Network* (GEAN), a graph neural network (GNN) architecture adapted to encode the augmented ProGraML (Cummins et al., 2021) graphs of programs. Therefore, we can run a few pass sequences selected by the model on an unseen program to obtain a good code size reduction. This enables us to find a good flag configuration that leads to 4.7% improvement on average, with just 45 compilation passes, a reasonable trade-off between the cost spent on trying compilation passes and the resulting performance gain.

We compare our approach with extensive baselines, including reinforcement learning (RL) -based methods such as PPO, Q-learning, and behavior cloning. We find that RL-based approaches operating on the original compiler pass space often suffer from unstable training (due to inaccurate value estimation) and sparse reward space and fail to generalize to unseen programs at inference. In comparison, our approach transforms the vast action space into a smaller one with much more densely distributed rewards. In this transformed space, approaches as simple as behavior cloning can be effective and generalizable to unseen programs.

2. Related Work

Graph structured data are present in numerous applications and it has been shown that taking advantage of this data can help us train very effective machine learning models. (Brauckmann et al., 2020) use abstract syntax trees and control flow graphs for learning compiler optimization goals. They show that using such graphs allows them to outperform state-of-the-art in the task of heterogeneous OpenCL mapping. (Guo et al., 2020) uses a transformer based model with a graph guided masked attention that incorporates the data flow graph into the training. They achieve state of the art performance in four tasks including code search, clone detection, code translation, and code refinement.

As a contender to graph neural networks, (Mialon et al., 2021) uses transformers to process graphs. They show that if we effectively encode positional and local sub-structures of graphs and feed them to the transformer, then the transformer can outperform the classical GNN models. They test their model on classification and regression tasks and achieve state of the art performance. In (Srinivas et al., 2020), they used an unsupervised model to learn embeddings of high dimensional pixel data using contrastive learning. They then use this embedding for downstream reinforcement learning tasks.

3. Methodology

3.1. Action space

The CompilerGym framework (Cummins et al., 2022) provides a convenient interface for the compiler pass ordering problem. The default environment allows choosing one of 124 discrete actions at each step corresponding to running a specific compiler pass. In this work we will use the term pass interchangeably with action. We fix episode lengths to 45 steps to match the setup in (Haj-Ali et al., 2020b). Given that our trajectories have a length of 45 steps, this means we have $124^{45} \sim 1.6 \times 10^{94}$ possible pass sequences to explore. To find an optimal pass sequence for a program, we can apply some existing reinforcement learning methods including Q learning like DQN (Mnih et al., 2015) and policy gradient like PPO (Schulman et al., 2017).

Pass Sequences However for this problem it turns out that certain pass sequences are good at optimizing many different programs (where “good” is defined as better than the compiler default `-Oz`). We found that constraining the action space to a learned set of pass sequences enables state of the art performance and also significantly reduces the challenge of exploration. This allows us to cast the problem as one of supervised learning over this set of pass sequences. We use the following algorithm to find a good set of pass sequences.

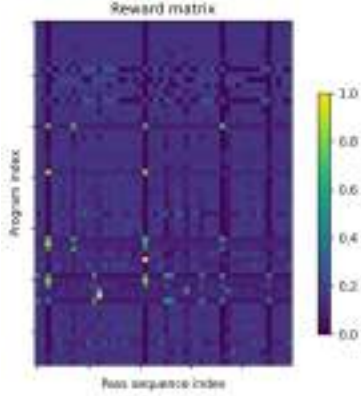


Figure 2. An exemplar reward matrix for 67 programs and 50 pass sequences. Most of the pass sequences do not lead to strong rewards, except for a few. On the other hand, certain pass sequences (i.e., columns) can lead to high rewards for multiple programs simultaneously and thus are good candidates for the coreset.

Suppose we have N programs and M promising pass sequences. Let $R = [r_{ij}] \in \mathbb{R}^{N \times M}$ be the reward matrix, in which $r_{ij} > 0$ is the *ratio* of the codesize of i -th program if applied with j -th pass sequence, compared to $-\infty$. $r_{ij} > 1$ means that the j -th pass sequence does *better* than $-\infty$ in codesize reduction for i -th program, and $r_{ij} < 1$ means it performs worse. The reward matrix is normalized per row, by the maximum reward for each program, so that the optimal pass sequence has reward of 1 for each program.

Then we aim to pick a subset S of K pass sequences, called the *coreset*, from all M pass sequences, so that the overall saving $J(S)$ is maximized:

$$\max_{|S| \leq K} J(S) = \sum_{i=1}^N \max_{j \in S} r_{ij} \quad (1)$$

Finding M candidate pass sequences. Note that there can be exponential number of pass sequences, and we cannot construct the entire reward matrix, instead we seed a list of candidate action trajectories. For this, we run a random policy on a subset of M (17500) selected training programs. For each selected program, we run E (200) episodes and pick the best pass sequence as one of the M candidates. If part of the best pass sequence leads to the same state, they are truncated so that the sequence becomes shorter. If multiple pass sequences yield the same reward we only retain the first after ordering them length-lexicographically. On average these last two steps reduce the length of the candidate pass sequences by 80%.

We then construct the reward r_{ij} by applying the j -th best pass sequence to program i , and comparing it with $-\infty$.

Finding the best coreset S with greedy algorithm. As

a function defined in subsets, $J(S)$ can be proven to be a nonnegative and monotone submodular function (See Appendix). While maximizing a submodular function is NP-hard (Ward & Živný, 2016), the following *greedy algorithm* is proven to be an efficient approximate algorithm that leads to fairly good solutions (Nemhauser et al., 1978). Starting from $S_0 = \emptyset$, at each iteration t , it picks a new pass sequence j_t as follows:

$$j_t := \arg \max_{j \notin S_{t-1}} J(S_{t-1} \cup \{j\}) \quad (2)$$

And $S_t \leftarrow S_{t-1} \cup \{j_t\}$ until we pick K pass sequences. We set $K = 50$ in this paper.

Given the discovered coreset S , we define a **generalized action** as a pass sequence in S . Applying a generalized action to a program means that we roll out the corresponding pass sequence on the program and return the best program state (i.e., having the highest cumulative reward) (it is feasible because we can cache the program state for each step).

3.2. Normalized Value Prediction

After discovering the “good” pass sequences (i.e., the coreset), we can turn the problem of the sequential decision-making on compiler passes into a problem of supervised learning. The target is to train a model to predict the best pass sequence conditioned on the program, where the label of the program is the index of the pass sequence that results in the largest code size reduction. However, one important observation we have is that there are typically multiple pass sequences in the coreset that all result in the largest code size reduction (see Figure 3 for the examples). Therefore, instead of using the single-class classification method with a cross entropy loss, we leverage the fact we have access to the values for all pass sequences. We predict the softmax normalized value of each pass sequence with a cross entropy loss detailed below. This approach is similar to behavior cloning (Pomerleau, 1988) but with soft targets over the coreset.

For a program p , we roll out all pass sequences in the coreset on it, obtaining a reward r_j^p for the j -th sequence (i.e., the highest cumulative reward observed during the rollout of the pass sequence), which forms a value vector $\mathbf{r}^p = [r_1^p, \dots, r_K^p]$. Then, the normalized values of the pass sequences are defined by

$$\mathbf{v}^p = \text{Softmax}(\mathbf{r}^p / T) \quad (3)$$

where T is a temperature parameter.

For an initial observation o^p of a program, our model outputs a probability distribution, $\mathbf{a} = f(o^p)$, over the pass sequences. The target of the training is to make $\mathbf{a}$ close to the normalized values of the pass sequences. To this end, we use the Kullback–Leibler (KL) divergence to supervise the

model, which can be reduced to the following cross entropy loss up to a constant term.

$$\mathcal{L}(\mathbf{a}^p, \mathbf{v}^p) = - \sum_{j=1}^K a_j^p \log v_j^p \quad (4)$$

3.3. Program Representations

Since we use the CompilerGym (Cummins et al., 2022) environments for program optimization, we exploit the program representations from CompilerGym, where program source code is converted to LLVM Intermediate Representation (IR) (Lattner & Adve, 2004) and several representations are constructed from the IR, including the ProGraML graph (Cummins et al., 2021), the Autophase feature (Haj-Ali et al., 2020b), and the Inst2vec feature (Ben-Nun et al., 2018). We choose to use the LLVM environment from CompilerGym because the LLVM ecosystem is a popular compiler infrastructure that powers Swift, Rust, Clang, and more.

Autophase We use the Autophase features Haj-Ali et al. (2020b) to build some baseline models for comparison, which will be detailed in Section 4.2. The Autophase feature is a 56-dimension integer feature vector summarizing the LLVM IR representation, and it contains integer counts of various program properties such as the number of instructions and maximum loop depth. we use an MLP to encode it and output a program representation.

ProGraML As a part of our main method, we leverage ProGraML (Cummins et al., 2021) graphs for training GNN models. ProGraML is a graph-based representation that encodes semantic information of the program which includes control flow, data flow, and function call flow. This representation has the advantage that it is not a fixed size - it does oversimplify large programs - and yet it is still a more compact format than the original IR format. We list three bullet points of the ProGraML graph below.

- **Node features** Each node in a ProGraML graph have 4 features described in Table 1. The “text” feature is a textual representation and the main feature that captures the semantics of a node. For example, it tells us what an “instruction” node does (e.g., it can be `alloca`, `store`, `add`, etc).
- **Edges features** Each edge in a ProGraML graph have 2 features described in Table 1.
- **Type graph** There is an issue with the ProGraML graph. Specifically, a node of type variable/constant node can end up with a long textual representation (for the “text” feature) if it is a composite data structure. For example, a struct (as in C/C++) containing dozens of data members needs to include all the

members in its “text” feature. In other words, the current ProGraML representation does not automatically break down composite data types into their basic components. Since there is an unbounded number of possible structs, this prevents 100% vocabulary coverage on any IR with structs (or other composite types). To address this issue, we propose to expand the node representing a composite data type into a type graph. Specifically, a pointer node is expanded into this type graph: `[variable] <- [pointer] <- [pointed-type]`, where [...] denotes a node and <- denotes an edge connection. A struct node is expanded into a type graph where all its members are represented by individual nodes (which may be further expanded into their components) and connected to a struct node. An array is expanded into this type graph: `[variable] <- [array] <- [element-type]`. The newly added nodes are categorized as type nodes and the edges connecting the type nodes are type edges. The type nodes and type edges constitute the type sub-graphs in the ProGraML graphs. In this manner, we break down the composite data structures into the type graphs that consist of only primitive data types such as `float` and `i32`.

3.4. Network Architecture

Since the Autophase feature can be encoded by a simple MLP, we discuss only the network architectures for encoding the ProGraML graphs in this section.

We use a graph neural network (GNN) as the backbone to encode the ProGraML graphs and output a graph-level representation. The GNN encodes the graph via multiple layers of message passing and outputs a graph-level representation by a global average pooling over the node features. The goal of graph encoding is to use the structure and relational dependencies of the graph to learn an embedding that allows us to learn a better policy. To this end, we experimented with several different GNN architectures such as Graph Convolutional Network (GCN) (Kipf & Welling, 2017), Gated Graph Convolutions Network (GGC) (Li et al., 2015), Graph Attention Network (GAT) (Brody et al., 2022), Graph Isomorphism Network (GIN) (Xu et al., 2019). To better capture the rich semantics of node/edge features in the ProGraML graphs, we propose Graph Edge Attention Network (GEAN), a variant of the graph attention network (Veličković et al., 2017). These GNNs leverage both the node and edge features, so we start by presenting how to embed the node and edge features.

Node embedding For the “text” features of the nodes, we build a vocabulary that maps from text to integer. The vocabulary covers all the text fields of the nodes in the graphs in the training set. The final vocabulary consists of

	Feature	Description
Node	type	One of {instruction, variable, constant, type }
	text	Semantics of the node
	function	Integer position in function
	block	Integer position in IR basic block
Edge	flow	Edge type. One of {call, control, data, type }
	position	Integer edge position in flow branching

Table 1. Features in the ProGraML graph representation which we augment with type information (changes highlighted). We ablate the augmentations in Section 4.5.

117 unique textual representations, and we add an additional item “unknown” to the vocabulary which denotes any text features that may be encountered at inference time and we have not seen before. The i -th textual representation is embedded using a learnable vector $v_i \in \mathbb{R}^d$, where d is the embedding dimension. The “type” feature is not used because it can be inferred from the “text” feature.

Edge embedding The edge embedding is the sum of three types of embedding as the following.

- **Type embedding** We have 4 types of edge flows, so we use 4 learnable vectors to represent them.
- **Position embedding** The “position” feature of an edge is a non-negative integer which does not have an upper bound. We truncate any edge positions larger than 32 to 32 and use a set of 32 learnable vectors to represent the edge positions.
- **Block embedding** We use the block indices of the two nodes connected by the edge to construct a new edge feature. The motivation is that whether the edge goes beyond an IR basic block can influence program optimization. Suppose the block indices of the source node and the target node of an edge are respectively b_i and b_j . We get the relative position of the two nodes with respect to IR basic blocks in the following way: $p_{block} = \text{sign}(b_i - b_j)$. If the edge connects two nodes in the same IR basic block, then p_{block} is 0. And $p_{block} = \pm 1$ indicates the edge goes from a block to the next/previous block. There are 3 possible values for p_{block} , so it is embedded using 3 learnable vectors.

The final embedding of an edge is the sum of its type, position, and block embedding vectors.

Graph mixup We note that the ProGraML graphs of two programs can be composed into a single graph without affecting the semantics of the two programs. And their value vectors can be added up to correctly represent the value vector of the composite graph. In this manner, we can enrich the input space to the GNNs and mitigate model overfitting for the normalized value prediction method.

Graph Edge Attention Network We introduce the GEAN in this paragraph and defer its mathematical details to the Appendix. There are two main differences between the GAT and GEAN. 1) GEAN adopts a dynamic edge representation. Specifically, GAT uses the node-edge-node feature to calculate the attention for neighborhood aggregation, while GEAN uses the node-edge-node feature to calculate not only the attention but also a new edge representation. Then, the updated edge representation is sent to the next layer for computation. Note that GAT uses the same edge embedding in each layer. We conduct an ablation study showing that the edge representation in GEAN improves the generalization of the model. 2) GAT treats the graph as an undirected graph while GEAN encodes the node-edge-node feature to output an updated node-edge-node feature, where the two updated node features represent the feature to be aggregated in the source node and the target node, respectively. This ensures that the directional information is preserved in the neighborhood aggregation.

3.5. Dataset Preparation

Overfitting issues could happen if training is performed on a small subset of programs, or the set of programs is not diverse enough. To mitigate this we find it helpful to create an aggregate dataset that uses many different public datasets as curated by CompilerGym. CompilerGym gives us access to 14 different datasets constructed using two different methods.

- **Curated** These are small collections of hand-picked programs. They are curated to be distinct from one another without overlap and are not useful for training. Typically programs are larger as they may comprise multiple source files combined into a single program. These are commonly used for evaluating compiler optimization improvements.
- **Uncurated** These are comprised of individual compiler IRs from building open source repositories such as Linux and Tensorflow. We also include synthetically generated programs, targeted for compiler testing (not optimization).

For our aggregate dataset we decided to holdout the entirety of the four curated datasets for use as an out-of-domain test set. This is important because they represent the types of programs we expect to see in the wild. We also split the uncurated datasets into train, validation, and test programs.

3.6. Evaluation

For all our metrics and rewards we leverage the IR instruction count as value we are trying to minimize. We also report metrics on each CompilerGym dataset as well as

Type	Dataset	Train	Val	Test
Uncurated	anghabench-v1	70,7000	1,000	2,000
	blas-v0	133	28	29
	github-v0	7,000	1,000	1,000
	linux-v0	4,906	1,000	1,000
	opencv-v0	149	32	32
	poj104-v1	7,000	1,000	1,000
	tensorflow-v0	415	89	90
	clgen-v0	697	149	150
	csmith-v0	222	48	48
	llvm-stress-v0	697	149	150
Curated	cbench-v1	0	0	11
	chstone-v0	0	0	12
	mibench-v1	0	0	40
	npb-v0	0	0	121
Total	-	728,219	4,495	4,683

Table 2. CompilerGym dataset types and training splits. The hand curated datasets are used solely to evaluate generalization to real world program domains at test time.

the mean over datasets to get a single number to compare overall results.

- The mean percent improved over $-Oz$ (**MeanOverOz**) defined as following:

$$\bar{I}^{Oz} = \text{MeanOverOz} := \frac{1}{|\mathcal{P}|} \sum_p \frac{I_p^{Oz} - I_p^{\pi_\theta}}{I_p^{\pi_\theta}}, \quad (5)$$

where p is a specific program from the set of programs $\mathcal{P}$ in the dataset. I_p^{Oz} is the number of IR instructions in the program after running the default compiler pass $-Oz$. $I_p^{\pi_\theta}$ is the number of IR instructions in the program after applying the policy under consideration. We can think of this as a simple average of the percent improvement over $-Oz$.

- We also look compare the geometric mean (**GMeanOverOz**) of final sizes across all programs relative to $-Oz$ to give a weighted comparison that is insensitive to outliers.

$$\bar{I}_G^{Oz} = \text{GMeanOverOz} := \left(\prod_p \frac{I_p^{Oz}}{I_p^{\pi_\theta}} \right)^{\frac{1}{|\mathcal{P}|}} \quad (6)$$

4. Experiments

4.1. Experimental Setup

For the methods in Table 3, we search over a set of hyperparameters, including the temperature T in Eq. 3, number of layers, the embedding dimension of the node/edge features, and the output dimension in the hidden layers in the MLPs. We select the best model of each method based on

the validation metric (validation MeanOverOz in 45 steps) and report the best model’s metrics on the test set.

4.2. Baseline Methods

Oracle We consider a brute-force search over the coreset in order to find the best pass sequence for a given program. This gives us an upper-bound of the downstream policy network. In our case the coreset has a total of 50 sequences (625 total passes).

Top-45 We also consider how well we would do if the oracle is only allowed to use the most popular pass sequences but limited to 45 passes. We use 45 passes because this is maximum allowed for our all other baselines and our proposed method.

RL-PPO We reproduce the Autophase (Haj-Ali et al., 2020b) pipeline by using the state-of-the-art RL algorithm PPO (Schulman et al., 2017) to learn a policy model. We have two program representations for training the RL models, including the Autophase feature and the ProGraML graphs (note that Haj-Ali et al. (2020b) only used the Autophase feature). The Autophase/ProGraML feature is sent to a GNN/MLP for feature encoding, which outputs a program-level embedding. Following Haj-Ali et al. (2020b), we add an additional action history feature to the RL pipeline, which is a histogram of previously applied passes. The vector of the histogram of action history is divided by 45 (i.e., the number of the total passes in our budget) for normalization. A 2-layer MLP is used to encode the action history to obtain a feature vector, which is concatenated with the program embedding extracted from the ProGraML graph or the Autophase feature. The concatenated feature is sent to a 2-layer MLP to output the action probability for the policy network. The value network (i.e., the critic) in our PPO pipeline mimics the policy network (i.e., the actor) in feature encoding and outputs a scalar to estimate the state values. The state values are the expectation of the discounted cumulative rewards where the reward in each step is the improvement over $-Oz$: $(I_p^{(t)} - I_p^{\pi_\theta})/I_p^{(t)}$, where $I_p^{(t)}$ denotes the current IR instruction count of the program p at time step t . This reward is reasonable since it makes the value approximation Markovian. At inference, an action is sampled from the output of the learned policy network at each time step until the total number of steps reaches 45.

Q-value-rank We consider each pass sequence in the coreset as a *generalized action* and train a Q network to predict the value of each generalized action. Recall that the value vector r^p is the highest cumulative reward observed during the rollout of each pass sequence in the coreset on program p . The Q-value-rank model is trained to approximate the value vector using a mean squared loss.

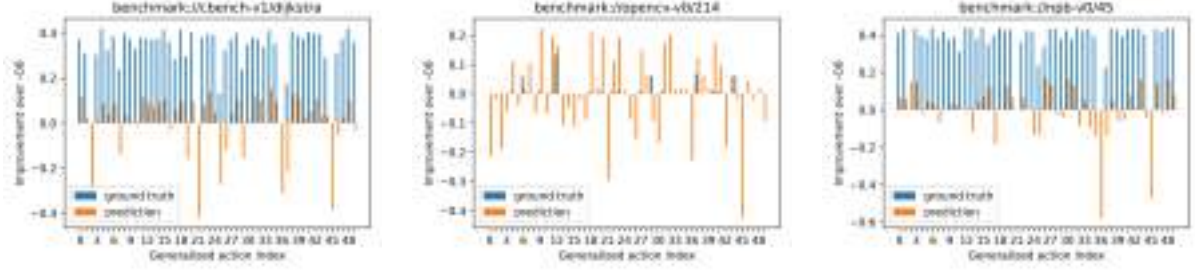


Figure 3. GEAN-Q-value-rank: ground truth of rewards and model predictions over the 50 generalized actions for three benchmarks.

BC We consider learning a standard behavior cloning model to predict the best pass sequences from the coreset, where the best pass sequence is defined as the following. As in the previous bullet point, the value vector is denoted by r^p . If there is only one i such that $r_i^p = \max_{j \in n} r_j^p$, then the classification label is i . If there are multiple such i 's (multiple pass sequences) that achieve the largest reward $\max_{j \in n} r_j^p$, then we order the corresponding pass sequences by length-lexicographic ordering. The classification label is selected to be the first one after the ordering. This ensures that our definition for the best pass sequence (among the coreset) for a program is unique and consistent. We use a standard cross entropy loss to train the single-class classification model.

NVP This is the normalized value prediction method described in Section 3.2.

The last three methods (i.e., Q-value-rank, BC, and NVP) share the same **inference protocol**. Note that they all output a vector $\mathbf{a}$ of length 50 whose entries correspond to the pass sequences in the coreset. At inference, we roll out the pass sequences with the highest values in $\mathbf{a}$ one by one until our budget of 45 passes is reached. Since the pass sequence has an average length of 12.5, typically 3 or 4 pass sequences are applied (anything beyond 45 passes will be truncated). For BC and NVP, we also tried sampling pass sequences using the model output $\mathbf{a}$, but that resulted in worse performance. Therefore, we stick to selection by maximum values.

4.3. Main Results

In Table 3 we present the main results of our experiments comparing our proposed method –NVP to various baselines. The test programs were completely held-out during both data-driven learning phases (pass sequence search and model training).

The results show that our model achieves strong performance over the prior method (Autophase-RL-PPO) proposed in (Haj-Ali et al., 2020b). Additionally, we can see that both the GEAN model and the normalized value prediction over the discovered coreset are needed to achieve the best performance within 45 passes. See Figure 6 in the

Method	#passes	$\bar{I}^{Oz}$	$\bar{I}_G^{Oz}$
Compiler (-Oz)	-	0%	1.000
Autophase-RL-PPO	45	-16.3%	0.960
GCN-RL-PPO	45	-12.2%	0.987
GGC-RL-PPO	45	-8.5%	1.000
GIN-RL-PPO	45	-11.3%	0.991
GAT-RL-PPO	45	-10.3%	0.999
GEAN-RL-PPO	45	-10.0%	0.997
GEAN-Q-value-rank	45	-0.3%	1.043
Autophase-BC	45	2.6%	1.043
GEAN-BC	45	2.1%	1.038
Autophase-NVP (Ours)	45	3.8%	1.054
GCN-NVP (Ours)	45	4.4%	1.058
GGC-NVP (Ours)	45	4.1%	1.056
GIN-NVP (Ours)	45	4.2%	1.056
GAT-NVP (Ours)	45	4.1%	1.055
GEAN-NVP (Ours)	45	4.7%	1.062
Top-45	45	-7.5%	0.992
Oracle	625	5.8%	1.075

Table 3. Evaluation results on **held-out test set** averaged over all datasets. $\bar{I}^{Oz}$ denotes per-program **MeanOverOz**, and $\bar{I}_G^{Oz}$ denotes **GMeanOverOz** over all programs. All methods except Compiler and Oracle baselines use 45 compiler optimization passes.

Appendix for a visualization of the improvement in program size over the 45 passes on programs from the holdout set.

The Oracle shows strong performance but requires a large number of interactions with the compiler. But, this shows that the pass sequence search generalizes to new unseen programs. This is somewhat unsurprising given that the compiler’s built-in hand-tuned pass list (-Oz) works reasonably well for most programs.

The performance of Top-45 by itself is weak showing that in order to achieve good results in a reasonable number of passes (45) we need to leverage a general policy and

search to select the most likely candidate pass sequences to evaluate.

4.4. Why Did the RL-PPO Baseline Fail?

We provide an empirical analysis of why the RL-PPO approaches obtain much lower performance compared to our NVP approaches. We *hypothesize* two possible reasons for the failures of RL-PPO. **1) Inaccurate state-value estimation results in a high variance in training.** In the PPO algorithm, we have a policy network (the actor) to output a probability distribution over the actions. And we have a value network (the critic) for estimating the state values, where the approximation is based on regressing the cumulative reward of trajectories sampled from the current policy (Schulman et al., 2017). The update of the policy network is based on the state value outputted by the value network. Inaccurate state value estimation results in a high variance in training the policy network. Due to the stochastic nature of the value estimation that stems from the Monte Carlo sampling in cumulative reward regression, it is difficult to analyze how accurately the value network approximates the ground truth state values (which are unknown even for the programs in the training set). We alleviate this issue by analyzing the Q-value-rank approach (as introduced in Section 4.2), which can be seen as a simplified version of the value approximation in PPO. The Q-value-rank approach is simpler because the values to estimate are deterministic (i.e., the value vector r^p is fixed for a program p). Moreover, since we consider the 50 pass sequences in our coreset as 50 generalized actions, the Q-value-rank approach can be seen as the value approximation in a PPO pipeline where a trajectory consists of only a single step over the 50 generalized actions. In this sense, the Q-value-rank approach is a simplified version of the regular value estimation in PPO. Figure 3 shows that the value estimation is inaccurate for programs in the held-out test set even for Q-value-rank approach. Therefore, it is even more challenging to estimate the state values in PPO. The inaccuracy leads to a high variance in training the policy network. **2) The reward is very sparse.** As shown in Figure 2, the rewards are very sparse. Therefore, the good states (i.e., the program states with a higher chance to be optimized in code size reduction) are rarely seen by the value/policy network during rollouts. Then, the value network does not have a good value estimation for those good states, and the policy network does not converge to output a good policy for them. We conjecture these two issues are the main reason for why the RL-PPO methods obtain the worst performance as shown in Table 3.

4.5. Ablation Studies

Ablation for GEAN-NVP We perform 3 ablation experiments for GEAN-NVP, where we remove graph mixup,

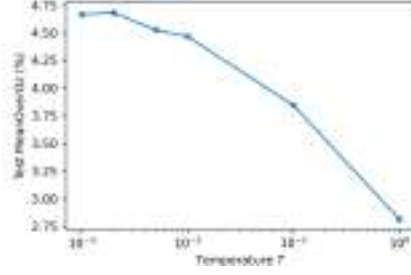


Figure 4. The effect of temperature on GEAN-NVP.

Method	Test MeanOverOz
GEAN-NVP	4.7% (0.0%)
- graph mixup	4.4% (-0.3%)
- edge embedding	4.4% (-0.3%)
- type graph	-5.3% (-10.0 %)

Table 4. Ablation on GEAN-NVP components.

mask the edge embedding, and remove the type graph, respectively. The results in Table 4 show that the test MeanOverOz metric drops after removing any of the three components. Specifically, the performance drops significantly after removing the type graph, which validates its importance.

The effect of the temperature The temperature parameter T in Eq. 3 controls how sharp the target distribution is. The distribution tends to be sharper as the temperature decreases. To analyze the influence of the temperature on the generalization of the model, We vary the temperature T in training the GEAN-NVP model and report the results in Figure 4.

5. Conclusions

In this paper, we develop a pipeline for program size reduction under limited compilation passes. We find that it is a great challenge to approximate the state values (i.e., the maximum code size reduction) for a diverse set of programs, so existing state-of-the-art methods such as proximal policy optimization (PPO) fail to obtain good performances. To tackle this problem, we propose a search algorithm that discovers a good set of pass sequences (i.e., the coreset), which generalizes well to unseen programs. Moreover, we propose to train a GNN to approximate the normalized state values of programs over the coreset, for which we propose a variant of the graph attention network, termed GEAN. Our pipeline of coreset discovery and normalized value prediction via GEAN perform significantly better than the PPO baselines.

A. Proofs

Lemma A.1. *The objective $J(S)$ defined in Eqn. 1*

$$\max_{|S| \leq K} J(S) = \sum_{i=1}^N \max_{j \in S} r_{ij} \quad (7)$$

(with the additional definition $J(\emptyset) = 0$), is a nonnegative and monotone submodular function.

Proof. Since $r_{ij} > 0$, it is clear that $J(S) \geq 0$ is nonnegative.

To incorporate the special case $J(\emptyset) = 0$, note that $J(S)$ can be written as

$$\max_{|S| \leq K} J(S) = \sum_{i=1}^N \max \left(\max_{j \in S} r_{ij}, 0 \right). \quad (8)$$

Let $\hat{r}_{ij} = r_{ij}$ and $\hat{r}_{i,0} = 0$, then in order to prove $J(S)$ is monotone and submodular, by additivity, we only need to prove $J_i(S) := \max_{j \in S \cup \{0\}} \hat{r}_{ij}$ is monotone and submodular.

For any $A \subseteq B$, it is clear that

$$J_i(A) = \max_{j \in A \cup \{0\}} \hat{r}_{ij} \leq \max_{j \in B \cup \{0\}} \hat{r}_{ij} = J_i(B) \quad (9)$$

So $J_i(S)$ is monotone.

To prove submodularity, for any $A \subseteq B$, we compare the quantity of $J_i(A \cup \{j\}) - J_i(A)$ and $J_i(B \cup \{j\}) - J_i(B)$ for $j \notin B$.

Case 1: $\hat{r}_{ij}$ is a maximum over the subset B . In this case, then $\hat{r}_{ij}$ is also a maximum over the subset A . Then $J_i(A \cup \{j\}) = J_i(B \cup \{j\}) = \hat{r}_{ij}$, since $J_i(A) \leq J_i(B)$, we have:

$$J_i(A \cup \{j\}) - J_i(A) \geq J_i(B \cup \{j\}) - J_i(B) \quad (10)$$

Case 2: $\hat{r}_{ij}$ is a maximum over A but not in B . Then $J_i(A \cup \{j\}) - J_i(A) \geq 0$, but $J_i(B \cup \{j\}) - J_i(B) = 0$. So Eqn. 10 still holds.

Case 3: $\hat{r}_{ij}$ is neither a maximum in A or in B . Then both $J_i(A \cup \{j\}) - J_i(A) = 0$ and $J_i(B \cup \{j\}) - J_i(B) = 0$. So Eqn. 10 still holds.

By definition of submodularity (Eqn. 10), we know $J_i(S)$ is submodular and so does $J(S)$. $\square$

B. GEAN Encoding

Our Graph Edge Attention Network (GEAN) has the following key features.

Attention with edge features We modify the attention mechanism in GAT to output an edge embedding and two node features for neighborhood aggregation. For clarity, we show a table containing the notations used in the GNN in Table 5. The feature update process can be mathematically defined by the following equations, where $M_i, i = 1, \dots, 5$ is an encoding fully connected layer.

$$X'_{ij}{}^{(t+1)} = M_1(X_i^{(t)}, E_{i \rightarrow j}^{(t)}, X_j^{(t)}), \quad (11)$$

$$a'_{ij}{}^{(t+1)} = M_2(X_i^{(t)}, E_{i \rightarrow j}^{(t)}, X_j^{(t)}), \quad (12)$$

$$X_{ji}^{(t+1)} = M_3(X_i^{(t)}, E_{i \rightarrow j}^{(t)}, X_j^{(t)}), \quad (13)$$

$$a_{ji}^{(t+1)} = M_4(X_i^{(t)}, E_{i \rightarrow j}^{(t)}, X_j^{(t)}), \quad (14)$$

$$E_{i \rightarrow j}^{(t+1)} = M_5(X_i^{(t)}, E_{i \rightarrow j}^{(t)}, X_j^{(t)}), \quad (15)$$

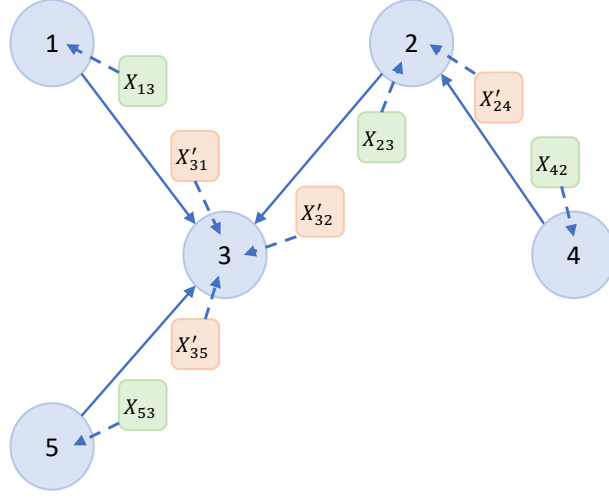


Figure 5. Graph attention. Circles denote nodes and solid arrows denote edges. Squares are the calculated features, and dash arrows represent feature aggregation. The orange/green squares denote the features to be aggregated in the target/source nodes of the edges. The edge embedding and the attention are not shown.

Notation	Meaning
$\mathcal{E}$	The set of edges in the graph
(i, j)	Edge from node i pointing to node j
$X_i^{(t)}$	Representation of node i at layer t
$E_{i \rightarrow j}^{(t)}$	Representation of the edge (i, j) at layer t
$X_{ij}^{(t)}$	Representation for node i associated with edge (i, j)
$X'_{ij}^{(t)}$	Representation for node i associated with edge (j, i)
$a_{ij}^{(t)}$	Raw attention associated with representation $X_{ij}^{(t)}$
$a'_{ij}^{(t)}$	Raw attention associated with representation $X'_{ij}^{(t)}$
$\alpha_{ij}^{(t)}$	Normalized attention associated with $a_{ij}^{(t)}$
$\alpha'_{ij}^{(t)}$	Normalized attention associated with $a'_{ij}^{(t)}$
$\mathcal{T}_i$	Target neighbors of node i : $\{j (i, j) \in \mathcal{E}\}$
$\mathcal{S}_i$	Source neighbors of node i : $\{j (j, i) \in \mathcal{E}\}$

Table 5. The notations in GEAN.

In words, the node-edge-node triplet, $(X_i^{(t)}, E_{i \rightarrow j}^{(t)}, X_j^{(t)})$, associated with edge (i, j) , is encoded by fully connected layers to output 5 features, including $X_{ij}^{(t+1)}$ and $a'_{ij}^{(t+1)}$ (a representation and attention to be aggregated in node i), and $X'_{ji}^{(t+1)}$ and $a_{ji}^{(t+1)}$ (a representation and attention to be aggregated in node j), and the updated edge representation $E_{i \rightarrow j}^{(t+1)}$. Note that the features to be aggregated to a target node are marked with the ', and those to a source node are without the ' (see Figure 5). After the feature encoding, we perform an attention-weighted neighborhood aggregation for each node, which can be mathematically described by the following equations.

$$\left[\alpha_{ij}^{(t+1)} \right]_{j \in \mathcal{T}_i} \parallel \left[\alpha'_{ij}^{(t+1)} \right]_{j \in \mathcal{S}_i} = \text{Softmax} \left[\left[a_{ij}^{(t+1)} \right]_{j \in \mathcal{T}_i} \parallel \left[a'_{ij}^{(t+1)} \right]_{j \in \mathcal{S}_i} \right] \quad (16)$$

$$X_i^{(t+1)} = \sum_{j \in \mathcal{T}_i} \alpha_{ij}^{(t+1)} X_{ij}^{(t+1)} + \sum_{j \in \mathcal{S}_i} \alpha'_{ij}^{(t+1)} X'_{ji}^{(t+1)} \quad (17)$$

where $\parallel$ denotes concatenation.

Learning Compiler Pass Orders using Coreset and Normalized Value Prediction

In comparison, GAT only outputs an attention score by encoding the node-edge-node triplet: $d_{ij}^{(t+1)} = (X_i^{(t)}, E_{i \rightarrow j}^{(t)}, X_j^{(t)})$, and the feature for neighborhood aggregation is only conditioned on the neighbors: $X_{ij}^{(t+1)} = \text{MLP}(X_j^{(t)})$. To summarize, our encoding approach can ensure that the GNN model is aware of the direction of the edge and that the edge embedding is updated in each layer, which helps improve the performance (as shown in Table 3).

C. Detailed Results

Dataset	Oracle	Top-45	Autophase-RL-PPO	Autophase-NVP	GEAN-RL-PPO	GEAN-NVP
anghabench-v1	0.7%/1.011	-1.0%/0.996	-15.9%/0.974	-0.2%/1.002	-0.8%/0.996	-0.0%/1.003
blas-v0	2.6%/1.028	-0.4%/0.997	-1.7%/0.984	2.1%/1.023	-1.0%/0.990	2.4%/1.026
cbench-v1	3.5%/1.041	-2.4%/0.984	-10.1%/0.925	-0.1%/1.008	-1.6%/0.998	2.2%/1.028
chstone-v0	9.3%/1.106	1.2%/1.016	1.3%/1.018	8.3%/1.095	5.4%/1.060	8.8%/1.101
clgen-v0	5.4%/1.060	3.1%/1.034	-0.5%/0.998	4.6%/1.051	0.3%/1.005	5.0%/1.056
csmith-v0	21.2%/1.320	-96.3%/0.851	-116.0%/0.954	21.1%/1.320	-124.6%/0.965	21.1%/1.320
github-v0	1.0%/1.011	0.2%/1.002	0.1%/1.001	0.9%/1.010	-0.2%/0.999	0.9%/1.010
linux-v0	0.6%/1.007	-0.4%/0.998	-0.5%/0.997	0.6%/1.006	-2.3%/0.989	0.6%/1.007
llvm-stress-v0	6.3%/1.087	-18.9%/0.885	-67.0%/0.731	0.7%/1.035	-17.5%/0.888	2.1%/1.045
mibench-v1	1.7%/1.020	0.0%/1.003	-2.8%/0.976	-5.8%/0.963	-0.3%/1.000	-0.1%/1.003
npb-v0	9.8%/1.159	5.7%/1.085	0.9%/1.035	5.1%/1.079	3.7%/1.068	5.5%/1.085
opencv-v0	5.2%/1.061	1.0%/1.013	0.5%/1.007	4.2%/1.051	0.3%/1.004	4.8%/1.057
poj104-v1	7.8%/1.105	3.9%/1.055	-17.5%/0.876	6.1%/1.080	-0.7%/1.008	6.3%/1.082
tensorflow-v0	6.1%/1.077	-0.2%/0.998	0.2%/1.004	5.9%/1.075	-0.2%/0.998	5.9%/1.075
Average	5.8%/1.075	-7.5%/0.992	-16.3%/0.960	3.8%/1.054	-10.0%/0.997	4.7%/1.062

Table 6. Detailed evaluation results on held-out test sets.

D. Trajectory comparison

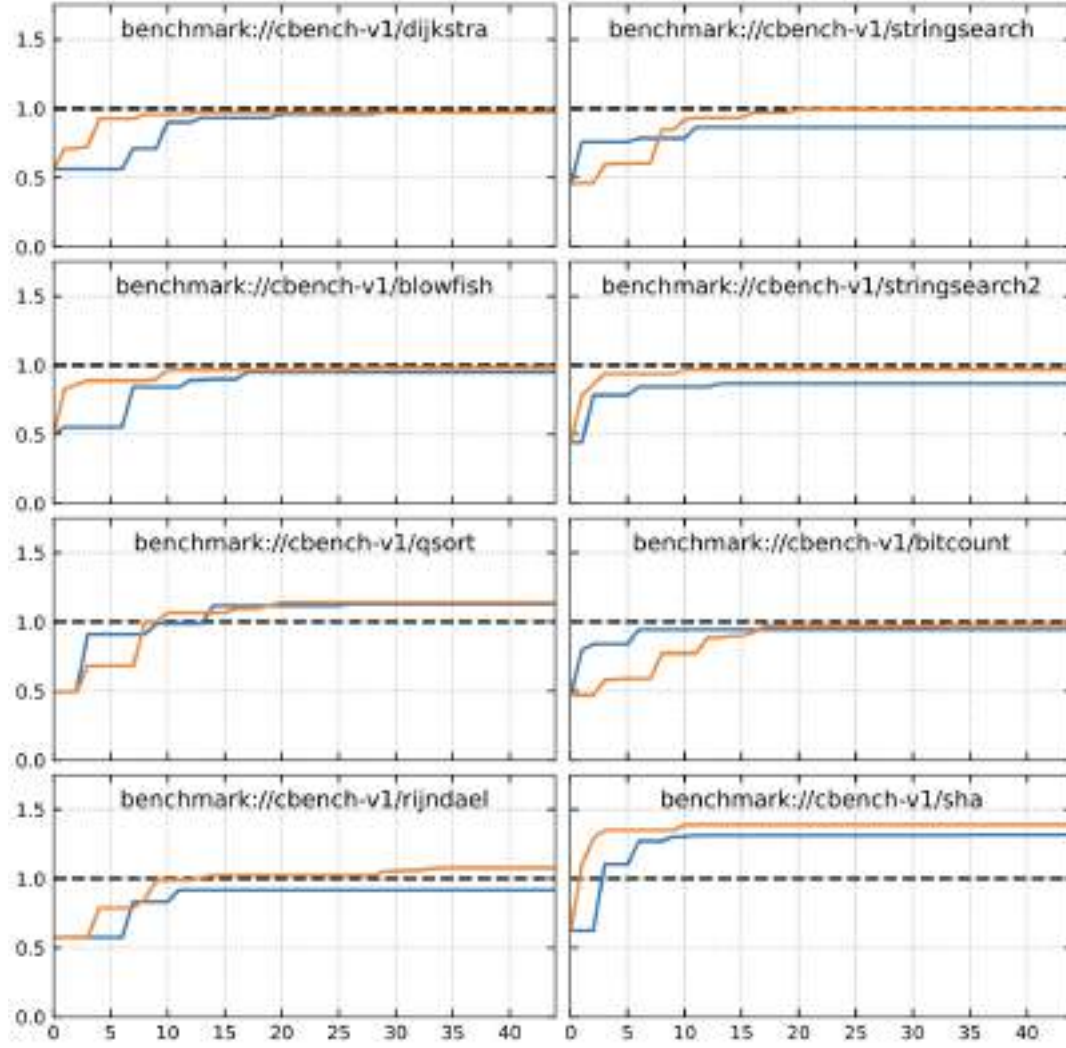


Figure 6. Program optimization example over many steps comparing the **Autophase-RL-PPO** (blue) approach with our **GEAN-NVP** (orange) approach. The dashed line represents the compiler default $-Oz$ performance and higher is better.

E. Compiler passes

Index	Flag	Index	Flag	Index	Flag
0	-add-discriminators	42	-globalsplit	84	-lower-expect
1	-adce	43	-guard-widening	85	-lower-guard-intrinsic
2	-aggressive-instcombine	44	-hotcoldsplit	86	-lowerinvoke
3	-alignment-from-assumptions	45	-ipconstprop	87	-lower-matrix-intrinsics
4	-always-inline	46	-ipsccp	88	-lowerswitch
5	-argpromotion	47	-indvars	89	-lower-widenable-condition
6	-attributor	48	-irce	90	-memcpypopt
7	-barrier	49	-infer-address-spaces	91	-mergefunc
8	-bdce	50	-inferattrs	92	-mergeicmps
9	-break-crit-edges	51	-inject-tli-mappings	93	-mldst-motion
10	-simplifcfg	52	-instsimplify	94	-sancov
11	-callsite-splitting	53	-instcombine	95	-name-anon-globals
12	-called-value-propagation	54	-instnamer	96	-nary-reassociate
13	-canonicalize-aliases	55	-jump-threading	97	-newgvn
14	-consthoist	56	-lcssa	98	-pgo-memop-opt
15	-constmerge	57	-licm	99	-partial-inliner
16	-constprop	58	-libcalls-shrinkwrap	100	-partially-inline-libcalls
17	-coro-cleanup	59	-load-store-vectorizer	101	-post-inline-ee-instrument
18	-coro-early	60	-loop-data-prefetch	102	-functionattrs
19	-coro-elide	61	-loop-deletion	103	-mem2reg
20	-coro-split	62	-loop-distribute	104	-prune-eh
21	-correlated-propagation	63	-loop-fusion	105	-reassociate
22	-cross-dso-cfi	64	-loop-guard-widening	106	-redundant-dbg-inst-elim
23	-deadargelim	65	-loop-idiom	107	-rpo-functionattrs
24	-dce	66	-loop-instsimplify	108	-rewrite-statepoints-for-gc
25	-die	67	-loop-interchange	109	-sccp
26	-dse	68	-loop-load-elim	110	-slp-vectorizer
27	-reg2mem	69	-loop-predication	111	-sroa
28	-div-rem-pairs	70	-loop-rollback	112	-scalarizer
29	-early-cse-memssa	71	-loop-rotate	113	-separate-const-offset-from-gep
30	-early-cse	72	-loop-simplifcfg	114	-simple-loop-unswitch
31	-elim-avail-extern	73	-loop-simplify	115	-sink
32	-ee-instrument	74	-loop-sink	116	-speculative-execution
33	-flattencfg	75	-loop-reduce	117	-slsr
34	-float2int	76	-loop-unroll-and-jam	118	-strip-dead-prototypes
35	-forceattrs	77	-loop-unroll	119	-strip-debug-declare
36	-inline	78	-loop-unswitch	120	-strip-nondebug
37	-insert-gcov-profiling	79	-loop-vectorize	121	-strip
38	-gvn-hoist	80	-loop-versioning-licm	122	-tailcallelim
39	-gvn	81	-loop-versioning	123	-mergereturn
40	-globaldce	82	-loweratomic		
41	-globalopt	83	-lower-constant-intrinsics		

Table 7. A list of LLVM compiler pass indices and their corresponding command line flag.

Quantum circuit optimization with deep reinforcement learning

Thomas Fösel,^{1,2,3} Murphy Yuezhen Niu,⁴ Florian Marquardt,^{1,2} and Li Li (李力)³

¹*Max Planck Institute for the Science of Light, Staudtstr. 2, 91058 Erlangen, Germany*

²*Physics Department, University of Erlangen-Nuremberg, Staudtstr. 5, 91058 Erlangen, Germany*

³*Google Research, Mountain View, CA 94043, USA*

⁴*Google Research, Venice Beach, CA 90291, USA*

A central aspect for operating future quantum computers is quantum circuit optimization, i.e., the search for efficient realizations of quantum algorithms given the device capabilities. In recent years, powerful approaches have been developed which focus on optimizing the high-level circuit structure. However, these approaches do not consider and thus cannot optimize for the hardware details of the quantum architecture, which is especially important for near-term devices. To address this point, we present an approach to quantum circuit optimization based on reinforcement learning. We demonstrate how an agent, realized by a deep convolutional neural network, can autonomously learn generic strategies to optimize arbitrary circuits on a specific architecture, where the optimization target can be chosen freely by the user. We demonstrate the feasibility of this approach by training agents on 12-qubit random circuits, where we find on average a depth reduction by 27% and a gate count reduction by 15%. We examine the extrapolation to larger circuits than used for training, and envision how this approach can be utilized for near-term quantum devices.

I. INTRODUCTION

The long-term goal of fault-tolerant large-scale quantum computing promises disruptive progress in multiple important areas of science and technology [1], such as decryption [2], database search [3], quantum simulation [4], and for optimization problems [5]. However, the considerable overhead required for fault-tolerant operations implies a strong incentive in the near term to explore applications that can already work on “noisy intermediate-scale quantum” (NISQ [6]) devices. With the help of suitable algorithms [7, 8], such devices will be able to produce results that outperform classical computers, despite a moderately high rate of errors (“noise resilience”). Recently, a quantum advantage has been demonstrated experimentally for some first benchmark problems on NISQ devices [9, 10].

Research on NISQ applications aims to deliver useful results while keeping the overall resources (number of qubits, computation time) required at a manageable level. Quantum circuit optimization (QCO) constitutes an essential step in addressing this challenge. Starting from a given quantum circuit, i.e., a sequence of quantum operations (gates) acting on a set of qubits, the goal of QCO is to find a logically equivalent circuit that is likely to reduce the probability of errors, by having an overall shorter runtime and using fewer gates (possibly with emphasis on gates of a certain type). This can be achieved mainly by compression (via combination or cancellation) and efficient parallelization of the quantum gates.

There has been important progress in QCO over the years, giving rise to a set of approaches like the family of T-par [11, 12], TOpt [13] and T-Optimizer [14], and QCO based on ZX calculus [15]. These approaches are inspired by the requirements of large-scale fault-tolerant quantum computation and therefore work on scenarios which are quite removed from what will be encountered in NISQ hardware devices. In fact, they have been formulated in a

hardware-independent way and rather operate on a global level.

What is especially needed for QCO on NISQ devices is, however, an approach that is able to take into account the optimization opportunities afforded by a more detailed consideration of the actual hardware (connectivity, gate set, etc.) [16]. Finding the right strategy for this task is a challenging problem. The benefits of any change in the quantum circuit are very context-dependent, especially if the circuit structure is irregular, and a greedy strategy (to always aim for the immediate improvement) is often not ideal.

Optimizing circuits by hand is very time-consuming, requires revising the strategy for every new platform, and can be prone to human errors. Hard-coded algorithmic strategies require considerable human implementation effort specific to the given architecture. In situations with complex decision-making, hard-coded strategies are also prone to unconsidered special cases impeding their ability to generate high-quality results. Optimization techniques like genetic algorithms or simulated annealing improve on these aspects, but lack computational efficiency because they do not select actions in a deliberate way, and they are unable to transfer the experience from earlier solved problem instances.

Optimizing strategies for complex decision-making problems is the goal of reinforcement learning (RL) [17], an important subfield of machine learning that goes beyond straightforward supervised learning. Based on repeated attempts, strategies which have been successful in the past are reinforced over time (exploitation), while alternatives are still occasionally tried out in a trial-and-error fashion (exploration). Due to the generality of this approach, RL is being employed in a broad and growing spectrum of applications, such as robotics [18], video games [19, 20], board games like chess and Go [21, 22], chemistry [23–25], neural architecture search [26], and chip placement [27]. Because RL discovers strategies autonomously and

therefore does not rely on a teacher, it often achieves super-human performance in situations where such a comparison can be made.

In recent years, RL has also been proposed for several problems in the field of quantum computing. Examples include quantum phase estimation [28], the design of quantum experiments [29], quantum control [30–33], quantum error correction [34–37] (alongside other machine learning approaches [38–41]), and quantum metrology [42, 43].

In this work, we introduce deep reinforcement learning for quantum circuit optimization. Our approach enables the computer to autonomously discover strategies for reducing the depth and gate count of quantum circuits, for arbitrary gate sets and connectivity. It allows to choose the optimization target at will and permits extrapolation of the discovered strategy to larger circuits. Due to its flexibility and generality, the RL approach proposed here has the potential to become a valuable component of the toolbox needed to unlock the power of NISQ devices in the near future.

II. TECHNIQUE

A. Quantum circuit optimization as reinforcement learning problem

The goal of RL is to discover strategies for decision-making problems. This is described by an “agent” interacting with the rest of the world, the “environment”. In several rounds, the agent receives information from the environment and, in response to this observation, chooses an action which alters the state of the environment. The agent is supposed to adapt its strategy so as to maximize a success measure, the “reward”. More information is provided in Sec. II C.

In the spirit of previous RL applications to quantum problems [28, 30–34, 42, 43], the obvious approach seems to let the agent build a circuit gate by gate to implement a certain target operation. However, this would come with two central problems here. First, it is extremely unlikely to find a suitable circuit by chance, so an untrained agent would in practice probably never see a positive reward signal. This problem is exacerbated by the fact that the gate set is typically not discrete, but gates can depend on continuous parameters. Second, in the particularly interesting quantum supremacy regime where the circuit cannot be simulated on a classical computer, there is the problem that even if one had found a valid circuit, verifying its correctness would be very hard and computationally expensive. Note that some tools like ZX calculus promise to arrive at a statement in polynomial time, but with the two possible results being positive or inconclusive whether two circuits are equivalent.

Therefore, we follow a different strategy that appears more promising: In QCO, it is common to start from a complete and correct, but typically inefficient circuit, and to progressively optimize it by applying a sequence of

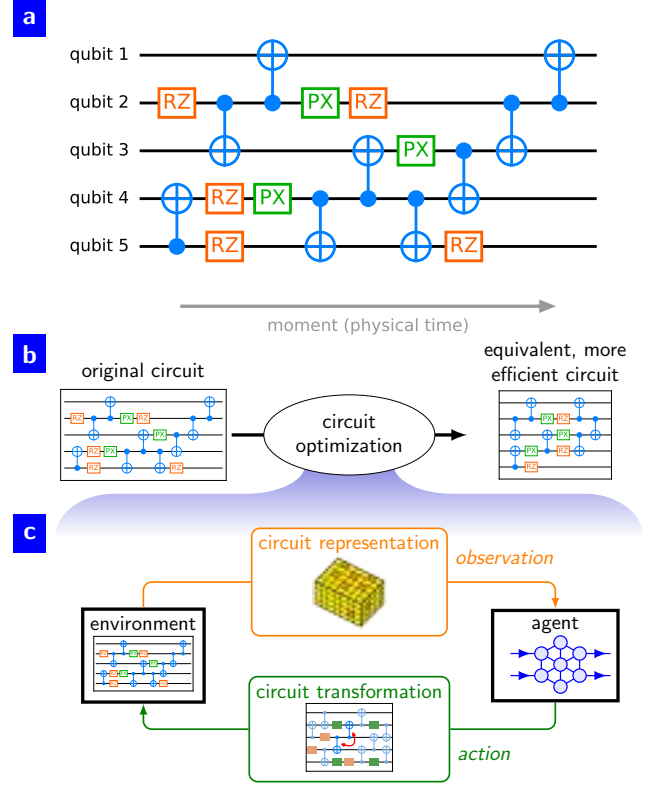


Figure 1. Overview. a) Diagram representation for quantum circuits. Each qubit is indicated with one line. The colored symbols represent operations (gates) on these qubits, with time increasing to the right. b) Quantum circuit optimization. For a given circuit, we aim to find a logically equivalent, but more efficient representation. c) Our reinforcement learning approach to quantum circuit optimization. Based on a diagram-like representation of the circuit, the agent, realized by a neural network, can choose between several circuit transformations to generate another, logically equivalent circuit; this process is repeated multiple times.

circuit transformations. However, it can be a formidable challenge to appropriately choose these transformations, and we make this decision the task of our agent. From the RL perspective, this means that *the states are the circuits* and *the actions are the circuit transformations*. By design, this approach immediately solves the challenge to finish with a correct, i.e., logically equivalent, circuit: we can preserve this property for the full process by allowing in each step only equivalence transformations. In addition, our approach is also scalable, i.e., it allows us to operate in the quantum supremacy regime: it is sufficient to verify equivalence for the few operations directly involved in an elementary circuit transformation, which is relatively cheap as long as all operations act only on a limited number of qubits.

Our general goal is to use RL to train a multi-purpose agent which afterwards will be able to optimize a wide class of circuits based on one given hardware architecture, without going through the RL procedure again in each

instance. Sometimes, though, it can be helpful to fall back to the more restricted approach, where an agent is trained to optimize one specific circuit only.

For our approach, we need to carefully distinguish between different notions of time: the physical time which refers to the execution order of the operations in the circuit, and the agent time in which the actions, here to modify the circuit, are applied. In contrast to comparable previous works on RL for quantum problems [28, 30–34, 42, 43], these two time notions are here independent. From the agent’s perspective, physical time is treated as an extra spatial dimension of the observation (cmp. Sec. II E). In addition, there is the training time along which the weights and thus the behavior of the agent changes.

B. Circuit transformations

Our goal is to optimize a circuit by applying a suitable sequence of circuit transformations. These transformations are based on local transformation rules. Such a rule could be to remove two subsequent operations that cancel each other; or, to reverse the order of two operations that commute (or anti-commute, as a global phase does not matter). Note that one given rule might be applicable to multiple locations in the circuit, yielding an independent transformation for each of them.

In our approach, we distinguish two kinds of transformation rules, which we refer to as “hard” and “soft” rules, and likewise call the transformations they induce hard and soft. Hard rules are those which are always advantageous, like the example of cancelling operations (if the goal is to shrink the circuit, cmp. Sec. II D). Soft rules, on the other hand, are those where the benefit of the transformation depends on the context. For example, for two commuting operations there might be a preferable order contingent on the surrounding operations, and dependent on the current configuration, it can make sense to exchange them or not. Also, it is not uncommon for soft transformations to be beneficial only in combination with several other hard or soft transformations.

In our approach, the decision of the agent is restricted to which soft transformation to apply next. This is always followed by a “pruning” step where automatically all permissible hard transformations are applied. In this sense, the decision of the agent can also be interpreted as not only selecting the soft, but implicitly also all ensuing hard transformations.

Currently, we determine and implement the small set of acceptable transformation rules by hand. The effort for this is manageable because only a moderate number of real-world quantum architectures exists, each with only a limited number of rules according to its specific gate set. As soon as we have arrived at a proper set of rules for one architecture, we can deal with any circuit on it.

C. Learning algorithm

The full RL training process is divided into several episodes. In our case, an episode corresponds to one optimization run for a specific circuit (which might change between episodes). Each episode itself is a sequence of T subsequent steps, in which the agent changes the state s_t with its action a_t into the new state s_{t+1} . For this, the agent receives a reward r_t ; in our case, the reward indicates the improvement of the circuit compared to the previous one. The objective in RL is to maximize the cumulative reward $\sum_{t=0}^{T-1} r_t$. This makes the return

$$R_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}, \quad (1)$$

where γ is a discount rate, an important quantity: For $\gamma = 1$, maximizing $\mathbb{E}[\sum_{t=0}^{T-1} r_t]$ would be equivalent to greedily maximizing $\mathbb{E}[R_t]$ in each time step; in practice, however, a value $\gamma < 1$ often makes learning more stable and efficient.

The agent is typically implemented by a neural network, or in general by an arbitrary parameterized ansatz. The interpretation of its output is defined by the RL algorithm, which also specifies how to train the agent’s parameters θ . In this work, we use proximal policy optimization (PPO [44]), from the family of advantage actor-critic (AAC) methods. An AAC agent computes two quantities: the policy $\pi_\theta(a|s)$, according to which the actions are probabilistically chosen; and an estimator for the state value $V_\theta(s)$, that is the expectation value for the return R when starting from state s and following policy π_θ . In our case, the meaning of the state value $V_*(s)$ for the best policy π_* is the further potential for optimizations of the circuit representing state s .

AAC methods cooptimize policy π_θ and state value V_θ during training [17]: Actions a_t are reinforced according to their advantage $r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)$, whose expectation value tells how much the average return $\mathbb{E}_{\pi_\theta}[R_t|s_t, a_t] = \mathbb{E}_{\pi_\theta}[r_t + \gamma V_\theta(s_{t+1})|s_t, a_t]$ for the chosen action a_t exceeds the average over all actions, that is $V_\theta(s_t)$. Simultaneously, the state-value estimator $V_\theta(s)$ is refined (and adjusted to changes in the policy π_θ) so as to satisfy the Bellman equation $V_\theta(s) = \mathbb{E}_{\pi_\theta}[r + \gamma V_\theta(s')|s]$. In the simplest case, this can be achieved by updating θ according to the learning gradient

$$g = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{T-1} (r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)) \frac{\partial}{\partial \theta} \left[\ln \pi_\theta(a_t|s_t) - \lambda V_\theta(s_t) \right] \right] \quad (2)$$

where the coefficient λ weights the cost term for the state value. PPO extends this basic AAC scheme in several ways for more powerful and efficient learning.

D. Reward

The central motive behind QCO is often to decrease the probability for potential errors to occur. More generally speaking, we always want to improve some arbitrary desirable property of a circuit, which is defined by the user. The reward, which decides towards which behavior RL steers the agent, needs to be constructed according to this goal. Let $q(s)$ be a function which quantifies the desirable property for a given circuit s , and is to be minimized. A successful agent is expected to progressively improve the circuit over the course of an episode, so the goal is to minimize $q(s_T)$ at the final step T . We drive the agent towards this behavior with an immediate reward of the form

$$r_t = -(q(s_{t+1}) - q(s_t)). \quad (3)$$

This provides better time resolution than a final reward, such as $r_t \sim -q(s_T) \cdot \delta_{t,T-1}$ or $r_t \sim -(q(s_T) - q(s_0)) \cdot \delta_{t,T-1}$, and therefore promises more efficient learning.

Now, we will derive a simple form for $q(s)$ which estimates the error probability for circuit s , or – to be precise – is a monotonously increasing function thereof. The quite simplistic, but still reasonable model behind it is to assume that each qubit loses information with a certain rate Γ while being idle, and that every gate induces additional errors. Let u_k indicate how much the k -th gate underperforms compared to decay with rate Γ during idle times. Then, the probability to execute a circuit error-free is given by

$$P_{\text{success}} = e^{-m\Gamma T} \cdot \prod_{k=1}^n u_k \quad (4)$$

where m is the number of qubits, n is the number of gates in the circuit, and T denotes the total runtime. Our goal to maximize P_{success} is equivalent to minimizing $q = -\ln P_{\text{success}} = m\Gamma T - \sum_{k=1}^n \ln u_k$.

During application of our RL approach, we will have to make concrete choices for the parameters, but we want to remain as general as possible. The least special assumption we can make is to attribute unit cost to all operations, i.e., an equal execution time $\alpha/(m\Gamma)$ and an equal underperformance factor $u_k = e^{-\beta}$. In this case, q can be expressed via the circuit depth d and gate count n :

$$q(s) = \alpha d(s) + \beta n(s). \quad (5)$$

Explicitly, we will set $\alpha = 1$ and $\beta = 0.2$ in Sec. III. However, we emphasize that $q(s)$ in Eq. (5) can be replaced with any function of a circuit, and because we employ model-free RL, nothing in the learning algorithm would need to be changed.

E. Representation of observations and actions

With perfect information about the environment state, i.e., the circuit, the agent can make the best decisions.

Therefore, we provide a complete description of the circuit as observation to the agent. In quantum computing, circuit diagrams are widely used to visualize circuits for humans. Our format is inspired by them, but adjusted to the conditions of a neural network: We reserve one input neuron for each possible operation in the circuit, and activate neurons depending on whether the corresponding operation is actually present. These input neurons are arranged on a 3D grid, whose axes correspond to qubit index, moment (time step) and gate class. For gate types with a discrete number of instances, each of them forms its own gate class. For gate types which depend on (continuous) control parameters, we distinguish separate classes for special parameter values (e.g., for Z rotations by $\pi/2$, π and $3\pi/2$), and group all remaining instances of this gate type into one class. This allows to represent quantum circuits for arbitrary choices of their continuous parameters, but to distinguish gates for which additional transformation rules apply. To characterize multi-qubit gates, in principle all qubit indices would have to be indicated. In a 1D chain of qubits where gates are allowed only between nearest neighbors, as in our examples in Sec. III, we can characterize two-qubit gates simply by their lowest-index qubit, and specify by the gate class which roles the qubits play (e.g., control vs. target qubit for a CNOT gate).

A particular challenge in this project was to design the output format to represent the policy of the agent. The basic problem is that the actions, i.e., the transformations of the circuit, are defined by the underlying transformation rule and all the gates affected by this transformation (which are characterized by their location in the circuit). Due to combinatorial explosion, the usual approach for discrete action spaces, to reserve one output neuron for every possible action, is not feasible here. One alternative would be to sort the transformations in a certain way and assign them consecutively to a queue of output neurons. However, this approach is not very promising either since the meaning for each output neuron would change for every circuit, potentially depending on aspects of very distant locations of the circuit, and therefore be very hard for the network to understand. Another alternative would be Q-learning where the agent, instead of computing the action value $Q(s, a)$ for state s and action a , computes $Q(s, s')$ for input state s and output state s' [24]; while conceptually this should work, this RL variant is less widely used and therefore not as well explored, and is restricted to Q-learning.

Instead, inspired by the representation of chess moves in [22], we follow a different approach where we associate transformations with certain locations in the circuit, in a way that uniquely characterizes which gates they affect. In our case, we use the following mapping: for a transformation involving a single gate, we indicate the moment and the smallest qubit index of the gate; for a transformation involving two gates, we indicate the smallest qubit index shared between both gates, and the moment of the first one. By providing multiple neurons per location to

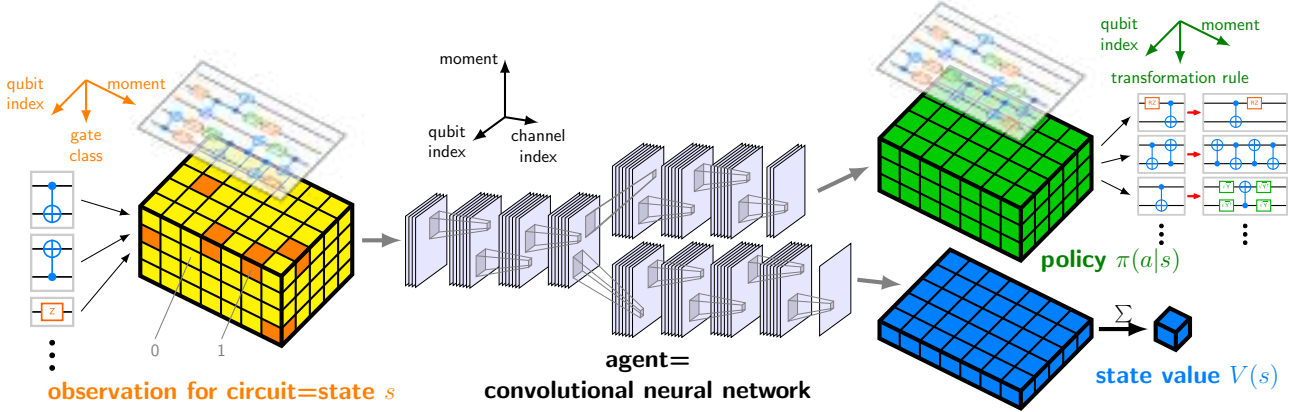


Figure 2. Deep convolutional network architecture of our RL agent. As observation, the agent receives a complete description of the state s , i.e., the quantum circuit. The input neurons are arranged on a 3D grid, whose axes correspond to qubit index, moment and gate class. This information is processed through a stack of multiple convolutional layers, where qubit index and moment are treated as spatial dimensions and the gate classes as input color channels. For the output, the agent computes two quantities: (i) The policy $\pi(s|a)$, according to which the actions a in state s are probabilistically chosen. Every action, i.e., circuit transformation, is mapped uniquely to one policy output neuron; the remaining neurons are disabled with an action mask. And (ii), the state value $V(s)$, which helps to update the policy $\pi(s|a)$ more efficiently during training. For us, $V(s)$ has the meaning of the optimization potential for the circuit.

indicate the underlying rule, we can achieve an injective mapping from transformations to output neurons (for the policy). Therefore, also these neurons are arranged on a 3D grid, whose axes correspond to qubit index, moment and transformation rule. There can be neurons to which no transformation is associated; we disable them with an action mask, whose value changes with the input circuit.

Besides solving the problem to keep the total number of output neurons at a moderate level, another central advantage of this format is that we can exclusively use convolutional layers [45] to process the observation into the policy, treating qubit and moment as spatial dimensions, and the remaining grid axis (gate class and transformation rule, respectively) as input “color” channels. Also to compute the state value, we use convolutional layers (with one output channel), and eventually average over the spatial dimensions. Fig. 2 illustrates the architecture of our deep convolutional network. The weight sharing in the convolutional layers contributes to efficient and robust learning, and a *fully* convolutional architecture will allow us to directly extrapolate to different circuit sizes (see Sec. III B).

F. RL problem classification

The RL problem in this article can be classified as a Markov decision process (MDP) with perfect information (since the circuit representation, which is given to the agent as its input, completely describes the state of the environment). The set of all circuits comprises the state space. The set of possible circuit transformations repre-

sents the action space, which is therefore discrete, and its size depends on the state, i.e., the circuit. The environment is deterministic: a fixed transformation (action) on a fixed circuit (state) always leads to the same outcome. Because the goal is to optimize a property which can be evaluated for any single given circuit, we can construct an immediate reward scheme (as opposed to situations where the reward is given only at the end of an episode).

III. RESULTS

In exploring the power of our RL approach, we need to select both a specific architecture (available gate set, processor layout, qubit connectivity) as well as the family of quantum circuits on whose optimization we want to focus.

Gate set For our simulations, we consider the gate set consisting of Z-Rotation, Phased-X and Controlled-Not (CNOT) gates. Together, they form a universal gate set. Whereas Z-Rotation and Phased-X are actually gate classes parameterized by 1 and 2 continuous variables, respectively, the CNOT is one fixed gate.

For our purposes, Z-Rotation, Phased-X and CNOT is a decent gate set because on the one hand, it induces a rich set of relatively simple transformation rules. On the other hand, they are quite similar to current real-world quantum hardware, such as Google’s Bristlecone (Z-Rotation, Phased-X and Controlled-Z gate [46]) and Sycamore (Z-Rotation, Phased-X and fermionic simulation gate [9, 47, 48]) processor: The CNOT gate differs from the Controlled-Z gate only by local gates (e.g., Hadamard or

Sqrt-Y) on the target qubit, and the fermionic simulation gate is a generalization of the Controlled-Z gate.

A. Training on random circuits

Dataset While a quite large number of quantum circuits for various applications and architectures can be found in publications of the recent years, to the knowledge of the authors a joint dataset of such circuits does not exist, and collecting them by hand would have been very tedious. At the stage of developing the technique and verifying its feasibility, we therefore decided to employ randomly generated circuits. Although the majority of them is likely not to be useful for actual quantum applications, they have the big advantage that they can be easily generated in large numbers. Due to the irregular structure of such circuits, the benefits of transformations are strongly context-dependent, and therefore random circuits constitute a case where formulating a generally applicable strategy would be enormously complex.

We now briefly describe our method to generate these random circuits. We consider 12 qubits arranged in a 1D line; two-qubit gates, here CNOTs, are only allowed between nearest neighbors. First, we build a sequence of 150 gates whose type (either CNOT or local) and the qubits it acts on (constrained by the connectivity) are chosen randomly. Because local gates might be decomposed into a Z-Rotation and a Phased-X gate, we obtain a slightly higher value for the mean number of hardware gates, $\langle n_1 \rangle \approx 159$, and for the mean depth we find $\langle d_1 \rangle \approx 48$. For many of these circuits, some hard rules (cmp. Sec. II B) can immediately be applied, i.e., there are some “trivial” optimizations; for example, two subsequent CNOTs on the same qubits directly cancel. To apply these transformations, which we refer to as pruning, reduces the mean depth to $\langle d_2 \rangle = 37.15 \pm 0.01$ and the mean gate count to $\langle n_2 \rangle = 115.27 \pm 0.03$. From here, we continue by applying random transformations (based on soft rules, each time followed by a pruning step), which turns out to make the circuit much larger and hence very inefficient. After 500 of these transformations, we find a mean depth of $\langle d_3 \rangle \approx 248$ and a mean gate count of $\langle n_3 \rangle \approx 508$.

We use these randomly expanded circuits as the training tasks for the RL agent, where each of them defines the starting point for one episode, i.e., for one optimization run. The random expansion step enhances the ability of the fully trained agent to generalize, by demanding it to deal well with a broad spectrum of both inefficient and efficient circuits, which will be encountered during every successful optimization run.

Simulated annealing As a benchmark for the results by the RL agent, we consider simulated annealing as an alternative approach, using an adapted Metropolis rule and an exponential cooling schedule. We apply simulated annealing to the randomly expanded circuits, i.e., on the same dataset as later for the agent. The results depend

strongly on the number of steps: Within 10000 steps, simulated annealing is able to merely reach the level after pruning (on average). Within 200000 steps, out of which roughly 15...20 % are actually accepted, a mean depth $\langle d \rangle = 27.35 \pm 0.08$ and gate count $\langle n \rangle = 105.15 \pm 0.32$ can be achieved.

RL Now, we apply the RL approach as described in Sec. II on our dataset of expanded random circuits. The hyperparameters have been optimized in a systematic search. One full learning process takes 6 to 7 days, on a node with 32 CPU cores.

We find that the learning process can be divided roughly into two phases (cmp. Fig. 3c): In the first ca. 50 epochs, the agent rapidly improves in minimizing the mean depth and gate count, to a level of $\langle d \rangle \approx 29$ and $\langle n \rangle \approx 100$ which is already considerably better than after pruning. In the remaining course of the training, the agent can slowly, but steadily improve towards $\langle d \rangle = 27.20 \pm 0.07$ and $\langle n \rangle \approx 97.86 \pm 0.33$ at around epoch 1000. Hence, the agent can clearly beat the results from simulated annealing, especially regarding the gate count. Because we use a new circuit in each training episode, no difference is to be expected between the performance on training and validation samples.

In Fig. 3d, we examine closer how a trained agent proceeds in optimizing a circuit. For this purpose, we choose 3200 circuits and plot how d and n evolve with every transformation. As Fig. 3d shows, the agent can rapidly improve the circuits during the first 150 to 200 steps. Afterwards, the curve is mostly flat, except for two effects: First, there are occasional excursions into lower-quality states, but the agent usually returns quickly to the previous level. Second, the agent sometimes manages to further optimize the circuit after some time, partially via passing some intermediate lower-quality states, i.e., to cross a reward barrier. Note that we did not give the agent the ability to terminate an episode.

Compared to the pruning level, we have seen that the agent achieves much better values for $\langle d \rangle$ and $\langle n \rangle$ on average. How effective the agent has actually become can be seen by comparing d and n for individual circuits between the agent and after pruning. In Fig. 3e, we plot this comparison for epochs 900 – 1000, i.e., for 3200 circuits in total. The agent achieves a reduction of both d and n for the vast majority of circuits, partially up to even more than 50 %. Only for 12 out of 3200 circuits, i.e., in around 0.4 % of the cases, the agent fails to achieve a better $q(s)$ than after pruning.

B. Extrapolation to larger random circuits

In Sec. III A, the agent has been trained and evaluated on 12-qubit random circuits. However, due to its fully convolutional architecture, this very same agent can – at least in principle – be directly used to optimize also circuits of a different size, in particular much larger ones. Now, we will test how well the agent can actually generalize its

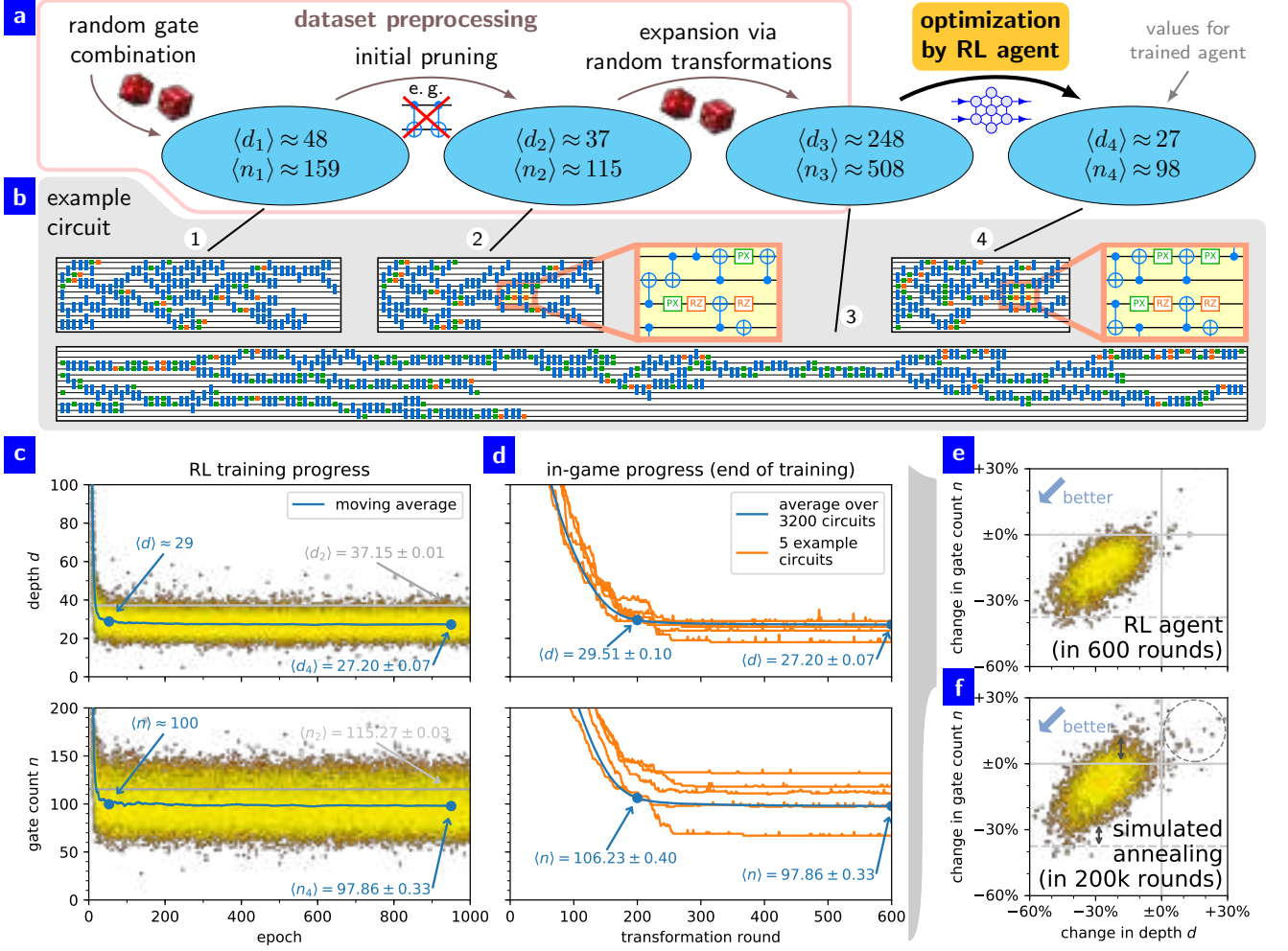


Figure 3. Training on random circuits. a) Circuit processing pipeline (see Sec. III A for details). After choosing an initial circuit by randomly combining gates, a pruning step follows where all “trivial” optimizations are applied. Afterwards, 500 random transformations are performed on this circuit, which turns out to significantly increase their depth d and gate count n . These expanded circuits are then used as the starting point of the episodes to train and evaluate the RL agent. b) Diagrams illustrating the evolution of one example circuit through this pipeline. c) Learning progress during training, demonstrating how the agent improves in reducing both the depth d (top) and the gate count n (bottom) of the circuits. The point cloud indicates, for all episodes during training, the corresponding quantity in the final time step. The blue curve shows the moving average over the latest 10% of epochs. For comparison, the gray line indicates the corresponding averages after pruning; already early in the training, the agent falls below this level for both quantities. d) In-game progress at the end of the learning process, showing for 5 episodes during the last epoch (orange) how the agent progressively optimizes (d, n) during an episode. The blue curve indicates the average over all episodes in the last 100 epochs of training. e) Relative improvement achieved by the RL agent, in reference to the corresponding circuit size after pruning. Each point corresponds to one episode during the last 100 epochs. f) Comparison with circuit optimization by simulated annealing (see Sec. III A for details). The graphical depiction and the considered circuits are equivalent to (e), which makes them directly comparable.

knowledge in this situation.

For this purpose, we reuse the scheme to generate random circuits as described in Sec. III A, except for changing two parameters: we increase the number of qubits from 12 to 50, and the number of initial gates from 150 to 2500. We find $\langle d \rangle = 199.25 \pm 0.08$ and $\langle n \rangle = 2655.3 \pm 1.2$ before pruning, and $\langle d \rangle = 156.67 \pm 0.07$ and $\langle n \rangle = 1940.2 \pm 1.6$ after pruning. Because we will not

use the circuits here to train the agent, we can skip the step to expand them by random transformations, whose purpose it was to feed to the agent also very inefficient circuits during training. Instead, the optimization by the agent starts here directly from the pruned circuits. As shown in Fig. 4b, the agent achieves to reduce $\langle d \rangle$ to 110.84 ± 0.07 and $\langle n \rangle$ to 1616.3 ± 2.0 within 2500 transformations. Remarkably, the reduction ratio in these

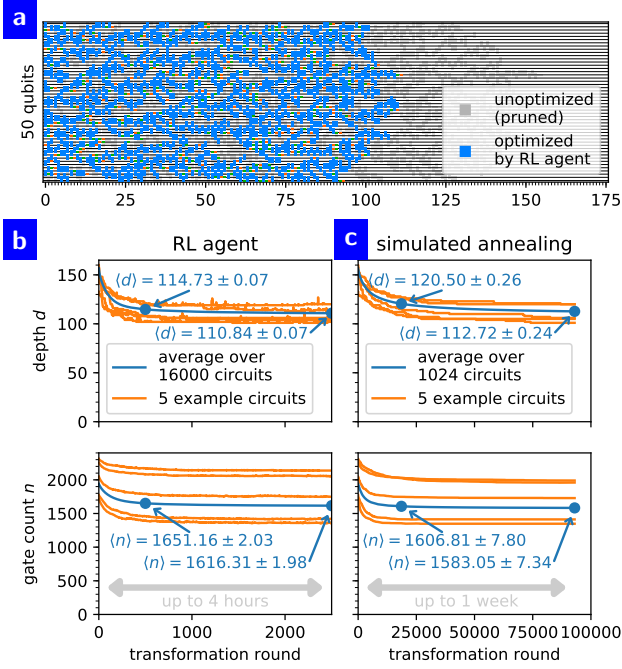


Figure 4. Extrapolation to 50-qubit random circuits. The agent has been trained on 12-qubit circuits (cmp. Fig. 3), no further learning updates are performed here. (a) shows the comparison between an unoptimized example circuit (after pruning) and the result of the optimization by the RL agent. (b) shows the progress of the agent in reducing depth d and gate count n over the course of 2500 transformations. (c) shows the corresponding curves for simulated annealing, which requires almost 100000 transformations to achieve a comparable degree of optimization (the computation was terminated after 1 week, at transformation 93000).

two quantities is comparable to the smaller circuits it has been trained on (cmp. Fig. 3).

Simulated annealing arrives at similar values, $\langle d \rangle = 112.72 \pm 0.24$ and $\langle n \rangle = 1583.0 \pm 7.3$, within 93000 transformations. These are much fewer transformations than required to optimize the smaller random circuits in Sec. III A, probably because here the random expansion step has been skipped. Nevertheless, 93000 transformations for each larger random circuits here have already taken one week (our termination criterion), which is comparable to the time needed to train an RL agent. Afterwards, this agent can optimize arbitrary circuits, in a relatively short time (3...5 h in this case).

Our results show that an agent can actually extrapolate its knowledge to larger circuits. More generally, they demonstrate that our approach, both with RL and simulated annealing, works deep in the quantum supremacy regime. Furthermore, this also highlights a situation where optimizing even a single circuit with simulated annealing needs already a runtime comparable to the full training of an RL agent and subsequently optimizing the particular circuit.

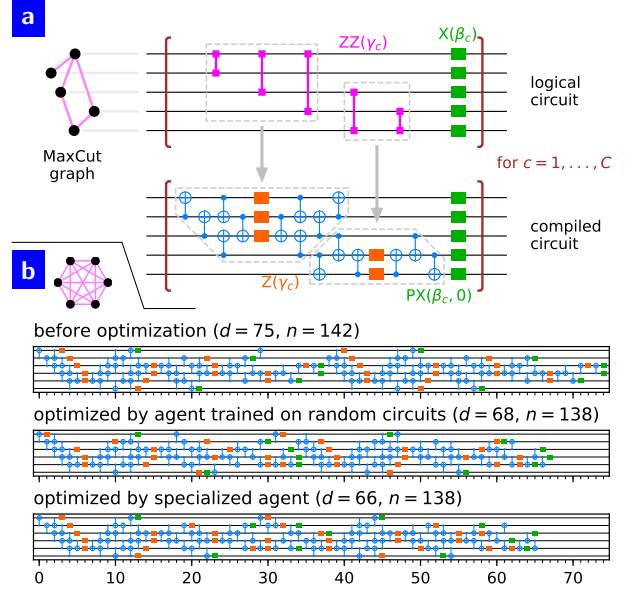


Figure 5. Optimization of QAOA-MaxCut circuits. (a) indicates how to translate the MaxCut problem for a graph into a quantum circuit following QAOA, and how to efficiently compile this logical circuit into our gate set. We display one of M cycles which form the full circuit, each with a different set of parameters (γ_c, β_c) whose values are refined during the QAOA algorithm. (b) shows the compiled circuit for $C = 2$ cycles and an all-to-all-connected graph with 6 nodes, which has depth $d = 75$ and gate count $n = 142$ (top). Using a generic agent trained on random circuits as in Fig. 3, we find (by postselection) improved circuits with $d = 68$ and $n = 138$ (middle). A specialized agent trained on this particular circuit can further optimize it to $d = 66$ and $n = 138$ (bottom).

C. QAOA-MaxCut circuit

As an example for a real-world quantum algorithm, we now consider the MaxCut problem. The goal is to arrange the nodes of an undirected, non-weighted graph into two groups such that the amount of cut edges is maximized. Finding the exact solution is an NP-hard problem. Following the quantum approximate optimization algorithm (QAOA [7]), approximate solutions can be found with the help of a quantum circuit consisting of repeated cycles of ZZ gates and local X rotations with variable angles [48] (cmp. Fig. 5a). We consider the same gate set as in the examples above, such that we can reuse the previously trained agent. Also, this covers the realistic situation where the native gates of the quantum algorithm do not necessarily match the native gates of the hardware. Fig. 5a shows an efficient compilation of this circuit into our gate set, where ZZ gates need to be decomposed into CNOTs and local Z rotations (local X rotations are a special case of Phased-X gates). Note that the variable angles of the gates do not affect the optimization strategy, as long as we assume these angles to be generic (i.e., not set to special values which would

allow additional optimizations).

Every graph corresponds to a different circuit. We restrict our analysis here to all-to-all connected graphs. While any such graph has trivial MaxCut solutions (nodes evenly distributed), the corresponding circuit is still useful for us to consider because the circuit for any other graph layout can be derived from it by removing some of the ZZ gates; therefore, it also provides a worst-case estimate. In Fig. 5b, we specifically consider a graph with 6 nodes and 2 cycles in the QAOA circuit.

First, we try to optimize this QAOA circuit with a generic agent trained on 12-qubit *random* circuits, as in Fig. 3. We find that on average, the agent manages to slightly reduce the gate count, although the depth is slightly increased. Even though this agent is, thus, not able to reliably optimize the circuit, it is still able to sometimes achieve an improvement: By postselecting over all time steps in multiple optimization runs, we find that the best circuit reduces the depth from 75 to 68 and, simultaneously, the gate count from 142 to 138 (cmp. Fig. 5b); note that different runs deviate because the policy is to some degree stochastic, and that the circuit quality fluctuates also with time. We observe that the optimizations happen at the interface between subsequent cycles, and at the end of the last cycle.

If we, however, train an agent via RL directly on this specific circuit, it does not only learn to reliably improve the circuit, but it also finds two further optimizations, each reducing the depth by 1. These optimizations are very hard to discover, as they require a lookahead by 7 and 9 transformations, respectively, where the search breadth is around 350. The resulting circuit, as shown in Fig. 5b, has a depth of 66 and a gate count of 138. To our knowledge, this is the most efficient realization on the gate set we provided (when including additional gates, one can arrive at an even shorter circuit [48]).

For these QAOA-MaxCut circuits, we start from an already quite efficient compilation, so the optimization potential is not very large here. Nevertheless, the agent is still able to find optimizations, including some which require complex transformation sequences and would thus be very hard to find by hand. On the general level, this example demonstrates that our approach can also be used to optimize real-world circuits, where the optimization potential might very well be larger than here. However, we have got reliable improvement and the best optimization quality only for agents trained on this circuit, which emphasizes the importance of a proper training dataset.

During early simulations, we made a noteworthy observation: Sometimes, the agent trained on these MaxCut circuits achieved optimizations far below a depth of 66 and a gate count of 138. It turned out that this happened only if the rotation angle for one cycle of ZZ gates, which was chosen at random, was relatively close to a multiple of π . In this case, the agent found a creative way to exploit that the closeness criterion for one of the transformation rules was too loose, which it used to partially or completely remove this cycle from the circuit. Although in

this particular case these transformations were not valid, the agent’s behavior shows that it would readily exploit the simplification potential in other complex circuits.

D. Discussion

Dataset We anticipate our RL agents to be considerably more powerful and reliable on real-world quantum circuits when trained on a dataset with similar properties, which is also indicated by our results in Sec. III C. In contrast to currently existing collections of quantum circuits, like [49], we need for our purposes first the circuits being compiled for specific hardware architectures, and second, also a significantly larger dataset size to fully train agents on them (based on Sec. III A, this can require in the order of 10000 to 100000 episodes).

We think of our future dataset to be generated as follows. We consider a set of various quantum algorithms, like QAOA [7], variational quantum eigensolver [8], Shor’s algorithm [2] (for small integers), and many more [50]. Many of these algorithms imply a large number of different circuits; for example, QAOA can be applied to multiple combinatorial optimization problems, and for each of them, the circuit changes with the concrete problem instance (e.g., graph layout) and the number of cycles. These circuits are described in a high-level, hardware-independent way. For every specific architecture (gate set, layout, connectivity), we would then generate an own dataset by compiling the circuits for them, which can be done using existing quantum libraries like Cirq [51].

Eventually, each of our agents will be trained on one of these architecture-specific datasets. If the dataset size is not yet sufficient for this, we can use data augmentation techniques (like mirroring the circuit in space and/or time), and to some degree also repeat circuits during training.

RL vs. simulated annealing In Sec. III A, simulated annealing achieved $98.48 \pm 1.06\%$ of the reduction in gate count n by the RL agent, but only $58.06 \pm 2.15\%$ in the depth d ; Sec. III B is more ambiguous, as simulated annealing achieved $109.67 \pm 2.44\%$ of the reduction in n by the RL agent, but only $95.88 \pm 0.57\%$ in d (all numbers in reference to the pruning level). Therefore, in these scenarios, RL tends to give overall slightly better results than simulated annealing.

Another aspect to consider in this comparison is runtime. After having been trained once on a certain class of circuits, an agent is considerably faster than simulated annealing in optimizing arbitrary circuits of this class (ca. 2 min vs. 1...3 d in Sec. III A, 3...5 h vs. 7 d in Sec. III B). Of course, for a fair comparison of the computational effort, it is necessary to also take into account the time needed for training the agent in the first place (up to 1 week), and eventually also the time to optimize the hyperparameters for both approaches. Therefore, if we take all those aspects into account, the question which method is less computationally expensive cannot be an-

swered in general, but depends on the class of circuits and previously solved problem instances – the only safe statement is that for a sufficient number of circuits to be optimized, the RL approach will be computationally cheaper.

Symbolic parameters In our current implementation, gates are represented numerically, which requires to consider fixed choices for the gate parameters. This is not directly a property of the RL approach presented in this work, but of our underlying quantum circuit framework [52]. With manageable effort, this framework can be extended to allow also gates parameterized by symbolic variables. This would simplify the optimization of entire classes of parameterized circuits, like for QAOA with its varying angles β_c and γ_c . So far, this is only indirectly possible by repeating the same sequence of transformations for differently parameterized circuits.

IV. CONCLUSION

In this article, we have introduced RL as a powerful and flexible approach to QCO. In particular, we have focused on the optimization of the local circuit structure to the available hardware resources, which is especially relevant for the NISQ era. The main challenge here is to discover suitable sequences of circuit transformations; the size of the search space and the lack of simple strategies in many situations, especially for circuits with an irregular structure, make this a difficult optimization problem. RL is well-suited to problems of this kind, and promises to simultaneously fulfill the following key features: *hardware-efficient*, i.e., that it can find the optimal solution given the available resources; *cross-platform*, i.e., that the approach works for different quantum architectures (gate sets, chip layouts, connectivities, etc.), without large migration costs; *autonomous*, i.e., that the only human effort is to specify the problem, and the optimization is done by the computer completely on its own, without requiring further interaction; and *reliable*, i.e., that the results are always correct and for a wide spectrum of circuits (close-to-)optimum.

For our approach to be successful, we had to address several key challenges: First, we have formulated QCO as a problem which can be efficiently addressed using RL (cmp. Secs. II A and II B). Second, we have developed efficient schemes to represent circuits for the network input and transformations for the network output (cmp. Sec. II E). Third, as prerequisite for applying techniques like RL or simulated annealing, we have implemented a Python framework to automate the process of identifying

all transformations within a circuit [52].

Both RL and simulated annealing turn out to be valid optimization techniques for the QCO problems considered in this work. In terms of optimization quality, the difference between both approaches in reducing depth and gate count were not larger than 10% (in reference to the pruning level), except for Sec. III A where simulated annealing achieved only $58.06 \pm 2.15\%$ of the depth reduction by the RL agent (cmp. Sec. III D). In terms of computational effort, our RL agents required up to 1 week of training time, but were afterwards considerably faster than simulated annealing in optimizing arbitrary circuits from the class they have been trained on (ca. 2 min vs. 1...3 d in Sec. III A, 3...5 h vs. 7 d in Sec. III B). We expect that RL can deal better with situations where decision-making becomes even harder than here. Therefore, RL seems to be, especially in the long run, the more promising approach.

So far, we have mostly considered randomly generated circuits to circumvent the lack of a sufficient dataset of realistic circuits; one central future direction will be to collect such a dataset. Several direct extensions of our work are straightforward to implement, yet potentially very fruitful for future applications, such as implementations for different gate sets, 2D geometries, etc. A (semi-)automated procedure to identify transformation rules, where machine learning could be beneficial, would further reduce the effort for including new gate sets. The optimization of parameterized circuit classes can be simplified by implementing symbolic variables as gate parameters in the underlying quantum circuit framework. Furthermore, the existing complementary QCO techniques which focus on optimizing the global circuit structure [11–15] could be combined in a two-stage process with our technique to optimize the local circuit structure.

We expect our approach to soon become a standard tool for optimizing circuits in NISQ experiments. Central practical advantages are the negligible additional effort to consider non-ideal circumstances, like inhomogeneous error rates or defect qubits, and the ability to customize the optimization criterion.

Data and code availability The code is publicly available under [52]. Other findings of this study are available from the corresponding author on reasonable request.

Acknowledgements T.F. thanks Google Research for hosting during his internship.

Competing interests The authors declare no competing interests.

Corresponding author Correspondence should be addressed to T.F.

-
- [1] Michael A Nielsen and Isaac Chuang. Quantum computation and quantum information, 2002.
 [2] Peter W Shor. Polynomial-time algorithms for prime fac-

- torization and discrete logarithms on a quantum computer. *SIAM review*, 41(2):303–332, 1999.
 [3] Lov K Grover. Quantum mechanics helps in searching for

Compiler Optimization for Quantum Computing Using Reinforcement Learning

Nils Quetschlich*

Lukas Burgholzer†

Robert Wille*‡

*Chair for Design Automation, Technical University of Munich, Germany

†Institute for Integrated Circuits, Johannes Kepler University Linz, Austria

‡Software Competence Center Hagenberg GmbH (SCCH), Austria

nils.quetschlich@tum.de

lukas.burgholzer@jku.at

robert.wille@tum.de

<https://www.cda.cit.tum.de/research/quantum>

Abstract—Any quantum computing application, once encoded as a quantum circuit, must be compiled before being executable on a quantum computer. Similar to classical compilation, quantum compilation is a sequential process with many compilation steps and numerous possible optimization passes. Despite the similarities, the development of compilers for quantum computing is still in its infancy—lacking mutual consolidation on the best sequence of passes, compatibility, adaptability, and flexibility. In this work, we take advantage of decades of classical compiler optimization and propose a *reinforcement learning* framework for developing optimized quantum circuit compilation flows. Through distinct constraints and a unifying interface, the framework supports the combination of techniques from different compilers and optimization tools in a single compilation flow. Experimental evaluations show that the proposed framework—set up with a selection of compilation passes from IBM’s Qiskit and Quantinuum’s TKET—significantly outperforms both individual compilers in 73% of cases regarding the expected fidelity. The framework is available on GitHub (<https://github.com/cda-tum/MQTPredictor>) as part of the Munich Quantum Toolkit (MQT).

I. INTRODUCTION

In classical computing, each *Central Processing Unit* (CPU) comes with its own *Instruction Set Architecture* (ISA)—describing the native operations the CPU can process. High-level programming languages such as C++, Rust, or Go are used to develop programs in a platform-agnostic way. These high-level descriptions are then *compiled* to a specific device’s ISA by a *compiler*. Prominent examples are LLVM [1] or GCC for C++, rustc for Rust, and gc or gccgo for Go. In each of these compilers, a plethora of passes for analyzing and transforming the source code is available. The “quality” of the code generated by compilers heavily depends on the selection of the individual passes applied during the compilation process as well as their execution order. Furthermore, one particular combination is highly unlikely to be the most effective sequence for every program on a particular machine. Even nowadays, after decades of classical compiler research, determining good sequences of compilation passes turns out to be a challenging task. In the literature, this problem is commonly referred to as *Phase Ordering*.

Research on the phase ordering problem already started roughly four decades ago [2]–[4] and is still very actively pursued. In the recent years, *autotuning* [5] (trying out different compilation parameters and evaluating the results with respect

to a particular metric) and machine learning [6] have shown promising results. A particularly promising branch of research is the application of *Reinforcement Learning* (RL [7], [8]) for this purpose. In [9], the authors show that their RL-based approach to phase ordering outperforms LLVM’s optimization scheme. Reinforcement Learning has also successfully been applied for inlining optimization [10] and vectorization [11].

The compilation of quantum circuits/programs follows a very similar scheme as the compilation of classical programs. All major quantum SDKs (such as IBM’s Qiskit [12], Quantinuum’s TKET [13], AWS Braket [14], or Microsoft’s Azure Quantum [15]) implement some sort of compilation flow for making circuits executable on particular devices (see, e.g., [16]). However, while research on classical compilers has been conducted for decades, compilers for quantum computing are still in early development. This manifests in several ways:

- 1) There is no mutual consolidation on what the best methods and combinations thereof are.
- 2) It is not straightforward to mix and match different options from various frameworks—creating something similar to a vendor lock-in.
- 3) Major SDKs can be quite slow in adopting the newest research findings—effectively restraining users from taking advantage of state-of-the-art methods.
- 4) Users can hardly customize the optimization goal or the target metric for the compilation process. Instead, they have to rely on predefined combinations of compilation passes provided by the SDKs—typically in the form of various optimization levels.

As a consequence, we are quickly heading towards the same problems as in classical compiler development and, despite the similarities, techniques from the classical realm cannot be straightforwardly applied in the quantum realm due to the inherent differences of both computational paradigms.

In this work, we aim to mitigate the drawbacks discussed above. Instead of trying to reinvent the wheel, the solution proposed in this work builds upon the decades of research on compiler optimization in the classical realm. To this end, we adapt an established classical technique to the quantum domain—modeling the compilation flow as a *Markov Decision Process* (MDP, [17]) and applying *Reinforcement Learning* to learn the best order in which the different compilation passes and optimizations can be applied.

The resulting framework (which is available on GitHub (<https://github.com/cda-tum/MQTPredictor>) as part of the Munich Quantum Toolkit (MQT)) is platform agnostic and easily extendable with further compilation methods from different SDKs and software tools. This is accomplished by defining distinct constraints that can be efficiently checked for each state in the process and a unifying interface for each compilation step. In contrast to slowly-adapting compilers in major SDKs, this allows to develop optimized compilation flows that can keep up with the fast-paced development and research in this domain.

Experimental evaluations demonstrate the feasibility of the proposed approach. Compared to the compilation flows offered by IBM's Qiskit and Quantinuum's TKET, the proposed framework—set up with a selection of compilation passes from both tools—is shown to outperform both individual compilers with respect to expected fidelity, critical depth, and their combination in 73%, 84%, 75% of cases.

The rest of this paper is structured as follows. Section II briefly reviews quantum circuit compilation. Then, Section III describes how compiler optimization for quantum circuits can be modeled as an MDP and how it can be tackled with reinforcement learning. Based on that, Section IV provides details on the implementation of the proposed framework and evaluates its performance. Finally, Section V concludes the paper.

II. BACKGROUND: QUANTUM CIRCUIT COMPILATION

Quantum programs are commonly described in the form of *quantum circuits*, i.e., sequences of quantum operations (also called quantum gates) acting on the qubits of a quantum system.

Example 1. A simple quantum circuit operating on three qubits (q_0, q_1, q_2) is shown in Fig. 1a. It contains four different kinds of gates: Two NOT gates (denoted as X), one Hadamard gate (denoted as H), one Z-rotation gate (denoted as R_z), and three two-qubit controlled-NOT or CNOT gates (with their control and target qubits denoted as $\bullet$ and $\oplus$, respectively).

Executing such programs on an actual *Quantum Processing Unit* (QPU) requires *compilation* conducted by *compilers*. They usually first employ device-independent optimizations to reduce the complexity of the original circuit description. Examples of such optimizations include gate-cancellation, gate-commutation, or high-level synthesis routines, e.g., proposed in [18]–[22].

Example 2. Consider again the circuit shown in Fig. 1a. Employing device-independent optimizations would, e.g., lead to the cancellation of the consecutive X gates on q_2 (highlighted by the green box) since $X \cdot X = I$.

After these initial optimizations, QPU-specific circuits are generated. To this end, each quantum processor offers a particular set of *native gates*, i.e., operations that can be directly executed on the device. Consequently, all non-native gates need to be *synthesized* using these native gates for the circuit to be executable. This synthesis step creates opportunities for further optimizations.

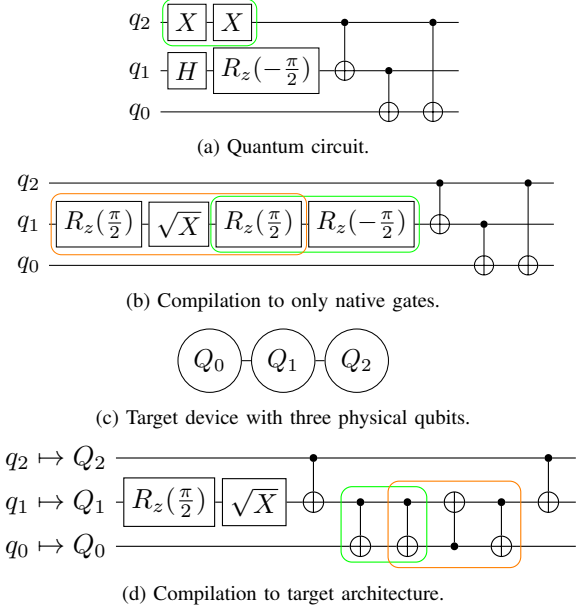


Fig. 1. Compilation of a quantum circuit to a targeted device.

Example 3. Recent devices offered by IBM provide a native gate-set consisting of R_z , $\sqrt{X}$, X, and CNOT gates. To execute the circuit from Fig. 1a on such a device, e.g., the Hadamard gate needs to be expressed in terms of these gates. A suitable decomposition into two $R_z(\frac{\pi}{2})$ and a $\sqrt{X}$ gate is shown in Fig. 1b (highlighted by the orange box). Incidentally, one of the $R_z(\frac{\pi}{2})$ gates can, then, be cancelled with the existing $R_z(-\frac{\pi}{2})$ gate (highlighted by the green box).

Some QPUs (e.g., those based on superconducting qubits or neutral atoms) only feature a limited connectivity between their qubits, so that multi-qubit gates can only be applied to qubits that are connected on the device. In these cases, a *mapping* step is required. Mapping is frequently split into *layouting*, i.e., assigning the circuit's logical qubits to the device's physical qubits, and *routing*, i.e., ensuring the connectivity constraints are satisfied by dynamically changing the assignment of logical to physical qubits. Again, optimizations may be applied after mapping the circuit to reduce the induced overhead.

Example 4. Assume the circuit from Fig. 1b shall be executed on an IBM device with three physical qubits (Q_0, Q_1, Q_2) arranged in a line (as depicted in Fig. 1c), i.e., only interactions between Q_0 and Q_1 as well as Q_1 and Q_2 are possible. Then, no layout exists that makes this circuit executable right away since the circuit contains interactions between all pairs of qubits. Thus, a SWAP gate (eventually realized as a sequence of three CNOT gates highlighted by the orange box) needs to be inserted during the routing step to make the circuit executable (as shown in Fig. 1d). In this case, some of the induced overhead can be reduced by cancelling two CNOT gates (highlighted by the green box).

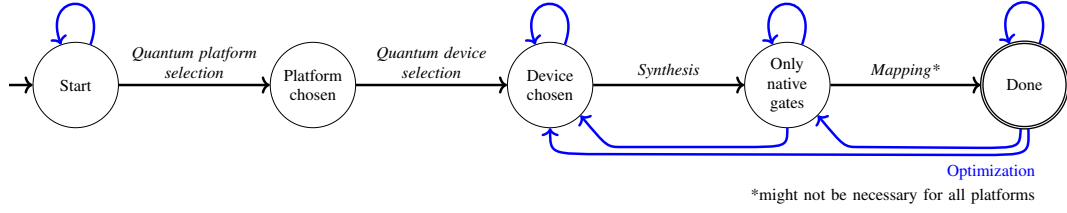


Fig. 2. Quantum compilation flow modeled as an MDP.

With the number of different synthesis, mapping, and optimization techniques for quantum circuits rapidly increasing, the landscape of compiler passes is becoming more and more complex—leading to several obstacles as discussed in Section I. In this work, we propose a framework that aims to overcome these obstacles.

III. PROPOSED FRAMEWORK

Fortunately, the underlying problem is not new per-se, since classical compilers have seen a similar trend in the past and lots of research has been conducted to find efficient solutions there. In the following, we describe how a classically-inspired reinforcement learning technique can be adapted to quantum circuit compilation. To this end, we first describe how to programmatically model quantum circuit compilation in order to, then, apply reinforcement learning to try to learn the best sequence of compilation passes depending on characteristics of the initial circuit—resulting in an *optimized compiler* for quantum circuits.

A. Modeling as a Markov Decision Process

The quantum circuit compilation flow reviewed above can be seen as a sequential procedure that takes an initial (high-level) quantum circuit and, step by step, applies certain actions/transformations to eventually arrive at a circuit representation that can be executed on a target device. Such a procedure can be formally modeled as a *Markov Decision Process* (MDP, [17]). To this end, it is split into states and corresponding actions that can be performed in each of the states. The model proposed in this work is illustrated in Fig. 2—with states being denoted as circles and actions as arrows. In the following, the process with its respective actions is described one by one.

Initially, the process starts in the “Start” state and stores the original quantum circuit to be compiled. At this point, there are two possible types of actions. On the one hand, device-independent *optimizations* can be applied to the circuit (indicated by the blue arrow). As reviewed in Section II, this includes techniques such as single-qubit gate fusion, gate cancellation and commutation, as well as high-level synthesis methods, e.g., proposed in [18]–[22]. After such an optimization, the process remains in the same state as before, but now stores a new optimized quantum circuit. On the other hand, the process can proceed to a different state by *Quantum platform selection* actions, e.g., choosing *IBM*, *Rigetti*, or *IonQ*—effectively fixing the targeted native gate-set. In the classical domain, this roughly corresponds to deciding, e.g., between *x86* and *ARM* architectures.

Once a platform has been chosen (i.e., the process is in the “Platform chosen” state), only a single type of action can be applied. Namely, the *Quantum device selection* actions offered by the selected platform. This step fixes the number of qubits and their topology, i.e., which qubits can interact with each other. Depending on the choice of the platform, a varying range of devices with different characteristics is available. At the time of writing, *IBM* for example offers 23 different quantum computers that each vary in their qubit count, their quantum volume, and their *Circuit Layer Operations Per Second* (CLOPS). In the classical domain, this roughly corresponds to targeting a specific CPU model family, such as *Intel Alder Lake* or *AMD Zen 4*.

After a particular device has been selected (i.e., the process is in the “Device chosen” state), the device-dependent compilation begins. The goal of the remaining steps is to produce a circuit that can be executed on the chosen device. To this end, the circuit needs to satisfy two conditions:

- 1) it must only use gates native on the selected platform, and
- 2) it must conform to the topology of the selected device.

While the first condition can be ensured by using *Synthesis* methods, e.g., as proposed in [23]–[26], the second condition is realized by corresponding *Mapping* methods, e.g., as proposed in [27]–[29]. Both of these tasks are highly non-trivial and there is a vast variety of methods available in major quantum SDKs as well as individual research tools. At the time of writing, *IBM*’s *Qiskit* alone offers four different layout and five routing algorithms—leading to 20 different mapping schemes. Once both conditions are satisfied, the process is in the “Done” state, i.e., the circuit is executable. Optimizations may be applied in any state during the device-dependent compilation. Depending on whether the resulting circuit satisfies none, the native-gates, or both constraints, the process respectively continues in the “Device chosen”, “Only native gates”, or “Done” state (again, indicated by the blue arrows).

Overall, the formulation described above leads to a modular framework with two characteristic traits:

- 1) Each state has *distinct constraints* that can be efficiently checked, e.g., whether the circuit only contains certain native gates or only acts on qubits connected on the device. Thus, it is straight-forward to determine the state after a certain action has been applied.
- 2) All actions have a *unified interface*, i.e., all of them use a quantum circuit as the main representation for their input and output. Moreover, the same format is used independent of the SDK or tool the actions originate from.

As a consequence, it is possible to mix and match compiler passes from different SDKs and tools and integrate them into a single *optimized* compiler. Such a framework also allows to mitigate vendor lock-ins and can help to keep up with the rapid pace of the field by allowing one to quickly integrate new methods into the overall compilation flow.

B. Compiler Optimization Using Reinforcement Learning

After a framework as described above is set up, it can be used as the basis for learning the best sequence of actions (i.e., compilation passes) depending on characteristics of the initial circuit. Motivated by its success in classical compiler optimization [9]–[11], we propose to apply reinforcement learning for this task. RL-methods aim to learn an *action policy* that describes which action to apply based on certain *observations* on the current state such that some cumulative *reward function* is maximized. In contrast to supervised machine learning methods, no training data must be provided. Instead, only an environment describing the underlying MDP (i.e., the set of states and corresponding actions), the features used as observations, and a corresponding reward function needs to be provided. Based on the rewards for certain actions in certain states, the action policy improves through the training process and learns how to maximize the expected cumulative reward.

In the proposed framework, observations are realized by extracting certain features from the quantum circuits in each state. In addition, a sparse reward function is used that reflects whether a final state has been reached and, then, quantifies the “quality” of the resulting circuit, e.g., in terms of the resulting gate count, circuit depth, or expected fidelity. This flexibility in the target metric allows one to design specialized compilers for dedicated use cases. After successful training, the learned action policy can be used as an *optimized compiler* for any quantum circuit.

C. Related Work

Machine learning has been successfully applied to quantum circuit design/compilation in the past. In [30], the authors propose a machine learning based layout and routing scheme and claim to be on-par with state-of-the-art methods. Another possible application is given in [31], where the graph structure of quantum circuits is used to predict their expected fidelities for certain devices considering their respective noise characteristics. An idea related to the one proposed in this work, which follows a more coarse-grained approach, has been demonstrated in [32]. There, a supervised machine learning model is used to predict good combinations of devices and existing compilation flows. In [33], RL has been applied to learn a strategy for applying individual gate transformation rules to optimize quantum circuits.

While these works either optimize one specific type of action or predict compilation options on a high abstraction level within one quantum SDK, the framework proposed in this work results in fine-grained steps that allow one to stitch together methods from multiple SDKs with a customizable optimization objective into a fully-fledged compilation flow.

IV. FEASIBILITY STUDY

In the following, the feasibility of the proposed framework is demonstrated. To this end, we first describe the particular instantiation that has been developed as part of this work. Then, the performance of the resulting optimized compiler is evaluated and compared against the state-of-the-art compilers available in IBM’s Qiskit [12] and Quantinuum’s TKET [13]. All developments and results are made publicly available on GitHub (<https://github.com/cda-tum/MQTPredictor>) as part of the Munich Quantum Toolkit (MQT)

A. Instantiation

The basis of the framework is formed by the open-source *OpenAI Gym* [34] library, which is used to set up an environment for the MDP described in Section III (as illustrated in Fig. 2). In this regard, the main endeavor is the selection and integration of all compilation passes that should be considered as actions in the learning process. As modeled in Fig. 2 we distinguish five kinds of actions: Quantum platform selection, Quantum device selection, Synthesis, Mapping, and Optimization. Due to the unified action interface of the proposed framework described earlier, methods from more than one SDK or toolkit can be incorporated. For each kind of action, suitable representatives from IBM’s Qiskit (version 0.39.2) and/or Quantinuum’s TKET (version 1.8.1) have been chosen—leading to the following list of available actions.

- *Platforms* and corresponding *Devices*:
 - IBM (superconducting): *ibmq_montreal* (27 qubits) and *ibmq_washington* (127 qubits)
 - Rigetti (superconducting): *Aspen-M-2* (80 qubits)
 - IonQ (trapped-ions): *Harmony* (11 qubits)
 - OQC (superconducting): *Lucy* (8 qubits)
- *Synthesis*: Qiskit’s *BasisTranslator*
- *Mapping*: Any combination of the following methods
 - Layout:
 - * Qiskit’s *TrivialLayout*
 - * Qiskit’s *DenseLayout*
 - * Qiskit’s *SabreLayout*
 - Routing:
 - * Qiskit’s *BasicSwap*
 - * Qiskit’s *StochasticSwap*
 - * Qiskit’s *SabreSwap*
 - * TKET’s *RoutingPass*
- *Optimization*:
 - Qiskit’s *OptimizeIqGatesDecomposition*
 - Qiskit’s *CXCancellation*
 - Qiskit’s *CommutativeCancellation*
 - Qiskit’s *CommutativeInverseCancellation*
 - Qiskit’s *RemoveDiagonalGatesBeforeMeasure*
 - Qiskit’s *InverseCancellation*
 - Qiskit’s *OptimizeCliffords*
 - Qiskit’s *Collect2qBlocks* + *ConsolidateBlocks*
 - TKET’s *PeepholeOptimise2Q*
 - TKET’s *CliffordSimp*
 - TKET’s *FullPeepholeOptimise*
 - TKET’s *RemoveRedundancies*

Inspired by the feature selection and demonstrated relevance in [32], the observations used to guide the reinforcement learning agent are based on seven features—namely the *number of qubits*, the *depth* of the circuit, and the five composite features of *program communication*, *critical-depth*, *entanglement-ratio*, *parallelism*, and *liveness* originally proposed in [35]. For the learning itself, three different reward functions have been implemented:

- 1) *Expected fidelity*: provides an estimate on the reliability of the circuit based on characteristics of the chosen device. A fidelity of 1 indicates an error-free result, while a fidelity of 0 indicates that the result is completely wrong.
- 2) *Critical Depth*: describes how many two-qubit gates in a quantum circuit lie along the critical path and contribute to the overall circuit depth. A critical depth close to 1 indicates a highly sequential quantum circuit. Since, a lower value is desirable, the corresponding reward function is defined as $1 - \text{critical depth}$.
- 3) *Combination*: adds up both mentioned reward functions (divided by two to result in a value between 0 and 1).

Finally, the *Proximal Policy Optimization (PPO)* algorithm [36] provided by *Stable-Baselines3* [37] is used for the reinforcement learning.

B. Results

To evaluate the performance of the framework instantiation described above, three models—one for each reward function—were trained on a selection of 200 circuits ranging from 2 to 20 qubits taken from the *target-independent* level of the MQT Bench library [38]. In total, 100,000 time steps were executed during the training for each reward function with runtimes of the order of hours. Then, the trained models were evaluated on the same circuits and the resulting quality in terms of the respective reward function was measured. To put this performance into perspective, all circuits are additionally compiled to the *ibmq_washington* device using Qiskit and TKET with the highest optimization level (*O3* for Qiskit and *O2* for TKET).

The results are summarized in Fig. 3. To this end, Fig. 3a, Fig. 3b, and Fig. 3c, respectively show the distribution of *absolute* reward differences of the proposed approach compared to Qiskit and TKET with respect to the three quality metrics. An *x*-value larger than zero indicates a positive improvement, e.g., a value of 0.2 for the fidelity means that the proposed approach managed to boost the fidelity by 20%. In an ideal scenario, all results would accumulate on the right-side of the zero-line.

The results demonstrate that, in the vast majority of cases, this is the case, i.e., the proposed method produces circuits of equal or better quality. In particular, the proposed method outperforms the Qiskit/TKET compiler in 73%/80%, 84%/86%, and 75%/78.5% of cases regarding expected fidelity, critical depth, and their combination.

The advantage of the proposed method becomes even clearer when looking at the average reward difference per benchmark algorithm (subsuming several instances with varying amounts of qubits) as shown in Fig. 3d, Fig. 3e, and

TABLE I
COMPARISON OF REINFORCEMENT LEARNING MODELS.

Model trained for...	Average result for...		
	Fidelity	Critical depth	Combination
Fidelity	0.48	0.27	0.37
Critical depth	0.18	0.47	0.33
Combination	0.45	0.33	0.39

Fig. 3f respectively. There, *y*-values higher than zero indicate a *net gain* in quality. Again, in the vast majority of cases, a *substantial* improvement can be observed. Compared to Qiskit/TKET, an average *absolute* increase in fidelity, critical depth, and their combination of 4.9%/10.7%, 22.6%/22.8%, and 5.5%/8.5% is achieved.

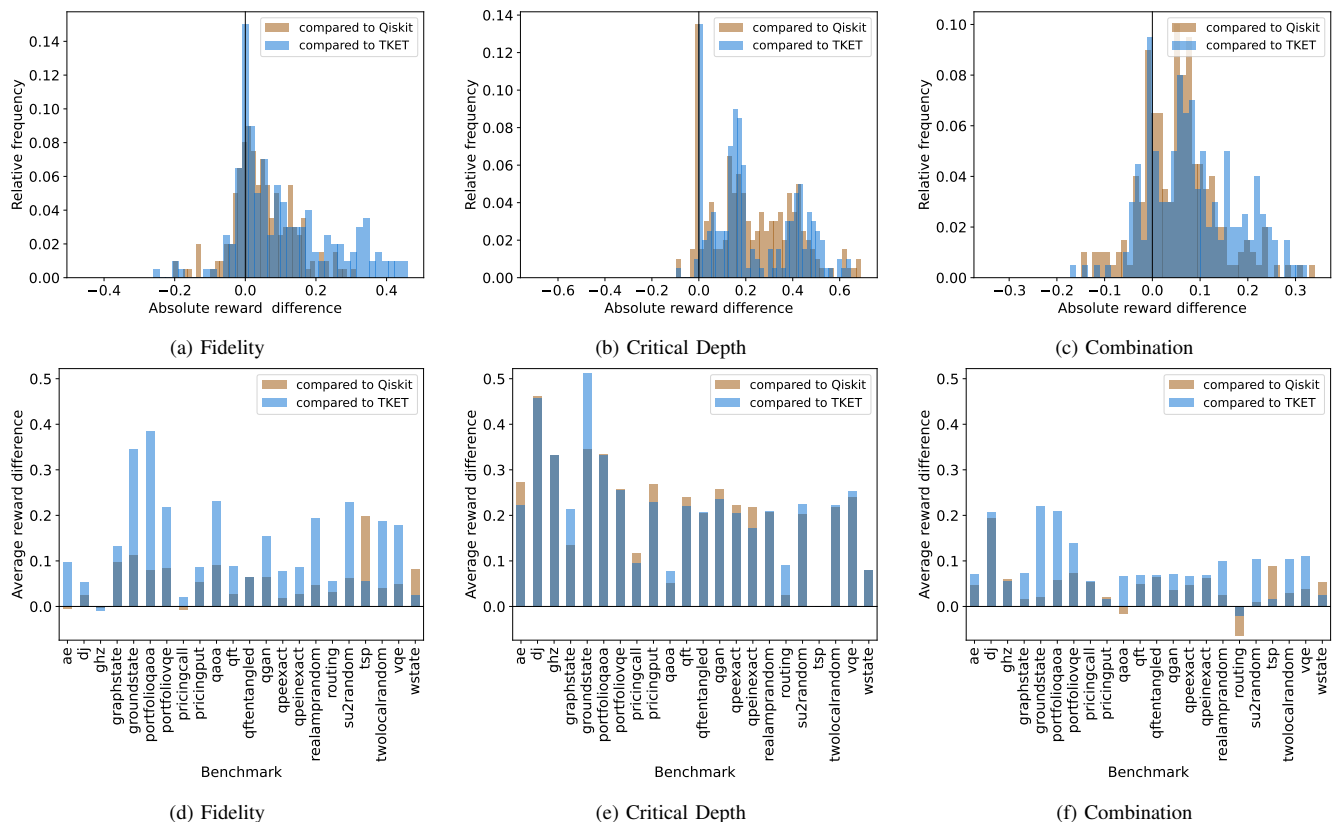
To further underline the proposed framework's ability to adapt to different target metrics, we additionally calculated the average reward for each metric by all three trained models—not only the one trained for the respective metric. The resulting numbers are denoted in Table I. They confirm that the model trained for a particular purpose indeed results in the compiled circuits with the highest rewards for all three cases.

V. CONCLUSIONS

Quantum circuit compilation flows are becoming more and more complex. This leads to a similar situation observed in classical compilation where there are so many options and it is unclear which options to choose and in which order to apply them. In this work, we proposed a framework that builds on the decades of research on classical compiler optimization to overcome these obstacles. To this end, the quantum circuit compilation flow was modeled as a Markov Decision Process with distinct constraints for each state and a unified interface for all actions. Based on that, reinforcement learning was applied to derive a complete compilation flow out of actions from multiple SDKs and toolkits. Experimental evaluations on the developed framework and three different optimization objectives demonstrate the advantage of the proposed method over Qiskit's and TKET's most optimized compilation sequences and the general feasibility of automatically creating compilations flows targeting a customizable objective. In 73%, 84%, 75% of the cases, the trained models outperformed Qiskit/TKET with an average *absolute* improvement of 4.9%/10.7%, 22.6%/22.8%, and 5.5%/8.5% regarding expected fidelity, critical depth, and their combination respectively. These results clearly demonstrate that adapting techniques from the classical realm can, indeed, yield substantial improvements in the quantum realm. By making the developed framework publicly available as open-source, we hope to lay the foundation for further exploration of this important area of research.

ACKNOWLEDGMENTS

This work received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation program (grant agreement No. 101001318), was part of the Munich Quantum Valley, which is supported by the Bavarian state government with funds from the Hightech Agenda Bayern Plus, and has been supported by the BMWK on the basis of a decision by the German Bundestag through project QuaST, as well as by the BMK, BMDW, and the State of Upper Austria in the frame of the COMET program (managed by the FFG).



REFERENCES

- REFERENCES
- [1] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization*, 2004.
 - [2] S. R. Vegdahl, "Phase coupling and constant generation in an optimizing microcode compiler," in *Workshop on Microprogramming*, 1982.
 - [3] D. B. Loveman and R. A. Faneuf, "Program optimization - theory and practice," *SIGPLAN Not.*, 1975.
 - [4] D. L. Whitfield and M. L. Soffa, "An approach for exploring code improving transformations," *ACM Trans. Program. Lang. Syst.*, 1997.
 - [5] A. H. Ashouri et al., "A survey on compiler autotuning using machine learning," *ACM Comput. Surv.*, 2018.
 - [6] H. Leather and C. Cummins, "Machine learning in compilers: Past, present and future," in *Forum on Specification and Design Languages*, 2020.
 - [7] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *Journal of Artificial Intelligence Research*, 1996.
 - [8] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
 - [9] Q. Huang et al., *Autophase: Juggling hls phase orderings in random forests with deep reinforcement learning*, 2020. arXiv: [2003.00671](https://arxiv.org/abs/2003.00671)
 - [10] M. Trofin et al., *MLgo: A machine learning guided compiler optimizations framework*, 2021. arXiv: [2101.04808](https://arxiv.org/abs/2101.04808)
 - [11] A. Haj-Ali et al., *Neurovectorizer: End-to-end vectorization with deep reinforcement learning*, 2019. arXiv: [1909.13639](https://arxiv.org/abs/1909.13639)
 - [12] Qiskit contributors, *Qiskit: An open-source framework for quantum computing*, 2023.
 - [13] S. Sivirajah et al., "TKET: A Retargetable Compiler for NISQ devices," *Quantum Science and Technology*, 2020.
 - [14] Amazon Braket Python SDK, Amazon Web Services, 2022. [Online]. Available: <https://github.com/aws/amazon-braket-sdk-python>
 - [15] Azure Quantum documentation, Microsoft, 2022. [Online]. Available: <https://learn.microsoft.com/en-us/azure/quantum/>
 - [16] R. Wille, R. Van Meter, and Y. Naveh, "IBM's Qiskit tool chain: Working with and developing for real quantum computers," in *Design, Automation and Test in Europe*, 2019.
 - [17] K. Bellman, "A markovian decision process," *Journal of Mathematics and Mechanics*, 1957.
 - [18] W. Hattori and S. Yamashita, "Quantum circuit optimization by changing the gate order for 2D nearest neighbor architectures," in *Int'l Conf. of Reversible Computation*, 2018.
 - [19] P. Niemann, R. Wille, and R. Drechsler, "Advanced exact synthesis of clifford-t circuits," *Quantum Information Processing*, 2020.
 - [20] G. Vidal and C. M. Dawson, "Universal quantum circuits for two-qubit transformations with three controlled-NOT gates," *Physical Review A*, 2004.
 - [21] T. Itoko et al., "Quantum circuit compilers using gate commutation rules," in *Asia and South Pacific Design Automation Conf.*, 2019.
 - [22] D. Maslov, G. Dueck, D. Miller, and C. Negrevergne, "Quantum circuit simplification and level compaction," *IEEE Trans. on CAD of Integrated Circuits and Systems*, 2008.
 - [23] B. Giles and P. Selinger, "Exact synthesis of multiqubit Clifford+T circuits," *Physical Review A*, 2013.
 - [24] M. Amy, D. Maslov, M. Mosca, and M. Roetteler, "A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits," *IEEE Trans. on CAD of Integrated Circuits and Systems*, 2013.
 - [25] D. M. Miller, R. Wille, and Z. Sasanian, "Elementary quantum gate realizations for multiple-control Toffoli gates," in *Int'l Symp. on Multi-Valued Logic*, 2011.
 - [26] S. Schneider, L. Burgholzer, and R. Wille, "A SAT encoding for optimal Clifford circuit synthesis," in *Asia and South Pacific Design Automation Conf.*, 2023.
 - [27] M. Y. Siraichi, V. F. dos Santos, S. Collange, and F. M. Q. Pereira, "Qubit allocation," in *Int'l Symp. on Code Generation and Optimization*, 2018.
 - [28] A. Zulehner, A. Paler, and R. Wille, "An efficient methodology for mapping quantum circuits to the IBM QX architectures," *IEEE Trans. on CAD of Integrated Circuits and Systems*, 2019.
 - [29] A. Matsuo and S. Yamashita, "An efficient method for quantum circuit placement problem on a 2-D grid," in *Int'l Conf. of Reversible Computation*, 2019.
 - [30] A. Paler, L. M. Sasu, A.-C. Florea, and R. Andonie, "Machine learning optimization of quantum circuit layouts," *ACM Transactions on Quantum Computing*, 2022.
 - [31] H. Wang et al., "Quest: Graph transformer for quantum circuit reliability estimation," 2022. arXiv: [2210.16724](https://arxiv.org/abs/2210.16724)
 - [32] N. Quetschlich, L. Burgholzer, and R. Wille, *Predicting good quantum circuit compilation options*, 2020. arXiv: [2210.08027](https://arxiv.org/abs/2210.08027)
 - [33] T. Fösel, M. Y. Niu, F. Marquardt, and L. Li, *Quantum circuit optimization with deep reinforcement learning*, 2021. arXiv: [2103.07585](https://arxiv.org/abs/2103.07585)
 - [34] G. Brockman et al., *OpenAI Gym*, 2016. arXiv: [1606.01540](https://arxiv.org/abs/1606.01540)
 - [35] T. Tomesh et al., *Supermarq: A scalable quantum benchmark suite*, 2022. arXiv: [2202.11045](https://arxiv.org/abs/2202.11045)
 - [36] J. Schulman et al., *Proximal policy optimization algorithms*, 2017. arXiv: [1707.06347](https://arxiv.org/abs/1707.06347)
 - [37] A. Raffin et al., "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, 2021.
 - [38] N. Quetschlich, L. Burgholzer, and R. Wille, *MQT Bench: Benchmarking software and design automation tools for quantum computing*. MQT Bench is available at <https://www.cda.cit.tum.de/mqtbench/> 2022. arXiv: [2204.13719](https://arxiv.org/abs/2204.13719)

Achieving Pareto-Optimality in Quantum Circuit Compilation via a Multi-Objective Heuristic Optimization Approach

Aleksandra Świerkowska^{*†}, Jorge Echavarria^{*}, Laura Schulz^{*}, Martin Schulz[†],

^{*}Leibniz Supercomputing Centre of the Bavarian Academy of Sciences and Humanities, Garching, Bavaria, Germany
{aleksandra.swierkowska, jorge.echavarria, laura.schulz}@lrz.de

[†]Technical University of Munich, Garching, Bavaria, Germany
{martin.w.j.schulz}@tum.de

Abstract—High Performance Computing-Quantum Computing (HPCQC) integration presents a promising yet challenging opportunity, particularly in the area of quantum circuit compilation and optimization, requiring further advancements in the field of Quantum Computing (QC). To address this, we introduce the Munich Quantum Compiler, a key component of the Munich Quantum Software Stack (MQSS). This compiler employs a heuristic-based approach to select a Pareto-optimal subset of optimizations in the form of LLVM passes for quantum circuits described in an LLVM-compliant Intermediate Representation (IR).

Index Terms—Quantum Computing, Multi-objective Optimization, Quantum Compilation, LLVM, QIR, MOEA, Genetic Algorithm, NSGA-II

I. INTRODUCTION

In recent years, Quantum Computing (QC) has demonstrated significant potential for achieving exponential performance improvements over classical algorithms for certain classes of computational problems [1]–[5]. In order to enable wider growth and development of the quantum potential, efforts towards High Performance Computing-Quantum Computing (HPCQC) integration have been initiated [6], [7]. Reaching their goal of providing seamless cooperation between classical and quantum parts of the system would allow to reach a new territory of research, mainly in the form of hybrid algorithms.

However, to be able to achieve a hybrid software stack, it is necessary to design highly sophisticated compilers capable of both: 1) providing effective optimizations for quantum circuits, such as reducing their size to allow execution before significant coherence degradation, and 2) adapting the quantum circuits to the unique capabilities and limitations of the available quantum accelerators. Furthermore, a quantum compiler should be able to support the common software stack for classical and quantum applications. For that reason, we decided to utilize Quantum Intermediate Representation (QIR) [8], an LLVM-compliant Intermediate Representation (IR) supporting interleaving quantum and classical instructions within a single program [9], as a mid-level quantum circuit representation in the Munich Quantum Compiler, a key component of the Munich Quantum Software Stack (MQSS) [10]–[12]. This approach allowed us to apply optimizations and other required

transformations to the circuit through the application of custom LLVM passes. Nevertheless, employing compilation schemes akin to the classical world, such as iteratively applying LLVM optimization passes, introduces classical challenges into the quantum domain, as we will elaborate.

The challenges associated with finding Pareto-optimal optimization subsets [13] and the sequence in which to apply them [14], also known as phase ordering, have been extensively studied in the classical compilation field. These are well-known NP-Hard problems [15]. The proposed solutions range from utilizing Genetic Algorithms (GAs) [16]–[18], for years deemed as state-of-the-art, through more modern approaches based on, for example, Machine Learning (ML) [19] or Reinforcement Learning (RL) [15], [20], [21].

Similarly, ML and RL are usually utilized regarding quantum compilation [22]–[25], mainly due to their efficient execution times. However, most of the proposed approaches in the literature focus on optimizing a singular objective, usually a complex figure of merit consisting of a weighted sum of multiple different objectives, such as depth or number of gates. Although the importance of some quantum metrics is well-established, the development of effective quantum performance metrics remains an active area of research [26]. To provide a change of metrics in a figure of merit in model-based compilers, each time a tedious and time-consuming model retraining is necessary.

The novelty of the quantum optimization approach proposed in the Munich Quantum Compiler lies in providing a multi-objective optimization through the utilization of a GA, more specifically, a non-dominated sorting-based Multi-Objective Evolutionary Algorithm (MOEA) called Non-dominated Sorting Genetic Algorithm II (NSGA-II). This approach yields a set of solution candidates belonging to the Pareto frontier, none of which is fully dominated by any other solution found. While GAs have proven to be highly effective in the classical domain, to the best of our knowledge, they have not yet been applied to the quantum version of this problem.

II. METHODS

Optimization is a crucial component of mostly every compiler, making the development of effective pass selection

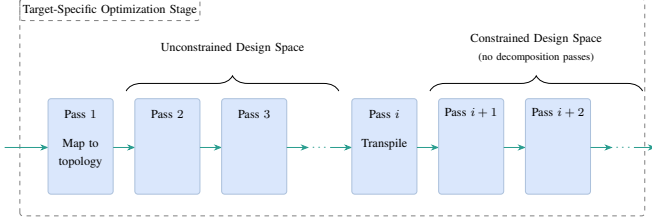


Fig. 1: Target-Specific Optimization Stage of the Munich Quantum Compiler.

strategies highly desirable. Although the number of strategies implemented over the years in various quantum programming frameworks, such as, for example, Qiskit [27], is constantly growing, the Munich Quantum Compiler distinguishes itself by utilizing a heuristic implemented in C++ to prioritize performance and efficiency. Furthermore, unlike tools like QIR Adaptor Tool (QAT) [28], which also apply a series of LLVM passes to QIR for optimizations, our compiler aims for seamless integration within HPCQC ecosystems.

The Munich Quantum Compiler aims to find a Pareto-optimal combination of LLVM passes to optimize multiple metrics simultaneously, such as the number of gates in the circuit or its depth. Given the varying characteristics of individual circuits and changing constraints throughout the pipeline execution, the effectiveness of pass subsets can differ. To address this problem, the Munich Quantum Compiler provides a subset tailored for each submitted quantum circuit by conducting a two-fold Design Space Exploration (DSE) to derive a Pareto-optimal solution. The two key stages of this process are the *Target-Agnostic Optimization Stage* and the *Target-Specific Optimization Stage*:

Target-Agnostic Optimization Stage: This stage is inherently simpler than the Target-Specific Optimization Stage, as it does not conduct an exhaustive DSE. Instead, it allows for a fully unconstrained exploration of the design space consisting of all the available target-agnostic optimization passes. These passes do not require any knowledge about the quantum accelerator intended for executing the optimized quantum circuit. Some examples include passes merging rotations, passes replacing specific sequences of gates, or passes removing pairs of inverse gates (UU^{-1}) as, for instance, two Pauli-X gates acting on the same qubit:

$$|\psi\rangle \text{---} \boxed{X} \text{---} \boxed{X} \text{---} \equiv |\psi\rangle \text{---} \boxed{I} \text{---}$$

Fig. 2: Applying two consecutive Pauli-X gates to a qubit is equivalent to applying an identity operation [29].

The primary goal of this stage is to optimize the quantum circuit to reduce its execution time in the subsequent, more complex stages of the MQSS.

Target-Specific Optimization Stage: This stage is significantly more intricate than the Target-Agnostic Optimization Stage, as it needs to not only provide optimizations but also other types of transformations, enabling the quantum circuit to

be executed on the target quantum accelerator. The configuration of passes we propose for this stage is presented in Fig. 1. This stage begins with the mapping of the quantum circuit qubits to the topology of the target accelerator. It is achieved by applying a pass utilizing, for example, the Munich Quantum Toolkit (MQT) QMAP [30] mapper, a tool capable of mapping the provided quantum circuit to the topology of the target architecture. Due to the expected potential overhead introduced by the mapper, this pass is followed by another DSE of optimization passes, including target-specific ones. Afterward, the quantum circuit is transpiled to the native gate set of the target quantum accelerator. Note that the above modifications should be sufficient to submit the circuit to the target quantum accelerator. However, we provide additional optimizations due to the critical importance of maximizing the circuit's quality for successful execution. To protect the transpiled circuit from introducing unsupported gates, these optimizations are constrained to exclude decomposition passes.

Fig. 3 presents a simplified example of the process occurring during the Target-Specific Optimization Stage. As shown, the additional application of optimizations following mapping and removal of unsupported gates allows us to maximally reduce gate overhead without compromising the mapping or gate set necessary for the successful execution of the circuit on the selected quantum device. Furthermore, utilizing a heuristic for DSE allows the application of a different decomposition, in this case, to each SWAP gate, depending on what is on each side for the maximal performance enhancement.

A. Multi-Objective Evolutionary Algorithm (MOEA)

To achieve high flexibility in objective selection, we opted to use the MOEA known as NSGA-II. The parameters and operators for NSGA-II detailed in the following were empirically chosen to best suit the design space.

a) *Encoding:* Binary encoding proved ineffective, as it can only encode whether a given pass should be applied, without specifying the sequence or the number of applications to the quantum circuit under optimization. In contrast, integer encoding allowed for both the removal and repetition of passes.

b) *Operators:* We decided on using a swap mutation [31] and two-point crossover [32] operators as, compared to other operators we tested, they demonstrated better effectiveness in improving the quality of the solutions.

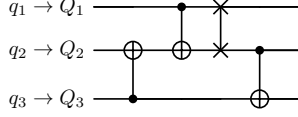
c) *Chromosome size:* The most promising results were observed with a chromosome size three times the number of available passes.

B. Design Space

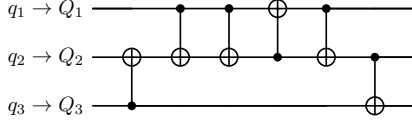
To introduce transformations to quantum circuits described in QIR, a collection of custom LLVM passes was created. The passes consist of both target-specific and target-agnostic transformations, which can be further divided into *commutation*, *decomposition*, *reduction*, and *structural* types. Each pass introduces a specific modification, such as the inverse gates removal shown in Fig. 2, contributing to enhancing the



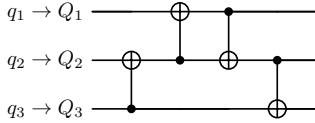
(a) The exemplary quantum circuit and the linear topology to which it needs to be mapped.



(b) Mapping the quantum circuit to the given topology results in the lack of a coupling mapping between q_1 and q_3 , because of which it is necessary to insert an additional SWAP gate in order to execute it on the selected quantum accelerator.



(c) Assuming that SWAP gates are not part of the native gate set of the target device, a target-specific optimization would apply a decomposition.



(d) Target-agnostic optimization would then remove the consecutive CNOT gates analogically as in Fig. 2 [29], bringing us to the optimized quantum circuit that can be executed on the target accelerator with minimized overhead.

Fig. 3: Circuit modifications occurring during the Target-Specific Optimization Stage for an exemplary fragment of a quantum circuit mapped to a linear topology $Q_1 - Q_2 - Q_3$ of a quantum accelerator.

quantum circuit's performance. To provide a possibly comprehensive optimization coverage, passes can overlap in the sets of gates they act on or operate counterproductively, with their actions negating each other. These mutually exclusive passes necessitate addressing the NP-Hard subset selection problem.

III. EMPIRICAL FINDINGS

A. Experimental Setup

a) *Design Space*: The design space for our experimentation consisted of 36 passes selected from those currently available for the Munich Quantum Compiler. For an optimization pass to be considered during the DSE, it had to meet the following criteria: 1) the pass must be capable of functioning autonomously, 2) it must be able to occupy any position in

the optimization pipeline, and 3) there must be no sequence of passes where the application of this pass compromises the integrity of the quantum circuit.

b) *Benchmark Suite*: To evaluate the effectiveness of our approach, we used a benchmark suite of 175 quantum circuits with up to 20 qubits, sourced from [33] and parsed into QIR.

c) *Objectives*: The performance of the Munich Quantum Compiler was evaluated based on multiple objectives: 1) the number of gates, 2) circuit depth, 3) entanglement ratio, defined as the ratio of two-qubit interactions to the number of all gate operations, 4) critical depth, defined as the ratio of two-qubit interactions on the longest path that sets the circuit depth to the number of all two-qubit interactions in the circuit, and 5) parallelism, defined as:

$$\left(\frac{n_g}{d} - 1\right) \frac{1}{n-1} \quad (1)$$

where n is the number of qubits, n_g the number of gates, and d is the circuit depth [26]. All of these objectives were minimized.

As research into metrics for quantifying quantum circuits evolves, the objectives may change. Our approach, however, can adapt to such changes instantly, without the need for retraining, unlike, for example, ML-based solutions. Moreover, in the future, we plan to extend the set of objectives to include circuit fidelity and *liveness* [26].

B. Results

Table I presents the results of each objective averaged over the benchmark suite. The number of gates and circuit depth are expressed as ratios relative to the corresponding measurements of the non-optimized circuits.

In a setup with all of the passes applied, the specific order was determined through a series of trials, during which we made strategic arrangements aiming to improve the performance. However, as presented in ("All passes" in Table I), this approach led to an increase in both the number of gates and depth compared to the original circuits. This increase is due to the introduction of unnecessary decompositions, which negatively affect the overall circuit quality.

In another approach, only a subset of these passes is selected ("Handpicked" in Table I). In this setup, the subset and the order of passes were again determined through iterative tries and based on prior knowledge, with each pass applied at most once. Although this approach mitigates the issues seen in the "All passes" setup, the improvements over the non-optimized circuit are negligible.

In order to compare the effectiveness of the MOEA-based approaches, from each set of solution candidates, a non-dominated candidate resulting in the lowest number of gates was chosen, as the number of gates is included in the calculations of most of the other objectives.

As one can observe, the application of NSGA-II with binary encoding, which involves a fixed order of a subset of passes, did not improve the average result, as shown in the row "NSGA-II bin.". Since the order of passes was fixed, the

TABLE I: The averages of the results obtained for each type of experimental setup. The numbers in bold highlight the lowest value achieved for each objective.

Setup	Gates ratio	Depth ratio	Ent. ratio	Critical depth	Parallelism
Non-optimized	1	1	0.5821	0.2083	0.5419
All passes	2.9805	2.4356	0.2147	0.1972	0.6074
Handpicked	0.9195	0.9697	0.5011	0.3535	0.5524
NSGA-II bin.	0.9195	0.9697	0.5011	0.3535	0.5524
NSGA-II int.	0.055	0.1378	0.0038	0.0114	0.3368

TABLE II: Comparison of average execution times of different NSGA-II setups for a single quantum circuit.

Setup	Avg. execution time [s]
NSGA-II bin.	4.514
NSGA-II int.	9.903

results quickly converged to those of the “Handpicked” setup. However, using integer encoding to determine the order of optimization passes and allowing for repetitions led to a notable performance enhancement even in the simplest setup with a population size of 8 individuals evolved over 8 generations. Utilizing a more sophisticated setup with a population size of 128 individuals evolved over 128 generations resulted in a significant improvement shown in the row “NSGA-II int.” in Table I.

Table II presents average execution times comparison for a single quantum circuit of NSGA-II setups utilizing different encodings. The “NSGA-II bin.” row presents the result for a setup with binary encoding with a population size of 32 individuals evolved over 32 generations. Analogically, the “NSGA-II int.” row shows the execution time of a setup with integer encoding ran with the same parameters.

IV. CONCLUSIONS

We propose the first multi-objective solution for quantum circuit optimization which employs NSGA-II. Using an optimized setup with integer encoding, allowing for repetitions of optimization passes, significantly improves the average fitness of the candidate solutions in a benchmarking suite compared to manually chosen optimization sets, with the number of gates improved by 94%, depth by 85.8%, entanglement ratio by 99.2%, critical depth by 96.8%, and parallelism by 39%. The compiler is easily adaptable to different requirements, allowing for the introduction of new objectives without the need to modify the optimization algorithm.

A. Future Work

The main downside of using MOEAs is the long execution time. As the future work on the Munich Quantum Compiler, we plan to explore various approaches with the potential of mitigating this issue:

a) *Predictor*: To prevent unnecessary execution time on converged candidate solutions, we propose training a *predictor model*, similar to the one introduced in [34], using the highly

optimized dataset generated by NSGA-II. This model can then predict the optimized objective values for a circuit, and reaching these values can serve as an additional stopping condition in the heuristic.

b) *Distributed Genetic Algorithm*: One way to increase efficiency would be to parallelize appropriate parts of the code, such as the application of passes to solution candidates and fitness evaluation. However, only certain parts of the code can be parallelized, as candidate selections must still be performed sequentially to ensure individuals are chosen from the entire undivided pool.

c) *Machine Learning*: On the one hand, a MOEA-based approach can generate Pareto-optimal candidate solutions but likely requiring higher execution times than, for example, ML-based techniques. On the other hand, ML-based approaches can yield results significantly quicker than MOEAs, but they tend to demand a substantial dataset of already highly optimized circuits. Access to the diverse training datasets generated from MOEA solutions can significantly improve the efficiency of ML-based methodologies, leading to superior results. That is, the results generated by the NSGA-II setup applied across a comprehensive set of quantum circuits could be utilized as a training dataset for a ML-based solutions. Successful application of ML models to the phase-ordering problem is well-documented in the literature [22], and it could potentially enhance the time-efficiency of the Munich Quantum Compiler.

REFERENCES

- [1] H. Liu, G. H. Low, D. S. Steiger, T. Häner, M. Reiher, and M. Troyer, “Prospects of quantum computing for molecular sciences,” *Materials Theory*, vol. 6, no. 1, p. 11, 2022. [Online]. Available: <https://doi.org/10.1186/s41313-021-00039-z>
- [2] A. Ajagekar and F. You, “New frontiers of quantum computing in chemical engineering,” *Korean Journal of Chemical Engineering*, vol. 39, no. 4, pp. 811–820, 2022. [Online]. Available: <https://doi.org/10.1007/s11814-021-1027-6>
- [3] P. S. Emani, J. Warrell, A. Anticevic, S. Bekiranov, M. Gandal, M. J. McConnell, G. Sapiro, A. Aspuru-Guzik, J. T. Baker, M. Bastiani, J. D. Murray, S. N. Sotiropoulos, J. Taylor, G. Senthil, T. Lehner, M. B. Gerstein, and A. W. Harrow, “Quantum computing at the frontiers of biological sciences,” *Nature Methods*, vol. 18, no. 7, pp. 701–709, 2021. [Online]. Available: <https://doi.org/10.1038/s41592-020-01004-3>
- [4] Y. Cao, J. Romero, and A. Aspuru-Guzik, “Potential of quantum computing for drug discovery,” *IBM Journal of Research and Development*, vol. 62, no. 6, pp. 6:1–6:20, 2018.
- [5] D. J. Egger, C. Gambella, J. Marecek, S. McFaddin, M. Mevissen, R. Raymond, A. Simonetto, S. Woerner, and E. Yndurain, “Quantum computing for finance: State-of-the-art and future prospects,” *IEEE Transactions on Quantum Engineering*, vol. 1, pp. 1–24, 2020.
- [6] M. Schulz, M. Ruefenacht, D. Kranzlmüller, and L. B. Schulz, “Accelerating HPC With Quantum Computing: It Is a Software Challenge Too,” *Comput. Sci. Eng.*, vol. 24, no. 4, pp. 60–64, 2022. [Online]. Available: <https://doi.org/10.1109/MCSE.2022.3221845>
- [7] A. Elsharkawy, X. M. To, P. Seitz, Y. Chen, Y. Stade, M. Geiger, Q. Huang, X. Guo, M. A. Ansari, C. B. Mendl, D. Kranzlmüller, and M. Schulz, “Integration of Quantum Accelerators with High Performance Computing - A Review of Quantum Programming Tools,” *CoRR*, vol. abs/2309.06167, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2309.06167>
- [8] QIR Alliance. (2021) QIR Specification. Also see <https://qir-alliance.org>. [Online]. Available: <https://github.com/qir-alliance/qir-spec>

Leveraging Machine Learning for Quantum Circuit Optimization

*Mathias Weiden
John D. Kubitowicz, Ed.*



Electrical Engineering and Computer Sciences
University of California, Berkeley

Technical Report No. UCB/EECS-2023-84

<http://www2.eecs.berkeley.edu/Pubs/TechRpts/2023/EECS-2023-84.html>

May 10, 2023

Abstract

In the era of noisy, non-fault tolerant, quantum hardware, reducing the number of operations in quantum programs is essential to ensure computations produce meaningful results. Unitary synthesis is a particularly promising technique which uses non-linear numerical optimization tools paired with heuristic-guided tree search algorithms to produce resource efficient implementations of quantum algorithms. This report illustrates how unitary synthesis can be used to optimize quantum circuits with many qubits. The TopAS algorithm demonstrates how minimizing both operation count and interaction complexity before mapping can produce better circuit implementations than the state of the art. This report also presents several discoveries that enable the use of machine learning in accelerating unitary synthesis. Namely, a canonical representation of unitary matrices, and an argument for the learnability of mappings between a specific class of unitary matrices and parameterized quantum circuits are presented. The QSeed algorithm shows how machine learning can be leveraged to accelerate the optimization of wide quantum circuits without sacrificing solution quality.

Leveraging Machine Learning for Quantum Circuit Optimization

Mathias Weiden

Spring 2023

1 Introduction

The idea to employ the seemingly unnatural powers of quantum mechanics for computation originated in Richard Feynman's 1981 lecture titled "Simulating Physics with Computers" [11]. Sixteen years later, in 1997, Simon's problem demonstrated the first oracle separation between quantum computers and probabilistic classical computers [25]. In that same year, Peter Shor showed that quantum computers can achieve super-polynomial speedups over classical computers for the prime number factoring problem [23]. Yet, the execution of these algorithms remains out of reach for modern quantum machines. The current Noisy Intermediate Scale Quantum (NISQ) era is defined by limited hardware that is extremely susceptible to environmental noise and decoherence [21]. Quantum error correction promises to elevate the field into an era of fault tolerance, but questions relating to the scaling of hardware capable of carrying out these complex error correction schemes remain unanswered [1]. Error mitigation, an ever widening suite of techniques used to reduce the effects of environment interference, serve as a NISQ era aid to these problems. Google's quantum supremacy experiments have confirmed the accuracy of a simple digital noise model, meaning just reducing the number of operations in quantum programs can dramatically improve the likelihood that computations are correct [3].

In this report, I will discuss common techniques used to optimize the operation count and depth of quantum circuits. I will focus on the use of unitary synthesis, a promising optimization technique. I will also present research efforts I have made in applying and improving unitary synthesis algorithms with machine learning.

This report is divided as follows: Section 2 surveys common techniques used for quantum circuit optimization and introduces unitary synthesis. Section 3 details the efforts I have made in applying machine learning to circuit optimization. Finally, Section 4 discusses promising future directions for the field of machine learning in circuit optimization.

2 Quantum Circuit Optimization

In this section, I will introduce preliminary concepts in quantum computing and the circuit optimization problem. I will also detail several widely used optimization techniques, and algorithms used for a specific style of optimization called unitary synthesis.

2.1 Preliminaries

Definition 2.1 (Qubits). A qubit is the base unit of quantum information. The qubit model assumes that a quantum system can be in any arbitrary superposition of two orthogonal states $|0\rangle$ and $|1\rangle$, so long as it satisfies

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad |\alpha|^2 + |\beta|^2 = 1$$

where $\alpha, \beta \in \mathbb{C}$. Given a system of n qubits and defining $N = 2^n$, the quantum state lies in a Hilbert space of size $\mathbb{C}^{N \times N}$. The composite state $|\phi\rangle$ of this n qubit system can be found by taking the n -fold tensor product

$$|\phi\rangle = \bigotimes_{i=1}^n |\psi_i\rangle.$$

Definition 2.2 (Quantum Gates). Operations on qubits must preserve the unit L2 norm of the quantum state. For this reason, operations on qubits must be unitary. Gates may act on one or multiple qubits, and they can be functions of real parameters. Two noteworthy quantum gates are the single qubit U3 gate

$$U3(a, b, c) = \begin{bmatrix} \cos(\frac{a}{2}) & -e^{ic} \sin(\frac{a}{2}) \\ e^{ib} \sin(\frac{a}{2}) & e^{i(b+c)} \cos(\frac{a}{2}) \end{bmatrix}$$

and the two qubit CNOT or CX gate

$$CNOT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

The gate set of $\{CNOT, U3\}$ is both universal for quantum computation and capable of creating any unitary matrix [19].

Quantum gates can be composed in parallel or sequentially. Parallel composition corresponds to a tensor product while sequential composition corresponds to a matrix multiplication.

Definition 2.3 (Quantum Circuits). A quantum circuit is a data structure that describes a quantum program. Qubits are represented as horizontal wires. Quantum gates are composed in sequences where time is assumed to flow from left to right. The single-qubit U3 gate appears as a squares, the multi-qubit CNOT gate spans the qubits on which it is applied. The width of a circuit

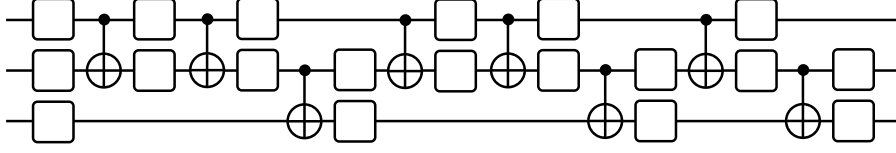


Figure 1: An example of a quantum circuit with U3 and CNOT gates.

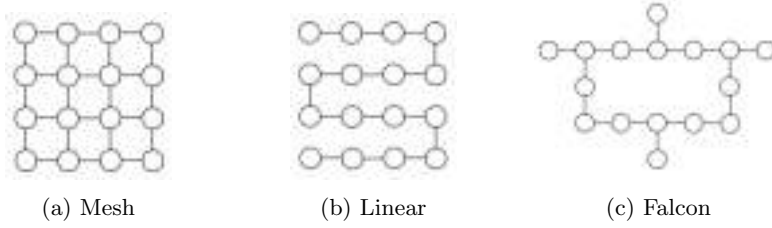


Figure 2: Examples of three common qubit topologies. The heavy hexagonal “falcon” style of topology is used in IBM’s superconducting machines. Meshes and lines are also common topologies.

refers to the number of qubits in the circuit, the depth refers to the number of time steps along the circuit’s critical path. Circuits that contain one or more parameterized gates (such as the U3 gate) are called Parameterized Quantum Circuits (PQCs). The unitary operation implemented by quantum circuit C is U_C . If C is a PQC parameterized by a vector $\theta \in \mathbb{R}^m$, then the unitary associated with C is denoted $U_C(\theta)$.

Although quantum circuits allow interactions between any pair of qubits, hardware is typically more limited. To account for this, a *qubit topology* can be used to describe more restrictive connectivities.

Definition 2.4 (Qubit Topology). A qubit topology is a graph $G = (V, E)$ where V is a set of vertices and E is a set of edges. Vertices $v \in V$ represent qubits, while edges $(u, v) \in E$ represent (hardware) allowed interactions between qubits.

The goal of quantum circuit optimization is to reduce the operation count and depth of quantum programs. This is necessary as NISQ era qubits are extremely sensitive to noise and are not error corrected. Certain quantum gates are more likely to introduce error than others. Multi-qubit gates, such as the CNOT, take far more time to execute and are far more error prone than single-qubit gates. Depending on the specific machine, some single-qubit gates may be more error prone and more difficult to calibrate than others [7]. Critically, the optimized version of a circuit must implement the same functionality as the original circuit.

2.2 Rule Based Circuit Optimization

Rule based circuit optimizations are a class of optimization techniques that make local gate transformations based on a pre-compiled series of transformation rules. Often, these sorts of algorithms are implemented with pattern recognition and replacement schemes [6]. Peephole optimization techniques (which are heavily used in classical compilation), where a window scans across the quantum circuit seeking opportunities for pattern replacement, are commonly used in practice [26, 2, 13].

This sort of optimization is fast, and is often very effective at reducing both circuit depth and gate counts, but, it has two inherent limitations. First, patterns for replacement must be identified beforehand, a process which is typically carried out by human observation. The space of possible circuit pattern-replacement pairs is vast, and the greatest opportunities for optimization have almost certainly not been fully explored. These replacement rules are human constructed and guided by heuristic and intuition. This restriction of peephole optimization has also been pointed out in classical compilers [4]. In response, tools such as CompilerGym [8] have been developed to use Reinforcement Learning (RL) to automate compiler optimization flows in classical compilers. The use of RL in peephole optimization for quantum computers has also been proposed.

Second, many of these rule based optimization techniques only consider very small width windows for optimization. Although fast, much opportunity for optimization is passed by because a more global perspective is not taken. Ultimately, the greatest issue with this particular aspect of rule based optimization is that typically small tweaks to imperfectly programmed algorithms are made. It is often the case that human written algorithm implementations are written in a way that is more understandable to humans, but this often results in non-optimal code. In these cases, it may instead be advantageous to rewrite instructions completely rather than make small changes to the existing code.

Both of these concerns are addressed in unitary synthesis optimization.

2.3 Unitary Synthesis

Unitary synthesis is a bottom-up method of implementing unitaries as quantum circuits. More formally,

Definition 2.5 (Unitary Synthesis). Given a unitary matrix $U \in \mathbb{C}^{N \times N}$, a unitary synthesis algorithm constructs a quantum circuit C that implements a unitary U_C satisfying

$$\|U - U_C\|_{HS} \leq \epsilon \quad (1)$$

where $\|\cdot\|_{HS}$ is the Hilbert-Schmidt norm defined as

$$\|A\|_{HS} = \text{Tr}(A^\dagger A). \quad (2)$$

Unitary synthesis describes the process of implementing a unitary as a quantum circuit in a way that allows for approximation. In the NISQ era, this is

a particularly interesting feature. Increasing the amount of approximation (or increasing the constant ϵ) typically reduces the number of gates in a circuit. So one can find a balance between error introduced by approximation, and error “saved” by gate reduction.

The Hilbert-Schmidt norm is one of the many metrics that can be used for this task, the most natural of which is the diamond norm [5]. However, the diamond norm is typically not used because of its extreme computational demands.

Synthesis can be performed with both parameterized and non-parameterized gate sets. This fact is known due to the Solovay-Kitaev theorem, which proves that discrete non-parameterized gate sets are capable of implementing any unitary to within some approximation factor [18]. For practicality, we typically assume a parameterized gate set such as $\{CNOT, U3\}$. Circuits expressed in this gate set are typically far less deep. Given a PQC, the process of solving for parameters to approximate a unitary is called *instantiation*. Expressed concretely,

Definition 2.6 (Instantiation). Given a target unitary U and a unitary $U(\theta)$ parameterized by $\theta \in \mathbb{R}^m$, instantiation solves the problem

$$\arg \min_{\theta} \|U - U(\theta)\|_{HS}. \quad (3)$$

The instantiation problem becomes exponentially more difficult as qubits are added to the circuit. This is because the size of the unitaries grow with $O(2^n)$. The parameterized unitary $U(\theta)$ is typically implemented with a PQC. Adding more gates to this circuit increases the number of parameters θ , which also adds difficulty.

2.3.1 QSearch

QSearch was among the first unitary synthesis algorithms [10]. QSearch approaches unitary synthesis by decomposing the problem into two operations: synthesis tree search and instantiation.

As input, QSearch accepts a qubit topology $G = (V, E)$ and a target unitary U . The edges $(u, v) \in E$ describe allowed operations in the synthesized circuit. Synthesis begins with a base circuit containing only U3 gates applied to each qubit. The circuit is instantiated to determine if it is a suitable approximation of the target unitary. At this point, the synthesis tree search process begins. A unit of gates is appended to the base circuit, one along each edge in the qubit topology. Circuits derived this way can be arranged in a tree, called the *synthesis circuit tree*. A node in the tree acts as a circuit prefix to its children; its children will only differ by gates appended to the end of the circuit. The process of synthesis can then be considered a search through the synthesis circuit tree. An example of synthesis circuit tree is depicted in Figure 3.

QSearch uses an A^* heuristic that considers the Hilbert-Schmidt distance to the target unitary as well as the current circuit’s gate count to guide the

synthesis tree search. Given a node in the tree, QSearch calls instantiation on each of the node’s children, and ranks them according to the heuristic.

Synthesis begins at the root node, then traverses through the synthesis circuit tree until a target circuit is reached. The number of instantiation calls made is at least as large as the number of circuits visited in this traversal. Other than wall-clock time, one can also measure the time needed to synthesize a circuit by measuring this path distance in the synthesis tree search.

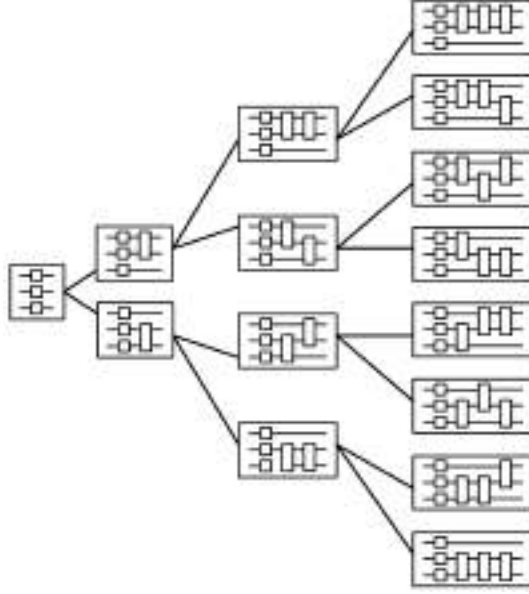


Figure 3: The synthesis circuit tree for a three qubit circuit with linear connectivity.

The details of QSearch’s CNOT gate count reduction compared to other rule-based methods can be found in [10]. QSearch’s bottom-up approach and allowed tunable approximation threshold result in unmatched gate reductions. This degree of optimization is possible because QSearch is capable of taking a broader view of the unitary being implemented. Redundancies and inefficiencies present in human written circuits are eschewed, and a globally efficient path to the target unitary is found. This can lead to rather non-intuitive results: Transverse Field Ising Model (TFIM) circuits, which simulate a material’s magnetization, can be compiled into constant depth circuits. These circuits iteratively append gates that compute the magnetization for new time steps in the simulation, but synthesis reliably finds a circuit with the same structure that summarizes the entire simulation.

Because it only applies gates along allowed edges in some qubit topology, circuits synthesized by QSearch can also be mapped to restrictive hardware.

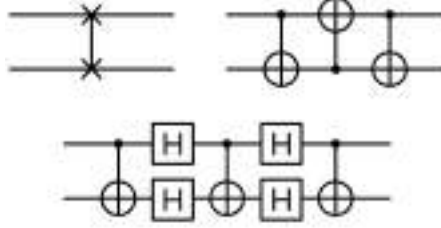


Figure 4: Three equivalent implementations of a quantum SWAP gate, which can be decomposed into three CNOT gates.

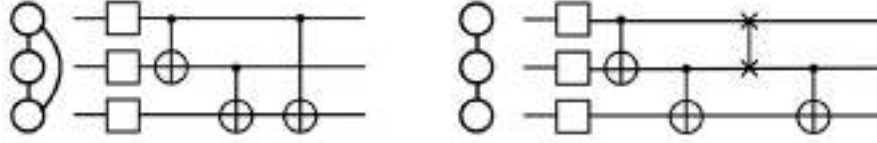


Figure 5: A quantum circuit that has been mapped to a restrictive qubit topology

Definition 2.7 (Mapped Quantum Circuits). A quantum circuit C is said to be mapped to a qubit topology $G = (V, E)$ if multi-qubit gates in C only occur along edges in E . A quantum circuit can be mapped to a qubit topology by the use of SWAP operations, which exchange the quantum state held by two qubits. The process of injecting SWAP gates into a circuit is called *routing*.

2.3.2 Synthesis Time Scaling

The run time of unitary synthesis algorithms such as QSearch grows exponentially as the number of qubits increases. Both stages, instantiation and synthesis tree search, contribute to this scaling. The time complexity of instantiation grows as $O(2^n)$ because the width of the target and parameterized unitaries grows exponentially. The run time of the tree search component also scales exponentially, as the minimum number of CNOT gates needed to implement an arbitrary unitary is $\frac{1}{4}[4^n - 3n - 1]$ [22]. Both components of QSearch's approach to synthesis result in synthesis times quickly becoming untenable.

2.3.3 Circuit Partitioning

In order to handle the run time scaling issues of bottom-up synthesis, wide quantum circuits must be broken into smaller subcircuits. This process is referred to as *circuit partitioning*.

Definition 2.8 (Circuit Partitioning). A partitioned quantum circuit is a quantum circuit C along with a sequence of sets S . Each set $B \in S$ is a set of gates

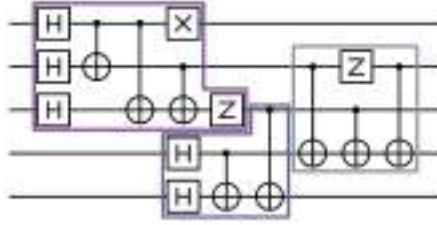


Figure 6: A five qubit quantum circuit that has been partitioned into three qubit subcircuits.

which appear in C . B is called a block, partition, or subcircuit. Blocks satisfy the properties

$$\forall B \in S \quad \bigcap B = 0 \quad \text{and} \quad \bigcup B = C \quad (4)$$

meaning each gate is a member of only one block, and all gates are a member of a block. To be a valid circuit partitioning, gates in block B must precede or succeed other gates in B . Valid partitionings only contain gates that act on interacting qubits. This means that the subgraph induced by some set of gates in B is connected. Typically, the width of partitions are limited to a value $k \in \mathbb{Z}$ called the *partition* or *block width*.

There are several strategies for partitioning quantum circuits. Their run times vary from growing with $\binom{n}{k}$, to linear in the number of gates. Should partitioning algorithms form blocks containing a similar number of gates? Should blocks greedily be formed so there are a few blocks with many gates? It remains unclear what the goal of a partitioning algorithm should be. The criteria for circuit partitioning is likely dependent on what optimization will be run, but this requires further investigation. Typically, it is the case that synthesis performs better when there are more CNOT gates in the circuit being synthesized. However, the time required to partition circuits is in practice far more important than this, so simple partitioning strategies are preferred.

To apply synthesis optimization to circuits wider than four (possibly five) qubits, circuit partitioning must be used. In a quantum compilation flow, the end goal is typically a series of instructions that can be run on real hardware. Optimized circuits must therefore be mapped to hardware. But, should optimization, such as synthesis, be applied before or after mapping?

2.3.4 QGo

QGo was the first wide quantum circuit synthesis algorithm [28]. QGo optimizes quantum circuits that have already been mapped to restrictive qubit topologies. This style of optimization is called post-mapping optimization, and the programmatic flow of QGo is illustrated in Figure 8. The guiding philosophy behind QGo is: use synthesis to combine redundant operations introduced by mapping with operations necessary for computation.

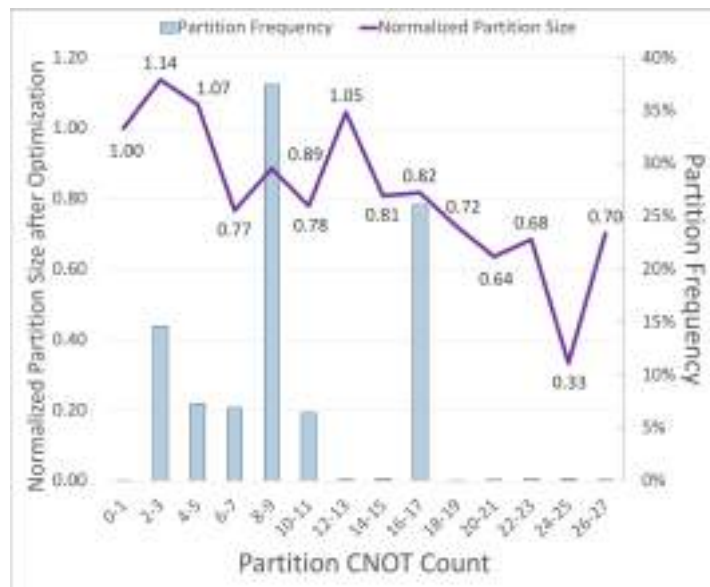


Figure 7: Synthesis performance vs. partition CNOT count. Also pictured is the distribution or frequency of blocks with that number of CNOTs in a set of partitioned benchmark circuits. A partition size of less than one indicates optimization occurred, lower is better.

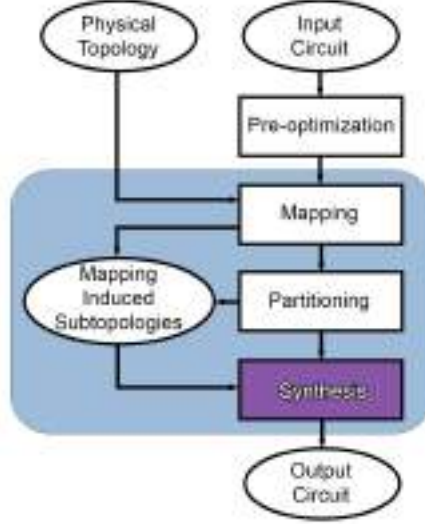


Figure 8: The program flow of a post-mapping synthesis algorithm such as QGo.

2.3.5 TopAS

The program flow of QGo begs the question, can further reductions in depth and gate count be achieved by applying synthesis before mapping? This “pre-mapping” style of optimization is illustrated in Figure 9. TopAS, a Topology Aware Synthesis algorithm for wide circuit optimization, is a pre-mapping synthesis algorithm [27].

The TopAS algorithm has one guiding principle: use synthesis to make the circuit mapping problem easier. Synthesis can be used to achieve this in two ways. First, reducing the number of operations in a circuit before mapping means fewer gates must be mapped, which typically results in better mapping algorithm performance. Second, restricting the complexity of operations in a circuit before mapping helps mapping performance, as simpler communication patterns require fewer mapping operations.

The first observation, that mapping algorithms do better when there are fewer gates, came by observing the performance of Qiskit’s implementation of the SABRE Swap algorithm on TFIM circuits mapped to mesh style topologies. The interactions in TFIM circuits are linear, meaning the circuit can be mapped to a linear qubit topology without the use of any SWAP operations. Because linear graphs are embedded within meshes, this implies that TFIM circuits can also be mapped to mesh topologies without the use of SWAP operations. However, our experiments showed that the SABRE mapping algorithm was unable to find a zero SWAP mapping of TFIM circuits, unless synthesis was performed beforehand.

The second observation, that mapping prefers simpler interactions, was

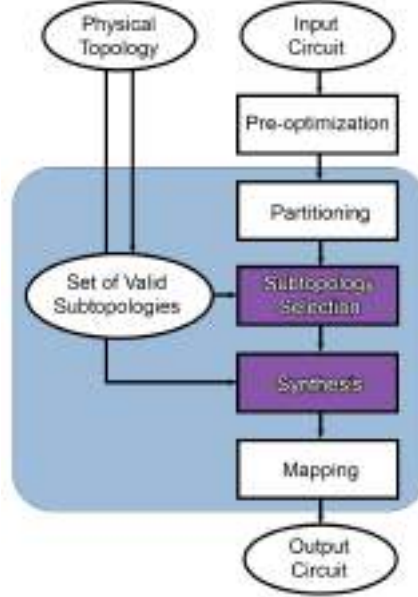


Figure 9: The program flow of a pre-mapping synthesis algorithm such as TopAS.

guided by a shift in perspective regarding mapping algorithms. Once partitions are formed, SWAPs can be divided into two categories: intra-partition SWAPs that exchange state within a partition, and inter-partition SWAPs that exchange qubits between partitions. Inter-partitions SWAPs are unavoidable. They describe movements of quantum information that are necessary for the correct sequence of interactions to take place. Intra-partitions SWAPs, however, describe a partition’s suitability for execution on that region of the qubit topology. If that partition were instead mapped to the region of the topology (called a subtopology) on which it is being executed, then no intra-partition SWAPs would be needed.

As mentioned in Section 2.3.1, QSearch accepts as input both a target unitary and a graph $G = (V, E)$ which describes the allowed interactions between qubits. This graph G is often denoted G_S and is called a *synthesis subtopology*. Subcircuits synthesized given a synthesis subtopology G_S are also mapped to that synthesis subtopology. Synthesis subtopologies differ from physical subtopologies in that physical subtopologies describe connected subgraphs in a qubit topology, whereas synthesis subtopologies need not have any connection to a real machine. Superconducting qubit machines are rarely fully connected, but synthesis subtopologies can be complete graphs. All order four synthesis subtopologies are illustrated in Figure 10.

The multi-qubit gates in a circuit also define a graph, called the *logical*

connectivity. Each edge in the logical connectivity corresponds to a multi-qubit gate between two qubits in a circuit. The weight of an edge in the logical connectivity corresponds to the number of times that interaction occurs. Logical connectivity graphs are often given the notation $G_L = (V, E)$, where E consists of weighted edges (w, u, v) for $w \in \mathbb{Z}$ and $u, v \in V$.

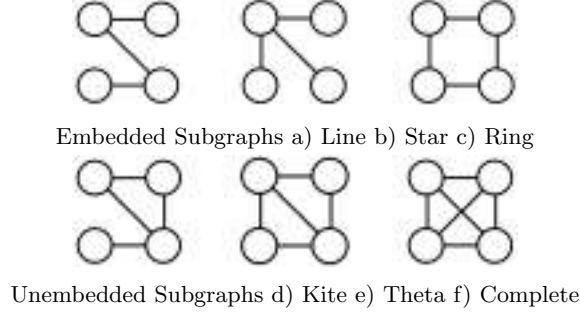


Figure 10: Possible order 4 synthesis subtopologies. The graphs a, b, and c are embedded within the mesh physical topology. Graphs d, e, and f are not. Only a and b are embedded in the falcon topology, while a is the only embedded subgraph of the linear topology.

When optimization is carried out before mapping, there is no a priori way of determining what synthesis subtopology each partition should be mapped to. Instead, TopAS uses a heuristic approach to select subtopologies for each partition. We observed that partitions synthesized to subtopologies that were similar to their unoptimized logical connectivity graphs typically saw greater reductions in gate counts. In order to quantify the similarity between a partition and a subtopology, we devised a graph similarity function inspired by machine learning kernel functions [15].

Definition 2.9 (Similarity Score). The similarity score function examines each edge in a synthesis subtopology G_S a logical connectivity G_L . From the edges, two vectors $v_L, v_S \in R^{k(k-1)/2}$ are computed. The vector v_S has a 1 at each location that corresponds to a present edge in G_S . Element i of v_L contains the weight w associated with edge i in G_L . The normalized inner product

$$\text{similarity}(v_S, v_L) = \frac{v_S^T v_L}{\sum_i v_L(i)} \quad (5)$$

quantifies the similarity between graphs G_S and G_L .

We also observed that using more restrictive synthesis subtopologies typically resulted in fewer routing operations in the final mapped and optimized circuits. Furthermore, selecting subtopologies embedded in the underlying qubit topologies also resulted in fewer total routing operations. These results are illustrated in Figure 11.

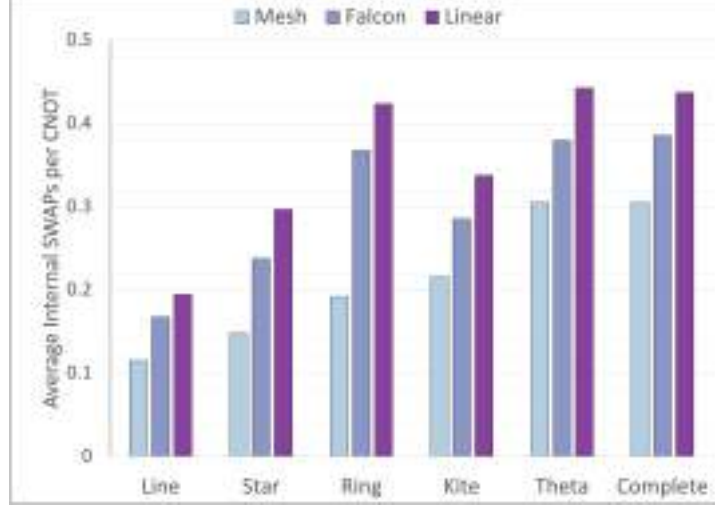


Figure 11: Simpler synthesis subtopologies result in fewer routing operations. Synthesis subtopologies embedded in the underlying qubit topology also result in fewer average routing operations. Internal SWAPs are intra-partition SWAPs.

So, the process of synthesis favors more densely connected synthesis subtopologies, as they are more similar to the logical connectivities of the circuits being synthesized, but the mapping algorithm performs better when more restrictive subtopologies are used. We devised a subtopology selection algorithm that combines and balances these two opposing demands. First, the set of allowed synthesis subtopologies is limited to those that are embedded in the underlying qubit topology. Then, the most restrictive of these subtopologies with the highest similarity score is used.

This subtopology selection policy allowed for TopAS to reach gate count reductions similar too, or just below those of QGo. This policy was able to reduce the number of intra-partition SWAPs, but something had to be done to reduce the number of inter-partition SWAP operations. The development of a *neighbor-aware* subtopology selection scheme was devised for this reason. Although partitioned subcircuits are synthesized independently, they are not executed in isolation. If a given partition can peek into a neighboring block to see what edges it uses, then the likelihood that a SWAP is inserted between blocks can be decreased by using edges that appear in both blocks. To accommodate this change, the weighted logical connectivity graph for each partition was modified to include shared edges in neighboring partitions. The inclusion of neighbor-awareness was the last piece of TopAS’ pre-mapping synthesis flow (see Figure 12). The subtopology selection algorithm is detailed in Algorithm 1.

Figures 13 and 14 show the CNOT gate count reduction and depth reduction of TopAS optimized circuits compared to Qiskit, Pytket, and QGo. Benchmarks

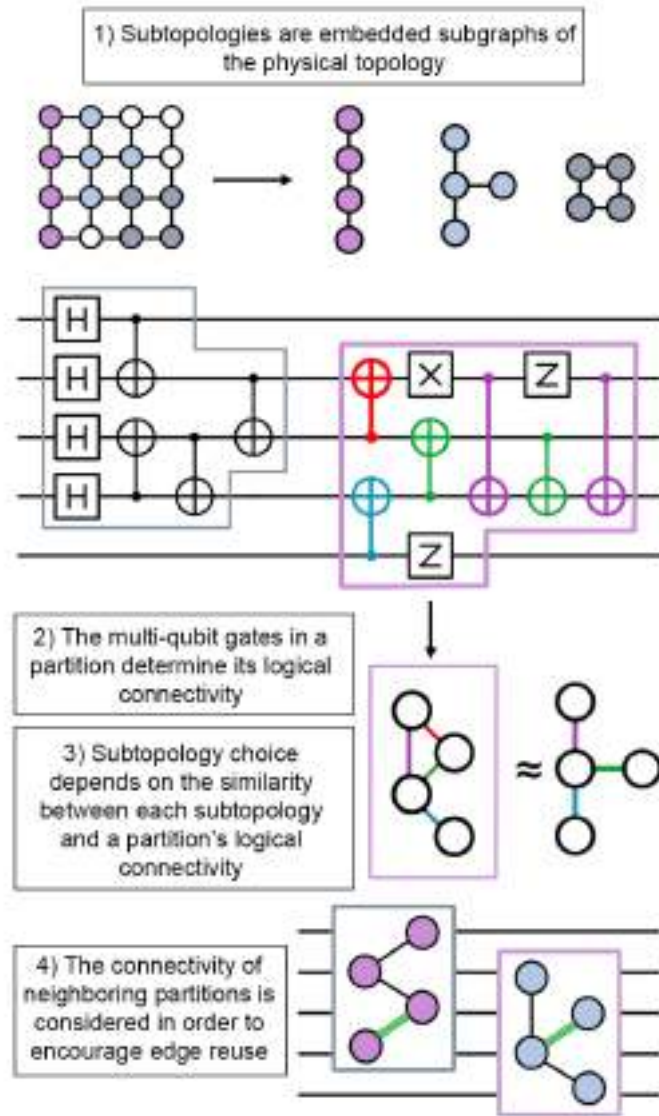


Figure 12: A high level view of the TopAS style implementation of pre-mapping synthesis optimization.

Algorithm 1 Subtopology Selection

Input: graph G_L , set of graphs $G_{neighbors}$, $bias$ function

Output: graph G_S

```
for all graphs  $G_n$  in  $G_{neighbors}$  do
  if  $shared\_edge$  in  $G_n$  and in  $G_L$  then
     $G_{overlap} \leftarrow shared\_edge$ 
  end if
end for
 $v_L \leftarrow$  edge weight vector for  $G_{overlap} \cup G_L$ 
 $candidates \leftarrow$  order  $\leq 4$  subgraphs in physical topology
for all graphs  $G_P$  in  $candidates$  do
   $v_P \leftarrow$  indicator vector for  $G_P$ 
   $score = \text{similarity}(v_P, v_L)$ 
  if  $bias(G_P) \times score > high\_score$  then
     $high\_score \leftarrow bias(G_P) \times score$ 
     $G_S \leftarrow G_P$ 
  end if
end for
return  $G_S$ 
```

are labeled so that the number following the benchmark name indicates the number of qubits in the circuit. The CNOT gate count and depth results reported are normalized by the values obtained for circuits optimized then mapped with Qiskit’s SABRE SWAP mapping algorithm.

TopAS is able to, on average, reduce CNOT count compared to Qiskit by 35.9% and Pytket by 24.3% when targeting a mesh qubit topology. For the falcon/heavy hexagonal qubit topology, TopAS outperforms Qiskit by 33.4% and Pytket by 19.1% on average. TopAS shows even greater reductions in circuit depth, averaging 38.6% and 37.6% shallower circuit compared to Qiskit and Pytket across all benchmarks.

TopAS is able to produce circuits with lower CNOT counts and depth than QGo. On average, TopAS outperforms QGo by 11.5% in CNOT count and 34.3% in depth when compiling to a mesh qubit topology. For the heavy hexagonal topology, TopAS does as well as QGo (only an average of 0.8% better) in CNOT count, but maintains an advantage of 37.3% in average depth reduction.

The TopAS project highlights an interesting fact: unitary synthesis is still able to outperform rule based optimization flows such as Qiskit and Pytket, even when a global view of the unitary being optimized is too large to do direct synthesis on. Synthesizing partitioned circuits may not produce the same results as direct, complete, synthesis, but by operating in a bottom-up manner, results are still impressive.

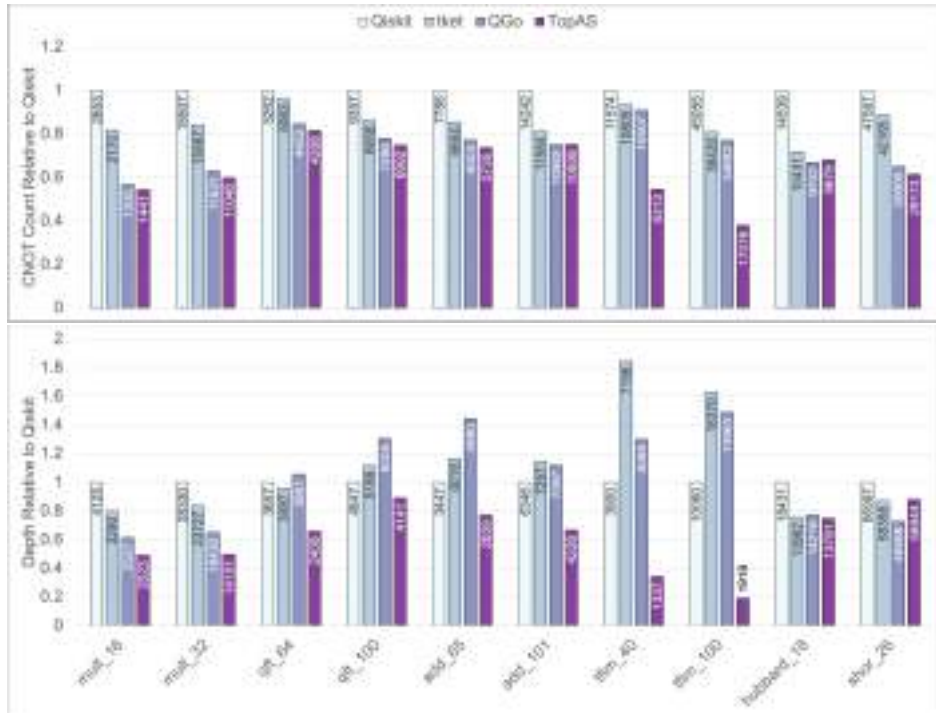


Figure 13: Comparison of relative CNOT count (top) and depth (bottom) for circuits mapped to the mesh qubit topology. CNOT count and depth is shown relative to optimized circuits mapped using Qiskit’s SABRE Swap algorithm. Both lower depth and CNOT count correspond to improved output fidelity on noisy machines.

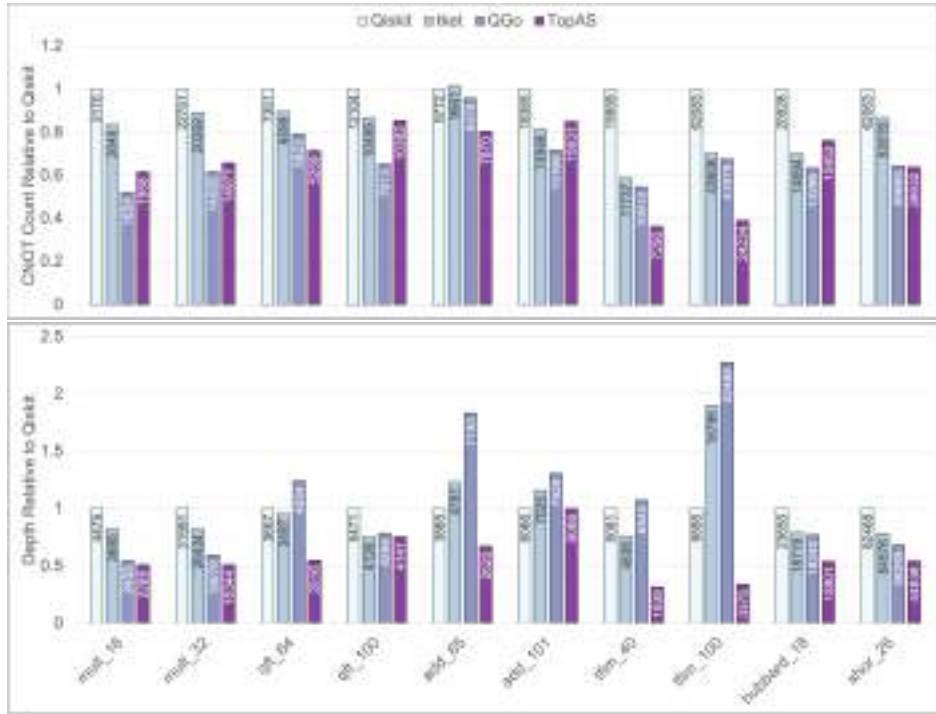


Figure 14: Comparison of relative CNOT count (top) and depth (bottom) for circuits mapped to the IBM style falcon/heavy hexagonal qubit topology. CNOT count and depth is shown relative to optimized circuits mapped using Qiskit’s SABRE Swap algorithm. Both lower depth and CNOT count correspond to improved output fidelity on noisy machines.

3 Using Machine Learning for Circuit Optimization

In this section, I will justify the use of machine learning in circuit optimization and present recent work in this field. I will discuss my findings relating to the dimensionality of unitary datasets of interest. Finally, I will discuss QSeed, a seeded synthesis algorithm that leverages machine learning to accelerate unitary synthesis optimization.

3.1 Related Work in Reinforcement Learning

Reinforcement Learning (RL), is a machine learning paradigm where agents learn policies based off rewards received from interacting with an environment. In classical compiling, RL is becoming a popular technique for discovering optimal policies for applying peephole optimizations. The CompilerGym and CompilerZoo tools both provide methods of training RL agents to guide compilation flows in order to minimize the number of machine instructions in programs [8]. RL gyms for quantum compilation do not exist, but they almost certainly will in the future, as this style of optimization has proven to be very powerful in the classical compilation space.

The work of [12] demonstrated how RL can be used to learn good peephole style replacement rules for randomly generated circuits. To my knowledge, this is the first project to have applied machine learning to quantum circuit optimization. I believe that RL policies trained to perform peephole optimization may ultimately become the de facto method of performing circuit optimization. However, I believe a weakness of this particular project is that these agents were trained to discover optimizations in random circuits. As I will show, random circuits, especially random unitaries, do not necessarily correspond to cases that we can or desire to optimize.

3.2 Dimensionality of Unitary Datasets

The manifold hypothesis is a conjecture of deep learning that posits that neural networks operate by learning low dimensional manifolds in high dimensional spaces. Although far from the first to propose the manifold hypothesis, the authors of [24] showed how a digit that is rotated, translated, or stretched still lies along the same manifold. Actually learning the shape of these manifolds is hard, but we can begin to identify some of their properties by looking at high level linear approximations of data. The Principle Component Analysis (PCA) specifically is a widely used technique for doing this [17]. PCA changes the basis of a set of data so that the new directions, or Principle Components (PCs), point in the directions of maximum variance in the dataset. When the dataset can be largely explained by a few PCs, neural networks typically perform well, as patterns of interest in the data are easier to learn in these cases [14].

For a fixed number of qubits, the space of unitaries is large. To present support for this claim, we can conduct an experiment to measure the dimensionality

of two sets of unitaries. First we prepare a set of random unitaries generated by PQC. These unitaries are generated by taking one of twelve candidate PQCs with parameters $\theta \in \mathbb{R}^m$, and uniformly sampling parameters $\theta_i \sim U[-\pi, \pi]$. Each of the twelve PQCs generate 10,000 unitaries. Next, we prepare a dataset that consists of about 203,000 unitaries taken from partitioned subcircuits. For each unitary, random and partitioned, we extract a *Pauli coefficient vector*.

Definition 3.1 (Pauli Coefficient Vector). Given a unitary $U \in \mathbb{C}^{2^n \times 2^n}$, there exists a Hermitian matrix $H \in \mathbb{C}^{2^n \times 2^n}$ such that

$$U = e^{iH}. \quad (6)$$

This matrix H can be decomposed into the linear combinations of all 4^n Pauli matrices. Concretely, we can construct a vector of Pauli matrices

$$\sigma_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \sigma_2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \sigma_3 = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \sigma_4 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$\vec{\sigma} = [\sigma_1 \otimes \sigma_1 \cdots \otimes \sigma_1 \quad \sigma_2 \otimes \sigma_1 \cdots \otimes \sigma_1 \quad \dots \quad \sigma_4 \otimes \sigma_4 \cdots \otimes \sigma_4]$$

such that,

$$H = \alpha \cdot \vec{\sigma} \quad (7)$$

where $\alpha \in \mathbb{R}^{4^n}$ and $\alpha_i \in (-\pi, \pi]$. The vector α is called the Pauli coefficient vector. The Pauli coefficient vector is a convenient way to represent matrices as representing a unitary matrix requires 4^n *complex* values, while the Pauli coefficient vector requires only 4^n *real* values.

After the Pauli coefficient vectors have been collected for each unitary, we can do two PCAs; one for the random and one for the partitioned unitaries. Figure 15 shows the cumulative variance explained as a function of the number of principle components considered. Because a small number of PCs describe the majority of the variance in the partitioned dataset (the first 16 explain 72.2%, the first 32 explain 90.7%), we expect patterns of interest to be learnable by a neural network. The random dataset's variance is spread even across its PCs. We would not expect that learning patterns of interest in this dataset is easy to do.

A natural way to think of what a PCA shows is how compressible a dataset is. For the partitioned unitary case, we could project our dataset into the space spanned by the first 32 principle components, then reconstruct the original dataset with 90.7% accuracy. To achieve this same level of accuracy on the random unitary dataset, the first 57 principle component directions must be kept. In this sense, the random unitary data is much higher dimensional than the partitioned unitary case.

This analysis shines a light on an important insight, random unitaries are very different from unitaries that come from circuits that do real computation. Random circuits (that implement random unitaries) are not representative of what we expect to see in real benchmarks.

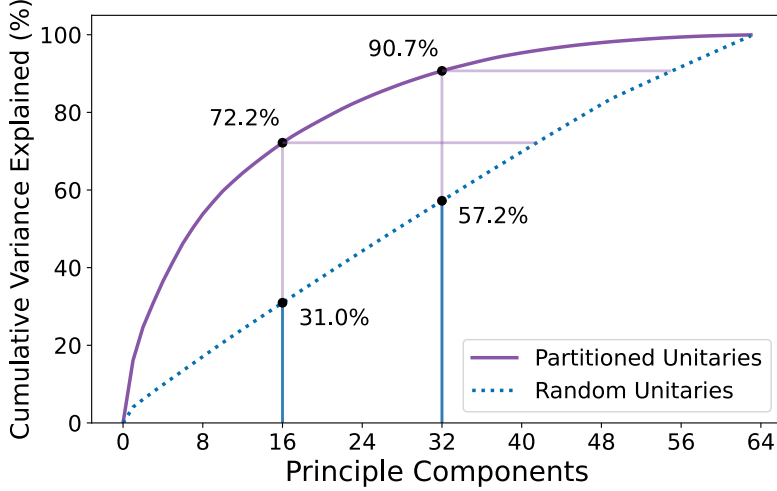


Figure 15: Cumulative variance explained by Principle Components of two datasets of three qubit unitaries. One is randomly generated, the other is made of partitioned unitaries from circuits in Table 1. The PCA of the partitioned unitaries implies these unitaries lie on a low dimensional manifold; there are patterns that can be identified in this dataset. The random unitaries do not exhibit identifiable low dimensional patterns. Learning on random unitaries is difficult, learning on partitioned unitaries is easier.

3.3 Learning a Mapping from Unitaries to Quantum Circuits

Now that I have established that we expect to be able to learn patterns of interest in partitioned unitaries, what can we do with this information?

Imagine that we have a series of n qubits, and a set of unitaries that these n qubits can implement $\mathcal{U}(2^n)$. We can also imagine a set $\mathcal{PQC}(n, K)$, which represents all PQCs with n qubits and K or fewer CNOT gates. We want to find a mapping

$$\phi : \mathcal{U}(2^n) \longrightarrow \mathcal{PQC}(n, K) \quad (8)$$

such that all $U \in \mathcal{U}(2^n)$ are mapped to the $PQC \in \mathcal{PQC}(n, K)$ that implements U with the fewest CNOTs. We assume that the set $\mathcal{PQC}(n, K)$ is large enough so that some PQC implements U with error at most ϵ .

Why might learning this mapping be useful? Because if we can, then we can replace the expensive synthesis tree search process described in Section 2.3.1 with much less expensive ML inference! Before this is possible, there are several things that must be addressed.

3.3.1 Expressivity of Parameterized Quantum Circuits

Much work has been done to quantify the expressivity of parameterized quantum circuits [16]. Many unitaries can be implemented by any given PQC, and many PQCs can implement any given unitary. It is important to note this because it makes the problem of finding the mapping $\phi : \mathcal{U}(2^n) \rightarrow \mathcal{PQC}(n, K)$ difficult. This explains why the PCA of random unitaries (Figure 15) shows that the variance is spread evenly across the principle components.

Another added difficulty of working with such expressive circuits is that it makes the problem of instantiation (Equation (3)) more difficult. To explain how we know this, we can again return to the example of the random unitary dataset. Here, we directly generate unitaries from a given PQC by randomly sampling parameters. But, if we have the unitary and the PQC, asking the instantiation function to solve for the parameters of that circuit that yield that unitary often fails. This means that even when we know the correct mapping of random unitaries to the PQCs that generated them, it is difficult to verify that this mapping is correct.

3.3.2 Canonical Representation of Unitaries

If two unitaries differ only by a global phase, states transformed by the two unitaries will always yield the same distribution of measurement results [19]. This means that for all intents and purposes, unitaries that differ only by a global phase are the same. However, if we want to apply numerical methods to unitaries, it is important that there is a common, canonical way of representing unitaries. Formally, we need a function f such that for unitaries U, U^* ,

$$\forall \theta \in \mathbb{R} \quad U = e^{i\theta} U^* \implies f(U) = U^*.$$

If $f(U) = U^*$ then the matrix U^* is the canonical representation of U . The algorithm for computing U^* is given by Algorithm 2. Performing these steps

Algorithm 2 Canonical Unitary Transformation

Input: unitary $U \in \mathbb{C}^{N \times N}$,

Output: unitary $U^* \in \mathbb{C}^{N \times N}$

Find $e^{i\theta} = |U|$

Compute $V = e^{i\theta} U = e^{i2\pi k/N} U^* \in \mathbb{SU}(N)$

$e^{i2\pi k/N}$ is the complex phase of first non-zero element in V

$U^* = e^{-i2\pi k/N} V$

return U^*

ensures that each unitary is mapped to a canonical special unitary. To my knowledge, this is the first algorithm that finds a canonical form for unitaries.

Previous works stop at the special unitary conversion step. Just doing this part is not enough to build global phase resilience into methods that do numerical computation on unitaries. I noticed by looking at many random examples of

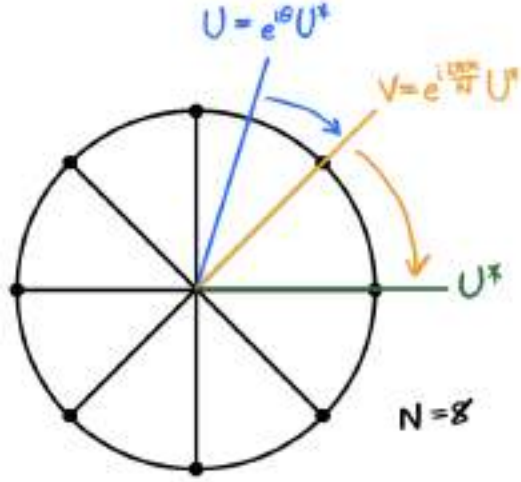


Figure 16: An illustration of the two steps in the canonical unitary transformation process.

this process, that for an $N \times N$ unitary, there were N different special unitaries that appeared. This lead me to notice that converting to a special unitary only removes the small phase differences between the different roots of unity (the blue arrow in Figure 16). In order to find a real canonical form, we need to always go to the same root of unity. A unitary will always have at least one non-zero element in each row and column. If we ensure that the first non-zero element of the first row has zero complex phase, we will always arrive at the same root of unity, no matter which root the special unitary conversion step took us to (orange arrow in Figure 16).

Even though this process is simple, I believe that it is a pretty significant tool to have. Unitary synthesis algorithms, and other numerical methods that work on unitary matrices, must choose cost functions that are resilient to global phase, otherwise intricacies of the numerical optimization process may skip over valid solutions to problems. Having a canonical representation is therefore significant because it broadens the scope of what can be done with unitary matrices. Cost functions need not be phase resilient anymore.

3.4 QSeed

Equipped with a canonical representation of unitary matrices and the knowledge that patterns of interest could be learned for partitioned unitary matrices, I began work on using machine learning to learn a mapping between unitaries and the PQCs that implement them. This work culminated in an algorithm called QSeed, a seeded synthesis algorithm that uses ML to predict seed circuits from which to begin synthesis. QSeed is able to achieve dramatic speedups compared

to QSearch because it decreases the time spent searching the synthesis circuit tree.

3.4.1 Why use ML for this task?

Before detailing the development behind QSeed, first I want to address why using ML might be advantageous for this task. If we want to learn a mapping from unitaries to PQCs, could we instead just observe many instances of unitaries that can implement PQCs and memorize those results? This approach would construct a database of previously seen unitary-PQC pairs that we could quickly interrogate. The problem with this approach is that it only allows us to do this mapping for unitary-PQC pairs that have already been observed.

Why might ML work any better than this database? A learned mapping is more robust than this database approach because the point of the learned mapping is that it can generalize to cases that have not yet been seen. This is useful because the number of possible unitaries we can see is vast (even when limiting ourselves to unitaries from partitioned circuits), and surely we cannot capture everything we might expect to see in a training dataset.

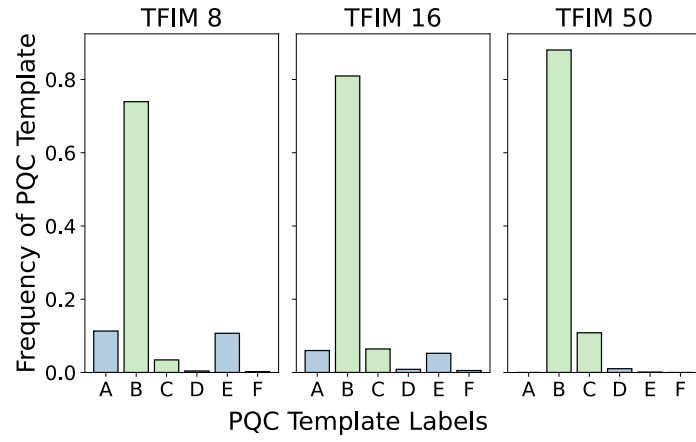
Why might we expect that learning this mapping is possible? We expect this is possible because there are common patterns that appear in similar quantum circuits. When synthesizing generated circuits from the same “family” but of different widths, the distribution of PQC templates that appear has similarities. This is illustrated in Figure 17. This figure shows how synthesized TFIM circuits of various widths all have similar looking distributions of used PQC templates, which are also illustrated.

Furthermore, circuits not from the same family, but that are still in some sense related, share common structure as well. Figure 18 shows how both QFT and Shor style modular exponentiation circuits have distributions of selected PQCs that look similar. The modular exponentiation (Shor) distribution look like the QFT distribution, with a extra mode around the template labeled “P”. The existence of these commonalities means that there is in fact some underlying structure or pattern to the mapping of unitary to PQCs. Now we can rest assured that there are in fact patterns of interest in our data that we can train a model to recognize.

3.4.2 Seeded Synthesis

If this mapping between unitaries and PQCs is learned, then the synthesis circuit tree search process could be almost entirely replaced. In actuality, it is unlikely that this mapping will be learned perfectly, so instead of immediately using the first PQC proposed, we simply begin synthesis from that circuit.

Definition 3.2 (Seeded Synthesis). Seeded synthesis accepts three inputs, a target unitary U , a qubit topology $G = (V, E)$, and a seed circuit C_{seed} . Instead of beginning the synthesis tree search process at the root node of the tree (which contains only single-qubit gates), synthesis begins at the circuit C_{seed} . CNOT



(a) PQC templates that appear in synthesized TFIM circuits of various widths. Circuits of different widths share characteristic modes around PQC's B and C.

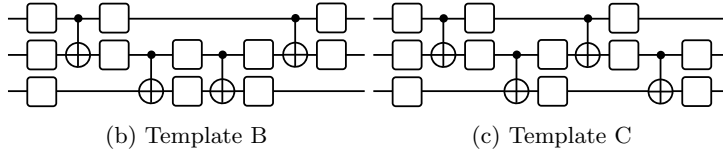


Figure 17: Histograms and examples of commonly repeated PQC templates that appear in optimized TFIM circuits.

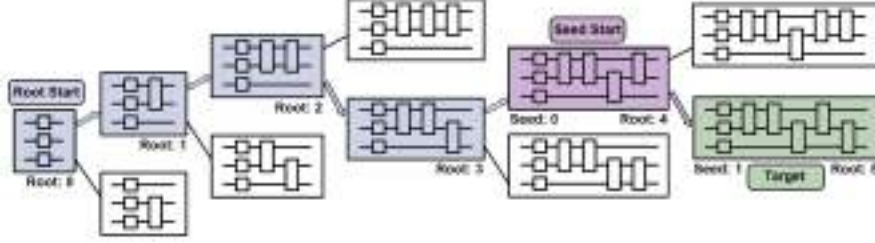


Figure 19: Seeded synthesis allows for the synthesis tree search process to begin at circuits closer to the target circuit node.

gates can now be both added and removed, which means that the tree search progresses both towards leaf nodes and the root node.

The purpose of the learned mapping between unitaries and PQCs is now simply to propose seed circuits from which the synthesis process can begin. Figure 19 shows why seeded synthesis can provide speedups compared to normal synthesis. QSearch always begins synthesis at the node marked “Root Start.” In this case, the path between the root and the “Target” has distance five. This means if the shortest path is taken, then at least six instantiation calls must be made. If we instead begin synthesis at the node marked “Seed Start,” because the path to the target has distance one, only two instantiation calls must be made. This example illustrates why we might expect to see large speedups in a seeded synthesis algorithm compared to QSearch.

3.4.3 Dataset Preparation

I took a supervised learning approach to learn the mapping in Equation (8). After collecting unitaries, computing their canonical forms (Section 3.3.2), and extracting their Pauli coefficient vectors (Definition 3.1), these datapoints still lacked labels. I approached this problem by enumerating all PQC templates with three qubits and up to eight CNOTs, then synthesizing each unitary. Each unitary was then paired with the integer label that corresponded to the PQC that implemented it. Because some unitaries appeared multiple times and some PQCs were more frequent than others, I normalized this dataset by removing repeated unitaries and randomly discarding examples from over represented PQCs. This ensured that whatever model I trained would not simply learn that some PQCs were more prevalent than others, but actually learn to propose PQCs based off the unitaries themselves.

The process of producing these labels is synthesis, and therefore labeling these unitaries is expensive.

3.4.4 The Recommender Model

A recommender is any function (learned or not) that proposes seed circuits for seeded synthesis. I chose to implement a recommender model with a neural network. More specifically, I chose an autoencoder style architecture. The results of the PCA of partitioned unitaries in Figure 15 motivated this choice. A purely linear projection of only half the total dimensionality of the original space captured the vast majority of the variance in the partitioned unitary dataset. I therefore believed that the underlying structure of these unitaries was simple, and that compressing their representation could add a degree of robustness to noise [14]. As the process of labeling the data for this task was expensive, I also wanted to ensure I could use unsupervised techniques to pre-train the model. Autoencoders can easily be pre-trained on the task of input reconstruction, so the recommender model was pre-trained to reconstruct input unitaries.

The output of the recommender model is a large vector where element i corresponds to how likely the model believes the PQC with label i is able to implement the input unitary. There were a total of 1199 possible output PQCs to select. To state this concretely, the output of the model is a vector $y \in \mathbb{R}^{1199}$, where

$$\text{softmax}(y_i) \in [0, 1] \quad (9)$$

represents the model’s confidence that PQC i can implement the input.

As this was meant primarily as a proof of concept, I elected to limit the width of unitaries and circuits to three qubits. I began by training a model which took as input the Pauli coefficient vector associated with each unitary, which is a vector in $\mathbb{R}^{64}$. Later on, I discovered that the process of converting unitaries into Pauli coefficient vectors was extremely expensive, and so the architecture was changed to instead operate directly on the canonical form of unitaries. As elements of the unitaries are complex, each element of the input unitary was split into real and imaginary parts. These new vectors $x \in \mathbb{R}^{128}$ were used as the new input.

The training and test datasets consisted of circuits from twelve different families of generated circuits. Each family contained multiple instances of that algorithm implemented at various circuit widths. The training and test data splits are shown in Table 1. Note that no Grover’s algorithm circuits of any size appear in the training dataset. The whole flow of the QSeed algorithm is depicted in Figure 20.

3.4.5 Comparing QSeed to QSearch

Recall that the goal of the QSeed algorithm is to accelerate unitary synthesis by avoiding spending too much time traversing the synthesis circuit tree. We can measure this by directly measuring the time it takes to synthesize circuits with both QSearch and QSeed. Importantly, we do not want to degrade the quality of our results for the sake of speed. Therefore, we must also ensure that

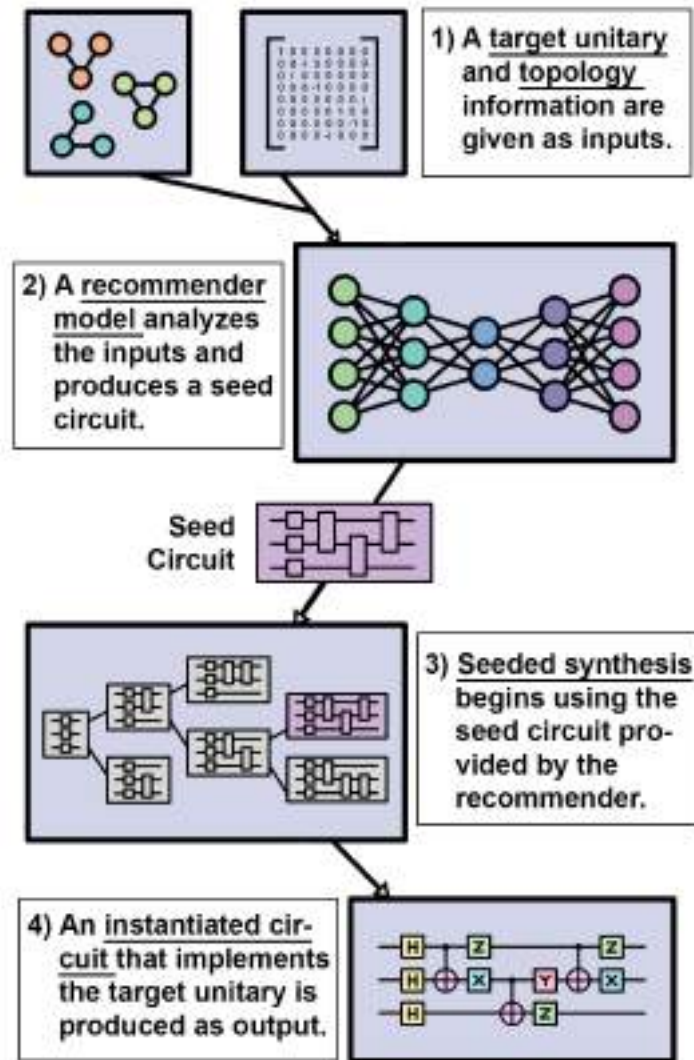


Figure 20: A high level depiction of the QSeed algorithm.

Circuit Name	Training Widths	Test Widths
add	17, 65	41
grover	-	10
heisenberg	4, 6, 7, 8, 16, 32, 64	5
hhl	8	6
hubbard	4, 18, 50	8
mult	8, 32, 64	16
qae	11, 33, 101	65
qft	3, 4, 8, 16, 32	64
qml	4, 25, 60, 108	128
qpe	6, 10, 14	18
shor	16, 32	64
tfim	3, 4, 5, 6, 7, 8, 16, 32,	64
vqe	12, 14	18

Table 1: Split of circuits withheld from recommender training for testing.

the number of CNOTs in the QSeed synthesized circuits is close to the number of CNOTs in the QSearch synthesized circuits.

Figure 21 summarizes the results of experiments run to compare the time to- and quality of- solutions for circuits optimized with QSearch and QSeed. Each of these circuits is first mapped with Qiskit’s SABRE SWAP mapping algorithm, then optimized with QGo style post-mapping synthesis (Figure 8). QSeed achieves an average speedup of $2.4\times$ compared to QSearch, while only suffering an average of 1.2% increase in CNOT count. Note that these test circuits contain unitaries not included in the training of the recommender model. The 64 qubit modular exponentiation circuit, a core component of Shor’s algorithm, has a speedup of $3.7\times$ (nearly 8000 seconds). The 18 qubit VQE circuit achieves a speedup of $2.3\times$. This particular benchmark is worth noting as classical-quantum hybrid algorithms have strict compilation time demands [20]. It is also worth noting that QSeed’s synthesis times include the time required to do inference/make seed recommendations.

Most impressive are the results for the compilation of the 10 qubit Grover’s algorithm circuit. No unitaries for the family of Grover’s algorithm circuits were included in training, so this benchmark is entirely alien to the recommender model. Still, a speedup of $3.2\times$ is achieved, and there are actually fewer CNOTs than in the QSearch optimized version.

3.4.6 Evaluating the Recommender Model

The results presented in Figure 21 demonstrate that using the recommender model and seeded synthesis provides substantial speedups over QSearch. However, one can imagine other recommender schemes. To ensure that our recommender model is performing as expected, we can do a deeper analysis of what is happening each time a partition is synthesized. To do this, we can observe the

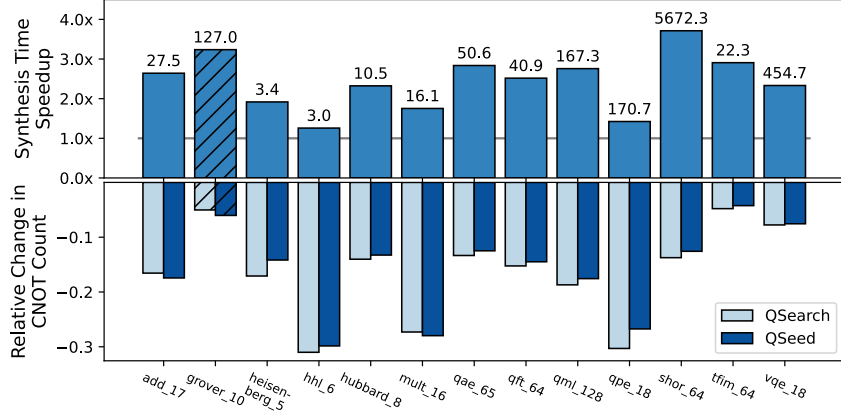


Figure 21: Comparing QSeed and QSearch. The top graph wall-clock synthesis time speedups. Larger speedups are better. The bottom graph shows relative changes in CNOT gate count relative to Qiskit level 3 optimized circuits. Lower relative change in CNOT counts are better.

number of instantiation calls made and the number of CNOTs in the optimized version of each block.

Recall that the number of instantiation calls is a proxy of how long synthesis will take, because each circuit along a path through the synthesis circuit tree must be instantiated. Figure 22 shows the distribution of the number of instantiation calls for the QSeed, randomly seeded, and QSearch approaches. The randomly seeded scheme simply randomly selects one of the 1199 possible seeds. QSearch acts like a seeded synthesis algorithm that always proposes the root as a seed. QSeed’s recommender scheme is able to propose seeds that require only a single instantiation call almost 90% of the time. The randomly seed case also typically requires very few instantiation calls. By measuring the expected number of instantiation calls, we can see that on average, QSeed and the randomly seeded scheme require almost $6\times$ fewer instantiations on average. This is why QSeed is able to synthesize unitaries faster than QSearch.

Figure 23 shows the number of CNOTs in the optimized blocks divided by the number of CNOTs in the original blocks. A value of 1 here indicates that no reduction in CNOTs has occurred, whereas a value less than 1 indicates the optimized subcircuit has fewer CNOTs. Lower values are better. Values higher than 1 indicate growth, which is undesirable. The expected reduction for QSeed closely matches that of QSearch. This is why we can expect that QSeed produces circuits with a similar number of CNOTs as QSearch. The randomly seeded scheme may offer speedups, but it averages an increase of 19% in CNOTs.

The randomly seeded results for these two experiments indicate that a randomly proposed PQC is likely to be capable of implementing a unitary, but

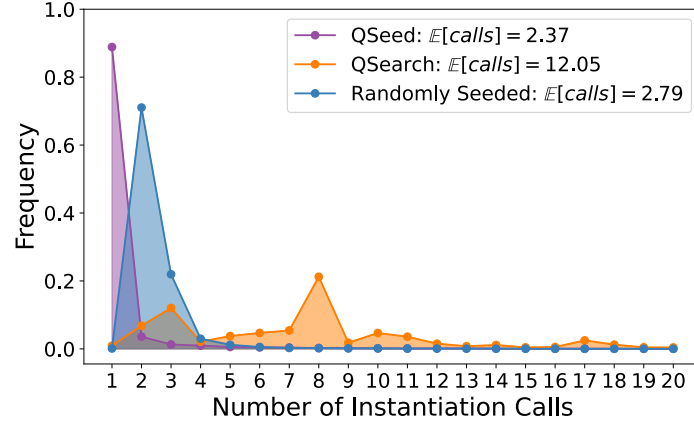


Figure 22: Distribution of instantiation calls made for various recommender schemes. QSeed uses the autoencoder style recommender model while the Randomly Seeded strategy randomly picks a seed circuit. Both are faster than QSearch as seen by the lower expected number of instantiation calls. The X-axis only shows up to 20 instantiation calls, but QSearch requires as many as 1704.

it is likely to be far deeper than is necessary. These experiments also show why QSeed is able to achieve speedups over QSearch while maintaining solution quality.

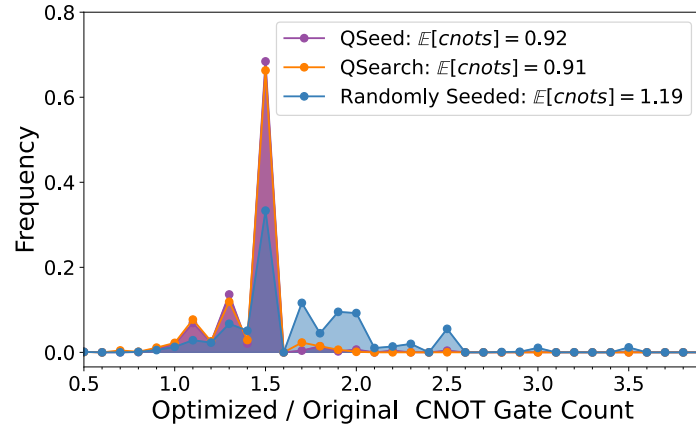


Figure 23: Distribution of relative CNOT gate counts for various recommender schemes. A value less than one corresponds to there being fewer gates in optimized block compared to the original block. Lower is better. The relative CNOT counts of QSeed closely match those of QSearch. The Randomly Seeded scheme results in circuit growth, as seen by the expected value being larger than 1.

4 Discussion and Future Directions

In this section, I will offer further discussion and describe future research directions that could further leverage machine learning for quantum circuit optimization.

4.1 Increasing Unitary Widths

Although promising, the results presented in Section 3 are limited to three qubit unitaries. A PCA of partitioned unitaries with four and five qubit widths show similar structure; therefore we expect that learning is still possible in these cases. Increasing the width of the unitaries operated on by the QSeed algorithm is limited by the lack of a labeled dataset of larger unitaries. Compiling such a dataset for the three qubit case was possible, but synthesis is much slower in these wider cases, so labeling becomes much more difficult.

When the width of the unitaries in the QSeed algorithm increase, so does the number of possible output PQC. This is a problem because it means the enumeration process used to exhaustively label PQCs is no longer viable. Luckily, it appears that not all PQCs are equally important. In the dataset of 203,000 partitioned unitaries. Only 415 appear more than 10 times. Of these, only 220 appear 100 or more times. I therefore expect that the number of PQCs that are widely used in the larger width cases to be similarly small.

The results presented in Figure 22 show that proposing any seed in the three qubit case often works, though not optimally. This is because the number of circuits capable of implementing a given unitary is high; there are many possible solution circuits. It seems that as widths increase, the complexity of both unitaries and circuits also increases substantially. This means that solution circuits become more sparse, and this random seed strategy is expected to perform far worse. This also means that a good recommender model is likely capable of demonstrating even larger speedups in these wider unitary cases.

4.2 Network Architectures for Learning on Unitaries

The selection of an autoencoder style neural network for the recommender model was motivated by the PCA results of Figure 15. There are likely architectures that are better suited to this task. This is especially true for cases where wider unitaries are considered, as the number of input parameters grows by 4 with the addition of each new qubit.

Convolutional Neural Networks (CNNs) are popular in computer vision tasks as they are translationally equivariant and particularly adept at working with data which exhibits spatial locality. CNNs are likely not well suited to the task of learning on unitaries, as the placement of a gate on the first of last qubit is in some sense very similar, but results in very differently structured unitaries.

Graph Neural Networks (GNNs) are a generalization of CNNs that have gained much popularity recently. It could be possible that using a GNN to learn a dense representation of a quantum circuit expressed as a Directed Acyclic

Graph could address the issues associated with the number of input parameters scaling with $O(4^n)$. Because unitaries of interest (non-random unitaries) are lower dimensional than their general counterparts (which use all 4^n parameters), it is worth investigating if one can learn a dense representation directly from a quantum circuit, without explicitly computing the full unitary.

4.3 Further Exploring the use of Reinforcement Learning

The work presented in [12] show that reinforcement learning can be used to learn good peephole style optimization techniques. I am interested to see whether these results can be further improved by exposing the agent exclusively to non-randomly generated circuits. I expect that this was not done originally because the number of non-random quantum circuits that do useful computation is rather limited.

I also wonder if a reinforcement learning agent can be trained to learn good synthesis tree search strategies. Paired with seeded synthesis, this could provide additional benefits over QSearch’s static A^* heuristic approach.

4.4 Learning to Incorporate Noise Resilience

How machine learning can be used to incorporate noise resilience is a research direction that also requires more attention. In [9], ML is used to learn a noise model for a machine. By executing a circuit consisting of only Clifford gates, and simulating this same circuit (which can be done efficiently as all gates are Cliffords), deviations in the expected and actual outcomes offer the necessary signals to train a neural network. I believe it is worth exploring other methods of predicting the amount of noise in or output fidelity of circuits executed on hardware.

I believe the role of machine learning in quantum circuit optimization, and quantum compilation as a whole, will surely grow over time.

References

- [1] D. Aharonov and M. Ben-Or, “Fault-tolerant quantum computation with constant error rate,” 1999.
- [2] G. Aleksandrowicz, T. Alexander, P. Barkoutsos, L. Bello, Y. Ben-Haim, D. Bucher, F. Cabrera-Hernández, J. Carballo-Franquis, A. Chen, C. Chen *et al.*, “Qiskit: An open-source framework for quantum computing,” *Accessed on: Mar*, vol. 16, 2019.
- [3] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. Brandao, D. A. Buell *et al.*, “Quantum supremacy using a programmable superconducting processor,” *Nature*, vol. 574, no. 7779, pp. 505–510, 2019.

MQT Predictor: Automatic Device Selection with Device-Specific Circuit Compilation for Quantum Computing

NILS QUETSCHLICH, Technical University of Munich, Germany

LUKAS BURGHOLZER, Technical University of Munich, Germany

ROBERT WILLE, Technical University of Munich, Germany and Software Competence Center Hagenberg GmbH, Austria

Fueled by recent accomplishments in quantum computing hardware and software, an increasing number of problems from various application domains are being explored as potential use cases for this new technology. Similarly to classical computing, realizing an application on a particular quantum device requires the corresponding (quantum) circuit to be *compiled* so that it can be executed on the device. With a steadily growing number of available devices—each with their own advantages and disadvantages—and a wide variety of different compilation tools, the number of choices to consider when trying to realize an application is quickly exploding. Due to missing tool support and automation, especially end-users who are not quantum computing experts are easily left unsupported and overwhelmed.

In this work, we propose a methodology that allows one to *automatically select* a suitable quantum device for a particular application *and* provides an *optimized compiler* for the selected device. The resulting framework—called the *MQT Predictor*—not only supports end-users in navigating the vast landscape of choices, it also allows *mixing and matching* compiler passes from various tools to create optimized compilers that transcend the individual tools. Evaluations of an exemplary framework instantiation based on more than 500 quantum circuits and seven devices have shown that—compared to both Qiskit’s and TKET’s most optimized compilation flows for all devices—the MQT Predictor produces circuits within the top-3 out of 14 baselines in more than 98% of cases while frequently outperforming any tested combination by up to 53% when optimizing for *expected fidelity*. Additionally, the framework is trained and evaluated for *critical depth* as another *figure of merit* to showcase its flexibility and generalizability—producing circuits within the top-3 in 89% of cases while frequently outperforming any tested combination by up to 400%. MQT Predictor is part of the *Munich Quantum Toolkit* (MQT) and publicly available as open-source on GitHub (<https://github.com/cda-tum/mqt-predictor>) and as an easy-to-use *Python* package (<https://pypi.org/p/mqt.predictor>).

1 INTRODUCTION

Quantum computing is an emerging computational technology and has seen considerable accomplishments in both hardware and software—especially in recent years. New quantum devices with an increasing number of qubits and higher fidelity rates are emerging almost on a daily basis. A similar trend can be observed for the development of quantum *Software Development Kits* (SDKs) such as, e.g., IBM’s Qiskit [1], Quantinuum’s TKET [2], Google’s Cirq [3], Xanadu’s PennyLane [4], or Rigetti’s Forest [5]. This sparks interest not only in academia but also in industry—with promising applications of quantum computing emerging in various domains, such as finance [6], optimization [7], machine learning [8], or chemistry [9]. However, realizing such quantum computing-based solutions remains a challenge that is not straight-forward to solve and, to date, requires several manual steps [10].

First, a suitable quantum device must be *selected* for the execution of the developed quantum algorithm. This alone is non-trivial since new quantum devices based on various underlying technologies emerge almost daily—each with their own advantages and disadvantages. There are hardly any practical guidelines on which device to choose based on the targeted application. As such, the best guess in many cases today is to simply try out many (if not all) possible devices and, afterwards, choose the best results—certainly a time- and resource-consuming endeavor that is not sustainable for the future.

Once a target device has been selected, the quantum circuit, which is typically designed in a device-agnostic fashion that does not account for any hardware limitations (such as a limited gate-set or limited connectivity), must be *compiled* accordingly so that it actually becomes executable on that device. This is similar to classical computing, where high-level, platform-agnostic code (e.g., written in C++) is being *compiled* to low-level, platform-specific machine code before being executed on a particular classical computer. Compilation itself is a sequential process consisting of a sequence of *compilation passes* that, step-by-step, transform the original quantum circuit so that it eventually conforms to the limitations imposed by the target device. Since many of the underlying problems in compilation are computationally hard (e.g., satisfying the connectivity limitations of a certain device with the least amount of overhead being NP-complete [11]), there is an ever-growing variety of compilation passes available across several quantum SDKs and software tools—again, each with their own advantages and disadvantages. As a result of the sheer number of options, choosing the best sequence of compilation passes for a given application is nearly impossible. Consequently, most quantum SDKs (such as Qiskit and TKET) provide easy-to-use high-level function calls that encapsulate “their” sequence of compilation passes into a single compilation flow. While this allows to conveniently compile circuits, it has several drawbacks. For one, it creates a kind of *vendor lock* that limits the available compilation passes to those available in the SDK offering the compilation flow. Furthermore, the respective compilation flows are designed to be broadly applicable and, hence, are neither device-specific nor circuit-specific. On top of that, no means are provided to optimize for a specific *figure of merit*—only the degree of desired optimization can be specified without any explicit optimization criterion.

Overall, this leaves anyone trying to realize a quantum computing application with more options than they could ever feasibly explore. What makes things worse is that the choice of the quantum device and the subsequent compilation flow heavily affect the quality of the resulting circuit and its execution. Choosing the right combination can often make the difference between a useful result and pure noise. This is especially unfortunate for end-users who are not experts in quantum computing and just want to use the technology to solve their application domain problem.

In this work¹, a new approach to *automate* the quantum device selection and according compilation is proposed to free end-users from this task and to prevent a scenario where quantum computers can only be utilized by quantum computing experts—hindering the overall adoption of that technology. To this end, we tackle this problem from two angles:

- (1) We propose a *prediction* method (based on *Supervised Machine Learning*) that, without performing any compilation, automatically predicts the most suitable device for a given application. This completely eliminates the manual and laborious task of determining a suitable target device and guides end-users through the vast landscape of choices without the need for quantum computing expertise.
- (2) We propose a method (based on *Reinforcement Learning*) that produces device-specific quantum circuit compilers by combining compilation passes from various compiler tools and learning optimized sequences of those passes with respect to a customizable *figure of merit*. This *mix-and-match* of compiler passes from various tools allows one to eliminate vendor locks and to create optimized compilers that transcend the individual tools.

Combining both of these methods into a holistic framework—the *MQT Predictor*—allows one to *automatically select* a suitable quantum device for a particular application while also providing an *optimized compiler* for the selected device and a customizable figure of merit.

¹Preliminary versions of this work have been published in [12], [13].

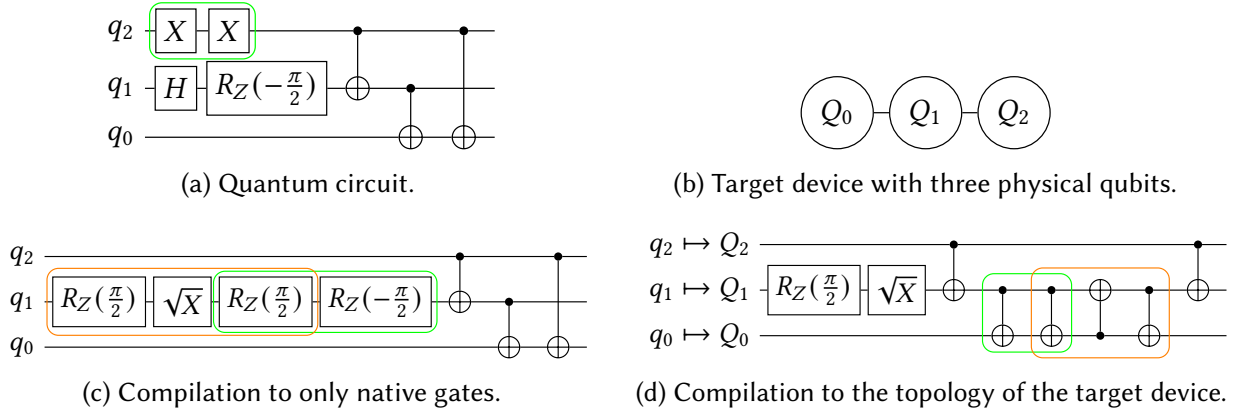


Fig. 1. Compilation of a quantum circuit to a targeted device.

To show its benefits, an instantiation of the proposed approach is implemented based on more than 500 quantum circuits ranging from 2 to 90 qubits and considering seven different quantum devices based on two qubit technologies. As a baseline to compare against, the most optimized compilation flows available in Qiskit and TKET have been used—leading to a total of $7 \times 2 = 14$ possible combinations of device and compiler. Compared to these reference results, the MQT Predictor produces circuits with an expected fidelity that is *on par with the best* possible result that could be achieved by trying out all combinations of devices and compilers. Specifically, the *automatic device selection* and *optimized circuit compilation* produced circuits within the top-3 in more than 98% of cases while frequently outperforming any tested combination by up to 53%. Additionally, MQT Predictor is trained and evaluated for *critical depth* as another figure of merit to showcase its flexibility and generalizability. The respective results show that the proposed method also achieves a similar performance—producing circuits within the top-3 in 89% of cases while frequently outperforming any tested combination by up to 400%. The corresponding framework (which is part of the *Munich Quantum Toolkit* (MQT) [14]) including the pre-trained models and classifiers is publicly available as open-source on GitHub (<https://github.com/cda-tum/mqt-predictor>) and as an easy-to-use *Python* package (<https://pypi.org/p/mqt.predictor>).

The rest of this work is structured as follows: In Section 2, quantum circuit compilation flows and related work are reviewed. Based on that, Section 3 introduces the proposed approach and the underlying methodologies with details given in Section 4 for the compilation using reinforcement learning and Section 5 for the device selection using supervised machine learning. The resulting combined approach is then described in Section 6, evaluated in Section 7, and discussed in Section 8. Section 9 concludes this work.

2 BACKGROUND AND MOTIVATION

To keep this work self-contained, this section gives a brief introduction to quantum circuit compilation, discusses its challenges, and reviews the related work.

2.1 Quantum Circuit Compilation

Solving any problem using quantum computing requires encoding the problem as a quantum algorithm. These algorithms are typically described as sequences of quantum operations (often referred to as *quantum gates*) that form a *quantum circuit*. Usually, one distinguishes between *single-qubit* gates affecting only one qubit and *multi-qubit* gates affecting more than one qubit at the same time.

EXAMPLE 1. *Fig. 1a shows a small exemplary quantum circuit with three qubits (q_0, q_1, q_2) and four different kinds of quantum gates acting on them. There are two single-qubit gates on q_2 (two NOT gates denoted as X) and two single-qubit gates on q_1 (a Hadamard gate and a Z-rotation gate denoted as H and R_Z , respectively). Additionally, there are multi-qubit gates between all possible qubit pairs, namely controlled-NOT (CNOT) gates denoted as $\bullet$ and $\oplus$ for their control, respectively, target qubit.*

The actual compilation can only be started as soon as a suitable quantum device is selected and, by that, its native gate-set and connectivity constraints are known. The device selection itself is already challenging due to several factors:

- Various underlying quantum technologies are currently pursued in parallel with no clear winner yet and all of them have their own advantages and disadvantages. *Ion trap*-based devices, e.g., feature an all-to-all connectivity but provide rather slow gate execution times while, e.g., *superconducting*-based devices provide fast gate execution times but usually come with a restricted connectivity.
- Even devices based on the same technology significantly vary in their hardware characteristics, such as qubit count, their respective error rates, coherence times, qubit connectivity, and gate execution times.
- On top of all that, the domain of quantum computing is fast-paced and constantly changing—quickly and frequently redefining the state of the art.

EXAMPLE 2. *Fig. 1b shows the topology of a three-qubit quantum device. This topology indicates that two-qubit gates may only be applied to (Q_0, Q_1) or (Q_1, Q_2) , but not to (Q_0, Q_2) .*

As introduced in Section 1, executing such quantum circuits on an actual quantum device requires compilation, which is usually conducted by *compilers* that are part of quantum SDKs as easy-to-use high-level function calls. However, compilation actually is a sequential process that can be roughly divided into three kinds of *compilation passes*: *Synthesis* passes compile a given quantum circuit to the native gate-set of a selected quantum device, *mapping* passes enforce its connectivity constraints, and *optimization* passes exploit various possibilities to reduce the circuit's complexity. Quantum SDKs usually offer a wide range of device-independent optimization passes with the goal of creating a more efficient quantum circuit which still realizes the same functionality by, e.g., gate-cancellation, gate-commutation, or high-level synthesis routines.

EXAMPLE 3. *Since $X \cdot X = I$, the circuit shown in Fig. 1a can be simplified by canceling the two subsequent X gates (denoted by the green box).*

Usually, those device-independent optimization passes are followed by the *device-dependent* compilation passes. Every quantum device can only directly execute a subset of all possible quantum gates—its so-called native gates. These native gates usually comprise a handful of single-qubit gates and one multi-qubit gate to become a *universal* gate-set. Therefore, all non-native gates must be *synthesized* into a sequence of native gates of that respective quantum device—a procedure that is NP-complete to solve in an optimal fashion [15]. Consequently, this alters the quantum circuit—leading to further room for optimization.

EXAMPLE 4. *Assume that the targeted device from Example 2 has a native gate-set consisting of R_Z , $\sqrt{X}$, X, and CNOT gates and that the circuit from Fig. 1a shall be executed on it. Only the Hadamard gate on qubit q_1 is not a native gate of the device and, hence, must be synthesized into a sequence of such gates. The orange box in Fig. 1c indicates one such decomposition into two $R_Z(\frac{\pi}{2})$ and a $\sqrt{X}$ gate. Again, this circuit alteration leads to further optimization potential since the two subsequent R_Z gates in the green box, again, cancel each other out.*

In addition to the limited gate-set, many quantum devices (e.g., based on such as superconducting qubits) only have a *limited connectivity* between their underlying qubits—requiring that all multi-qubit gates must be placed on qubits which are actually connected on the respective device. To adapt a quantum circuit with gates operating between arbitrary qubits to such a limited connectivity, a *mapping* pass is necessary. This pass is often split into *layouting* and *routing*. The former assigns each (logical) qubit of the circuit to a (physical) qubit on the device and the latter ensures that all connectivity constraints are satisfied by changing the qubit assignment throughout the circuit, e.g., using *SWAP* gates—a procedure that is, again, NP-complete to solve in an optimal fashion [11]. Then, again, optimization passes can be applied to reduce the complexity of the resulting quantum circuit.

EXAMPLE 5. Consider again the circuit shown in Fig. 1c which shall be executed on the three-qubit quantum device shown in Fig. 1b. Since the quantum circuit contains multi-qubit gates between all qubit pairs, there is no trivial layout that satisfies the connectivity constraints throughout the entire circuit. Therefore, the circuit must be mapped by introducing a SWAP gate as denoted in the orange box in Fig. 1d (again, compiled into its native gate-set sequence of three CNOT gates). Although the resulting quantum circuit is now fully executable, a successive optimization pass can, again, eliminate two subsequent CNOT gates.

Although the compilation process is rather straight-forward conceptually—consisting of *synthesis*, *mapping*, and *optimization* passes—it comes with practical challenges. For each step, a variety of different techniques and approaches have been proposed, such as [15]–[20] for synthesis, [21]–[29] for mapping, and [30]–[33] for optimization. Therefore, end-users are easily overwhelmed with selecting the *best* combination of provided SDKs and respective compilation passes—not even mentioning the effort of becoming acquainted with multiple (often poorly documented) SDKs and, respectively, needed conversions between them. Every SDK usually implements a subset of these compilation passes with varying degrees of effectiveness and efficiency. Those passes are provided as pre-configured, fixed sequences of compilation passes (for various levels of desired optimization)—independent of the given quantum circuit.

2.2 Related Work

A handful of techniques have already been explored to support the quantum device selection and compilation. In [34]–[37], approaches that compile a given circuit for *all* devices in a brute-force fashion and compare their resulting quality (also referred to as *autotuning*) are proposed. A similar approach to select the best sequence of compilation passes by evaluating all possible combinations has been proposed in [38]. Furthermore, in [39], a methodology has been proposed to determine whether compiled circuits can be reliably executed on certain devices to give some guidance. The same authors have further explored how *Multi-Criteria Decision Analysis* methods can be used to select the most suitable compiled circuit from a given set of all compiled circuits [40] and how machine learning can be used to predict the goodness of a device and compiler combination [41].

Supervised Machine Learning (ML) techniques have also been applied to quantum compilation, e.g., in [42], an ML-based layout and routing scheme is proposed, while in [43], an ML model is trained to predict the optimization potential of a given quantum circuit to decide whether to apply (potentially costly) optimization passes or not. Furthermore, *Reinforcement Learning* (RL) has been explored in this context, e.g., in [44] where a strategy for applying individual gate transformation rules to optimize quantum circuits is learned or in [45] where an RL model is trained to learn how to estimate the circuit fidelity for specific devices. Moreover, environments to train RL models for various compilation tasks are proposed in [46]—similar to the environment proposed in [13].

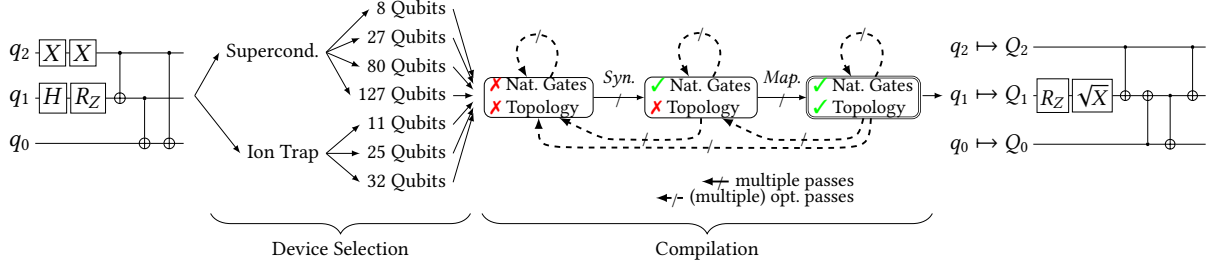


Fig. 2. The device selection and compilation process.

In classical compilation, machine learning has been explored for a much longer time which led to sophisticated compiler optimization techniques with overviews given in, e.g., [47], [48] and established these methods as state-of-the-art techniques in this domain. Additionally, even a combination of these techniques has been explored in [49] by using a machine learning model to determine promising areas of optimization space which then can be further investigated using autotuning. On top of that, reinforcement learning approaches targeting the LLVM [50] compiler emerged especially in recent years with encouraging results [51]–[53].

So far, many of the approaches toward both quantum device selection and compilation [34]–[40] follow a naive brute-force approach that, obviously, is not a long-term solution due to the steadily increasing number of quantum devices and compilers. Furthermore, existing ML-based tools to support end-users [41]–[45] do not consider the entire compilation or do not provide a broad coverage in terms of supported devices, range of qubits, or considered algorithms, and frequently lack the necessary means to scale to significantly more devices and compilers. On top of that, most approaches focus on using current compilers *as-is* and hardly make use of the sophisticated ML-based approaches for classical compiler optimization—leading to lots of untapped potential.

3 GENERAL IDEA

In this work, we propose a new approach to *automate* the device selection and compilation process that learns from previously conducted compilations and *predicts* the most suitable target device for a given quantum circuit *as well as* provides an *optimized compiler* for the selected device. To this end, this section motivates the proposed approach and describes its general ideas, the chosen methodology, and its utilization.

3.1 Proposed Approach

In an ideal scenario, a quantum circuit would be compiled for all possible sequences of compilations on all possible devices to determine the best compilation flow. However, since compilation comes with considerable complexity both in time and resources, this is practically infeasible.

EXAMPLE 6. Assume that we want to find the best compiled version of the quantum circuit shown in Fig. 1a given access to seven different devices (based on two qubit technologies) and various compiler passes provided by multiple SDKs as illustrated in Fig. 2. Manually determining the best possible combination of options is not only challenging, but close to impossible due to the number of possible combinations—which, in theory, is infinitely large due to backward loops in the compilation process that are introduced by optimization passes. End-users could just choose any (random) path from the left-hand side of Fig. 2 to the right-hand side. However, this is highly unlikely to yield a sufficiently good result (not to dare, the best possible result).

In the proposed approach, we first consider finding the best sequence of *compilation* passes for a particular device as an isolated problem that is independent of the device selection task itself. To this end, compilation is modeled as a sequential (*Markov Decision*) process of synthesis, mapping, and optimization passes for a given native gate-set and qubit connectivity. Using *Reinforcement Learning* (RL), respective models can be trained to *learn* the sequences that lead to the most promising compilation result for each supported device according to some *figure of merit* such as expected fidelity or critical depth. These models can then be used as optimized black-box compilers for the respective devices.

Based on these optimized compilers, the task of choosing the most promising device is tackled. To this end, *Supervised Machine Learning* (ML) is applied by training an agent that *learns* the best suited device for a given circuit assuming that it will be compiled using the trained RL models. Thus, a holistic approach is created that takes a to-be-compiled quantum circuit as input and predicts which quantum device to use, compiles it accordingly using the trained RL model, and returns the compiled circuit with detailed compilation information. All of this is conducted in a fully automated fashion which is especially helpful for end-users who are not experts in quantum computing. Additionally, compilation passes from various sources can be integrated—effectively eliminating vendor locks by mixing and matching compiler passes from various SDKs.

To learn how to *best* compile quantum circuits, it is necessary to give the notion of “best” a meaning by defining a *figure of merit* to optimize for. As this is an essential part of the proposed approach, more details on this way of customizing the compilation are provided in the following.

3.2 Figures of Merit

In general, the figure of merit can be completely customizable and consider *anything* from (compiled) circuit characteristics such as gate counts to *soft* factors—applicable to the device selection—such as actual execution costs or the queue length for a specific device vendor. Nevertheless, a comprehensive figure of merit should consider at least the characteristics of the compiled quantum circuit and the characteristics of the selected quantum device. Therefore, the respective hardware information needs to be collected, such as, e.g., gate/readout fidelities, gate execution times, or decoherence times. While for some devices, this information may be publicly available, for other devices, it may be estimated from comparable devices, previous records, or insider knowledge.

EXAMPLE 7. Consider the figure of merit that takes into account the following two aspects:

- (1) If the selected device is not large enough to execute a given quantum circuit with respect to its number of qubits, the worst-possible score is assigned.
- (2) If the device is large enough, the evaluation score is calculated using the formula:

$$\mathcal{F} = \prod_{i=1}^{|G|} \mathcal{F}(g_i) \prod_{j=1}^m \mathcal{F}_{RO}(q_j)$$

with $\mathcal{F}(g_i)$ being the expected execution fidelity of gate g_i on its corresponding qubit(s), $\mathcal{F}_{RO}(q_j)$ being the expected execution fidelity of a measurement operation q_j on its corresponding qubit and $|G|$ respectively m being the number of gates and measurements in the compiled circuit.

This figure of merit determines an estimate of the probability that a quantum circuit will return the expected result, the so-called expected fidelity, which ranges between 0.0 and 1.0 with higher values being better.

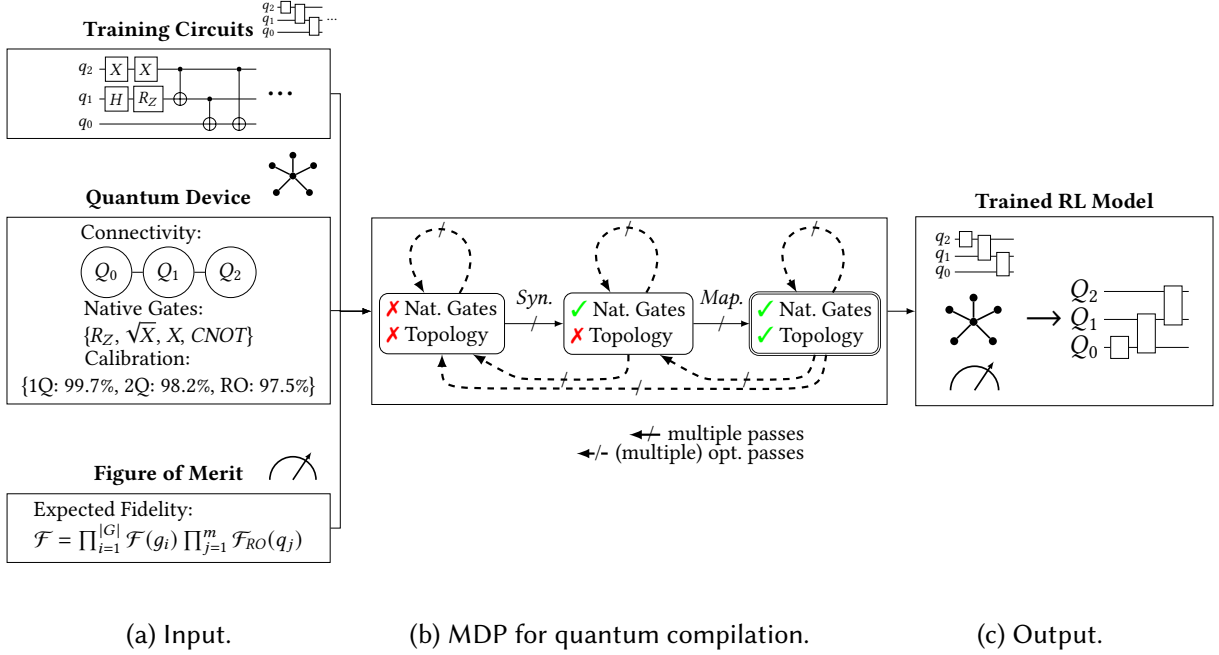


Fig. 3. Training process of an RL model for quantum compilation.

However, the expected fidelity introduced above is just one exemplary figure of merit. A potential alternative could be the *critical depth* (taken from [54])—a measure to describe the percentage of multi-qubit gates on the longest path through a compiled quantum circuit (determining the depth). A respective value close to 1 would indicate a very sequential circuit while a value of 0 would indicate a highly parallel one. On top of that, even combinations with customizable weighting are possible, such as 25% of the expected fidelity and 75% critical depth. Furthermore, different figures of merit could be used for the device selection and the respective compilation itself. While for the former, soft factors such as actual execution costs or queue lengths might be sensible to consider as well, the latter should only focus on technical factors that describe the quality of the compilation itself.

In the following, detailed descriptions of both the RL-based approach to the compilation and the ML-based approach to the device selection are given in Section 4 and Section 5, respectively. Afterwards, Section 6 describes the combination of both techniques into a holistic framework.

4 COMPILATION USING REINFORCEMENT LEARNING

Compilation, fortunately, is not new per-se, since classical compilers have seen a similar trend of increasing complexity and variety in the past. Instead of reinventing the wheel, we propose to make use of the decades of research on classical compiler optimization and model quantum circuit compilation in a similar fashion. Based on that, classical reinforcement learning is used to learn optimized sequences of compilation passes for a given figure of merit. The resulting model can then be used as a compiler.

4.1 Quantum Circuit Compilation Modeled as a Markov Decision Process

Quantum circuit compilation as described in Section 2.1 can be interpreted as a *Markov Decision Process* (MDP, [55]) that transforms a high-level circuit into an executable circuit by applying synthesis, mapping, and optimization passes. For that, the compilation process is broken down into *states* and corresponding *actions* that can be executed in those states to advance the process. Each

state captures which of the requirements for a circuit to be executable are satisfied—specifically, whether the circuit only contains native gates and whether it matches the topology of the device. This results in three states²:

- (1) The circuit contains non-native gates and does not respect the device topology,
- (2) The circuit only contains native gates but does not respect the device topology, and
- (3) The circuit only contains native gates and respects the device topology, i.e., it is executable.

The resulting MDP is depicted in Fig. 3b—with states being denoted as rounded rectangles and actions as arrows. In the following, the individual states and actions will be described in more detail alongside an example.

Given a high-level quantum circuit and a target device (determining the native gate-set and qubit topology), compilation typically begins in the state where the circuit contains non-native gates and does not respect the device topology. In this state, two kinds of actions are possible: Either a *synthesis action* that transforms the circuit so that it only contains gates native to the target device, or an *optimization action* that applies any kind of optimization to the circuit.

EXAMPLE 8. *To make this modeling more tangible, consider again the compilation flow shown in Fig. 1 with the initial circuit depicted in Fig. 1a. Since it comprises a non-native gate, the compilation starts from the left-most state in Fig. 3b. First, an optimization pass is applied that cancels the two redundant X gates. Since the circuit still contains non-native gates, this does not change the overall state—as indicated by the dashed self-loop above the state. Then, a synthesis pass is applied, which yields the circuit depicted in Fig. 1c and advances the state of the MDP to the next state.*

Whenever the circuit only contains native gates but does not yet respect the device topology, three different actions are possible: First, the circuit could be mapped using any *mapping action*—making the circuit executable and, thus, advancing the MDP to the final state. Additionally, the circuit may be optimized again. Depending on whether the optimization pass preserves the native gates, the state after the action either stays the same (*gate-set-preserving optimization action*) or goes back to the previous, non-native gates state (*general optimization action*).

EXAMPLE 9. *Consider again the synthesized quantum circuit from Fig. 1c. As before in Example 8, a similar optimization is conducted to cancel the two rotation gates. Since the resulting circuit still only consists of native gates, this optimization does not change the state of the MDP—again represented by a dashed self-loop in Fig. 3b. Subsequently, a mapping action is applied that leads to the quantum circuit depicted in Fig. 1d and advances the state of the MDP to the final state.*

In the final state, both the device’s native gate-set and qubit connectivity constraints are fulfilled and any circuit in this state is already executable. However, additional *optimization* actions can be applied to further reduce complexity. Again, those actions might violate any of the two constraints and the circuit might end up in a state where it is not executable any more.

EXAMPLE 10. *The quantum circuit resulting from Example 9 is already executable. However, another optimization action is applied to the circuit depicted in Fig. 1d to cancel two further gates and, by that, reducing the complexity. Since this does not violate the device’s connectivity, the circuit remains executable and the MDP state does not change—again represented by a dashed self-loop in Fig. 3b.*

²We purposely omit the state where the circuit respects the topology but contains non-native gates as there are hardly any compilation passes available that take advantage of this specific combination of conditions.

This MDP formulation enables a modular framework based on two properties:

- (1) Each of the constraints that characterize an MDP state is *efficiently computable* (via a single traversal of the circuit).
- (2) All actions have a *unified interface* based on a common intermediate representation (IR) for their input and output—independent of the origin of a certain action.

The former ensures that, at any stage during the compilation, state transitions within the MDP can be computed efficiently. The latter ensures interoperability between different SDKs and makes it possible to *mix and match* compilation passes provided by multiple SDKs. Consequently, they can be combined and integrated in a single compilation framework—mitigating vendor lock-ins, allowing one to quickly adapt and integrate further compilation passes, and helping to keep up with the fast-paced development in quantum computing software.

4.2 Learning Optimal Compilation Sequences Using Reinforcement Learning

In classical compilation, different problems have been tackled very successfully using *Reinforcement Learning* (RL) [51]–[53]. In general, these techniques aim to learn an *action policy* that determines which actions should be applied based on *observations* of the current state with the goal of maximizing a cumulative *reward function*. In contrast to providing labeled training data (as in *Supervised Machine Learning*, ML), a *training environment* representing the underlying MDP including its states and actions must be defined.

To avoid reinventing the wheel, we also propose using such an RL approach for quantum circuit compilation based on the MDP shown in Fig. 3b. To this end, three things must be provided as input (which are depicted in Fig. 3a):

- (1) *Training Circuits*: Used in the training process to learn an action policy.
- (2) *Quantum Device*: Defines the native gate-set, the qubit connectivity, and certain characteristics (such as gate fidelities) for the compilation process.
- (3) *Figure of Merit*: Defines the evaluation score to guide the learning process.

An *RL agent* can then be employed to learn a corresponding action policy that maximizes the expected cumulative reward. For the training itself, a *sparse* reward function is used, i.e., as long as the circuit is not executable, the reward is always zero, and whenever the circuit becomes executable, the chosen figure of merit (e.g., expected fidelity as described in Section 3.2) is used to assign a score to the compiled circuit.

The resulting trained RL model then acts as *compiler* itself that can compile any given quantum circuit for the target device it was trained for—as illustrated in Fig. 3c. This creates a compiler that is fine-tuned for a specific device, optimizes for a customizable figure of merit, and factors in the input circuit when determining which compilation passes to apply—three major advances over current quantum circuit compilers.

5 DEVICE SELECTION USING SUPERVISED MACHINE LEARNING

As discussed in Section 2.1, selecting the best quantum device for a particular application is itself a challenging task. This section describes how that task can be interpreted as a classification problem and how supervised machine learning can predict the most promising qubit technology and device for a given quantum circuit—therefore, automatically make this decision for an end-user.

5.1 Quantum Device Selection Modeled as a Classification Task

A naive approach to selecting the best quantum device for a given quantum circuit would be to first compile it for all devices using a specific compiler, e.g., the trained RL models from Section 4. Afterwards, the resulting compiled circuits must be evaluated according to some *figure of merit*

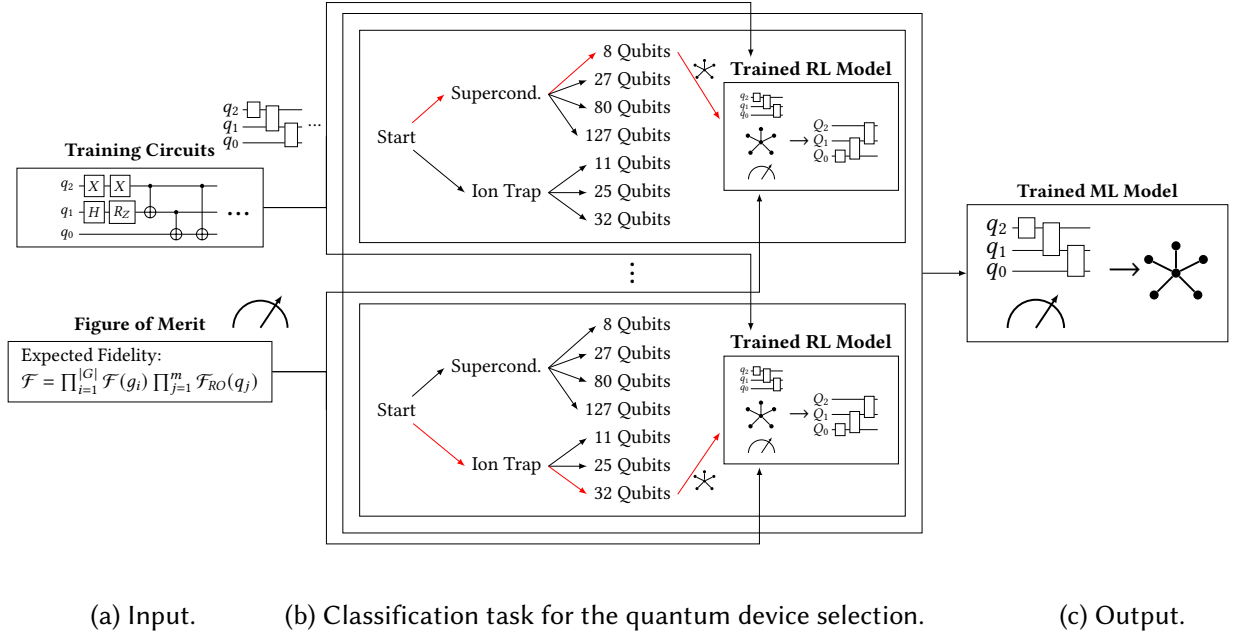


Fig. 4. Training process of an ML model for the quantum device selection.

to identify the most promising device. However, doing this for each and every to-be-compiled quantum circuit is practically infeasible since compilation is a time-consuming task and the number of available devices is steadily growing.

EXAMPLE 11. *There are various underlying technologies how to realize quantum computers such as, e.g., superconducting- and ion trap-based qubits. Even within a technology, respective devices may significantly vary in their characteristics such as the connectivity scheme, native gate-set, and fidelity rates. Assuming that an end-user has access to seven devices based on two technologies, the task of determining the best device for a given circuit and figure of merit already becomes challenging since seven different compilations would have to be conducted—as insinuated in Fig. 4b.*

Therefore, we propose an approach to *learn* from previously conducted compilations and interpret the selection of the most promising device for a circuit and figure of merit as a statistical *classification* task—a perfect fit for supervised machine learning. By that, end-users are freed from answering the following questions themselves:

- Which *qubit technology* is best suited for the application at hand?
- Which particular *device*, e.g., from IBM or Rigetti, fits the quantum algorithm best?

5.2 Predicting a Quantum Device Using Supervised Machine Learning

Supervised Machine Learning (ML) has proven a promising methodology in classical compilation (as described in [47], [48]) but also in various other domains, such as, e.g., medicine, cyber security, and predictive analytics [56]. A crucial part to utilize this technology is gathering suitable training data which are representative for the whole problem domain space to facilitate generalizability—in this case, representative training circuits covering a broad range of applications. Subsequently, each training circuit must be compiled for *all* possible devices—using the trained RL models that act as compilers—to be able to identify the most promising according to the chosen figure of merit as shown in Fig. 4b. That device then acts as the *classification label* and, together with the training circuit itself, constitutes one *training sample*. To generate all the *labeled training data* necessary

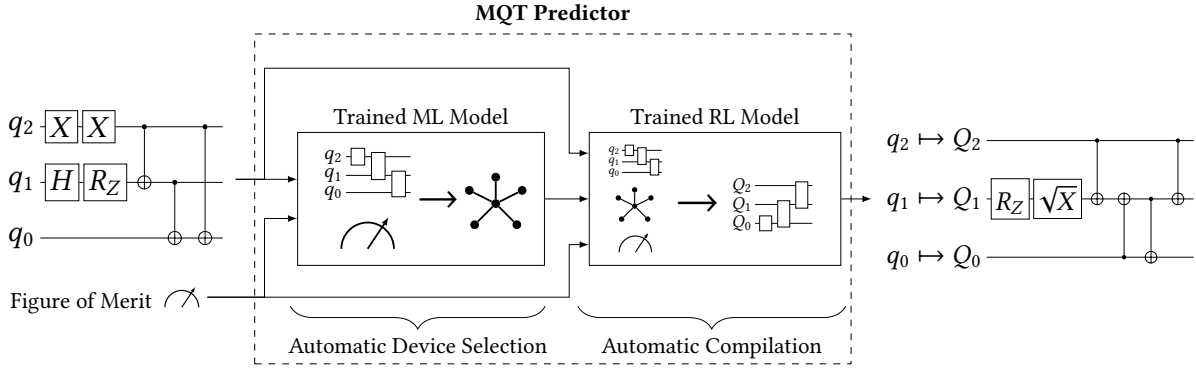


Fig. 5. Automatic compilation flow selection and execution using MQT Predictor.

to train respective ML models, this process is repeated for each training circuit and requires two things that must be provided (which are depicted in Fig. 4a):

- (1) *Training Circuits*: Similar to the RL models, used here to generate labeled training data³.
- (2) *Figure of Merit*: Determines the most promising device based on all conducted compilations.

Each labeled circuit must then be transformed into a *feature vector* to be suitable for training a classifier. There are many degrees of freedom when choosing the respective feature and it is very much an active field of research to determine features that best characterize a given quantum circuit. In practice, these features should at least include the number of qubits and some statistics about the interactions of gates in the circuit, such as depth, parallelism, or interaction frequency.

Subsequently, the actual ML model can be trained. This methodology with the respectively trained ML model (as illustrated in Fig. 4c) then acts as a tool to automatically predict the most promising quantum device for a given circuit and figure of merit. As a result, the user gets a recommendation that is circuit-specific, tuned for a specific figure of merit, and requires no compilation at all—again, three major advances over the established workflow.

6 PROPOSED APPROACH: MQT PREDICTOR

Together with the trained RL models from Section 4, the trained ML model automatically solves the device selection and compilation process—forming a holistic framework. In the following, this framework, called the *MQT Predictor*, and its implementation are described in more detail.

6.1 Approach

The proposed approach is shown in Fig. 5. It takes a quantum circuit and a customizable figure of merit as input. Then, the framework itself predicts which quantum device is the most promising using the trained ML model—in *realtime* without conducting any compilation at all. The outcome of this is the predicted device. Based on that, the respectively trained RL model compiles the given circuit for the predicted device optimizing for the chosen figure of merit. Finally, it returns the compiled quantum circuit. By that, the MQT Predictor completely frees end-users from the daunting tasks of selecting a proper quantum device and finding a good compiler and can be used with just a single *Python* function call—providing the same ease of use as the state-of-the-art compilers such as Qiskit [1]. On top of that, it combines the advantages of both individual techniques into a single powerful tool.

EXAMPLE 12. Consider again the original scenario shown in Fig. 2, where an end-user wants to execute the circuit shown again on the left-hand side of Fig. 5. Instead of manually finding the

³To reduce the effort needed for regenerating labeled training data after changes, all compiled circuits including their evaluation score are persistently stored in a database.

most promising device and compiler passes, the circuit can be simply fed into the MQT Predictor framework together with the expected fidelity (as defined in [Section 3.2](#)) as its figure of merit. Then, the MQT Predictor predicts a respective device and compiles the circuit accordingly using the trained RL model—returning the compiled circuit shown on the right-hand side of [Fig. 5](#).

The following sections describe the particular instantiation of the framework that has been implemented as part of this work.

6.2 Implementation of Automatic Compilation Using RL

The training environment described in [Section 4](#) and shown in [Fig. 3b](#) is built on top of the open-source library *OpenAI Gym* [57]. For that, all compilation passes from different sources that should be considered in learning promising compilation sequences must be implemented through a *unified interface*.

To this end, *Qiskit's QuantumCircuit* is used as the *intermediate representation*. Therefore, a compilation pass can be incorporated if the following requirement is fulfilled: Either it must provide an interface for both the input and the compiled output circuit that supports *Qiskit's QuantumCircuit* or a parser that supports *OpenQASM* (version 2 or 3)—which, then, can be used to create and serialize a *Qiskit's QuantumCircuit* again.

As modeled in [Fig. 3](#), three different types of actions are considered: synthesis, mapping, and optimization passes. Using the described unified interface for both input and output of a compilation action, compilation passes from various quantum SDKs can easily be incorporated. To demonstrate this, representative compilation passes from IBM's *Qiskit* (version 0.43.3) and Quantinuum's *TKET* (version 1.17.1) have been integrated—leading to the following list of available actions.

- *Synthesis: Qiskit's BasisTranslator*
- *Mapping: Qiskit's Sabre mapping or any combination of the following methods*
 - Layout:
 - * *Qiskit's TrivialLayout*
 - * *Qiskit's DenseLayout*
 - * *Qiskit's SabreLayout*
 - Routing:
 - * *Qiskit's BasicSwap*
 - * *Qiskit's StochasticSwap*
 - * *Qiskit's SabreSwap*
 - * *TKET's RoutingPass*
- *Optimization:*
 - *Qiskit's Optimize1qGatesDecomposition*
 - *Qiskit's CXCancellation*
 - *Qiskit's CommutativeCancellation*
 - *Qiskit's CommutativeInverseCancellation*
 - *Qiskit's RemoveDiagonalGatesBeforeMeasure*
 - *Qiskit's InverseCancellation*
 - *Qiskit's OptimizeCliffords*
 - *Qiskit's Collect2qBlocks + ConsolidateBlocks*
 - *Qiskit's O3 Fixpoint Optimization Routine*
 - *TKET's PeepholeOptimise2Q*
 - *TKET's CliffordSimp*
 - *TKET's FullPeepholeOptimise*
 - *TKET's RemoveRedundancies*

For the training process, more than 500 training circuits from *MQT Bench* [58] on the target-independent level have been used ranging from 2 to 30 qubits. Lastly, a maskable version of the *Proximal Policy Optimization (PPO)* algorithm [59] provided by *Stable-Baselines3* [60] is used for the reinforcement learning training process itself and also takes care of potential infinite sequences due to the backward loops within the MDP.

6.3 Implementation of Automatic Device Selection Using ML

For the quantum device selection, the ML model described in Section 5 and shown in Fig. 4b is trained using scikit-learn [61]. In this regard, seven different underlying algorithms are implemented and evaluated in [12]—with grid-searched and 5-fold cross-validated parameter values—to determine the most promising one:

- *Random Forest* [62]
- *Gradient Boosting* [63]
- *Decision Tree* [64]
- *Nearest Neighbor* [65]
- *Multilayer Perceptron* [66]
- *Support Vector Machine* [67]
- *Naive Bayes* [68]

In [69], an overview of today’s use of those techniques is given. All seven machine learning classifiers have been experimentally evaluated. Since the *Random Forest* classifier has given the best results, it is used in the following as the ML algorithm.

As indicated in Fig. 2, seven devices from two qubit technologies are considered as representatives:

- Superconducting: four devices with 8, 27, 80, and 127 qubits
- Ion Trap: three devices with 11, 25, and 32 qubits

To generate respective training data, more than 500 training circuits from *MQT Bench* [58] on the target-independent level are used (ranging from 2 to 90 qubits) with a 70/30 train-test-split. The compilations to generate the labeled training data are conducted based on the trained RL models acting as respective compilers.

6.4 Quantum Circuit Representation

Both the RL and the ML training process require quantum circuits to be represented as so-called *feature vectors*—a vectorized representation based on integer and floating point values that makes them suitable for training a respective model. Similar to the figure of merit described in Section 3.2, these features can be arbitrarily complex. In this instantiation, various characteristics are used to describe a quantum circuit for both models: the *number of qubits*, the *depth* of the circuit, and the five composite features of *program communication*, *critical-depth*, *entanglement-ratio*, *parallelism*, and *liveness* originally proposed in [54]:

- *Program Communication*: Metric to measure the average degree of interaction for all qubits. A value of 1 indicates that each qubit interacts at least once with all other qubits.
- *Critical Depth*: Metric to measure how many of all multi-qubit gates are on the longest path (defining the depth of a quantum circuit). A value of 1 indicates that all multi-qubit gates are on the longest path.
- *Entanglement Ratio*: Metric to measure how many gates in a quantum circuit are multi-qubit gates. A value of 1 indicates that the quantum circuit consists of only multi-qubit gates.
- *Parallelism*: Metric to measure how much parallelization within the circuit is possible due to simultaneous gate execution. A value of 1 describes large parallelization.

- *Liveness*: Metric to measure how often the qubits are idling and waiting for their next gate execution. A value of 1 describes a circuit in which there is a gate execution on each qubit at each time step.

Furthermore, for the ML model, the number of gates for each gate type according to the *OpenQASM 2.0* specification [70] are used as features. For the RL model, this was omitted to improve the learning efficiency, since the gate count features have been shown to be less representative than the composite ones [12].

7 EVALUATION

Based on the instantiation described above, this section demonstrates the feasibility of the MQT Predictor framework by means of numerical evaluation. All used pre-trained models and classifiers are publicly available as open-source on GitHub (<https://github.com/cda-tum/mqt-predictor>) and as an easy-to-use *Python* package (<https://pypi.org/p/mqt.predictor>).

7.1 Setup

To assess the quality of the proposed framework, it is evaluated on more than 150 benchmarks from 2 to 90 qubits taken from *MQT Bench* [58] using the target-independent level circuits and the MQT Predictor instantiation described in Section 6. For each of those evaluation circuits—which have not been part of the (ML) training circuits—the tool predicts the most promising target device and its optimized compiler to compile the given circuit to the selected device. To assess quality, two figures of merit are used:

- (1) The expected fidelity as introduced in Section 3.2 and
- (2) The critical depth describing the ratio of multi-qubit gates on the longest path of the compiled quantum circuit⁴.

Note that the used figures of merit are examples and the MQT Predictor framework allows end-user to provide *any* metric to optimize for. In this case, the focus rather lies on technical factors to determine both the best device and compilation sequence while it might be sensible to also factor in soft factors such as the execution costs or the queue length for the device selection. Since the focus of this work is to provide a framework that optimizes for *any* given figure of merit, the following evaluations focus on how well the framework optimizes for these exemplary figures of merit and not on how suited these figures of merit are to assess the compilation task itself.

To generate baseline values, all evaluation circuits are compiled for all seven currently supported devices using both Qiskit’s and TKET’s most optimized (pre-configured and fixed) compilation settings *O3* respectively *O2*—leading to 14 different compiled circuits as baselines.

7.2 Results

Fig. 6 shows the results obtained by using the proposed framework to optimize the expected fidelity. Each tick on the x-axis denotes one evaluation circuit that is assessed using the 14 baseline compilation flows (7 devices $\times$ 2 compilers) and the MQT Predictor. Benchmarks leading to only zero values have been excluded from all evaluations. Since (maximally) 15 values per tick would be hard to read, the graph is reduced to only show the

- MQT Predictor evaluation score in blue,
- best evaluation score in green,
- median evaluation score in yellow, and
- worst evaluation score in red.

⁴Those ratios are actually subtracted from 1 such that higher values indicate a more parallel quantum circuit that is assumed to be better.

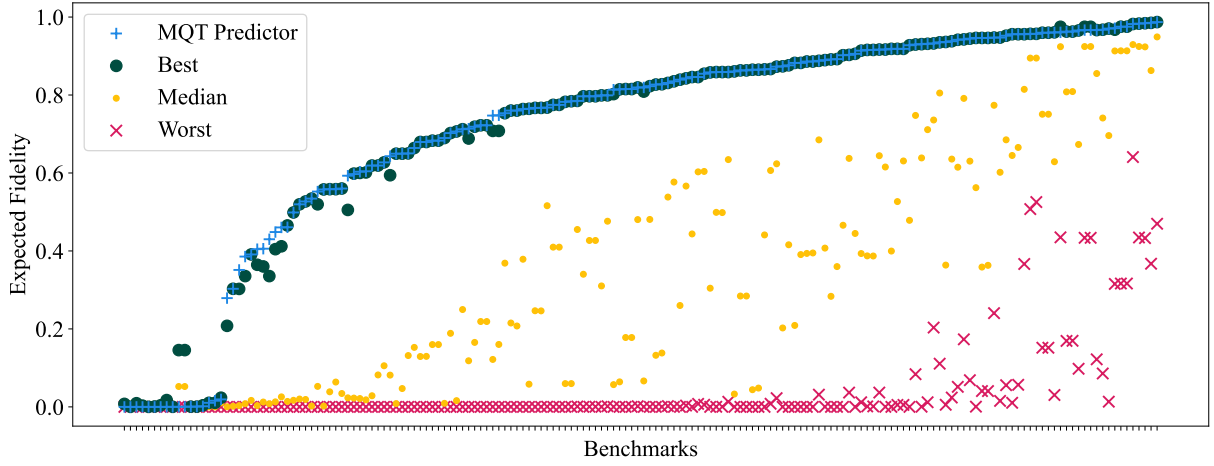


Fig. 6. Evaluation of the expected fidelity. All benchmarks are sorted by their MQT Predictor score for better readability.

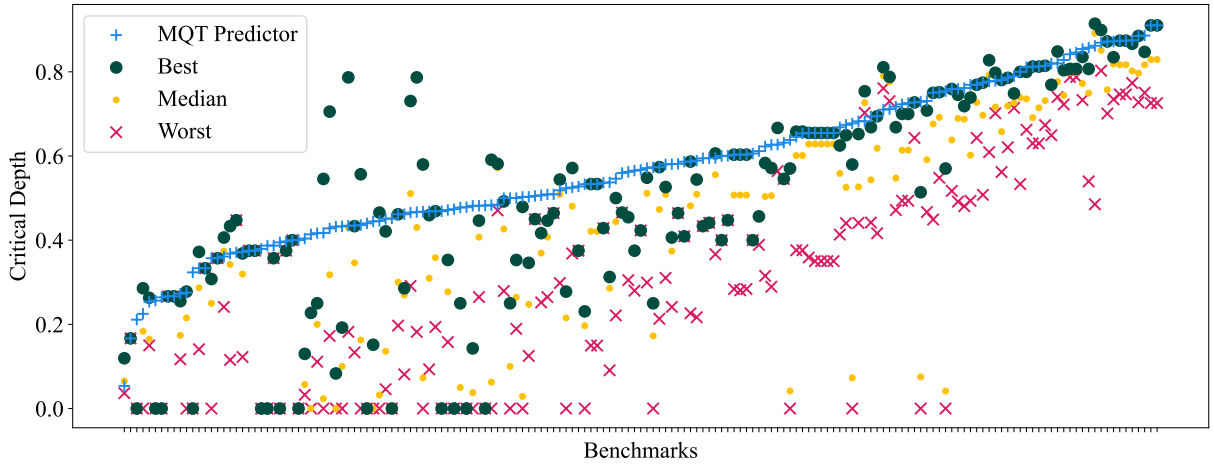


Fig. 7. Evaluation of the critical depth. All benchmarks are sorted by their MQT Predictor score for better readability.

In this setup, the proposed method is capable of producing compiled quantum circuits that are within the top-3 of all 14 baselines in over 98% of cases while frequently outperforming all baselines by up to 53%. Therefore, especially end-users who are not experts in quantum computing can use it to reliably and automatically compile quantum circuits whose quality otherwise can only be achieved by many manual experiments—often infeasible due to the necessary time effort and expert knowledge. These results also clearly highlight that choosing the right device and compiler can make the difference between a successful execution and obtaining completely random results.

To demonstrate the ability to optimize for customizable figures of merit, we additionally set up the framework to optimize for critical depth and show the respective results in Fig. 7. Again, benchmarks only leading to zero values have been excluded from the evaluations. Since it is not possible to define a custom figure of merit/optimization criterion for both Qiskit and TKET, the same compiled baseline evaluation circuits are used. The respective graph has been reduced in a similar fashion as described above. For 89% of cases, the compiled circuits produced by MQT Predictor are within the top-3 of the 14 baselines while frequently outperforming all baselines by up to 400%.

Table 1. Comparison of the influence of the figure of merit.

Model trained for...	Average result for...	
	Exp. Fidelity	Critical depth
Exp. Fidelity	0.67	0.42
Critical depth	0.12	0.55

These results describe the performance from the end-users’ perspective, who use the MQT Predictor framework to automatically select the most promising device and compile accordingly. To provide more insight into the underlying training data, the ideal device distribution for the evaluated benchmarks is explored in [Appendix A.1](#). Additionally, the impact of both the device selection as well as the compilation has been examined in an isolated fashion in [Appendix A.2](#) and [Appendix A.3](#), respectively.

To further investigate the frameworks ability to optimize for customizable figures of merit, the average evaluation scores for both figures of merit are calculated for both respectively trained frameworks. The resulting numbers are denoted in [Table 1](#) and confirm that the framework trained for a particular figure of merit indeed results in the compiled circuits with the highest evaluation scores for both cases.

7.3 Generalizability

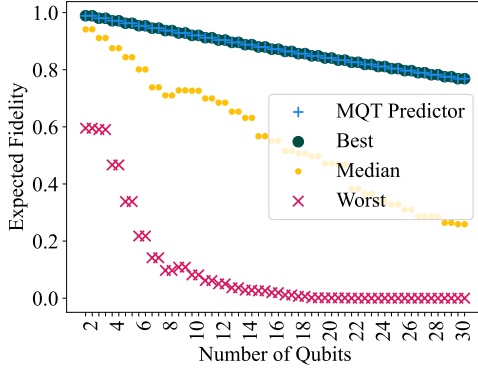
The performance of machine learning models, in general, is highly dependent on the quality of the training data. Therefore, training data should be as heterogeneous as possible to cover a wide variety of circuits with different characteristics to achieve a high *generalizability*. For that purpose, a multitude of different algorithms are considered in the training data for both the ML and RL models described above—ranging from quantum circuit building blocks such as, e.g., the *quantum fourier transform* and *amplitude estimation* towards rather complex quantum circuits such as, e.g., ground state estimation—all taken from *MQT Bench* [58].

To estimate how well the trained MQT Predictor works for not only the quantum circuits that are part of the training data but also for other algorithms that have not been considered during training, a further evaluation has been conducted. To evaluate generalizability, quantum circuits for the *Greenberger–Horne–Zeilinger* (GHZ) state, which have not been considered during training, are evaluated in a similar fashion as before in [Section 7.2](#). The resulting scores for expected fidelity and critical depth are denoted in [Fig. 8a](#) and [Fig. 8b](#), respectively.

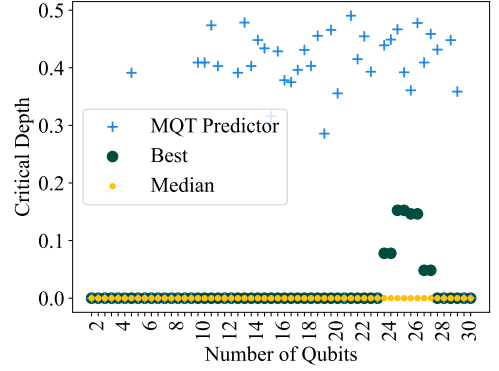
Taking into account both figures of merit, a performance similar to the results described in [Section 7.2](#) is achieved. Considering the expected fidelity, MQT Predictor leads to the same performance as the best baseline for all cases. In case of the critical depth as a figure of merit, MQT Predictor even outperforms all baselines in most cases. It should be noted that the best baseline results for critical depth frequently are 0.0 (since *all* two-qubit gates are on the longest path), while the MQT Predictor achieves significant improvements in many of these cases. Although this is only a small evaluation of the generalizability of the proposed methodology, it indicates that it also works well in untrained scenarios.

8 DISCUSSION

Classical machine learning has shown tremendous capabilities to solve problems from various domains such as classical compilation and beyond. However, its potential success inherently depends on gathering a sufficiently large amount of high-quality training data and, by that, it is not per-se well suited for changes, which often require to re-train the underlying machine learning models. This section specifically gives guidance how the MQT Predictor deals with such changes in various ways.



(a) Expected fidelity for GHZ circuits.



(b) Critical depth for GHZ circuits.

Fig. 8. Generalizability evaluation for GHZ circuits which were not part of the training processes.

8.1 Changes in Hardware Information used in Figure of Merit

Quantum computers are frequently calibrated to maximize the actual gate fidelities, e.g., used in the figure of merit introduced in Section 3.2. Thus, these calibration updates affect the fidelity data used throughout both the RL and ML training as part of the cost function. Consequently, the models need to be re-trained. Unfortunately, it is not feasible to always generate all training data from scratch whenever such a calibration is conducted since it may occur on a daily (or even hourly) basis, while the training of the proposed framework instantiation took approximately 24 hours on a consumer MacBook Pro. As with any AI solution currently, a natural way to speed up the training time would be to utilize supercomputers, e.g., using *High Performance Computing* clusters. Future work may focus on machine learning techniques to adjust existing models to new circumstances that do not involve re-training from scratch.

8.2 Additional Compilation Passes and Figures of Merit

The quantum software ecosystem is moving fast and quantum SDKs frequently release new versions of their toolchains. In addition, many new research methods for quantum circuit compilation are proposed regularly. In order to keep up with this pace, the MQT Predictor has been designed with a unified and modular interface that allows one to easily add and/or update compilation passes. The good news is that the degree of change—whether just another optimization pass has been added or whether all of it changed—does not affect the overall effort. The bad news is that any such change still requires re-training of all RL and ML models. Also here, future work may focus on finding ways to re-train models without starting from scratch.

A similar effort is necessary when changing or adapting the figure of merit. Since both the RL and the ML models optimize for it during training, again, the whole MQT Predictor framework must be re-trained and the persistently stored data must be refreshed.

8.3 Changes in Supported Quantum Devices

New quantum devices become available on a regular basis. Fortunately, adding a new device to the proposed framework is rather easy. It is only necessary to train the respective RL model and to determine the respective evaluation score based on the chosen figure of merit for all training circuits. Afterwards, the ML model can be re-trained in minutes. This is enabled by persistently storing the compiled circuits and evaluation scores of previously conducted compilations as part of the training data generation process. On the contrary, no additional effort is needed to exclude devices—either in case of a temporary downtime due to, e.g., calibration, or a permanent deprecation. These devices can just be masked out without any further effort necessary.

9 CONCLUSION

Using quantum computing as a technology to *just* solve a problem from any kind of application domain is challenging, since a suitable quantum device must be selected to solve a problem encoded as a quantum circuit and the circuit must be compiled accordingly. Deciding which is the best device and how to best compile for a certain application—according to a *figure of merit*—currently requires expert knowledge in quantum computing. So far, especially end-users from application domains are often left unsupported and overwhelmed due to missing automation and tool support. This situation is even worsened, since the result quality heavily fluctuates with the particular choice of options and often makes the difference between a useful result and a useless one due to too much noise.

In this work, we proposed the MQT Predictor framework to automate the selection of suitable target devices and the subsequent compilation. By that, end-users are freed from this task with the goal of preventing a scenario where quantum computers can only be utilized by quantum computing experts—hindering the overall adoption of the technology. The proposed methodology allows learning on the basis of previously conducted compilations of quantum circuits for various devices. For that, the problem is tackled from two angles: Using *Reinforcement Learning*, optimal compilation pass sequences are learned for all available devices—mixing and matching compilation passes from various sources and quantum SDKs. Furthermore, using *Supervised Machine Learning*, a prediction of a quantum device is made for a given quantum circuit according to customizable figures of merit such as, e.g., the *expected fidelity* of the to-be-compiled quantum circuit.

The proposed framework has been implemented using more than 500 quantum circuits from 2 to 90 qubits for seven devices from two qubit technologies and has shown to yield compiled circuits that are within the top-3 in more than 98% of cases while frequently outperforming any tested combination by up to 53% when considering the expected fidelity as the figure of merit. Furthermore, trained and evaluated for *critical depth* as another figure of merit, the performance was within the top-3 in 89% of cases while frequently outperforming any tested combination by up to 400%. The corresponding framework (which is part of the *Munich Quantum Toolkit* (MQT)) including its pre-trained models and classifiers is publicly available on GitHub (<https://github.com/cda-tum/mqt-predictor>) and as an easy-to-use *Python* package (<https://pypi.org/p/mqt.predictor>).

These results clearly demonstrate that adapting technologies from the classical realm and applying them to quantum computing can significantly foster the progress of this new and promising technology. Especially since quantum computers are mostly utilized by experts at the moment due to the challenges within the current workflow for realizing quantum application on real machines. With this work, we hope to take a step forward in simplifying the utilization of quantum computers for end-users from application domains and the development of optimized compilers within the community.

ACKNOWLEDGMENTS

This work received funding from the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation program (grant agreement No. 101001318), was part of the Munich Quantum Valley, which is supported by the Bavarian state government with funds from the Hightech Agenda Bayern Plus, and has been supported by the BMWK on the basis of a decision by the German Bundestag through project QuaST, as well as by the BMK, BMDW, the State of Upper Austria in the frame of the COMET program, and the QuantumReady project within Quantum Austria (managed by the FFG).

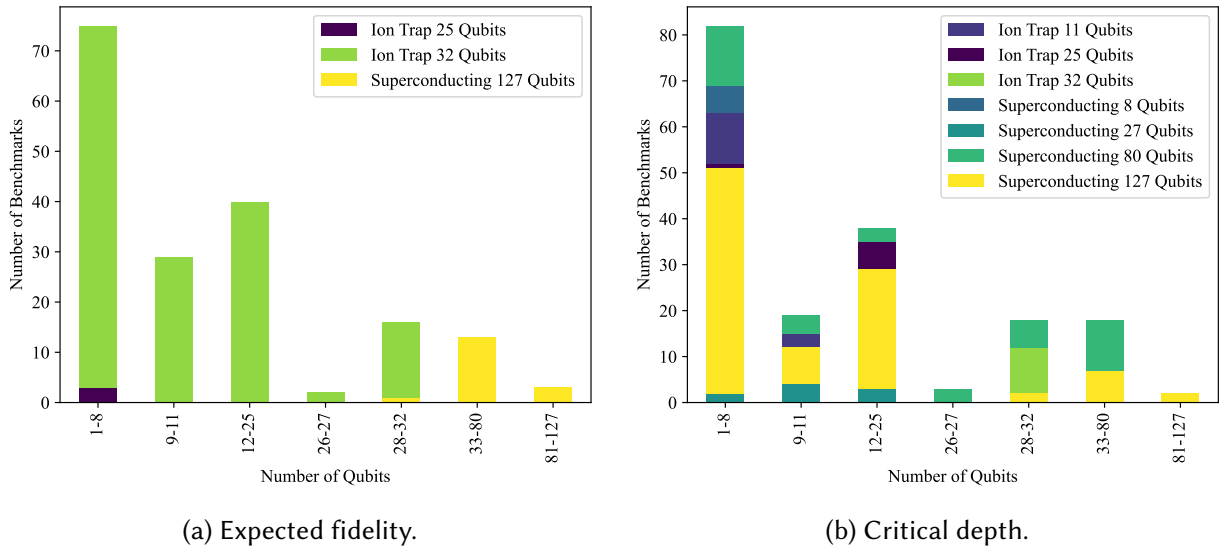


Fig. 9. Device distributions.

A APPENDICES

A.1 Device Distribution

To better understand the problem of selecting the best device, the underlying ground truth data used to train the MQT Predictor is further investigated. To this end, the frequency with which each device is selected as the best performing one is investigated. For that, each benchmark is compiled for all supported devices using the respectively trained RL compilers and the one leading to the best result is determined. The results are visualized in Fig. 9. Furthermore, the benchmarks are grouped by their number of qubits such that all benchmarks within one group can be executed on the same number of devices—leading to seven groups since currently seven devices are supported with a decreasing number of available devices for an increasing number of qubits.

When using the expected fidelity as the figure of merit, the results are rather distinct as visualized in Fig. 9a. For most benchmarks, the ion trap device with 32 qubits results in the best performance. Therefore, the rule of thumb known by quantum computing experts—taking an ion trap device when possible—has been confirmed. However, when all ion trap devices’ capacities are exceeded, the largest superconducting device is best even when a smaller device would have been available as well. This showcases that another quantum expert rule of thumb—taking always the smallest but fitting device—cannot be confirmed by our experimental data.

When using the critical depth as the figure of merit, the device distribution is significantly more diverse as visualized in Fig. 9b⁵. Here, all seven devices are chosen at least sometimes and there is no apparent correlation between the best performing device and the benchmark groups since most devices result in the best performance for more than just one group. Consequently, no clear guidance can be derived from that—underlining the importance of software tools to guide this decision automatically.

⁵The benchmarks used for the evaluation differ between both figures of merit and, therefore, the number of benchmarks in each group differs as well.

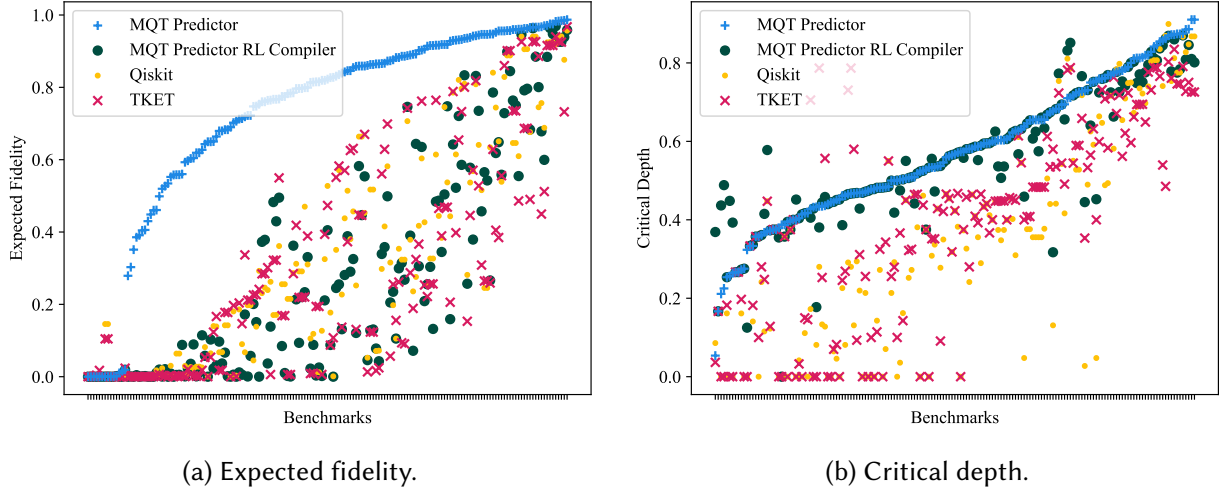


Fig. 10. Isolated evaluation of the device selection using ML.

A.2 Detailed Evaluation: Device Selection Using ML

The MQT Predictor framework automatically selects the most promising device and compiles accordingly. To assess the effect each of the two tasks has, they are evaluated in an isolated fashion—starting with the device selection and its respective results shown in Fig. 10. To this end, the largest device (superconducting device with 127 qubits) is used as the default device to fit all benchmarks. Then, the benchmarks are compiled for it using Qiskit, TKET, and the MQT Predictor RL compiler and, similarly to Fig. 6 and Fig. 7, the benchmarks are sorted by their MQT Predictor score for better readability. The resulting performances are then compared against the full MQT Predictor approach including the device selection.

For the expected fidelity, the results are visualized in Fig. 10a—showing a distinct gap between the results of the full MQT Predictor approach including the device selection compared to all three baselines using the default device. This highlights the importance of selecting the most suitable device, since that can make the difference between a successful execution and obtaining completely random results. Furthermore, it is interesting to see that there is no clear winner among Qiskit, TKET, and the MQT Predictor RL compiler.

In the case of critical depth, the situation is different as visualized in Fig. 10b since the full MQT Predictor approach less frequently leads to the best result. This suggests that the default device is a good choice for critical depth and the influence between it and the best one is smaller than it is the case for the expected fidelity. Also, this matches with Fig. 9b since often the largest device is the most suitable in the ground truth data. Furthermore, the MQT Predictor RL compiler approach outperforms both Qiskit and TKET in most cases—showcasing, that both are not optimizing for critical depth but optimize for a different implicit underlying criterion closer to the expected fidelity.

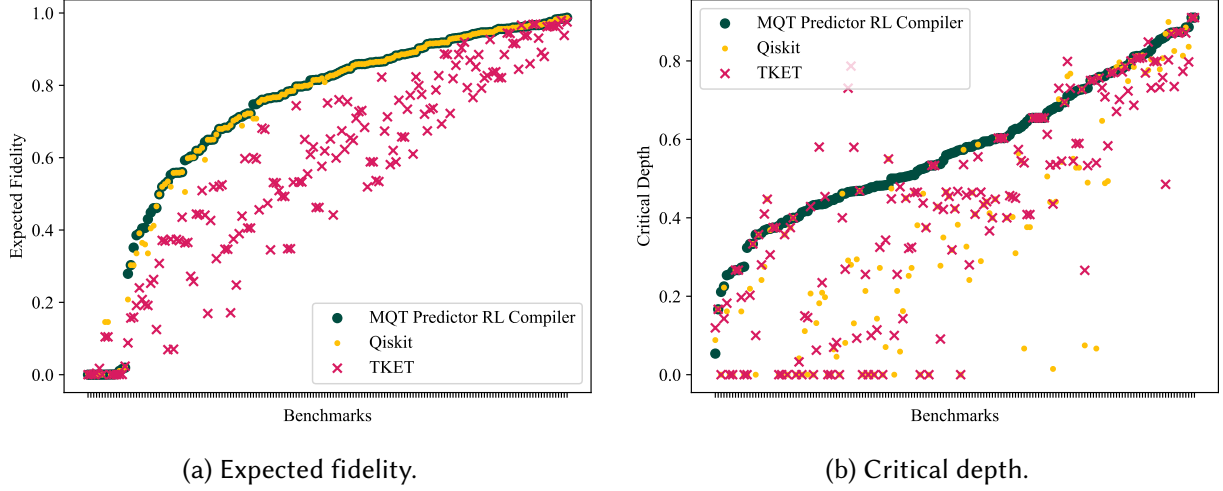


Fig. 11. Isolated evaluation of the compilation using RL.

A.3 Detailed Evaluation: Compilation Using RL

To assess the compilation quality of the RL part of the MQT Predictor framework, it is evaluated in a similar and isolated fashion as the device selection described in [Appendix A.2](#). To this end, the device is set to be the one predicted for each benchmark and the effect of the different compilers is evaluated in [Fig. 11](#). For that, the same compilers as before are used: Qiskit, TKET, and the MQT Predictor RL compiler—while, in this case, the latter corresponds to the full MQT Predictor approach because the default device is the predicted one. Therefore, this evaluation setup results in plots that show a subset of the data points plotted in [Fig. 6](#) respectively [Fig. 7](#) in which the compilers are evaluated for all seven devices while here only the predicted one is used. The benchmarks are sorted by their MQT Predictor RL compiler scores for better readability (which, in this case, corresponds to the scores of the full MQT Predictor approach).

For the expected fidelity visualized in [Fig. 11a](#), the MQT Predictor RL compiler outperforms TKET and Qiskit, although there is only a small performance gap to the latter since Qiskit performs better than TKET for the predicted device. Compared to [Fig. 10a](#) where the default device was the largest superconducting one, this was not the case. Therefore, apparently the compilation quality significantly depends on the device—either by Qiskit being better for the predicted one (which is often an ion trap one), TKET being worse, or both. Nevertheless, the plot shows the (small) improvement of the MQT Predictor compilation—potentially growing when including more compilation passes from further providers.

In the case of the critical depth visualized in [Fig. 11b](#), the performance differences between Qiskit and TKET are not as distinct as before. Furthermore, the performance difference of the MQT Predictor RL compiler compared to both of them is even larger—highlighting the impact it has.

MQT Bench: Benchmarking Software and Design Automation Tools for Quantum Computing

Nils Quetschlich¹, Lukas Burgholzer¹, and Robert Wille^{1,2}

¹Chair for Design Automation, Technical University of Munich, Germany

²Software Competence Center Hagenberg GmbH (SCCH), Austria

Quantum software tools for a wide variety of design tasks on and across different levels of abstraction are crucial in order to eventually realize useful quantum applications. This requires practical and relevant benchmarks for new software tools to be empirically evaluated and compared to the current state of the art. Although benchmarks for specific design tasks are commonly available, the demand for an overarching cross-level benchmark suite has not yet been fully met and there is no mutual consolidation in how quantum software tools are evaluated thus far. In this work, we propose the *MQT Bench* benchmark suite (as part of the *Munich Quantum Toolkit*, MQT) based on four core traits: (1) cross-level support for different abstraction levels, (2) accessibility via an easy-to-use web interface (<https://www.cda.cit.tum.de/mqtbench>) and a Python package, (3) provision of a broad selection of benchmarks to facilitate generalizability, as well as (4) extendability to future algorithms, gate-sets, and hardware architectures. By comprising more than 70,000 benchmark circuits ranging from 2 to 130 qubits on four abstraction levels, MQT Bench presents a first step towards benchmarking different abstraction levels with a single benchmark suite to increase comparability, reproducibility, and transparency.

1 Introduction

Quantum computing has gained a lot of attention due to its promising applications and the advances in quantum computing hardware. But still, designing quantum algorithms often means to manually implement quantum circuits on the gate-level—similar to assembly language in classical computing. Hence, there is an urgent need to enhance the workflow of designing and testing quantum algorithms. Without software tools that aid in the design of quantum algorithms, the underlying hardware may not be efficiently utilized. Thus, researchers and developers have already proposed tools for various design tasks, such as quantum circuit simulation [1]–[13], compilation [14]–[34], or verification [35]–[50]. This already led to comprehensive quantum circuit design flows realized through toolkits such as IBM’s Qiskit [51], Google’s Cirq [52], or Rigetti’s Forest [53]—in addition to numerous tools and methods developed by other researchers and engineers in the field.

Generally, these software tools operate on and across different levels of abstraction and all tackle computationally hard problems. As a result, they have to compromise between resource demands and result quality. Usually, when a new software tool is proposed, its performance is empirically evaluated. The question how to evaluate the performance of a software tool is very challenging and not yet fully answered—especially for tools solving NP-complete problems. The most common approach is to run certain problem instances, so-called benchmarks, and compare the performance of the newly proposed method against state-of-the-art methods regarding a certain characteristic, e.g., run-time or solution quality.

Nils Quetschlich: nils.quetschlich@tum.de

Lukas Burgholzer: lukas.burgholzer@tum.de

Robert Wille: robert.wille@tum.de

Currently, this benchmarking is conducted using a wide variety of different benchmark suites—each with a specific focus. While some benchmark suites, e.g., [54], [55], provide benchmarks on higher abstraction levels, other benchmark suites focus on lower abstraction levels, e.g., [56], [57]. Although the intended target levels are properly covered, the demand for a cross-level benchmark suite is not fully met yet. As another consequence, there is no mutual consolidation which benchmarks to use for empirical evaluations yet—leading to lower comparability, reproducibility, and transparency.

In this paper, we propose *MQT Bench*—the benchmark suite from the *Munich Quantum Toolkit* (MQT) which explicitly aims to address those drawbacks. It aims at providing a first step towards benchmarking the whole quantum software stack with a single benchmark suite by offering the same benchmark algorithms on different levels of abstraction. To realize such a benchmark suite, several challenges have to be tackled:

- **Distinct requirements on different levels:** Benchmarks have to fulfill certain requirements depending on the abstraction level, e.g., only gates of a device’s native gate-set are allowed.
- **Accessibility:** To foster adoption of the benchmark suite, it needs to be as easy to use as possible.
- **Generalizability:** Providing a comprehensive set of benchmark algorithms to cover as many use cases as possible.
- **Extendability:** Future algorithms, gate-sets, and architectures should be easy to integrate.

MQT Bench has been developed with these challenges in mind and is based on four core traits:

1. **Cross-level benchmarking:** Provision of benchmarks on four abstraction levels.
2. **Accessibility:** To simplify the usage of MQT Bench, we provide a web interface (<https://www.cda.cit.tum.de/mqtbench>) and a Python package in addition to the open-source repository on GitHub (<https://github.com/cda-tum/mqt-bench>).
3. **Algorithm selection:** Broad selection of different algorithms ranging from building blocks, i.e., Quantum Fourier Transform (QFT), to applications, i.e., Grover’s algorithm.
4. **Extendability:** Algorithms, gate-sets, and hardware architectures are easily extendable and integrable into MQT Bench.

By this, MQT Bench aims to improve the comparability, reproducibility, and transparency of empirical evaluations for the whole quantum software stack.

The rest of this work is structured as follows: In [Section 2](#), we review the necessary basics of the quantum computing compilation flow and the respective software stack to keep this work self-contained. Afterwards, [Section 3](#) reviews the state of the art on how this quantum software stack is currently evaluated and benchmarked—motivating the benchmark suite proposed in this paper. Based on that, [Section 4](#) introduces the resulting benchmark suite and its core traits, before the generated benchmarks are evaluated in [Section 5](#). [Section 6](#) concludes this work.

2 Background

To keep this paper self-contained, this section gives a brief overview of the quantum circuit compilation flow with its different abstraction levels and the respective quantum software stack. The levels mentioned in this section are inspired by the structure proposed by the openQASM 3.0 specification [58] and can be found similarly in many other compilation flows as well.

2.1 Quantum Circuit Compilation Flow

Similar to the classical domain, executing a conceptual quantum algorithm on an actual device requires compiling it to a representation that adheres to all constraints imposed by the hardware. Usually, quantum algorithms are initially developed and tested on a hardware-agnostic level. Generally, this is described as a quantum circuit that consists of high-level building blocks without any restrictions to a certain gate-set or hardware architecture and is defined as the *algorithmic level*.

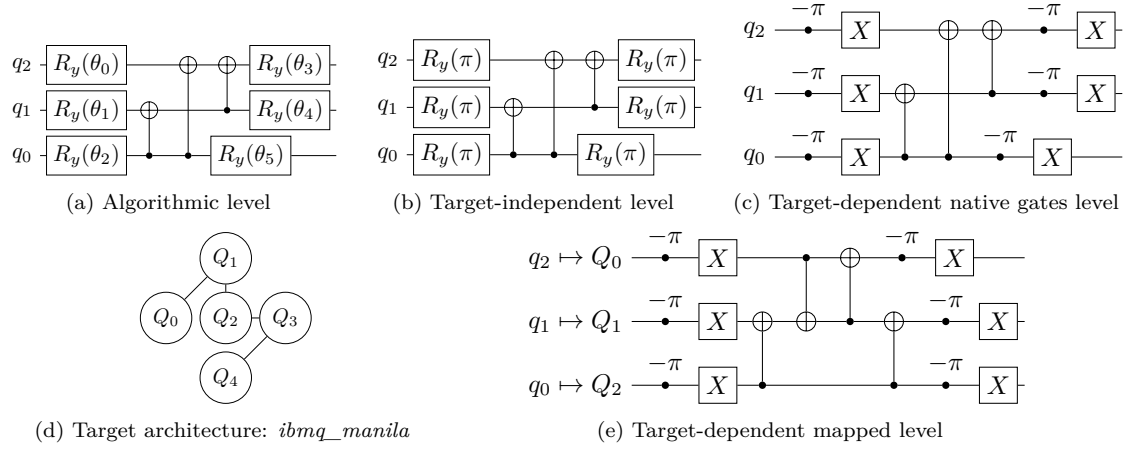


Figure 1: Compilation of a Variational Quantum Algorithm.

Example 1. Variational Quantum Algorithms (VQAs) are an emerging class of quantum algorithms with a wide range of applications. A respective circuit is depicted in Fig. 1a and shows an example of an ansatz function frequently used for Variational Quantum Eigensolvers (VQEs), a subclass of VQAs. On this abstraction level, the circuit is parameterized by the angles θ_i of the six single-qubit gates.

The first step in realizing a conceptual quantum algorithm on the algorithmic level for a particular problem is to synthesize the high-level building blocks and optimize the resulting representation independently of the actual target architecture. In analogy to a classical compiler, this involves tasks such as constant propagation and folding, gate modifier evaluation, synthesis, loop unrolling, and gate simplification. The resulting representation is defined as the *target-independent level*.

Example 2. VQAs are hybrid quantum-classical algorithms, where the parameters of the quantum ansatz are iteratively updated by a classical optimizer analogous to conventional gradient-based optimization. Consider again the circuit from Fig. 1a. Assuming that these parameters have been determined, e.g., $\theta_i = \pi$ for $i = 0, \dots, 5$, they are now propagated and the resulting quantum circuit is shown in Fig. 1b.

Today’s quantum devices impose several constraints on the circuits that may be executed on them. Thus, a target-dependent compilation phase is necessary. For one, devices only provide a particular set of native gates—typically consisting of an entangling gate (like the CX gate) and some family of single-qubit gates. Therefore, the gates of a circuit need to be translated to this native gate-set, which is typically followed by an optimization pass that aims to reduce the introduced overhead. This representation level is defined as the *target-dependent native gates level*.

Example 3. Different quantum computer realizations support different native gate-sets. In our example, we consider the `ibmq_manila` device as the target device that natively supports I , X , $\sqrt{X}$, R_z , and CX gates. Consequently, the R_y gates in Fig. 1b have to be converted using only these native gates. In this case, they are substituted by a sequence of R_z (denoted as $\bullet$ with a phase of $-\pi$) and X gates as shown in Fig. 1c.

In addition to the limited gate-set, today’s devices (at least those based on superconducting qubits) only feature limited connectivity between their qubits. Consequently, the circuit’s logical qubits need to be *mapped* to the targeted device’s physical qubits, so that multi-qubit gates are only applied to qubits directly connected on the device. Since such a mapping can rarely be determined in a static fashion, i.e., globally for the whole circuit, the mapping has to change dynamically throughout the circuit, which is frequently referred to as mapping or routing. Again, this is followed by a round of optimization in order to minimize the overhead caused by the compilation. This representation level is defined as the *target-dependent mapped level*.

Example 4. Consider again the scenario from [Example 3](#). The architecture of the `ibmq_manila` device is shown in [Fig. 1d](#) and defines between which qubits a two-qubit operation can be performed. Since the circuit shown in [Fig. 1c](#) contains CX gates operating between all combinations of qubits, there is no mapping directly matching the target architecture’s layout. As a consequence, a non-trivial mapping followed by a round of optimization leads to the resulting circuit shown in [Fig. 1e](#). This is also the reason for the different sequence of CX gates compared to the previous example.

At this point, the circuit is ready to be sent to the quantum computer’s backend for execution. From there, the individual gates are scheduled, linked to a particular calibration, and the resulting circuit is eventually passed to the target machine code generator. The resulting binaries are then forwarded to the execution engine to orchestrate the quantum computation on the actual device.

2.2 Quantum Software Stack

Quantum software is needed on all levels reviewed above in order to aid designers in eventually realizing useful quantum applications. In the following, we review three of the core design tasks for software tools.

Today, quantum computers are a scarce resource with limited availability and capability. Until more devices become available that are larger and less prone to errors, classical means to simulate quantum algorithm circuits (also known as *quantum circuit simulators*) are essential for fostering the development of quantum applications. Additionally, and in contrast to actual quantum computing devices, quantum circuit simulators allow for detailed insights into the quantum states throughout the circuit execution since the state’s amplitudes are explicitly tracked during the simulation. On actual hardware, information can only be extracted in the form of measurements from the final quantum state, which each yields a classical bit string according to the distribution described by the state’s amplitudes. Such simulators can be used on the highest possible level of abstraction, since they can be designed in a way that does not require restrictions on the circuit’s gate-set or the qubits’ connectivity, e.g., as proposed in [\[1\]–\[13\]](#). Furthermore, they might also be used on the lowest possible level in order to perform noise-aware simulations to estimate how a circuit is likely to perform on an actual device, e.g., as proposed in [\[59\]–\[62\]](#). Either way, on both abstraction levels, quantum circuit simulation is non-trivial since corresponding representations of states and operations, in general, require exponential space or runtime—requiring powerful software tools.

Due to the immense complexity of the tasks involved in quantum circuit compilation (as it has been illustrated in [Section 2.1](#)), e.g., mapping being NP-complete [\[63\]](#), manually conducting compilation often is not an option. Consequently, efficient software tools are needed across all levels of abstraction of the aforementioned flow in order to realize a conceptual algorithm on an actual device. Thus, various compilation tools have been proposed, e.g., [\[14\]–\[34\]](#).

During compilation, an algorithm’s description is considerably altered and transformed. Naturally, it is of utmost importance to ensure that the originally intended functionality is preserved through all levels of abstraction. This procedure is called verification. While the underlying principle of comparing the transformations represented by different quantum circuits is conceptually simple, the exponential size of the underlying matrices makes this problem challenging—it has been shown to be QMA-complete [\[64\]](#). Due to the increasing complexity of today’s compilation flows, tools for verifying their results become increasingly important. Examples of verification tools have been proposed in [\[35\]–\[50\]](#).

These design tasks demonstrate the wide variety of software tools operating on and across different levels of abstraction—leading to comprehensive quantum circuit design flows realized through toolkits such as IBM’s Qiskit [\[51\]](#), Google’s Cirq [\[52\]](#), or Rigetti’s Forest [\[53\]](#). At the same time, all these tasks have in common that they have an immensely large complexity. Therefore, a multitude of techniques have been proposed for each task—each with its own trade-off between resource demand and quality of the result. It is key in the development of scientific methods to empirically evaluate and compare their performances on practical, relevant benchmarks.

3 Benchmarking

The benchmark suite proposed in this paper is not the first and certainly not the last collection of quantum circuit benchmarks. In this section, we review some of the existing suites and their respective foci. Afterwards, we discuss how MQT Bench further complements this.

3.1 Current State of the Art

Providing a comprehensive set of benchmarks to satisfy the needs across all levels of the quantum circuit compilation flow is a challenging task. Several approaches to provide benchmark suites for some of the levels within the quantum circuit compilation flow have already been proposed and a non-exhaustive overview is given in the following:

Application-Oriented Performance Benchmarks for Quantum Computing [54]: Starting with an example located on the algorithmic level, this benchmark suite rather focuses on high-level descriptions. At the moment of writing, it provides 13 benchmark algorithms which can be generated with a variable number of qubits via jupyter notebooks using multiple software compilation flows and are classified into four categories of complexity. The resulting circuits are available as high-level Python objects without any further compilation steps.

SupermarQ [55]: Similarly, SupermarQ also focuses on the algorithmic level by providing benchmarks with an adjustable qubit range for eight algorithms as high-level Python objects via a Python package. Additionally, six feature vectors are proposed to describe the characteristics of the benchmarks.

QASMBench [56]: This benchmark suite focuses on the target-independent level. It offers numerous quantum circuits with a wide but fixed range of both the number of qubits and depth. It classifies the benchmarks into three categories of sizes and three categories of algorithm classes. All circuits are available in an intermediate representation according to the openQASM 2.0 specification [65].

RevLib [57]: This benchmark suite provides a wide variety of reversible circuits in an intermediate representation format specified by the authors. Classical functions are frequently embedded into reversible circuits in order to use them in quantum algorithms, e.g., oracle functions or Boolean building blocks such as the modular exponentiation in Shor’s algorithm [66]. In addition, decomposed versions of this benchmark suite have found widespread use in evaluating the performance of mapping tools, e.g., in [32]–[34]. Since reversible circuits merely form a subclass of quantum circuits and do not employ quantum mechanical effects such as superposition and entanglement, they do not serve as adequate benchmarks for the whole stack.

3.2 Motivation

All of the mentioned benchmark suites have in common that they each target a specific abstraction level within the quantum circuit compilation flow. Although the respectively intended target level is well covered, the demand for a cross-level benchmark suite is not fully met yet. Additionally, so far, there is no mutual consolidation which benchmarks to use for empirical evaluations of software tools. This results in a lower comparability, reproduceability, and transparency of results and may even cause confusion because benchmarks believed to realize the same particular task are quite frequently realized in a completely different fashion, e.g., a benchmark called "grover" could realize Grover’s algorithm with any particular oracle. Consequently, the demand for a benchmark suite that spans the whole stack of abstraction levels constitutes the main motivation of this contribution. In order to realize such a benchmark suite, several challenges have to be tackled:

- Distinct requirements for benchmark circuits on different levels: The closer a level is to the actual computing hardware, the more requirements have to be fulfilled, e.g., only gates from the device’s native gate-set are allowed and the connectivity of the hardware architecture must be considered.
- Accessibility: In order to facilitate its adoption, the benchmark suite needs to be as easy to use as possible. While most benchmark suites already provide open-source access, running and adapting the code to the user’s needs, e.g., obtaining a selection of relevant benchmarks, frequently is tedious or impossible and a solution that keeps this hurdle low is desirable.

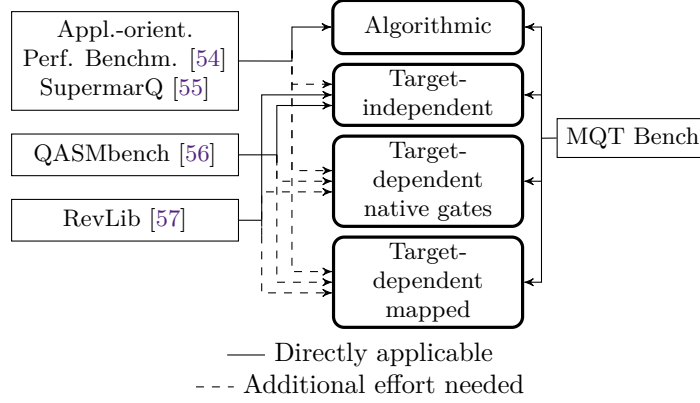


Figure 2: Benchmarking software tools with and without MQT Bench.

- Generalizability of a software tool: This can only be guaranteed if it is evaluated on a broad set of benchmarks, relating to several characteristics, e.g., the number of qubits, circuit depth, or the application domain.
- Extendability: In addition to that, quantum computing is rapidly advancing and the extendability of the benchmark suite to future algorithms, gate-sets, and architectures becomes even more important.

MQT Bench has been developed with these challenges in mind and aims to be a first step towards covering the whole quantum software stack with a benchmark suite based on the following four core traits (which are described in detail in Section 4):

1. Cross-level benchmarking: All benchmarks are provided on four abstraction levels reviewed in Section 2.1.
2. Accessibility: A website is provided (<https://www.cda.cit.tum.de/mqtbench>) to simplify the usage of MQT Bench as much as possible. Additionally, all benchmarks can also be generated on-demand using our Python package. While most users' needs should be covered by this, we also give access to the open-source repository on GitHub (<https://github.com/cda-tum/mqt-bench>).
3. Algorithm Selection: Broad selection of different benchmarks with parameterizable characteristics ranging from building blocks, i.e., QFT, to applications, i.e., Grover's algorithm.
4. Extendability: MQT Bench is easily extendable with respect to available benchmarks, native gate-sets, and hardware architectures.

Benchmarking quantum software tools and compilation flows with the same benchmarks aims to aid comparability, reproducibility, and transparency of empirical evaluations. How this leads to more consistent testing is shown in the following example.

Example 5. Consider a full quantum compilation flow across all four levels which shall be developed and tested as depicted in Fig. 2. On the left-hand side of the levels, the current state of the art of benchmarking is sketched. To comprehensively test each level, different benchmark suites have to be adapted and used. In contrast, MQT Bench replaces all of those and enables testing software tools with the same benchmarks throughout the whole quantum circuit compilation flow as shown on the right-hand side.

The screenshot shows the TUM MQT Bench website interface. It features a navigation bar at the top with links: TUM MQT Bench, Benchmarks, Level and File Descriptions, Benchmark Descriptions, More on our Work, and Legal Information. The main content area is divided into four columns:

- Algorithm Level:** A note stating "Currently, only available using the Python package. Details can be found on our [GitHub page](#)." Below this is a "Download" section showing "Number of selected benchmarks: 0" and a note about downloading benchmarks as a zip archive. A blue button labeled "Download selected benchmarks" is at the bottom.
- Target-independent Level:** A section titled "Select the used compiler:" with radio buttons for Qiskit and TKET.
- Target-dependent Native Gates Level:** A section titled "Select targeted native gate-set:" with radio buttons for IBM, Rigetti, OQC, IonQ, and Quantinuum. Below this is "Select the used compiler:" with radio buttons for Qiskit and TKET, and a sub-section for optimization levels (Opt. 0 to Opt. 3) if Qiskit is selected.
- Target-dependent Mapped Level:** A section titled "Select a targeted device:" with radio buttons for IBM (Mumbai, Montreal, Eagle, Aspen-M2, Osprey), OQC (Lucy, Hydra), IonQ (Harmony, Eagle, Zenita), and Quantinuum (H1, H2). Below this is "Select the used compiler:" with radio buttons for Qiskit and TKET, and a sub-section for optimization levels (Opt. 0 to Opt. 3) if Qiskit is selected, and a sub-section for placement settings (Graph Placement, Line Placement) if TKET is selected.

Figure 3: User interface of the MQT Bench website.

4 MQT Bench

In this section, we describe in more detail how each of the four core traits mentioned above is implemented.

4.1 Cross-Level Benchmarking

As exemplarily illustrated in Section 2, software development in quantum computing takes place on various levels. While there arguably are numerous possible levels, MQT Bench focuses on four as a balanced measure between specificity and generizability—inspired by the structure proposed in the openQASM 3.0 specification [58]. Providing these benchmarks on all of the four abstraction levels is the main contribution of this work. More precisely, the following levels are covered:

1. *Algorithmic level:* In this format, all kinds of quantum gates may be used and subsumed into high-level building blocks. Loops are possible and the circuit structure might be described by constants, variables, and mathematical expressions. This level provides benchmarks on the most generic level and specific values are assigned to its variables, e.g., the number of iterations of a loop. The number of qubits can be specified.
2. *Target-independent level:* Here, loop unrolling, constant folding and propagation are conducted. Also, naive gate simplification is enforced. Again, the number of qubits can be adapted through the website and the used compiler can be selected.
3. *Target-dependent native gates level:* On this level, a particular native gate-set is chosen and the circuit is transpiled and optimized accordingly. Here, the native gate-set, the compiler used and its settings can be specified.
4. *Target-dependent mapped level:* Finally, a dedicated architecture layout is chosen and the circuit is mapped to the targeted device such that it satisfies all its connectivity constraints and becomes executable on the device. Similar to the previous level, the used compiler and its settings are selectable—in addition to the target device.

All relevant information of a generated benchmark is denoted in short in the file name and in detail within the file itself.

4.2 Accessibility

MQT Bench strives to meet the needs of many. To accomplish this goal, it has to be as user-friendly as possible. On the one hand, this involves the benchmark library itself. All of the high-level, algorithmic benchmark scripts and routines to generate representations for the individual levels are publicly available on GitHub (<https://github.com/cda-tum/mqt-bench>). This allows for complete transparency on how the respective circuits have been generated. Additionally, a Python package is provided such that each benchmark can be generated on-demand.

On the other hand, and most importantly, this involves the way in which users interface with the benchmark suite. While most benchmark suites are provided as an accumulation of benchmark files—making it hard to extract the desired set of benchmarks, MQT Bench includes an easy-to-use, no-coding-required web interface (<https://www.cda.cit.tum.de/mqtbench>) that provides the means to filter the large number of pre-generated benchmarks according to the specific needs of the user. This web interface and its underlying server software is also part of our Python package, such that every user can utilize the same interface locally without the need to access our publicly available server. A screenshot of the website’s user interface and its configuration options is shown in Fig. 3.

4.3 Algorithm Selection

To provide a broad spectrum of different benchmarks, MQT Bench comprises most of today’s de-facto standard quantum algorithms. This includes building blocks, e.g., Quantum Fourier Transform (QFT) and the Greenberger-Horne-Zeilinger state preparation (GHZ), up to higher-level algorithms, e.g., Grover’s [67] and Shor’s [66] algorithm. At the time of writing, MQT Bench comprises the following benchmarks:

- Amplitude Estimation (AE)
- Deutsch-Jozsa (DJ) algorithm
- Graph state preparation
- GHZ state preparation
- Grover’s algorithm
- Quantum Approximation Optimization Algorithm (QAOA)
- Quantum Fourier Transformation (QFT)
- Quantum Phase Estimation (QPE)
- Quantum Walk
- Random circuit
- Shor’s algorithm
- Variational Quantum Eigensolver (VQE)
- Three VQA ansatz functions with randomly initialized parameters: Two Local, Real Amplitudes and Efficient SU2
- W-State preparation

Additionally, MQT Bench provides several application benchmarks specifically targeting variational quantum algorithms, since those algorithms are especially promising in the NISQ-era [68]. To this end, we use the classification provided by IBM Qiskit’s application levels: Optimization, Machine Learning, Finance, and Nature:

- *Optimization*: Travelling salesman problem and vehicle routing
- *Machine Learning*: Quantum Neural Network (QNN)
- *Finance*: Portfolio optimization and option pricing
- *Nature*: Ground state estimation

4.4 Extendability

Quantum computing is a rapidly evolving area of research and, especially in recent years, numerous promising algorithms and applications have emerged. Any benchmark suite has to be extendable and must continuously evolve in order to remain relevant. MQT Bench is designed to be easily extendable in a multitude of ways:

- *Algorithms*: New applications and algorithms can be easily integrated into MQT Bench by providing the corresponding algorithmic level description. The generation of all other levels and options (such as the native gate-sets, architectures, and compilers with their settings) is automatically handled by the proposed library.
- *Native gate-sets*: So far, native gate-sets of superconducting quantum computers from IBM, Rigetti, and Oxford Quantum Circuits are provided—in addition to the gate-sets of IonQ’s and Quantinuum’s ion trap-based quantum computers. In the future, additional gate-sets (such as those of Google’s and or AQT’s quantum computers) can be realized by adopting the necessary compilation routines in the proposed library.
- *Hardware architectures*: All major players currently rely on a modular platform for the architecture of their devices in order to further scale their quantum computers. It is straightforward to extend MQT Bench’s list of available architectures to accommodate future architectures.

Although the four levels MQT Bench is based on cover today’s main use cases, the number of abstraction levels most likely is going to increase in the future. While the mapped level has been considered the last one before a circuit is sent to the quantum computer for execution, pulse-level programming is envisioned to give even more control to software developers as proposed in [69]. On the other end of the spectrum, higher-level abstractions and programming languages will be required to foster the adoption of quantum technology. Currently, a programmatic description of the benchmarks is needed on the algorithmic level. An even higher level where this programmatic description will be automatically derived from is also envisioned. *QUARK* [70], a framework for quantum computing application benchmarking consisting of four levels where the lowest level considers the whole compilation flow discussed in Section 2.1, constitutes one step towards this direction.

5 Evaluation

At the time of writing, MQT Bench (*v1.0.0*) considers two compilers, five native gate-sets, and seven devices ranging from 8 to 127 qubits. In this section, we provide an overview of several characteristics and statistics of the resulting pre-generated benchmarks. To this end, the following metrics are illustrated in Fig. 4:

Number of qubits: A first broad overview is given in Fig. 4a, which shows the relative frequency of the generated benchmarks per number of qubits and per compiler. The relative frequency of benchmarks is decreasing with an increasing number of qubits due to fewer devices being available for higher qubit numbers and the maximal generation time for a benchmark file being exceeded more frequently. Furthermore, the benchmark generation using the *TKET* compiler generally took more time than *Qiskit*—leading to a smaller number of generated benchmarks, especially on the target-dependent mapped level.

Distribution of target-dependent mapped level benchmarks: The distribution of the target device for the mapped level is shown in Fig. 4b, with the number of qubits denoted in brackets. As expected, the larger the device, the more benchmarks are created for it. On the target-dependent native gates level, there is an equal distribution with respect to the number of benchmarks created for each of the five native gate-sets.

Distribution of benchmark characteristics: In Fig. 4c, six different characteristics with values between 0 and 1 are evaluated for all pre-generated benchmarks. Five of those characteristics (Program Communication, Critical Depth, Entanglement Ratio, Liveness, and Parallelism) have been proposed in [55]. Furthermore, the percentage of multi-qubit gates is evaluated.

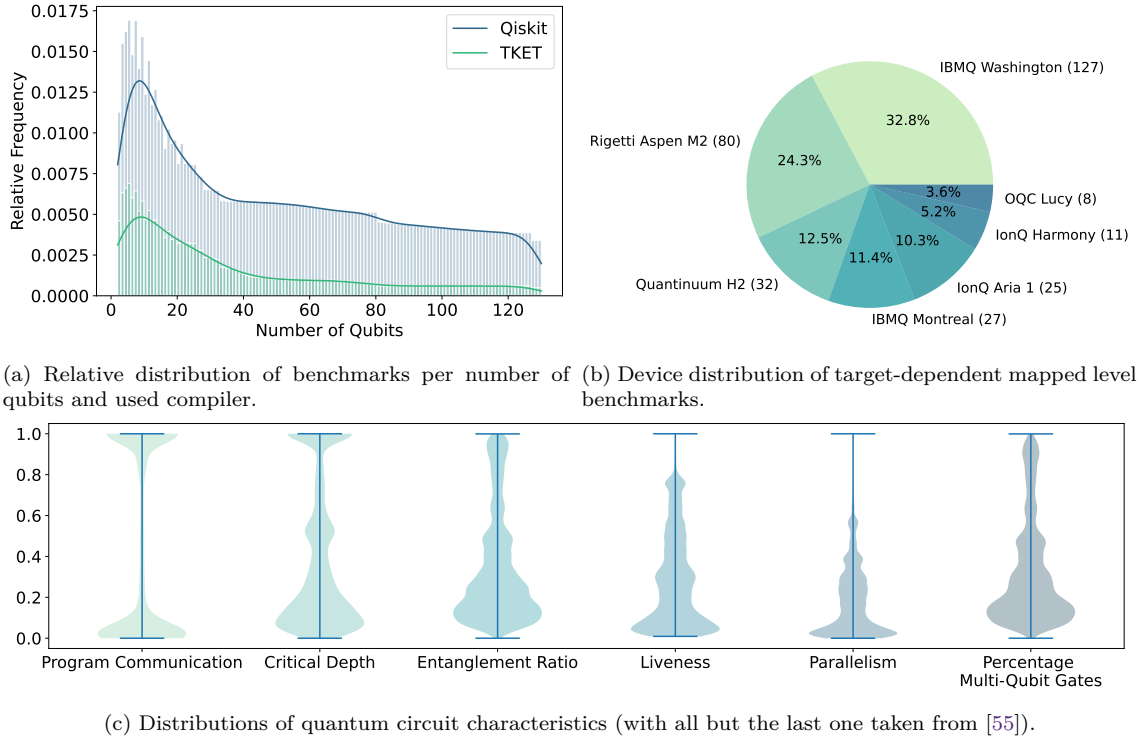


Figure 4: Insights into the provided benchmarks and their characteristics.

6 Conclusion

In this work, we proposed *MQT Bench*—a quantum circuit benchmark suite (as part of the *Munich Quantum Toolkit*, MQT) comprising different algorithms, compilers, native gate-sets, and target devices—resulting in more than 70,000 benchmark circuits ranging from 2 to 130 qubits on four abstraction levels. To keep this large number of benchmarks manageable, we provide an easy-to-use web interface (<https://www.cda.cit.tum.de/mqtbench>) allowing users to filter the benchmarks according to their needs. *MQT Bench* is also provided as a Python package (including the server software to start the web interface locally), such that each of the benchmarks can be easily generated on-demand. Furthermore, we give access to the open-source repository on GitHub (<https://github.com/cda-tum/mqt-bench>). By this, *MQT Bench* presents a first step towards serving a single benchmark suite for the whole quantum software stack—facilitating empirical evaluations of quantum software tools that are comparable, reproducible, and transparent.

Acknowledgments

This work received funding from the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation program (grant agreement No. 101001318), was part of the Munich Quantum Valley, which is supported by the Bavarian state government with funds from the Hightech Agenda Bayern Plus, and has been supported by the BMWK on the basis of a decision by the German Bundestag through project QuaST, as well as by the BMK, BMDW, and the State of Upper Austria in the frame of the COMET program (managed by the FFG).